

EasyCrypt User Guide

Riverside Research

Version 2.0

Nov 2025

This material is based upon work supported by the Defense Advanced Research Projects Agency (DARPA) and Naval Information Warfare Center Pacific (NIWC Pacific) under N66001-22-C-4020 and HR001122C0185.

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the DARPA and NIWC Pacific.

Distribution A: Approved for Public Release

Revision Sheet

Release No.	Date	Revision Description
1.0	11/14/2022	Initial Release as EasyCrypt Reference for Programmers
2.0	11/30/2025	Subordinated sections <i>Types</i> through <i>Axioms and Lemmas</i> under new section, <i>The Ambient Logic</i> , and expanded and reorganized them substantially Rewrote <i>Ambient Logic Proofs</i> Consolidated material on modules into a new section, <i>Programs and Program Logics</i> Consolidated most material on theories to a new section, <i>More on Theories</i> Added new section, <i>Pragmas</i> Substantially updated the <i>Quick Reference</i> appendix

Table of Contents

1	Introduction	1
2	Running EasyCrypt	2
3	Theories, Scripts and Steps	5
4	The Ambient Logic	6
4.1	Types, Functions and Expressions.....	6
4.2	Operators	10
4.3	Abstract Operators, Axioms and <code>Bool</code>	11
4.4	Rounding Out Types, Expressions and Axioms	14
4.4.1	Let Expressions.....	14
4.4.2	Conditional Expressions	14
4.4.3	Tuple Types	14
4.4.4	Record Types.....	15
4.4.5	Concrete Types.....	16
4.4.6	Symbolic Operator Names	17
4.4.7	Abbreviations	18
4.4.8	Shorthand Notations for Axioms	19
4.4.9	Scopes, Namespaces and Printing	19
4.5	EasyCrypt Library, Part 1	22
4.5.1	Pervasive	22
4.5.2	Logic, Part 1.....	23
4.5.3	AllCore and Core	23
4.5.4	Int and IntDiv.....	23
4.5.5	Real.....	25
4.6	Inductive Types	26
4.6.1	Syntax.....	26
4.6.2	Semantics	27
4.6.3	Operators on Inductive Types.....	29
4.6.4	Match Expressions	30
4.6.5	Destructors.....	31
4.6.6	Limitations of Inductive Types (Optional)	32
4.7	Polymorphic Operators.....	34
4.8	Polymorphic Types (Type Constructors)	37

4.9	Type Classes and Instances	37
4.10	EasyCrypt Library, Part 2	38
4.10.1	Logic, Part 2	38
4.10.2	List	41
4.10.3	Finite	43
4.10.4	FSet	44
4.10.5	FMap	45
4.10.6	Xint	47
5	Ambient Logic Proofs	48
5.1	Goals	48
5.2	Proof Tactics and Steps	50
5.2.1	The Simplify Tactic	51
5.2.2	The Done and Trivial Tactics	54
5.2.3	The Reflexivity Tactic	55
5.2.4	The Admit Tactic and Abort	55
5.3	Using the Context	55
5.3.1	The Introduction Tactical	56
5.3.2	Reverting an Assumption	60
5.3.3	The Clear, Subst and Assumption Tactics	62
5.3.4	The Pose Tactic	63
5.4	Leveraging Lemmas I: The Rewrite Tactic	64
5.4.1	Searching for and Locating Lemmas	64
5.4.2	Basic Rewriting	65
5.4.3	Generating Subgoals	68
5.4.4	Rewriting Using a Hypothesis	69
5.4.5	Rewriting from Right to Left	70
5.4.6	Occurrence Selectors	71
5.4.7	Rewriting in a Hypothesis	72
5.4.8	Specifying the Instantiation (Proof Terms)	74
5.4.9	Specifying the Match	77
5.4.10	Rewriting with an Anonymous Lemma	78
5.4.11	Rewriting with Multiple Proof Terms	81
5.4.12	Rewriting with a Choice of Proof Terms	84
5.4.13	Subst versus Rewrite	86

5.4.14	Hints	86
5.4.15	Folding and Unfolding	92
5.5	Case Analysis: The Case Tactic	96
5.5.1	The Case Tactic with No Arguments	96
5.5.2	The Case Tactic with One Argument.....	98
5.6	Conjunctive Goals: The Split Tactic	101
5.7	Leveraging Lemmas II: The Apply and Exact Tactics	102
5.7.1	Reasoning Backward	102
5.7.2	The Apply Tactic with No Arguments.....	108
5.7.3	Proof Terms Again (Optional)	109
5.7.4	The Apply Tactic with Multiple Arguments.....	112
5.7.5	Reasoning Forward	112
5.7.6	The Apply Tactic with an Anonymous Lemma	117
5.7.7	Proof by Induction: The Elim Tactic	118
5.8	The Smt Tactic.....	123
5.9	Combining Steps with Tacticals.....	125
5.9.1	The Try Tactical	125
5.9.2	The Do Tactical.....	126
5.9.3	The Alternative Tactical	127
5.9.4	The Chain Tactical	127
5.9.5	Variations on a Theme	129
5.9.6	The By Tactical	130
5.9.7	Tactical Precedence	131
5.9.8	Bullets.....	132
5.10	The Introduction Tactical Revisited	132
5.10.1	Applying a Lemma to the Top Assumption	137
5.10.2	The Case Introduction Pattern.....	139
5.10.3	Introduction Patterns in Odd Places	147
5.11	Specialized Proof Tactics.....	148
5.11.1	The Congr, Left and Right Tactics.....	148
5.11.2	The Exists Tactic	150
5.11.3	The Algebra Tactics	151
5.11.4	The Change Tactic	152
5.11.5	The Progress Tactic	153

5.11.6	The Solve Tactic.....	155
5.12	The Generalization Tactical.....	155
5.12.1	Generalization Patterns	156
5.12.2	Instantiating a Lemma	157
5.12.3	Abstracting an Expression.....	162
5.13	The Full Syntax of the Move Tactic	165
5.14	The Full Syntax of the Case Tactic.....	166
5.15	The Full Syntax of the Elim tactic	168
5.16	The Full Syntax of the Apply and Exact Tactics	169
5.17	The Full Syntax of the Rewrite Tactic.....	170
5.18	Generalization Tactics	171
5.18.1	The Have Tactic	171
5.18.2	The Suff Tactic.....	177
5.18.3	The Gen Have Tactic.....	177
5.18.4	The Wlog Tactic.....	180
6	Programs and Program Logics	182
6.1	Modules	182
6.1.1	Statements.....	183
6.1.2	Code Positions.....	187
6.1.3	Glob.....	188
6.1.4	Defining Modules by Exception	189
6.2	Module Types and Functors.....	191
6.3	Reasoning about Memories and Modules.....	195
6.4	Probability Distributions	199
7	Hoare Logic Proofs	201
7.1	Symbolic Execution	202
7.2	Loop Invariants.....	215
7.3	Random and Procedure Call Assignments	221
7.4	Abstract Procedure Call Assignments	225
7.5	Other Tactics	228
8	Probabilistic Hoare Logic Proofs	229
9	Probabilistic Relational Hoare Logic Proofs	229
10	Other Program Logic Tactics	229
11	More on Theories.....	229

11.1	Sub-Theories	229
11.2	Abstract Theories	231
11.3	Theory Cloning	231
11.4	Polymorphic (ish) Theories	234
11.5	Subtype Declarations	242
11.6	Ring and Field Instances.....	243
11.7	Sections.....	245
12	Pragmas.....	246
13	Appendix: Quick Reference.....	248
13.1	Lexical Details.....	248
13.1.1	Reserved Words	249
13.1.2	Symbolic Operator Names	249
13.2	Type Expressions and Declarations.....	251
13.3	Operator and Predicate Declarations	251
13.4	Typed Expressions.....	253
13.5	Axioms and Lemmas	254
13.6	Ambient Logic Proofs	256
13.6.1	Proof Tactics.....	256
13.6.2	Proof Tacticals.....	264
13.6.3	Introduction Patterns.....	265
13.6.4	Generalization Patterns	267
13.6.5	Proof Terms.....	267
13.6.6	Occurrence Selectors	268
13.6.7	Focus Indices	268
13.6.8	Repetition Markers	268
13.7	Print, Search and Locate	269
13.8	Modules	269
13.9	Module Types and Functors.....	273
13.10	Theories and Sections	274
13.11	Pragmas.....	278
13.12	Hoare Logic Tactics.....	279
13.12.1	Program Reasoning Tactics	279
13.12.2	Program Transformation Tactics.....	281
13.12.3	Program Specification Tactics	284

13.12.4	Automatic Tactics.....	284
13.13	Probabilistic Hoare Logic Tactics.....	285
13.13.1	Program Reasoning Tactics	285
13.13.2	Program Transformation Tactics.....	286
13.13.3	Program Specification Tactics	286
13.13.4	Automatic Tactics.....	287
13.14	Probabilistic Relational Hoare Logic Tactics.....	287
13.14.1	Program Reasoning Tactics	288
13.14.2	Program Transformation Tactics.....	289
13.14.3	Program Specification Tactics	290
13.14.4	Automatic Tactics.....	291
13.14.5	Advanced Program Tactics.....	291
13.15	Other Program Logic Tactics	291
13.15.1	Program Reasoning Tactics	291
13.15.2	Program Transformation Tactics.....	292
13.15.3	Program Specification Tactics	292
13.15.4	Advanced Program Tactics.....	293

List of Figures

Figure 1 Visualizations of the Type <code>intlist</code>	32
Figure 2 Visualizations of the Type <code>tree</code>	33
Figure 3 Visualizations of the Type <code>tree1</code>	33
Figure 4 Visualizations of the Type <code>tree2</code>	33
Figure 5 Visualizations of the Type <code>tree3</code>	34
Figure 6 Emacs running EasyCrypt in Proof General.....	50
Figure 7 The Do Tactical	126
Figure 8 Execution of $\tau_1; \tau_2; \tau_3$	128
Figure 9 Execution of $\tau_1. \tau_2. \tau_3$	129

List of Tables

Table 1 Regular Expressions for Renaming	232
Table 2 List of Pragmas	247

1 Introduction

EasyCrypt is an interactive computer program for stating and proving properties of probabilistic programs. It defines two distinct but interlocking languages or formalisms. The first is a simple imperative programming language called the module language. A module consists of a set of global variables (i.e., visible to other modules) and a set of procedures. A procedure consists of a set of local variables and a sequence of statements. Statements fall into a small set of conventional imperative statement types (assignment, `if`, `while`). One form of assignment statement, denoted `<$`, provides a primitive operator for sampling a value from a probability (sub-) distribution. Here are two sample modules from the reference manual, with minor editing:

```
(* Load (require) the theories [Bool] and [DBool]. [Bool] defines
 * the exclusive-or operator [(^^)], and [DBool] defines the
 * uniform distribution on [bool], which is named [{0,1}]. *)
require import Bool DBool.

(* G1.f() yields a randomly chosen Boolean value *)
module G1 = {
  proc f() : bool = {
    var x : bool;
    x <$ {0,1}; (* sample x from the distribution {0,1} *)
    return x;
  }
}.

(* G2.f() yields the exclusive-or of two randomly chosen
 * Boolean values *)
module G2 = {
  proc f() : bool = {
    var x, y : bool;
    x <$ {0,1}; y <$ {0,1};
    return x ^^ y;
  }
}.
```

The other language is logic. EasyCrypt has an “ambient” logic based on polymorphic typed lambda calculus. It has an elaborate type system, including inductive types and type inference, and treats functions as first-class values. EasyCrypt implements an interactive proof system with proof tactics and links to external automated theorem provers. EasyCrypt also has three sub-logics for reasoning about modules:

- Hoare logic (HL) for stating pre- and postcondition pairs of executing a procedure
- Probabilistic Hoare logic (pHL) for stating the probability of a probabilistic procedure’s execution resulting in a postcondition holding
- Probabilistic, relational Hoare logic (pRHL) for stating that the probability distributions of the outputs of a pair of probabilistic procedures are identical

Here are two sample lemmas, with their proofs, about the two modules presented above.

```
(* PRHL judgement relating [G1.f] and [G2.f.] [= {res}] means
```

```

    * the result of G1.f has same probability distribution as the
    * result of [G2.f] *)
lemma G1_G2_f : equiv[G1.f ~ G2.f : true ==> ={res}].
proof.
  proc. seq 0 1 : true. rnd {2}. skip. smt.
  rnd (fun (z : bool) => z ^^ x{2}). skip. smt.
qed.

(* [G1.f] and [G2.f] are equally likely to return true: *)
lemma G1_G2_true &m :
  Pr[G1.f() @ &m : res] = Pr[G2.f() @ &m : res].
proof. byequiv. apply G1_G2_f. trivial. trivial. qed.

```

EasyCrypt’s computational model is very simple. The statement types have formal semantics, so they can be reasoned about, but executed only symbolically. Memory consists of a single global map from variable names to values, including local procedure variables and parameters. Notably, it does not consist of an array of bytes; it does not have an execution stack; recursive procedures are not allowed; it does not have a memory heap for dynamically allocating objects; it does not have pointers or reference types. Further, the module language does not support encapsulation, packages, information hiding or any of that; it is not object-oriented, though a module can be thought of as a global singleton object, if one must. It has no support for input and output, character sets or strings.

The ambient logic resembles a functional programming language, except, again, it is not executable: it’s for proving theorems. Still, your skills in functional programming will come in handy.

The module language can refer to types and operators defined in the ambient logic for declaring variables and writing expressions, respectively. Thus, a good bit of a module, such as the right-hand sides of assignment ($\leftarrow$ and $\leftarrow \$$) statements, is technically logic. Conversely, the three sub-logics of the logic language can refer to modules, procedures, global variables and the contents of memory for stating and proving lemmas about them.

This User Guide describes the module and logic languages without assuming extensive background in programming or logic, but a basic understanding of the principles of both. For readers well-versed in software development, I point out places where that background might lead to unconscious assumptions that are false. These excursions are brief and should not distract readers who lack this background (bias).

The EasyCrypt reference manual is in the GitHub repository <https://github.com/EasyCrypt/easycrypt-doc> (which is sister to EasyCrypt/easycrypt). Only Chapters 1 to 3 are filled in: 4-7 are headings only. It is terse and can be difficult to understand if you’re not well-versed in logic and proof assistants. It is also somewhat out of date. On the other hand, it was written by an expert in both cryptography and formal logic, unlike this guide. The EasyCrypt Tutorial focuses on using the language to specify security properties formally, presenting much of the language along the way, then mentions some theorems to prove while providing no hints as to how.

2 Running EasyCrypt

This guide does not cover installing EasyCrypt. Instructions for that are in the GitHub repository.

You can run EasyCrypt in batch mode from the command line with a file name or interactively from within Emacs to execute a file one line at a time, etc. I really don't like Emacs, but this mode is useful enough to put up with it, though it is far from a full-blown Integrated Development Environment (IDE). This mode uses a general framework for running proof assistants in Emacs called Proof General.

It turns out you can use interactive mode from the command line, too, which is not as useful as Proof General but is good to try at least once. The first command you should run is

```
$ easycrypt -help
Usage: easycrypt [command] [options...] [args...]

* Available commands: compile, cli, config, runtest, why3config
...
```

This is valuable information, if a bit terse. Let's expand it a bit. To run EasyCrypt interactively from the command line, do

```
$ easycrypt cli # or you may omit "cli"
>> Copyright (c) 2012 IMDEA Software Institute
>> Copyright (c) 2012 Inria
>> Copyright (c) 2017 Ecole Polytechnique
>> Copyright (c) EasyCrypt contributors (see AUTHORS)
>> Distributed under the terms of the MIT license
>> Standard Library (theories/**/*ec):
>>     Distributed under the terms of the MIT license
>>
>> GIT hash: r2025.02-17-gb3f6888
[0|check]>
```

At this point, you can type EasyCrypt code. After each successful step (see *Theories, Scripts and Steps*), the integer in the prompt goes up.

- To exit EasyCrypt, enter

```
exit.
```

including the period. You can also type the EOF character (CTRL-D on Linux, CTRL-Z on Windows).

- Most EasyCrypt output is written to standard error. The following is written to standard output:
 - The "[n|check]>" prompt.
 - Output of `print` commands.

To run EasyCrypt with an input file, do

```
$ easycrypt compile Script.ec # or you may omit "compile"
```

While this runs, it displays a line of internal statistics. If there are no errors, the command completes with no output (unless the file contains `print` commands) and creates a JSON file named `Script.eco`. This is not an object or executable file: the `compile` command doesn't actually compile anything; it just checks it. You can prevent creation of the `eco` file with

```
$ easycrypt -no-eco Script.ec
```

If your file requires theories that are not in the EasyCrypt library or the current folder, you can tell it where to look:

```
$ easycrypt -I ../../common Script.ec # -R searches recursively
```

You may specify as many `-I` and `-R` options as you want.

If you want to know where EasyCrypt thinks the EasyCrypt library is, or other configuration details, do

```
$ easycrypt config
git-hash: r2025.02-17-gb3f6888
load-path:
  <system>@/home/robert/.opam/5.1.1/lib/easycrypt/theories
...
why3 configuration file
  </home/robert/.config/easycrypt/why3.conf>
EasyCrypt configuration file
  <none>
known provers: Z3@4.8.14, Alt-Ergo@2.2.0 [evicted:inconsistent/
overridden=false]
Commands PATH: /home/robert/.opam/5.1.1/lib/easycrypt/commands
System PATH:
...
```

I show the highlighted text because this happened to me once. I don't know why the Alt-Ergo prover got evicted but it is easy enough to prevent:

```
$ easycrypt -no-evict Alt-Ergo Script.ec
```

The `why3config` command reconfigures Why3. It assumes you want the Why3 configuration file in `~/.config/easycrypt` rather than in Why3's default location. You should create this folder:

```
$ mkdir ~/.config/easycrypt
$ easycrypt why3config
...
Save config to /home/rpg/.config/easycrypt/why3.conf
```

but you can specify the Why3 default one (or any other location) instead:

```
$ easycrypt why3config -why3 ~/.why3.conf
...
Save config to /home/rpg/.why3.conf
```

A note on syntax. EasyCrypt is a formal notation with a syntax and a semantics. I shall present the syntax only by example: the grammar can be found in the source code. EasyCrypt's use of white space, periods, commas, semi-colons and other punctuation between elements of lists (e.g., of steps, declarations, statements, parameters and so on), no doubt for subtle parsing-related reasons, seems arbitrary and even capricious at times and takes getting used to. Error handling is minimal: you get "parse error" no matter what syntactic rule you broke, semantic errors are no better and the first error halts processing. This is fine in interactive mode but in batch mode, not so much. Programmers have come to expect far more from their compilers and IDEs.

3 Theories, Scripts and Steps

At the top level, EasyCrypt code is organized into **theories**. Theories are used to bundle a set of related entities, such as a type, operators and predicates over the type, axioms and lemmas over the operators and predicates, and proofs of the lemmas. Each theory has its own namespaces for types, operators, modules and so on (see *Scopes, Namespaces and Printing*). Theories may contain other theories.

Top-level theories, which are also called **scripts**, are stored in text files and the name of the file is the name of the theory plus a file extension, which is “.ec” for *concrete* theories and “.eca” for *abstract* theories (see *More on Theories*). Theory names (and hence the names of files that contain them) must be valid EasyCrypt identifiers that begin with an uppercase letter (see *Lexical Details*). Top-level theory names are also subject to the rules governing file names in the host operating system. Scripts can be organized into folders but EasyCrypt takes no notice of this except when searching for a file. In particular, if scripts in different folders have the same name, only one of them will be used (whichever one is found first). In effect, top-level theories (scripts) live in a single, global namespace regardless of the folder structure, and duplicate names are silently ignored.

A theory consists of a sequence of **steps** terminated by periods. A step may

- declare types
- declare operators or predicates
- define an abbreviation or notation
- declare a module or module type
- state an axiom or lemma
- declare a type class or instance of one
- delimit the beginning or end of a proof
- apply a proof tactic
- require (load), import or export theories
- clone (copy) a theory
- delimit the beginning or end of a nested theory or a section of a theory
- set the value of a pragma or SMT option
- provide a hint to a proof tactic
- print a type, operator, predicate, axiom, lemma, module, module type or theory
- search for axioms and lemmas containing one or more expressions
- locate an axiom, lemma, operator or type

Note. Absent from this list are global variable and procedure declarations, because they occur only in modules.

Before a script can use a top-level theory (i.e., one defined in another file), it must first **require** it, which loads the file. Nested theories do not have to be required. Multiple theories can be required in one step by listing them separated by spaces:

```
require Int Bool IntDiv.
```

An entity declared in a theory is referenced using a **qualified name** consisting of the theory name, a period and the element’s name (nested theories require nested qualified names). Qualification requires that any symbolic operator names it uses can only be applied in prefix notation (e.g., `Int . (+) a 2`). A script can **import** a theory (after requiring it, if necessary) to make its elements usable without

qualification, as long as the unqualified name is unambiguous, with the side effect that prefix, infix or mixfix operators may be used as such (e.g., `a + 2`).

```
import Int Bool IntDiv.
```

Top-level theories may be required and imported in one step:

```
require import Int Bool IntDiv.
```

A nested theory can be imported once its parent theory has been required:

```
require import Ring.  
import IntID.          (* [IntID] is a sub-theory of [Ring] *)
```

Require is effectively transitive: when a theory is loaded, whatever that theory requires is necessarily also loaded. Best practice, however, is to require everything a given script needs. When a theory that has already been loaded is required, it is not loaded again.

Import is not transitive: a script that imports a theory must still use qualified names for entities contained in whatever theories that theory imports. However, a theory can also **export** a theory. This has the same effect as importing it, but in addition, whenever one theory imports (or exports) another, it automatically imports (or exports) its exports as well (and therefore does not even have to require them!). For example,

```
<in B.ec>  
require export A.  (* B can use A w/o qualified names *)  
<in C1.ec>  
require import B.  (* C1 can use A & B w/o qualified names *)  
<in D1.ec>  
require import C1. (* D1 can use C1 w/o qualified names but  
                  cannot use A or B w/o importing them *)  
  
<in C2.ec>  
require export B.  (* C2 same as C1 *)  
<in D2.ec>  
require import C2. (* D2 can use A, B & C2 w/o qualified names *)
```

Much more will be said about theories in *More on Theories*.

4 The Ambient Logic

There's a lot to cover before we get back to modules and program logic. This section describes the ambient logic and the following one presents proofs in it.

4.1 Types, Functions and Expressions

A **type** can be thought of as a finite or infinite set of **values**. EasyCrypt is strongly typed, meaning all values have types. Types are used in both logical and computational contexts. The ambient logic is a typed lambda calculus and the module language “inherits” the type system of the logic. The EasyCrypt parser does type inference, so in practice types can often be omitted (though clarity often suffers).

The types in type theory are similar to the types in programming languages, but they are also different and we are going to dwell on this point longer than may seem necessary. Types (and values) are a primitive notion in type theory, meaning they are assumed to exist but have no formal definition, only

informal explications (such as “a type is a set of values”). Moreover, values can only belong to one type. That is, there is no subtype relation between types, or in object-oriented terms, no subclass relation, no “is-a” and no inheritance. One cannot take the union or intersection of two types.

Types in imperative languages are normally tied to specific representations on actual computers, such as integers in 2’s complement binary with specific numbers of bits, IEEE single- and double-precision floating point numbers, or character sets such as 7-bit ASCII, various 8-bit extensions and Unicode, which can be encoded in UTF8, UTF16 or other encodings. These “basic” types can be combined in various ways, typically including arrays (which often includes the string type as a special case), structures (also called records) and objects. Languages and compilers vary in how much of the computer representation is exposed to the user.

In EasyCrypt, representation is not an issue. Values are written as expressions and what they “really” are is undefined. There are no implicit type conversions—a value of one type can only be converted to another type by an explicit, user-defined operator.

Functional languages often do a better job of hiding representation concerns and present a more mathematical view of types to the user but must still be concerned with being efficient. EasyCrypt does not concern itself with efficiency or even being executable, only in proving lemmas. Thus, there is no concept of lazy versus eager evaluation, for example.

The simplest kind of type is just a name, giving no hint as to what values it may contain.

```
type t.
```

The name can be any legal identifier. This kind of type is called an **abstract type**. Multiple abstract types can be declared in one step as a list separated by commas:

```
type t1, t2, t3.
```

For the sake of further discussion, let’s assume `int` is the type of all integers without worrying about how it can be defined in EasyCrypt. Let’s further assume that the symbols 0, 1, 2 and so on represent `int` values and that the expression `x + y` (where `x` and `y` have type `int`) represents the usual addition operation. We will come back to all of this later.

A **function** is a rule for computing one value from another. This is the lambda operator of untyped lambda calculus, $\lambda x.e$, where `x` is a **variable** and `e` is an **expression** that may refer to `x`. The variable `x` is called the **parameter** of the function and `e` is called its **body**. In typed lambda calculus, both `x` and `e` have types. The type of `x` is called the **domain** of the function and the type of `e` is called the **codomain**. All functions are **total**, meaning they are defined for every element of the domain. A common example of a **partial** function is division, where division by zero is not allowed. We will see much later how such edge cases can be handled.

A function is written in EasyCrypt as

```
fun (x : t) => e
```

For example, this function adds 7 to an `int`:

```
fun (x : int) => x + 7
```


If the type of x can be inferred from how it is used in e , its type may be omitted, along with the parentheses:

```
fun x => x + 7
```

Some notes on variables. The name of a function's parameter does not matter: the two expressions

```
fun (x : int) => 7 + x
fun (y : int) => 7 + y
```

denote exactly the same function. The variables x and y are called **formal variables**, meaning the exact symbol used is arbitrary. Functions that differ only in the choice of a variable name are called **alpha-equivalent** and the process of renaming a variable throughout a function is called **alpha conversion**. The variables in an expression like $7 + x$ on its own are called **free**. When the expression is placed inside a function, the function is said to **bind** the variables, which are then called **bound**. You may see error messages about free variables in your code, which usually means you used the wrong symbol in an expression, forgot to list a symbol as a parameter or messed up your parentheses.

A function is a “first-class object,” meaning it is “just a value.” Therefore, it has a type. The type of a function is written

```
t1 -> t2
```

where $t1$ is the domain (the type of the parameter) and $t2$ is the codomain (the type of the body). Therefore, the function above has type

```
int -> int
```

A function is **applied** to an expression (called an **argument**) using juxtaposition:

```
(fun (x : int) => x + 7) 15
```

This requires that the argument (15) have the same type as the parameter (x), which is `int` in this case. Anything else is an error. The type of the application expression is the function's codomain.

Beta (β) reduction is the process of simplifying a function application symbolically by “doing” what the function says and thereby eliminating it. This is a mechanical process of replacing every occurrence of the parameter in the body with the argument:

```
(x + 7) [x := 15] = 15 + 7
```

where $[x := 15]$ denotes substitution. There are numerous fussy details about substitution, such as when the argument is already using the variable x (one simply alpha converts the function to use a different name before substituting). At this stage of our discussion, any further reduction of this expression to, say, 22, is out of scope.

Since functions are values, a function can be the body of a function:

```
fun (x : int) => fun (y : int) => y + x
```

The type of this function is

```
int -> (int -> int)
```

If this function is applied to 7 (an `int`) and β -reduced, the result is

```

(fun (x : int) => fun (y : int) => y + x) 7 →β
(fun (y : int) => y + x) [x := 7] =
fun (y : int) => y + 7

```

which has type `int -> int`, as expected. This function can then be applied to 15, say, which we saw β -reduces to `15 + 7`. Successive applications would be written

```

((fun (x : int) => fun (y : int) => y + x) 7) 15

```

A note on the EasyCrypt parser. Function application is left-associative, so successive applications can be written with fewer parentheses:

```

(fun (x : int) => fun (y : int) => y + x) 7 15

```

In contrast, `->` is right-associative, so the type

```

int -> int -> int

```

is parsed as

```

int -> (int -> int)

```

The parameter of a function can have a function type and therefore a function can be passed as an argument to a function. For example,

```

fun (f : int -> int) => f 15

```

The type of this function is

```

(int -> int) -> int

```

The body of this function applies the parameter to the argument 15. If this function is applied to `fun (x : int) => x + 7` and β -reduced, the result is

```

(fun (f : int -> int) => f 15) (fun (x : int) => x + 7) →β
(f 15)[f := (fun (x : int) => x + 7)] =
(fun (x : int) => x + 7) 15 →β
15 + 7

```

This also demonstrates that, in general, the types

```

t1 -> (t2 -> t3)
(t1 -> t2) -> t3

```

are not the same.

You may have noticed that

```

fun (x : int) => fun (y : int) => y + x

```

behaves like a function with two parameters:

```

(fun (x : int) => fun (y : int) => y + x) 7 15 →β
(fun (y : int) => y + 7) 15 →β
15 + 7

```

The first application produces a function with one argument and the second produces a value that is not a function. EasyCrypt encourages this viewpoint by allowing the following abbreviated forms:

```
fun (x : int) (y : int) => y + x
fun (x : int, y : int) => y + x
```

Moreover, since in this case $t_1 = t_2$, you can further abbreviate it as

```
fun (x y : int) => y + x
```

Note that a list of parameter names of the same type is *not* separated by commas. These abbreviations do not affect the type of the function, which remains `int -> (int -> int)`.

All of the β reductions above have applied to an expression as a whole. Beta reduction can be extended to apply to subexpressions (function applications nested in some larger expression) in the obvious way. The extension has numerous “nice” properties, including *termination* (you can only reduce an expression a finite number of times) and *confluence* (if an expression has two function applications, it doesn’t matter which one you do first). The fully reduced form of an expression is therefore unique and is called **beta normal form**. There are other kinds of reduction that will be mentioned as we go and we will generically say one expression **reduces to** another (or write $M \rightarrow N$ with no subscript) to mean a sequence of zero or more reductions of various types.

4.2 Operators

Anonymous functions like those in the previous section are all well and good but are inconvenient to repeat over and over. It would be nice to be able to give them names (not unlike the formal variable f in one example). In addition, sometimes one wants to posit an abstract function without saying exactly what function it is. For these reasons, EasyCrypt has **operators**, which are simply *named functions*.

For example, the “ $x + 7$ ” function above can be given a name like this:

```
op plus7 : int -> int = fun (x : int) => x + 7.
```

This declaration is a *step* (see *Theories, Scripts and Steps*), a “complete sentence,” as it were, with a period at the end. This syntax is a “long” form that is never seen in practice. The following short form looks much more like an imperative language and it is easy to forget we’re still talking logic:

```
op plus7 (x : int) : int = x + 7.
```

The type after the colon is the codomain or *return type*, which must be the type of the expression following the `=`. Using type inference, we can write

```
op plus7 x = x + 7.
```

An operator is applied to an argument in the same way as a function, and β -reduction works the same way, too, except it’s called **δ -reduction** (δ for *definition*):

```
plus7 15  $\rightarrow_\delta$  (x + 7) [x := 15] = 15 + 7
```

Programmers, note that this is *not* the syntax used in most imperative programming languages, which is `plus7(15)`. You’ll mess that up many times and grow familiar with the error messages that result.

The name of an operator can be used anywhere the corresponding function can be used. For example, we can define an operator equal to the “ $f\ 15$ ” function above,

```
op apply15 (f : int -> int) : int => f 15.
```

and apply it to the `plus7` operator like this:

```
apply15 plus7
```

The type of this expression is `int` and it $\delta\delta$ -reduces to `15 + 7`.

Operators can have **tags**, which are a list of identifiers in square brackets after `op`. The only identifiers with meaning, however, are `full`, `lossless`, `opaque`, `smt_opaque` and `uniform`. See *The Smt Tactic* and *Probability Distributions* for what they mean.

4.3 Abstract Operators, Axioms and Bool

All of the operators in the previous section were **concrete**, meaning they had bodies. An **abstract operator** has a name and a type, but no body. For example, in abstract algebra, a *monoid* is a set equipped with an associative binary operation and an identity element. For example, the nonnegative integers with addition form a monoid, the identity element being 0. We can begin describing a monoid as follows:

```
type t.  
op plusm : t -> t -> t.  
op idm : t.
```

Note that the type of `idm` is not a function type. There are two ways to think about this. One is just to say that an operator doesn't have to be a function: it can have any type and simply represents a value of that type, just as `plusm` represents a value of type `t -> t -> t`.

To emphasize this viewpoint, the keyword `const` is synonymous with `op` but can only be used with non-functional types. Thus, we can write

```
const idm : t.
```

but

```
const plusm : t -> t -> t.      (* illegal *)
```

produces the error message

```
this operator type is (unifiable) to a function type
```

Non-functional operators are also called *nullary* operators, by analogy with *unary* and *binary*.

The other viewpoint is to say that `idm` is a function with an implicit domain that contains only one value. EasyCrypt has such a type, `unit`, whose only value is `tt`, and we can use it explicitly to declare

```
op idm : unit -> t.
```

With this definition, we would have to write `idm tt` every time we wanted to use it. It's not quite the same, but it is worth keeping this near equivalence in mind.

To finish the definition of a monoid, we need to state the associative and identity laws. In general, we need the ability to assert that an expression is true. Such an assertion is called an **axiom**. The axioms for a monoid are

```

axiom A (a b c : t) : plusm a (plusm b c) = plusm (plusm a b) c.
axiom left_id (a : t) : plusm idm a = a.
axiom right_id (a : t) : plusm a idm = a.

```

An axiom has a name that is an identifier, an optional list of parameters using the same syntax as operators and functions, a colon and an expression whose type is a **truth value**. An axiom states that, for any choice of values for the parameters, the body of the axiom is true.

So what is a truth value? In untyped logic, there is a distinction between truth values and other values. Propositional logic deals only with truth values: all symbols (propositional variables) are either true or false and the various connectives (and, or, not, implies) compute truth values from other truth values. Predicate logic introduces two new kinds of symbols: functions and predicates (a.k.a. relations). A function produces an untyped value and a predicate produces a truth value. Otherwise, they are the same. Some varieties of predicate logic further assume a built-in predicate for equality of values, $=$. Equality is a *congruence*, which goes beyond the definition of an equivalence relation to mean two values are indistinguishable in all contexts and thus interchangeable.

Typed logics *almost* replace the notion of truth values with the Boolean type. The EasyCrypt library defines the type `bool` and constants `true` and `false` as if it were just another type, and the ambient logic requires that the body of an axiom be an expression of type `bool`. This produces a chicken-and-egg problem: how do you write axioms about `true` and `false` (and operators such as `and` and `or`) without presupposing the very type you are using to give axioms meaning? The answer is that, of necessity, the type `bool` is built-in to the logic itself and is something of a “first among equals.”

EasyCrypt also retains the concept of a predicate, though in practice it is almost indistinguishable from an operator with codomain `bool`. For example, if we posit a function `mod` of type `int -> int -> int` that calculates the modulus of a number relative to a base (e.g., `mod 5 3 = 2`), then we can define a predicate for asserting that a specified `int` value is even like this:

```

pred even (x : int) = mod x 2 = 0.

```

The type of `even` is `int -> bool` and this declaration is just a syntactic variant of

```

op even (x : int) : bool = mod x 2 = 0.

```

Using either definition, `even 5` reduces to `false` and `even 6` reduces to `true`. EasyCrypt also assumes every type `t` has an operator (or predicate) $(=)$: `t -> t -> bool`, so one can also say `even 5 = false` and `even 6 = true`. Equality is another “first among equals” among operators: it is built-in to the logic.

Predicates have a few quirks, but as they are also superfluous, you can safely ignore them. The basics above are sufficient for understanding any you might see in the EasyCrypt library.

A note on Boolean. Programming languages generally treat Boolean as just another type, if they define it at all, but build it into the semantics of `if` and `while` statements, among others. Many still treat the integer 0 as meaning false and any non-zero value as true. Type theory certainly doesn’t allow that.

Two other constructs specific to truth values (`bool`) are the **quantifiers** `forall` and `exists`. The syntax is

```

forall (x : t), φ

```

```
exists (x : t),  $\varphi$ 
```

where φ denotes an expression of type `bool` that may reference the formal variable `x`. Like a function, a quantifier *binds* its variable over its body, φ . Unlike a function, the variable does not represent an input from which a value is derived. Instead, the meaning is the standard one: `forall` means that for any choice of `x`, φ is true, while `exists` means that there is at least one choice of `x` that makes φ true.

Nested quantifiers of the same kind can be abbreviated using the same syntax as nested anonymous functions: instead of

```
forall (x : int), forall (y : int), forall (z : bool), ...
```

you can write any of

```
forall (x : int, y : int, z : bool), ...
forall (x y : int, z : bool), ...
forall (x y : int) (z : bool), ...
```

and similarly for `exists`.

A parameter of an axiom has the same meaning as a `forall` quantifier. For example, the following axioms are equivalent:

```
axiom A (a b c : t) : plusm a (plusm b c) = plusm (plusm a b) c.
axiom A' (a b : t) :
  forall (c : t), plusm a (plusm b c) = plusm (plusm a b) c.
axiom A'' :
  forall (a b c : t), plusm a (plusm b c) = plusm (plusm a b) c.
```

A **lemma** has the form of an axiom and even the same meaning, but instead of being an assumption, it has to have a **proof**. For example, in a *commutative monoid* the binary operation is commutative as well as associative. One conventionally drops the axiom for right identity since it is now easily proved to follow from commutativity and left identity. A theory for a commutative monoid is therefore:

```
type t.
op plusm : t -> t -> t.
op idm : t.
axiom A (a b c : t) : plusm a (plusm b c) = plusm (plusm a b) c.
axiom C (a b : t) : plusm a b = plusm b a.
axiom left_id (a : t) : plusm idm a = a.
lemma right_id (a : t) : plusm a idm = a.
proof. (* to be covered later *) qed.
```

Equipped with types, abstract operators and axioms, we are now equipped to define many interesting structures beyond monoids, such as groups, rings, fields, vectors, matrices, polynomials, partial orders, lattices and more computer-ish *abstract data types*, such as finite sets, bags, lists, stacks, queues, trees, maps and so on. Then we can switch to the module language and explore algorithms for searching, sorting, encryption and all that.

Instead of that, there are several odds and ends about types, expressions and axioms to cover and then we'll take a first look at some of the theories in the EasyCrypt library.

4.4 Rounding Out Types, Expressions and Axioms

The next several sections discuss additional constructs or aspects of types, expressions and axioms in no particular order.

4.4.1 Let Expressions

A **let expression** is an alternative syntax for function application:

```
let x : t = e2 in e1
```

is equivalent to

```
(fun (x : t) => e1) e2
```

and might be more readable. For example, consider this (rather contrived) nested function applied to three arguments:

```
(fun (x y : int, z : bool) => y) 0 1 false
```

This can be written as nested let expressions as

```
let x : int = 0 in
  let y : int = 1 in
    let z : bool = false in
      y
```

Both expressions reduce to 1, but for some reason, reduction of a let expression is called **zeta (ζ) reduction**.

4.4.2 Conditional Expressions

A **conditional expression** is built from a `bool` expression and two expressions of the same type. There are two equivalent syntaxes:

```
x < 0 ? -x : x
if x < 0 then -x else x
```

Both of these examples reduce to `-x` if `x < 0` evaluates to `true` and to `x` otherwise. The type of these expressions is `int -> int`. Either can be used, for example, to define an operator in the obvious way:

```
op abs (x : int) : int = if x < 0 then -x else x.
```

Reduction of a conditional expression with a known condition is a kind of **iota (ι) reduction**. For example, the expression `if false then -x else x` reduces to `x`.

4.4.3 Tuple Types

A **tuple type** is a type with values consisting of ordered lists (*tuples*) of values. It is written as a list of two or more types separated by `*`. For example, the type

```
int * bool
```

is the type for all ordered pairs where the first element has type `int` and the second has type `bool`. A tuple type is also known as a **product type**, by analogy with Cartesian products of sets.

A tuple is written as a list of expressions between parentheses and separated by commas. For example,

```
(3, 2)
```

is a value of type `int * int`. Given a tuple, the individual components can be retrieved by applying a *projection*, which are numbered starting at 1, so for example,

```
(3, 2).`1
```

denotes the first element of the tuple, which is 3.

The `*` operator does not associate. That is, the following are distinct types:

```
t1 * t2 * t3 are triples of the form (a, b, c)
(t1 * t2) * t3 are pairs of the form ((a, b), c)
t1 * (t2 * t3) are pairs of the form (a, (b, c))
```

A tuple type can be the domain of a function, operator or predicate with multiple parameters. For example,

```
fun (xy : int * int) => xy.`2 + xy.`1
```

is the function that takes a pair of `int` and returns their sum. This function can be applied to an argument and reduced like this:

```
(fun (xy : int * int) => xy.`2 + xy.`1) (7, 15) →
(xy.`2 + xy.`1) [xy := (7, 15)] →
(7, 15).`2 + (7, 15).`1 →
15 + 7
```

Compare this to the nested function from *Types, Functions*, which we said was “like” a function with two parameters. Types of the form `t1 -> t2 -> t3` and `(t1 * t2) -> t3` are formally equivalent, so which to choose is a matter of taste. The nested function version is said to be the *curried* form of the tuple version, after the American mathematician and logician Haskell Brooks Curry (for whom three programming languages are also named: Haskell, Brook and Curry). The curried form is arguably more flexible and less cumbersome to write and is often preferred.

Similarly, a tuple type can be the codomain of a function or operator to return multiple results. For example,

```
op sq_cube (x : int) : int * int = (x * x, x * x * x).
```

computes both the square and cube of an integer.

4.4.4 Record Types

A **record type** is similar to a tuple type, except the elements (or *fields*) of the record are named and their order is not significant. A record type is defined by listing its **field projections** and their types between braces and separated by semi-colons (a semi-colon is also permitted after the last one):

```
type coordinates = {x : real; y : real; z : real}.
```

There is no shorthand for defining multiple field projections of the same type. Field names share a namespace with operators and predicates, so no other operator, predicate or field projection in the same theory can use the names `x`, `y` or `z`. See *Scopes, Namespaces and Printing* for more about namespaces.

A value of a record type (a record) is written as a list of equalities between braces with bars and separated by semi-colons (a semi-colon is also permitted after the last one). For example,

```
{| z = 4.0; y = 3.0; x = 2.0 |}
```

is a value of the `coordinates` type defined above. This is why field projection names have to be unique within a theory—so EasyCrypt can uniquely infer the type of a record value. As said previously, the order of the fields does not matter and was deliberately changed here.

A record can also be derived from another record by specifying just the fields that are different. For example, if `YouAreHere` has the value above and you move 5 units along the `y`-axis, we can write your new location as

```
{| YouAreHere with y = 8.0 |}
```

An individual field of a record value can be obtained by applying one of its field projections, so for example,

```
{| z = 4.0; y = 3.0; x = 2.0 |}.`x
```

denotes the real value projected by `x`, which is `2.0`. It is also legal to apply `x` as an operator, as in

```
x {| z = 4.0; y = 3.0; x = 2.0 |}
```

For example, another way to modify `YouAreHere` as above is to write

```
{| YouAreHere with y = YouAreHere.`y + 5.0 |}
```

Every record type automatically has an operator for constructing a value from a list of field values. Its name is “`mk_`” followed by the type name. For example, the `coordinates` type automatically has

```
op mk_coordinates (x' y' z' : real) : coordinates =
  {| x = x'; y = y'; z = z' |}.
```

Since this is an operator, the order of the arguments matters and follows the order given in the type declaration. The record above can therefore be constructed as the expression

```
mk_coordinates 2.0 3.0 4.0
```

Every record type automatically has an induction axiom. Its name is the type name followed by “`_ind`”. For example, the `coordinates` type automatically has

```
axiom coordinates_ind :
  forall (P : coordinates -> bool),
    (forall (i i0 i1 : real), P {| x = i; y = i0; z = i1; |}) =>
      forall (c : coordinates), P c.
```

This says that if any choice of three real values produces `coordinates` that satisfy `P`, then all `coordinates` satisfy `P`. We will discuss proof by induction in detail later.

4.4.5 Concrete Types

In addition to abstract, tuple and record types, a type can be defined in terms of other types. For example, if a particular type expression is going to be used many times, it might be convenient to give it a name:

```
type int_pair = int * int.
type int_binop = int -> int -> int.
```

These names are now synonymous with their corresponding types. They are called **concrete types**.

4.4.6 Symbolic Operator Names

We have seen that operators (and predicates) are applied to arguments using juxtaposition, with the operator name first (called *prefix* notation, as in `plusm x y`). In both mathematics and programming, however, many binary operators are conventionally written as symbols in *infix* notation (e.g., $x + y$) while other operators use an idiosyncratic syntax, such as `|x|` for absolute value or `a[i]` for an element of an array (these are sometimes called *mixfix* notations). To accommodate such usage, EasyCrypt allows an operator (but not a predicate) to have a **symbolic operator name** instead of an identifier. The rules for symbolic operator names are different for binary, unary and mixfix operators. The naming rules are, frankly, Byzantine:

Binary operators

- A backward slash (`\`) followed by a sequence of letters, digits, underscores and apostrophes can be used as a binary infix operator, such as `x \in m` or `S1 \subset S2`. These are called **named operators**.
- A sequence of the symbols `=`, `<`, `>`, `/`, `\`, `+`, `-`, `*`, `|`, `:`, `&`, `^` and `%` surrounded by backticks (```) can be used as a binary infix operator, such as `a `<` b` or `a `==//&=` b`.
- *Selected* sequences of these symbols without backticks can be used as binary infix operators. Almost any sequence of these symbols can be declared as an operator, but many will give parse errors when you try to use them infix. See *Symbolic Operator Names* in the appendix for details.

Common examples include

- `/\` and `&&` for “Boolean and” or the “meet” operation of a lattice (such as set intersection)
- `\/` and `||` for “Boolean or” or the “join” operation of a lattice (such as set union)
- `^` and `^^` for “Boolean xor” and for exponentiation
- `%/` for integer division
- `%%` for integer modulo
- `::`, `:::`, `::::` and so on for connectives, like adding an item to a list
- `<=` for any partial or total order, and `<` as the strict (not equal) version
- `=`, `==`, `===` and so on for any equivalence relation
- `+`, `++`, `+++` and so on for anything vaguely like addition, such as list concatenation

Unary operators

- A named operator can be used as a unary prefix operator, such as `\rev s` or `\conj z`.
- The symbols `!`, `+` and `-` can be used as unary prefix operators, such as `!b` (used for Boolean not) or `-5` (used for int and real negation).
- Since prefix notation is the default, the benefit is that these names are also assigned precedence levels.

Mixfix operators

- There are exactly four mixfix operators. Their names contain underscores to indicate where arguments go when they are applied. Typical usage is as follows:
 - `[]` is a nullary operator (a constant) for *empty* or *null* values, such as an empty list.
 - ``|_|` is a unary operator for *magnitudes* or *norms*, such as the absolute value of an integer (``|x - y|`), length of a vector or determinant of a matrix.
 - `_.[_]` is a binary operator for accessing part of an object by an *index*, such as an element of a list by position (`a.[i]`) or of a map by a key (`map.[key]`).
 - `_.[_<-_]` is a ternary operator for adding or overriding part of an object with an *index* and a *value*, such as `a.[i<-x]` or `map.[key<-value]`. These return a new object with one element changed—they do not modify the object. In the module language, however, `m.[k<-v];` is treated as a shortcut for the assignment `m <- m.[k<-v];`.
 - White space is allowed around and within arguments but not between symbols: ``| x |` and `map.[ key <- value ]` are legal, but ``|x|` and `a.[i]` are not.

When a symbolic operator name is used to declare an operator or use it as a value, additional punctuation is needed to indicate the intended usage (`list`, `map` and `complex` are notional types, for now):

```
op (+) (x y : int) : int.          (* binary infix operator *)
op (\in) (x : int, s : list) : bool. (* binary infix operator *)
op [-] (x : int) : int.            (* unary prefix operator *)
op [\conj] (z : complex) : complex. (* unary prefix operator *)
op "_.[_]" (m : map, k : key) : map. (* mixfix operator *)
```

This syntax is also required in qualified names:

```
Int.(+) (Int.[-] x) y
```

and can be used to apply the operator in normal prefix notation:

```
(+) ([-] x) y
```

where both are equivalent to

```
-x + y
```

4.4.7 Abbreviations

An alternative or abbreviated syntax can be provided for applying an operator that has already been defined. For example, suppose you wanted to use the binary infix operator `(++)` as an option for the monoid operator `plum`. You can define the abbreviation in either of these ways:

```
abbrev (++) (a b : t) : t = plum a b.
abbrev (++) = plum.
```

The difference is that the first one says `(++)` does the same thing as `plum`, while the second says it equals `plum`. This affects the `search` command but apparently nothing else. See *Searching for and Locating Lemmas*.

You can then write the associativity axiom like this:

```
axiom A (a b c : t) : (a ++ (b ++ c)) = ((a ++ b) ++ c).
```

Importantly, the symbol `(<>)` is an abbreviation for “not equal to”: one can write

```
! (a = b)
```

or one can write

```
a <> b
```

See also *Scopes, Namespaces and Printing*, below.

4.4.8 Shorthand Notations for Axioms

EasyCrypt has some shorthand notations for declaring axioms.

To declare an abstract operator with an axiom in one step, you can write:

```
op N : {int | 0 <= N} as N_ge0.
```

which has the same meaning as

```
op N : int.  
axiom N_ge0 : 0 <= N.
```

This works with operands of any type and with any `bool`-valued expression, such as

```
op f : {t1 -> t2 | is_additive f} as f_is_additive.  
op k : {int -> real | forall i, 0%r < k i <= 1%r} as k_in_unit.
```

A shorthand for an axiom that repeats the definition of a concrete operator is exemplified by

```
op sq_cube (x : int) : int * int = (x * x, x * x * x)  
  axiomatized by sq_cube_def.
```

which has the same meaning as the declaration followed by the axiom:

```
op sq_cube (x : int) : int * int = (x * x, x * x * x).  
axiom sq_cube_def :  
  forall (x : int), sq_cube (x) = (x * x, x * x * x).
```

This may seem redundant, but the definition and the axiom are used differently in proofs.

4.4.9 Scopes, Namespaces and Printing

A theory defines a **scope** for the names (identifiers and symbols) declared within it. This ties each occurrence of a name to its containing theory and means the same name can be used in multiple theories.

Within a theory there are also many **namespaces**. Names in different namespaces can be the same, but within a namespace all names must be distinct. There are six namespaces:

- type
- op – operators, predicates, field projections and abbreviations
- axiom – axioms and lemmas
- module
- module type
- theory – meaning sub-theories

Top-level theories occupy a single, global namespace of their own. As stated previously, EasyCrypt is not able to detect when two top-level theories in a file system have the same name.

The **print** step displays the declaration of a name. For example,

```
print coordinates.
```

displays

```
* In [type declarations]:  
  
type coordinates = { x : real; y : real; z : real; }.
```

If there had been an operator named `coordinates` and also a lemma, it would have displayed them as well. You can optionally specify a namespace and see only declarations for names in that space, which provides a convenient way to remember what the namespaces are. For example,

```
print type coordinates.
```

displays

```
type coordinates = { x : real; y : real; z : real; }.
```

and

```
print op plus7.
```

displays

```
op plus7 (x : int) : int = x + 7.
```

Since field projections and predicates share a namespace with operators, you can also do

```
print op x.  
print op even.
```

to display

```
op x : coordinates -> real = < 1-th projection of coordinates >.  
pred even (x : int) = mod x 2 = 0.
```

For symbolic operator names, don't forget to use the appropriate punctuation:

```
print op (+).  
print op [\conj].  
print op "_.[_]".
```

Abbreviations also share a namespace with operators. For example,

```
print op (++).
```

displays

```
abbrev (++) (a b : t) : t = plusm a b.
```

The `print` step recognizes a few other categories in addition to namespaces. These will be mentioned as they occur and are summarized in the appendix.

The `print` step does not display declarations exactly as they were entered:

- It has its own ideas about spaces, line breaks and indentation
 - The line width can be set from the command-line:

```
$ easycrypt -pp-width n
```

The default is 80.

- It displays `const` as `op`
- It displays applications of record constructors (`mk_t v1 ...`) as record values (`{| f1 = v1; ...|}`)
- It displays applications of record projections (`f1 v`) in postfix notation (`v.`f1`)
- It displays full type information, which is handy for debugging troublesome code
- It does not display type arguments for polymorphic operators (`f<:t>`), which would also be handy for debugging troublesome code (see *Polymorphic Operators*)
- It does not display tags when printing operators
- It eliminates as many parentheses as it can using precedence and associativity rules
- It displays axioms and lemmas using `forall` rather than parameters
- It uses abbreviations wherever possible
 - Exception: if an abbreviation is declared with the `-printing` tag, it is ignored when printing. For example,

```
abbrev [-printing] (++) (a b : t) : t = plusm a b.
```

For example, given the declaration

```
const test = x (mk_coordinates 2.0 3.0 4.0).
```

the step

```
print test.
```

displays

```
op test = {| x = 2.0; y = 3.0; z = 4.0; |}.`x.
```

As another example, the monoid axiom `C` above is displayed as

```
axiom C: forall (a b : t), plusm a b = plusm b a.
```

but once the abbreviation `(++)` is defined, it changes to

```
axiom C: forall (a b : t), a ++ b = b ++ a.
```

Finally, the built-in abbreviation for “not equal to” is printed as its own definition:

```
abbrev (<>) ['a] (x y : 'a) : bool = x <> y.
```

I assume this is a minor bug.

A **binder** (an operator, predicate, axiom or lemma declaration or a function, quantifier or let expression) also defines a scope for its parameters/variables that is limited in extent to its body, as does a module or module type for its procedures, a module for its global variables and a procedure for its parameters and local variables. Variables, parameters and procedures can only be printed, however, by printing the declaration that contains them.

4.5 EasyCrypt Library, Part 1

EasyCrypt starts as “empty” as possible, with no pre-defined types, operators or axioms except for minimal support for truth values and a parser that can read numbers and lists. The first theories it loads from the library are in the prelude folder. Theories in this folder are loaded automatically, so there is no need to `require` them (this can be overridden with the `-boot` command line option). We will start with two of them. You wouldn’t learn C, Python or Java without also learning the standard libraries, would you?

4.5.1 Pervasive

The `Pervasive` theory declares the following fundamental abstract types and operators, all of which seem to have some degree of built-in support:

```
type unit.
op tt : unit.
type bool.
op false, true : bool.
op [!] : bool -> bool.
op (||) : bool -> bool -> bool. (* short-circuit OR *)
op (\/) : bool -> bool -> bool. (* OR *)
op (&&) : bool -> bool -> bool. (* short-circuit AND *)
op (/\\) : bool -> bool -> bool. (* AND *)
op (=>) : bool -> bool -> bool. (* implies *)
op (<=>) : bool -> bool -> bool. (* Boolean equality or iff *)
op (=) ['a] : 'a -> 'a -> bool. (* equality for any type
                                * See Polymorphic Operators *)
abbrev (<>) (x y : 'a) = ! (x = y). (* not equals for any type *)
type int.
type real.
type 'a distr. (* See Probability Distributions *)
op mu : 'a distr -> 'a -> bool -> real. (* ditto *)
op witness ['a] : 'a. (* See Polymorphic Operators *)
```

Note that none of these have axioms in this theory. They are “stubs” to be elaborated in other theories or by other means, such as integration with SMT solvers.

The Boolean operators `&&` and `||` are logically equivalent to `/\\` and `\\/`, respectively, but model the “short-circuit” operators found in many programming languages: they only evaluate the second argument if the result is not already determined by the value of the first. A common idiom, for example, is “`n < len && a[n] != 0`,” where the array access `a[n]` is not performed if the range check on `n` fails, since `false /\\ p` is always `false`. When the theorem prover has to consider the arguments of `p && q` separately, it treats them as `p` and `p => q`, while `p || q` is treated as `p` and `!p => q`.

Note that `/\\` and `=>` are related in a way reminiscent of tuple and functional types. Expressions of the form `A => (B => C)` and `(A /\\ B) => C` are formally equivalent, so which to choose is a matter of taste but the “curried” form is often preferred. For example, rather than write

$$A /\\ B /\\ C /\\ D => E$$

we often prefer to write

$$A => B => C => D => E$$

Note that, to the parser, `=>` is right-associative, just like `->`, while `/\` is left-associative (unlike the tuple syntax `*`, which is non-associative) and has a higher precedence than `=>`.

4.5.2 Logic, Part 1

This theory contains much that we are not prepared to discuss yet but it also contains over a hundred low-level lemmas involving `bool`, including such classic gems as

```
lemma implybT (b : bool) : b => true.
lemma implyFb (b : bool) : false => b.
lemma neg_imply (a b : bool) : !(a => b) <=> a /\ !b.
lemma negb_and (a b : bool) : !(a /\ b) <=> !a \/ !b.
lemma negb_or (a b : bool) : !(a \/ b) <=> !a /\ !b.
lemma contra (b c : bool) : (c => b) => (!b => !c).
```

It also defines the XOR operator,

```
op (^) (b1 b2 : bool) : bool = if b1 then !b2 else b2. (* XOR *)
```

with 19 lemmas about it.

If you find yourself working on a proof and need to do some low-level manipulations on `bool` expressions, look here.

4.5.3 AllCore and Core

Located in the `core` folder, the `AllCore` theory is purely a convenience. It requires and exports five very commonly used theories: `Core`, `Int`, `Real`, `Xint` and `Ring.IntID`.

The `Core` theory defines no types and only two operators but has some additional lemmas about things defined in the `Logic` theory. None of them seem especially critical, but I guess you never know.

4.5.4 Int and IntDiv

The `int` type is defined in `Pervasive` but is of limited use without the additional declarations in `Int`. `Int` is in the `datatypes` folder.

The parser recognizes arbitrary-precision non-negative numeric literals of type `int` written in the usual way, as sequences of digits: `0`, `1`, `23` and so on. The unary `[-]` operator is used to produce negative `int` values, such as `-0` (which equals `0`, of course), `-1` and `-23`, but only after the `Int` theory has been required and imported.

The `Int` theory contains 71 axioms and 52 lemmas. Operators are

- `add` (abbreviated `(+)`) – addition
- `opp` (abbreviated `[-]`) – additive inverse (opposite or negative)
- `mul` (abbreviated `(*)`) – multiplication
- `lt` (abbreviated `(<)`) – less-than comparison
- `le` (abbreviated `(<=)`) – less-than-or-equal-to comparison
- `absz` (abbreviated `"`|_|"`) – absolute value
- `(-)` – subtraction (an abbreviation for `x + (-y)`)
- `b2i` – converts `bool` to `int` as `false = 0`, `true = 1`
- In sub-theory `IterOp`, three polymorphic operators (see *Polymorphic Operators*) for iterating a function a specified number of times:

- `iteri n opr x` – applies the function `opr : int -> 'a -> 'a` for $i = n - 1$ down to 0 starting with `x`. For example,
 - `iteri 0 opr x = x` (opr applied 0 times)
 - `iteri 1 opr x = opr 0 x`
 - `iteri 2 opr x = opr 1 (opr 0 x)`
 - `iteri 4 (fun i x => i + x) 5 = 3 + (2 + (1 + (0 + 5)))`
- `iter n f x0` – applies the unary function `f : 'a -> 'a` n times starting with `x0`.
 - `iter 0 f x0 = x0` (f applied 0 times)
 - `iter 1 f x0 = f x0`
 - `iter 2 f x0 = f (f x0)`
 - `iter 4 (fun x => 2 * x) 5 = 2 * (2 * (2 * (2 * 5)))`
- `iterop n opr x z` – applies the binary function `opr : 'a -> 'a -> 'a` $(n - 1)$ times starting with `x`, or if $n = 0$ it returns `z`
 - `iterop 0 opr x z = z` (opr applied -1 times, so to speak)
 - `iterop 1 opr x z = x` (opr applied 0 times)
 - `iterop 2 opr x z = opr x x`
 - `iterop 4 (+) 5 1 = 5 + (5 + (5 + 5))`
- `odd z` – returns whether `z` is odd (not divisible by 2)
 - `odd 0 = odd 2 = ... = false` (also -2, -4 and so on)
 - `odd 1 = odd 3 = ... = true` (also -1, -3 and so on)
- `min a b = if (a < b) then a else b`
- `max a b = if (a < b) then b else a`

Many more lemmas about these operators are in the `StdOrder.IntOrder` theory.

`IntDiv` is in the `algebra` folder. It defines commonly used infix binary operators that relate to integer division:

- `(% /)` – the integer quotient, so `15 % / 4 = 3`
 - `m % / d` is an abbreviation for `(edivz m d) .`1`, so you must use `search edivz.` to locate lemmas involving `(% /)`.
 - Note that `m % / 0 = 0` for all `m`, including `0 % / 0 = 0`
- `(%%)` – the integer remainder, so `15 %% 4 = 3`
 - `m %% d` is an abbreviation for `(edivz m d) .`2`, so you must use `search edivz.` to locate lemmas involving `(%%)`.
 - Note that `m %% 0 = m` for all `m`, including `0 %% 0 = 0`
- `(% |)` – whether one `int` evenly divides another, so `4 % | 15 = false`

Other operators include

- `gcd` – greatest common divisor of two ints, so `gcd 12 18 = 6`
- `coprime` – whether two ints are coprime (their `gcd = 1`), so `coprime 15 4 = true`

- `inv` – a multiplicative inverse of `a` with respect to `p`; the inverse is not unique, so `inv 4 15` might return 4 because $(4 * 4) \% 15 = 1$. Returns `a` if there is no inverse.
- `prime` – whether an `int` is prime, so `prime 15 = false`

4.5.5 Real

The `real` type is defined in `Pervasive` but is of very limited use without the additional declarations in `Real`. `Real` is in the `datatypes` folder.

The parser recognizes arbitrary-precision non-negative numeric literals of type `real` written in the usual way, as sequences of digits with a decimal point: `0.1`, `3.14159` and so on, but only after the `Real` theory has been required and imported. The decimal point must be preceded and followed by a digit: `3.` and `.3` are illegal. The unary `[-]` operator is used to produce negative `real` values, such as `-0.0` (which equals `0.0`, of course), `-1.5` and `-3.333`.

All `real` literals are derived from `int` literals using the `from_int` operation. The symbolic operator name `(%r)` is an abbreviation for `from_int`. Oddly, this form is not listed as a legal symbolic name and it is used postfix in spite of having parentheses around it. Literals are printed as mixed fractions:

```
real pi_2 = 3.14.
print op pi_2.
```

displays

```
op pi_2 : real = 3%r + 7%r / 50%r.
```

Operators on `real` values are

- `add` (abbreviated `(+)`) – addition
- `opp` (abbreviated `[-]`) – additive inverse (opposite or negative)
- `mul` (abbreviated `(*)`) – multiplication
- `inv` – multiplicative inverse (reciprocal)
- `lt` (abbreviated `(<)`) – less-than comparison
- `le` (abbreviated `(<=)`) – less-than-or-equal-to comparison
- `(>)` – an abbreviation for `(<)` with its arguments reversed
- `(>=)` – an abbreviation for `(<=)` with its arguments reversed
- `"`|_|"` – absolute value
- `(-)` – subtraction (an abbreviation for `x + (-y)`)
- `(/)` – division (an abbreviation for `x * (inv y)`)
- `b2r b` – an abbreviation for `if b then from_int 1 else from_int 0`
- `nonempty E` – whether a function `E : real -> bool` is true for any values
- `ub E z` – whether `z` is an upper bound for all values for which `E : real -> bool` is true
- `has_ub E` – whether `(ub E)` is nonempty
- `has_lub E` – whether `E` is nonempty and `(ub E)` is nonempty
- `lub E` – the smallest value that is an upper bound for a function `E : real -> bool`
- `intp x` – the largest integer `i` such that `i <= x`
- `floor x` – the largest integer `i` such that `i <= x`
- `ceil x` – the smallest integer `i` such that `x <= i`
- `exp x n` (abbreviated `(^)`) – x^n where `x` is `real` and `n` is `int`

Many lemmas about these operators are in the `Real` theory and many more are in the `Real.RField` sub-theory and the `StdOrder.RealOrder` theory.

4.6 Inductive Types

4.6.1 Syntax

An **inductive type** is an abstract type declared by listing one or more **constructors** between square brackets and separated by vertical bars (a bar is also permitted before the first one):

```
type suit = [Spades | Clubs | Diamonds | Hearts].
type int_or_bool = [First of int | Second of bool].
type intlist = [nil | cons of (int * intlist)].
```

A constructor is an abstract operator whose codomain is the type being defined. In these examples, `Spades` and `nil` are constants (or nullary constructors) of type `suit` and `intlist`, respectively. For the others, the token `of` followed by a type expression indicates the type of the parameter, so `First` has type `int -> int_or_bool`, `Second` has type `bool -> int_or_bool` and `cons` has type `(int * intlist) -> intlist`.

Constructors belong to the `op` namespace. For example, the step

```
print op Diamonds.
```

displays

```
op Diamonds : suit = < 3-th constructor of suit >.
```

As a reminder, this means a constructor cannot have the same name as an operator, predicate, field projection, abbreviation or other constructor in the same theory. A constructor's name can be a symbolic operator name and used accordingly.

You may already be wondering if `cons` can be declared in curried form rather than with a domain that is a tuple, so that we can write `cons 1 (cons 2 nil)` instead of `cons (1, cons (2, nil))` or even write `cons 1` as an expression of type `intlist -> intlist`. To appreciate the difficulty this poses, consider the attempt

```
type intlist = [nil | cons of (int -> intlist)].    (* no cigar *)
```

The type of `cons` is now

```
(int -> intlist) -> intlist
```

which is *not* the intended nested function,

```
int -> (intlist -> intlist)
```

A special syntax is provided to produce the desired effect:

```
type intlist = [nil | cons of int & intlist].
```

Note that parentheses are *required* for `cons of (int * intlist)` and *forbidden* for `cons of int & intlist`. The `'&'` syntax is also used to declare curried abstract predicates. It is needed because inductive types and predicates both involve a function type with an implicit codomain.

Inductive types encompass multiple constructs common in imperative programming languages:

- *Enumerated types* or *enumerations* (a list of constants, such as `suit`)
- *Variant types* (such as `int_or_bool`), also known as a *disjoint union*, *coproduct* or *sum* type
- *Recursive types* (such as `intlist`), which are commonly implemented with pointers (linked lists, trees, etc.)

More generally, they are known as *algebraic data types*. Constructors are also known as *injections*, *inclusion maps* or *tags*. Inductive types support well-founded recursion and induction. More will be said on this shortly.

4.6.2 Semantics

An inductive type has two implicit properties that eliminate the need for many axioms. First, the set of expressions containing only the constructors of an inductive type (and their arguments) collectively account for all the values of the type, which is exemplified by the slogan, “no junk” (there are no other values). Second, every value is distinct, which is exemplified by the slogan, “no confusion” (no value is represented by more than one expression). Another way to say this is that inductive types have *initial semantics*.

That is, the type `suit` has exactly the four values listed, no two of which are equal. To say this explicitly would take many axioms:

```
axiom all_suits : forall (s : suit), s = Spades \/ s = Clubs \/
  s = Diamonds \/ s = Hearts.
axiom s_is_not_c : Spades <> Clubs.
axiom s_is_not_d : Spades <> Diamonds.
axiom s_is_not_h : Spades <> Hearts.
axiom c_is_not_d : Clubs <> Diamonds.
axiom c_is_not_h : Clubs <> Hearts.
axiom d_is_not_h : Diamonds <> Hearts.
```

The type `int_or_bool` has an infinite number of values: for each `int n`, there is a corresponding `int_or_bool` value denoted `First n` and there are two more values, `Second false` and `Second true`. Axioms for this type would be

```
axiom all_int_or_bools : forall (ib : int_or_bool),
  (exists (n : int), ib = First n) \/
  (exists (b : bool), ib = Second b).
axiom f_is_not_s :
  forall (n : int, b : bool), First n <> Second b.
axiom f_inj : forall (m n : int), First m = First n => m = n.
axiom s_inj : forall (a b : bool), Second a = Second b => a = b.
```

Axioms `f_inj` and `s_inj` state that `First` and `Second` are *injective* operators: distinct inputs produce distinct outputs. To see this more clearly, consider the contrapositive of `f_inj`, which is `m <> n => First m <> First n`.

The type `intlist` is also infinite and it has a recursive definition. Its values (using the tuple version) include `nil`, `cons (1, nil)`, `cons (2, nil)`, ..., `cons (1, cons (1, nil))` and so on for every finite sequence of `cons` applied to a succession of `ints` and ending with `nil`. Its “no junk” and “no confusion” axioms are left as an exercise for the reader. (I had to say that at least once in this guide.)

You cannot change this behavior. If you add an axiom like

```
axiom Idem (a : int, l : intlist) :  
  cons (a, cons (a, l)) = cons (a, l).          (* Don't do this *)
```

in an attempt to make `intlist` behave more like a set, or anything else that would imply that two distinct constructor expressions denote the same value, all you accomplish is to make your theory inconsistent.

A note from the field. The axioms above are for explanatory purposes and cannot be used or printed, but something like them surely exists under the covers, as it were. I once defined an inductive type with a few hundred constants and in a proof EasyCrypt had to determine if a variable was equal to a particular one. I let it think about that for several minutes before giving up and killing the process. Apparently, it was working its way through the equivalent of hundreds of thousands of axioms stating that each constant was not equal to any other. Pro tip: keep the number of constructors under, say, 50.

For every inductive type, EasyCrypt automatically generates two (very real) axioms whose names are the name of the type followed by “_ind” and “_case”. These are called *induction axioms*, by analogy with mathematical induction over the natural numbers. Induction over the constructors of an inductive type is also called *Noetherian induction* (after [Emmy Noether](#)), *well-founded induction* or *structural induction*. For `suit`, these axioms are

```
axiom suit_ind:  
  forall (P : suit -> bool),  
    P Spades => P Clubs => P Diamonds => P Hearts =>  
      forall (s : suit), P s.  
axiom suit_case:  
  forall (P : suit -> bool),  
    P Spades => P Clubs => P Diamonds => P Hearts =>  
      forall (s : suit), P s.
```

Since the constructors for type `suit` are all constants, the two axioms end up being the same. They state that any property `P` that is true of `Spades`, `Clubs`, `Diamonds` and `Hearts` is true of all `suits` (because there are no other values: no junk).

The induction axioms for the recursive type `intlist` (with the curried form of `cons`) are a bit more interesting:

```
axiom intlist_ind:  
  forall (P : intlist -> bool),  
    P nil =>  
      (forall (i : int) (i0 : intlist), P i0 => P (cons i i0)) =>  
        forall (i : intlist), P i.  
axiom intlist_case:  
  forall (P : intlist -> bool),  
    P nil =>  
      (forall (i : int) (i0 : intlist), P (cons i i0)) =>  
        forall (i : intlist), P i.
```

Each one starts with `P nil` as the base case, followed by the induction hypothesis, followed by the conclusion that `P` is true of all `intlists`. The `intlist_ind` axiom has the “classic” induction

hypothesis where if P is true of a list $i0$, it is also true of all lists that add one element to $i0$. The `intlist_case` axiom has a simpler induction hypothesis that just states P is true of all lists constructed by `cons`.

We will see in *Case Analysis: The Case Tactic* and *Proof by Induction: The Elim Tactic* how induction axioms are used in proofs.

4.6.3 Operators on Inductive Types

A concrete operator over an inductive type is defined by cases, with one case per constructor. Each case consists of a **pattern** and a corresponding expression that denotes the value of the operator for all arguments matching that pattern. A pattern consists of a constructor applied to the appropriate number of **pattern variables**. For example, using the curried version of `intlist`, one can define

```
op size (xs : intlist) =
  with xs = nil => 0
  with xs = cons y ys => 1 + size ys.
```

This definition has two patterns for matching `xs`, which are `nil` and `cons y ys`, where `y` and `ys` are pattern variables. For example, to reduce the expression

```
size (cons 0 (cons 1 nil))
```

the argument is matched to the pattern in the second case with `y = 0` and `ys = (cons 1 nil)`. The value for that case is `1 + size (cons 1 nil)`. This call to `size` matches the second pattern again, this time with `y = 1` and `ys = nil` and a value of `1 + size nil`, where `size nil` matches the first pattern and has value 0. The recursive calls unwind to give the final result, `1 + (1 + 0)`. This procedure is called ***iota (i) reduction*** (*i* is for *inductive*).

The `size` operator for the tuple version of `intlist` looks like this:

```
op size (xs : intlist) =
  with xs = nil => 0
  with xs = cons y => 1 + size y.`2.
```

where the pattern variable `y` is (inferred to be) a tuple. (NOTE: I swear this used to work but in the r2022.04-198-beb00ea4 and later versions it gives a “syntactic termination check failure” error.) A pattern cannot use a literal tuple such as

```
with xs = cons (y, ys) => 1 + size ys. (* doesn't work *)
```

This results in a parse error at the left parenthesis.

For functional programmers, the `size` function can be written to use *tail recursion* by introducing an auxiliary operator with an accumulator:

```
op size' (accum : int, xs : intlist) =
  with xs = nil => accum
  with xs = cons y ys => size' (accum + 1) ys.
op size (xs : intlist) = size' 0 xs.
```

Tail-recursive functions have nice computational properties, which are not relevant here.

If an operator has multiple inductive type parameters, its cases can be based on any one of them or on many at once using a list of patterns separated by commas. An example using just one is the concatenation operator,

```
op cat (s1 s2 : intlist) =
  with s1 = nil => s2
  with s1 = cons x tail => cons x (cat tail s2).
```

Later on, we will prove that `cat` is associative and `nil` is an identity for it, making `(intlist, concat, nil)` a monoid. We can also prove that all lists can be built from `[]`, singleton lists `(cons (x, nil))` and `cat` (akin to “no junk” and allowing induction), but not that all such lists are distinct: a list can be broken into sublists multiple ways, in general (“confusion” happens).

An example with cases based on two parameters is the `subseq` operator, which determines whether the elements of one list occur in the same order in a second list, such that `(1, 2, 3)` is a subsequence of `(4, 1, 5, 2, 6, 3)` but not of `(4, 1, 5, 3, 6, 2)`. Its declaration is

```
op subseq (s1 s2 : intlist) : bool =
  with s2 = nil, s1 = nil => true
  with s2 = nil, s1 = cons _ _ => false
  with s2 = cons x s2', ys = nil => true
  with s2 = cons x s2', ys = cons y s1' =>
    subseq (if x = y then s1' else s1) s2'.
```

Note the use of `_` in the second case: the value for this case does not depend on the arguments passed to `cons`, so the pattern variables were left anonymous. A single underscore is used in many other contexts where the value of an expression does not matter but a placeholder is needed.

A predicate cannot be defined over an inductive type by cases; one must use a `bool`-valued function, such as `subseq`.

EasyCrypt can verify syntactically that the recursion is well-founded (always terminates). For example, the first three cases of `subseq` are base cases and the fourth one always recurses with a shorter list for `s2`. If this check fails, the declaration is rejected.

4.6.4 Match Expressions

A **match expression** is analogous to a switch or case statement in other languages but chooses among the constructors of an inductive type. For example, here is an operator that converts an `int_or_bool` to an `int` by shifting non-negative integers to the right to make room for `true` and `false` (see also [Hilbert's Hotel](#)).

```
op to_int (ib : int_or_bool) : int =
  match ib with
  | First n => if 0 <= n then n + 2 else n
  | Second b => if b then 1 else 0
  end.
```

A match expression is obviously very similar to defining an operator over an inductive type. What differs is that it begins with `match e with`, where `e` is any expression of a type that is an inductive type; the cases are separated by vertical bars (the one before the first case is optional); and it ends with `end`. Note that the period in the example above terminates the declaration—it is not part of the match

expression. The most important difference, of course, is that a match expression can be used anywhere an expression is allowed.

A match expression must cover all the constructors of the inductive type. There is no shorthand, such as “else” or “default,” and no “fall-through” between cases.

4.6.5 Destructors

When an inductive type is declared, a partial projection operator (a.k.a. a **destructor**) is generated automatically for each constructor. These provide a way to extract the arguments passed to a constructor, although in many cases a `match` expression is handier. For type `Suit`, above, the destructors are all very similar because the constructors are all constants:

```
op get_as_Spades (the : Suit) : unit option =
  match the with
  | Spades => Some tt
  | Clubs => None
  | Diamonds => None
  | Hearts => None
  end.
op get_as_Clubs (the : Suit) : unit option =
  match the with
  | Spades => None
  | Clubs => Some tt
  | Diamonds => None
  | Hearts => None
  end.
op get_as_Diamonds (the : Suit) : unit option =
  match the with
  | Spades => None
  | Clubs => None
  | Diamonds => Some tt
  | Hearts => None
  end.
op get_as_Hearts (the : Suit) : unit option =
  match the with
  | Spades => None
  | Clubs => None
  | Diamonds => None
  | Hearts => Some tt
  end.
```

The `int_or_bool` type illustrates destructors for non-recursive constructors with parameters:

```
op get_as_First (the : int_or_bool) : int option =
  match the with
  | First i => Some i
  | Second b => None
  end.
op get_as_Second (the : int_or_bool) : bool option =
  match the with
  | First i => None
```



```

| Second b => Some b
end.

```

The `intlist` type illustrates a destructor for a recursive constructor:

```

op get_as_nil (the : intlist) : unit option =
  match the with
  | nil => Some tt
  | cons x => None
  end.

op get_as_cons (the : intlist) : (int * intlist) option =
  match the with
  | nil => None
  | cons x => Some x
  end.

```

The destructor for the curried form of `cons` is virtually identical.

4.6.6 Limitations of Inductive Types (Optional)

Due to the implicit axioms of the “no confusion” rule, the signatures of the constructors rigidly dictate the “shape” of the resulting inductive type. The `intlist` type is a linear structure with a (rather useless) `nil` marking one end. Figure 1 shows how the lists `nil`, `cons 3 nil` and `cons (cons 5 nil)` might be visualized as very unbalanced binary trees. The lines do not represent pointers but merely connect a constructor with its arguments.

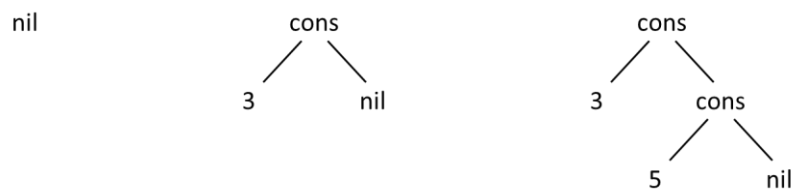


Figure 1 Visualizations of the Type `intlist`

This is a happy coincidence because lists are extremely useful. We already saw that a bag or a set cannot be defined by adding axioms to an inductive type. (There are other ways.)

If we define an inductive type with an operator that has two recursive references, we get a general binary tree and there is no way to “flatten” it into a list by adding axioms. For example, we might generalize `intlist` like this:

```
type tree = [ empty | join of tree & int & tree ].
```

Figure 2 depicts the trees corresponding to the expressions `empty`, `join empty 3 empty`, `join (join empty 3 empty) 5 empty` and `join empty 3 (join empty 5 empty)`, respectively. The last two illustrate the non-associativity of `join`.

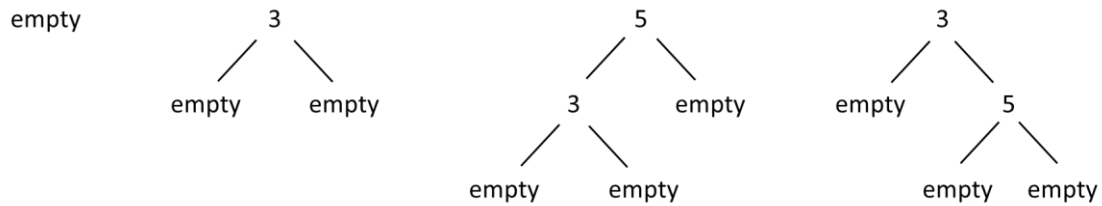


Figure 2 Visualizations of the Type tree

In short, `tree` consists of finite binary trees with an `int` at each interior node and `empty` at all the leaves. Like `nil`, the `empty` leaves don't really contribute anything in terms of how the `ints` are arranged, but we are stuck with them.

An alternative definition is

```
type tree1 = [ leaf int | join of tree1 & tree1 ].
```

Figure 3 depicts the trees corresponding to the terms `leaf 3`, `join (leaf 3) (leaf 5)` and `join (leaf 3) (join (leaf 5) (leaf 7))`. In short, `tree1` consists of finite binary trees with `ints` as the leaves and no `empty` nodes. Internal nodes hold no data, just the tree's structure.

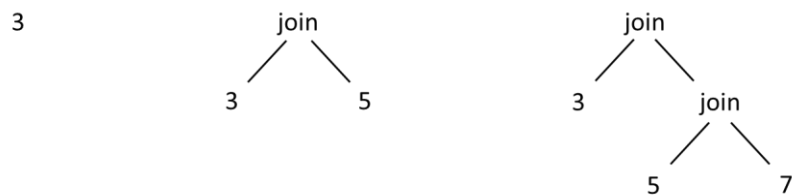


Figure 3 Visualizations of the Type tree1

The `tree1` type has no value representing an empty tree, which is inconvenient. We could add one:

```
type tree2 = [ empty | Tree of tree1 ].
```

The extra root node is no worse than having `nil` at the end of a list.

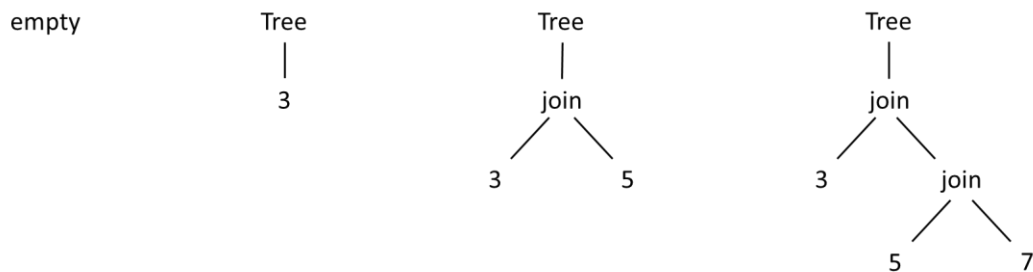


Figure 4 Visualizations of the Type tree2

Yet another alternative we might consider could be termed "all of the above":

```
type t = [ empty | leaf int | join of t & int & t ].
```

Figure 5 depicts the trees corresponding to the expressions `empty`, `leaf 3`, `join (leaf 3) 5 empty`, `join (join empty 3 empty) 5 empty`. Now we have binary trees with an `int` at each interior node and leaves that are `ints` or `empty` trees.

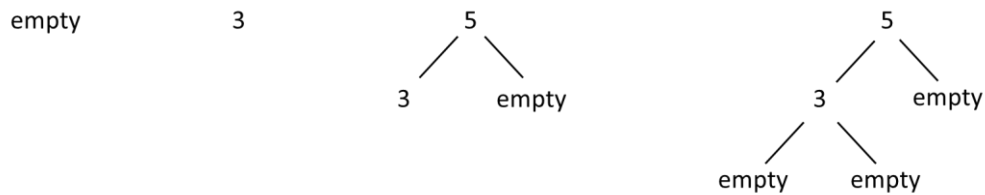


Figure 5 Visualizations of the Type `tree3`

A big problem with this type is that multiple trees contain the same `ints` in the same relative positions: they just have varying numbers of `empty` leaves. We would like to ignore the `empty` leaves and treat such sets of trees as equal, but as we’ve seen, we can’t. This structure records more than we want it to.

The point is, with two you get binary trees, with three you get ternary trees, and so on. Is it possible to define an inductive type for “shapeless” types, such as sets or bags, or so-called *mobiles*, which are commutative trees, so each internal node can “rotate” in place? Not directly, but the `FSet` theory defines an abstract type based on `List` that “ignores” the linear order it imposes (see `FSet`). You can also define sets or bags from scratch, using explicit “no junk”, “limited confusion” and induction axioms.

4.7 Polymorphic Operators

All of the operators discussed so far have had specific types, such as `int` or `t -> t -> t`. There are concepts we would like to express, however, that apply to all types or to types of a particular form, such as the generic function type `A -> B`, where `A` and `B` can be any type. Examples include

- Identity functions. These can be defined for a specific type, but we often think of “the” identity function for any type.
- Identity elements (a.k.a. units) such as 0 for addition or 1 for multiplication.
- Properties of functions, such as being injective or associative, so we can assert that a function over arbitrary types has this property or that.
- Function composition: a function `f : A -> B` and a function `g : B -> C` can be composed to form a function `g ∘ f : A -> C = g (f a)`.
- Functions over polymorphic types, such as the size of a list regardless of the type of the elements of the list (this one has to wait until *Polymorphic Types (Type Constructors)*).

The ambient logic has **polymorphic** operators that let us work with such concepts. The type of a polymorphic operator includes **type variables** that can be instantiated with any type. A type variable name is an apostrophe followed by an identifier, so it can never be mistaken for anything other than a type variable. The scope of a type variable is limited to the declaration that contains it. Regular variables are sometimes called **term variables** to distinguish them.

A note on OOP. In many programming languages, two functions can have the same name as long as their parameters have different types. This is called function *overloading* and a compiler can figure out which function is meant by examining the types of the arguments. In object-oriented programming languages, a subclass can *override* a method defined in a superclass to have a different implementation, such that at runtime a call to the method first checks the type of the object and then *dispatches* to the

correct implementation. Overloaded and/or overridden functions are sometimes called *polymorphic*, but the polymorphic operators in EasyCrypt are very different. A type variable can be replaced with *any* type, not just a select few. It's all or nothing. There is also no notion of choosing among competing implementations—concrete polymorphic functions have only one definition, which applies regardless of how type variables are instantiated. They can also be abstract and have axioms.

For example, a generic identity operator can be declared as

```
op id ['a] (x : 'a) : 'a = x.
```

The type of `id` is `'a -> 'a`. Types that differ only in the name of a type variable denote the same type, just as operators and functions that differ only in the name of a parameter do. Thus,

```
op id' ['z] (x : 'z) : 'z = x.
```

denotes the same polymorphic operator as `id`.

Functions can be polymorphic, too, as long as the type variables they use are declared somewhere earlier. Here is `id` declared in the “long” syntax but still declares the type variable used by the function:

```
op id ['a] : 'a -> 'a = fun (x : 'a) => x.
```

The explicit type variable declaration `['a]` after the operator name can be inferred just by using it and can therefore be omitted. For example,

```
op id (x : 'a) : 'a = x.
```

Multiple type variables are separated by commas and the order doesn't matter. Here is function composition, which is declared in the `Logic` theory:

```
op (\o) ['a, 'b, 'c] (g : 'b -> 'c) (f : 'a -> 'b) (x : 'a) : 'c
  = g (f x).
```

Note that its name makes it an infix operator: one writes `f \o g`.

Predicates can also be polymorphic. Many useful properties of functions are defined in `Logic`, some as predicates and some as `bool` operators, including these on unary operators

```
op injective ['b, 'a] (f : 'a -> 'b) : bool =
  forall (x y : 'a), f x = f y => x = y.
op surjective ['b, 'a] (f : 'a -> 'b) : bool =
  forall (x : 'b), exists (y : 'a), x = f y.
pred cancel ['b, 'a] (f : 'a -> 'b) (g : 'b -> 'a) =
  forall (x : 'a), g (f x) = x.
pred bijective ['b, 'a] (f : 'b -> 'a) =
  exists (g : 'a -> 'b), cancel f g /\ cancel g f.
pred involutive ['a] (f : 'a -> 'a) = cancel f f.
```

and these on binary operators

```
pred associative ['a] (f : 'a -> 'a -> 'a) =
  forall (x y z : 'a), f x (f y z) = f (f x y) z.
pred commutative ['a, 'b] (f : 'a -> 'a -> 'b) =
  forall (x y : 'a), f x y = f y x.
```

```

pred idempotent ['a] (f : 'a -> 'a -> 'a) =
  forall (x : 'a), f x x = x.
pred left_id ['a, 'b] (e : 'a) (f : 'a -> 'b -> 'b) =
  forall (x : 'b), f e x = x.
pred right_id ['a, 'b] (e : 'b) (f : 'a -> 'b -> 'a) =
  forall (x : 'a), f x e = x.

```

among many others. Notice how `commutative`, `left_id` and `right_id` use two type variables for maximum generality.

Polymorphic operators or predicates are essentially functions from types to operators or predicates. They cannot be used until the type variables have been given types. The syntax for this is `f<:τ>`, where `f` is a polymorphic operator or predicate with one type variable and `τ` is a type. For example, a monoid can now be defined a bit more compactly like this:

```

type t.
op idm : t.
op plusm : t -> t -> t.
axiom A : associative<:t> plusm.
axiom left_idm : left_id<:t, t> idm plusm.
axiom right_idm : right_id<:t, t> idm plusm.

```

With type inference, however, it is rarely necessary to provide type arguments explicitly. For example, the presence of `idm` and `plusm` in these axioms imply that `'a = 'b = t`, so `<:t>` and `<:t, t>` can safely be omitted. The step

```
print axiom A.
```

displays

```
axiom A: associative plusm.
```

omitting how the type variable was instantiated even though otherwise `print` shows all inferred types.

Here is a rare instance where a type argument is required because no other information is available:

```
op int_id = id<:int>.
```

Another way to provide the needed information is

```
op int_id : int -> int = id.
```

A very important polymorphic operator is declared in the `Pervasive` theory:

```
op witness ['a] : 'a. (* All types are inhabited in EC *)
```

Given any type, `witness` returns a value of that type. It does not specify which one, and I don't think there is a way to determine which one it returns. Its importance is largely theoretical: as the comment states, the existence of `witness` is tantamount to an axiom that states all types contain at least one value, perhaps something like

```
axiom no_empty_types ['a] : exists (x : 'a), true.
```

The typed lambda calculus does not make this assumption, in general. If empty types are allowed to exist, then the `witness` function does not.

Witnesses find practical use as default values for otherwise partial functions, such as what to return when an array is accessed with an index that is out of bounds.

4.8 Polymorphic Types (Type Constructors)

Types can also be defined using type variables. For example, a list is a list is a list, regardless of the type of the elements in the list. Such **polymorphic types** are functions from types to types and are also called **type constructors**. Type constructors are applied to type arguments to produce types.

A type constructor is declared with the type variables listed before the name. Two or more type variables are listed between parentheses and separated by commas. The rest of the syntax is the same as for type declarations, using the type variables as type expressions. Here is an example of a polymorphic tuple type, functional type, record type and inductive type, respectively:

```
type 'a pair = 'a * 'a.
type ('a, 'b, 'c) binfun = 'a -> 'b -> 'c.
type ('a, 'b) rec = { x : 'a; y : 'b }.
type ('a, 'b) either = [ First of 'a | Second of 'b ].
```

Type constructors are applied to type arguments the same way, by listing the arguments before the name (postfix). Lists of arguments are written in parentheses and separated by commas:

```
type complex = real pair.
type complex_binfun = (complex, complex, complex) binfun.
type age_weight = (int, real) rec.
type number = (real, int) either.
```

Polymorphic types necessarily have polymorphic operators, predicates and constructors:

```
op fst ['a] (x : 'a pair) : 'a = x.`1.
op snd ['a] (x : 'a pair) : 'a = x.`2.
pred self_equal ['a] (x : 'a pair) = fst x = snd x.
```

The inductive type constructor `either`, above, has polymorphic constructors `First of 'a` and `Second of 'b` and polymorphic induction axioms `either_ind` and `either_case`. The inductive type `number` has constructors `First of real` and `Second of int`. It does not have its own induction axioms, but `either_ind` and `either_case` apply to it.

Operators and predicates are defined over constructed types (i.e., type constructors instantiated with types) in the usual ways. For example,

```
op [\conj] (z : complex) : complex = (fst z, -snd z).
```

4.9 Type Classes and Instances

A **type class** is a set of types with common properties, which are defined in terms of operators they must have and axioms the operators must satisfy. For example, an algebraic structure such as a monoid, group, ring or field can be defined as a type class. **Instances** of a type class can then be declared, such as the monoids `(int, 0, (+))`, `(int, 1, (*))`, `('a list, [], (++)` and so on. A type class can also be a class of type constructors, such as the functor, monad and applicative design patterns.

The terminology suggests object-oriented programming, but that’s not the intent. Rather than a class being a type and instances being objects (values), a type class is a set of types and an instance is a particular type, with values of the instance being a third level. A type class is analogous to an interface, concept or trait in the object-oriented programming language of your choice.

The EasyCrypt library has no examples of type classes. Oddly, however, it does have instances of three built-in type classes: `field`, `ring` and `bring` (Boolean ring). These are used by the `algebra` proof tactic and its kin. See *Ring and Field Instances* and *The Algebra Tactics*.

Theories and cloning provide a different notion of bundling and instantiation. See *More on Theories*. Type classes are an emerging feature that is not fully fleshed out.

4.10 EasyCrypt Library, Part 2

4.10.1 Logic, Part 2

`Option` is a polymorphic inductive type that extends an arbitrary type with one additional value, named `None`.

```
type 'a option = [None | Some of 'a].
```

For example, the type `int option` contains the values `None` and, for every `n : int`, `Some n`, while the type `bool option` contains the values `None`, `Some false` and `Some true`. An `option` type can be used to define a partial function by extending the codomain with `None` and returning it whenever the function is undefined on its domain. We will see examples of this when discussing the `List` theory, below.

Operators on `'a option` include subtle variations on a couple themes. These three take a function with domain `'a` and apply it to an argument of type `'a option`:

- `oapp ['a 'b] f d ox : 'b` – apply the function `f : 'a -> 'b` safely to the optional value `ox : 'a option`: if `ox` is `None`, return the default value `d : 'b` and if `ox` is `Some x`, return `f x`.
 - `oapp` “adapts” `f` to accept `'a option` rather than `'a` and still return `'b`.
 - The value provided as `d` might be a “sentinel” value that `f` otherwise never returns or might be `witness<: 'b>`
- `omap ['a 'b] f ox : 'b option` – apply the function `f : 'a -> 'b` safely to the optional value `ox : 'a option`: if `ox` is `None`, return `None`, and if `ox` is `Some x`, return `Some (f x)`.
 - `omap` “adapts” `f` to accept `'a option` rather than `'a` and return `'b option` rather than `'b`.
 - `omap` only returns `None` if `ox` is `None`.
- `obind ['a 'b] f ox : 'b option` – apply the function `f : 'a -> 'b option` safely to the optional value `ox : 'a option`: if `ox` is `None<: 'a>`, return `None<: 'b>`, and if `ox` is `Some x`, return `f x`.
 - `obind` “adapts” `f` to accept `'a option` rather than `'a` and still return `'b option`.
 - `obind` returns `None` if `ox` is `None` or if `f x` is `None`.

These two operators “unwrap” a value of `'a option` to a value of `'a`.

- `odflt ['a] d ox : 'a` – if `ox : 'a` option is `None`, return the default value `d : 'a` and if `ox` is `Some x`, return `x`.
- `oget ['a] ox : 'a` – call `odflt` with `witness<: 'a>` as the default value.

Note that `'a` option with the `Some` constructor and `obind` operator constitutes a *monad*.

The Logic theory also defines a set of operators on sets of values of a type `'a`. A set `A` is represented by its so-called **characteristic function** `p : 'a -> bool` where $x \in A \iff p\ x$.

```
(* The empty set *)
op pred0 ['a] = fun (x : 'a) => false.
(* The universal set *)
op predT ['a] = fun (x : 'a) => true.
(* Set intersection *)
op predI ['a] = fun (p1 p2 : 'a -> bool) (x : 'a) =>
    p1 x /\ p2 x.
(* Set union *)
op predU ['a] = fun (p1 p2 : 'a -> bool) (x : 'a) =>
    p1 x \/ p2 x.
(* Set complement *)
op predC ['a] = fun (p1 : 'a -> bool) (x : 'a) => ! p x.
(* Set difference *)
op predD ['a] = fun (p1 p2 : 'a -> bool) (x : 'a) =>
    p1 x /\ ! p2 x.
(* Singleton set, {c} *)
op pred1 ['a] = fun (c x : 'a) => x = c.
(* Union of a set & a singleton set *)
op predU1 ['a] = fun (c : 'a) (p : 'a -> bool) (x : 'a) =>
    x = c \/ p x.
(* Complement of a singleton set *)
op predC1 ['a] = fun (c x : 'a) => x <> c.
(* Difference of a set & a singleton set *)
op predD1 ['a] = fun (p : 'a -> bool) (c x : 'a) =>
    x <> c /\ p x.
```

The theory does something similar with relations, where a relation is a set of ordered pairs and its characteristic function has type `'a -> 'b -> bool`.

```
(* The empty relation *)
op rel0 ['a 'b] = fun (a : 'a) (b : 'b) => false.
(* The universal relation *)
op relT ['a 'b] = fun (a : 'a) (b : 'b) => true.
(* Relation intersection *)
op relI ['a 'b] = fun (p1 p2 : 'a -> 'b -> bool) (a : 'a)
    (b : 'b) => p1 a b /\ p2 a b.
(* Relation union *)
op relU ['a 'b] = fun (p1 p2 : 'a -> 'b -> bool) (a : 'a)
    (b : 'b) => p1 a b \/ p2 a b.
(* Relation complement *)
op relC ['a 'b] = fun (p : 'a -> 'b -> bool) (a : 'a) (b : 'b) =>
    ! p a b.
```



```

(* Relation difference *)
op relD ['a 'b] = fun (p1 p2 : 'a -> 'b -> bool) (a : 'a)
                    (b : 'b) => p1 a b /\ ! p2 a b.
(* Singleton relation *)
op rel1 ['a 'b] = fun (ca : 'a) (cb : 'b) (a : 'a) (b : 'b) =>
                    ca = a /\ cb = b.
(* Union of a relation & a singleton relation *)
op relU1 ['a 'b] = fun (ca : 'a) (cb : 'b) (p : 'a -> 'b -> bool)
                    (a : 'a) (b : 'b) => (ca = a /\ cb = b) \/ p a b.
(* Complement of a singleton relation *)
op relC1 ['a 'b] = fun (ca : 'a) (cb : 'b) (a : 'a) (b : 'b) =>
                    ca <> a /\ cb <> b.
(* Difference of a relation & a singleton relation *)
op relD1 ['a 'b] = fun (p : 'a -> 'b -> bool) (ca : 'a) (cb : 'b)
                    (a : 'a) (b : 'b) => (ca <> a /\ cb <> b) /\ p a b.

```

The theory goes on to define properties of homogeneous relations (relations over values of one type):

```

type 'a rel : 'a -> 'a -> bool.
pred total ['a] (R : rel) = forall x y, R x y \/ R y x.
pred transitive ['a] (R : rel) =
  forall y x z, R x y => R y z => R x z.
pred symmetric ['a] (R : rel) = forall x y, R x y = R y x.
pred antisymmetric ['a] (R : rel) =
  forall x y, R x y /\ R y x => x = y.
pred pre_symmetric ['a] (R : rel) =
  forall x y, R x y => R y x.
pred reflexive ['a] (R : rel) =
  forall x, R x x.
pred irreflexive ['a] (R : rel) =
  forall x, ! R x x.

```

The final part of the Logic theory to be highlighted is the choiceb operator:

```

op choiceb ['a] (P : 'a -> bool) (x0 : 'a) : 'a.

axiom choicebP ['a] (P : 'a -> bool) (x0 : 'a) :
  (exists x, P x) => P (choiceb P x0).

axiom choiceb_dfl ['a] (P : 'a -> bool) (x0 : 'a) :
  (forall x, !P x) => choiceb P x0 = x0.

axiom eq_choice ['a] (P Q : 'a -> bool) (x0 : 'a) :
  (forall x, P x <=> Q x) => choiceb P x0 = choiceb Q x0.

axiom choice_dfl_irrelevant ['a] (P : 'a -> bool) (x0 x1 : 'a) :
  (exists x, P x) => choiceb P x0 = choiceb P x1.

axiom funchoice ['a 'b] (P : 'a -> 'b -> bool) :
  (forall x, exists y, P x y)
=> (exists f, forall x, P x (f x)).

```

Given a predicate `P`, the `choiceb` operator returns a value that satisfies it, unless there is no such value. Like `witness`, which value is returned is not specified, and I don't think there is a way to determine which one it returns unless it is known by other means that there is only one. Unlike `witness`, `choiceb` does not guarantee that a value satisfying `P` exists, only that if one does, it can construct one.

The `choiceb` operator is again of largely theoretical interest but does find practical use in the EasyCrypt library when a desired value can be described (i.e., by a predicate) but there's no obvious way to construct one. For example, given any function `f`, a partial inverse can be defined like this:

```
op pinv (f : 'a -> 'b) (y : 'b) : 'a option =
  if exists x, y = f x
  then Some (choiceb (fun x, y = f x) witness)
  else None.
```

which says, for any value in the codomain of `f`, if there are domain values for which `f` returns that value, the partial inverse will return one of them (as chosen by `choiceb`) and otherwise it will return `None`. Since the `choiceb` occurs in a context where such a domain value is known to exist (i.e., the `then` clause), it will never return `witness`.

4.10.2 List

The `List` theory is in the `datatypes` folder. It is the largest theory in the library.

The polymorphic list inductive type is

```
type 'a list = [
  | "[]"
  | (::) of 'a & 'a list
].
```

where the infix operator `"[]"` represents the empty list and the symbolic infix binary operator `(::)` represents a list formed by adding an element (conventionally to the “front” or “head”) of another list. Note that `(::)` is curried.

EasyCrypt has some built-in support for lists. In particular, the parser interprets the expression

```
[1; 2; 15; -3]
```

as the `int list`

```
1 :: 2 :: 15 :: -3 :: []
```

and similarly for `s : 'a, [s]` as the list `s :: []` of type `'a list`.

The list type has 56 operators, including

- o `size s : int` – the number of elements in `s`
- o `head z0 s : 'a` – the first element of `s`, or `z0` if `s = []`
- o `ohead s : 'a option` – the first element of `s` as `Some x` or `None` if `s = []`
- o `behead s : 'a list` – `s` with its head removed, or `[]` if `s = []`
- o `(++) s t : 'a list` – the concatenation of `s` and `t`
- o `rcons s x : 'a list` – `s` with `x` added to the end (just before `[]`)

- o `last z0 s : 'a` – the last element of `s`, or `z0` if `s = []`
- o `belast z0 s : 'a list` – `s` with its last element removed and `z0` added to the front, or `[]` if `s = []` (Not what you were expecting? Me, either.)
- o `nseq n x : 'a list` – a list consisting of `n` repetitions of `x`, or `[]` if `n <= 0`
- o `nth x0 s n : 'a` – the element of `s` at index `n`, where the head has index 0 and the last has index `(size s) - 1`, or `x0` if `n < 0` or `n >= size s`.
- o `onth s n : 'a option` – the element of `s` at index `n` as `Some y`, where the head has index 0 and the last has index `(size s) - 1`, or `None` if `n < 0` or `n >= size s`.
- o `mem s x : bool` – whether `x` occurs in `s`
- o `find p s : int` – the index of the first element of `s` that satisfies `p x`, where `p : 'a -> bool`, or `(size s)` if there is no such element
- o `count p s : int` – the number of elements of `s` that satisfy `p x`, where `p : 'a -> bool`
- o `filter p s : 'a list` – the list of elements of `s` that satisfy `p x`, where `p : 'a -> bool`
- o `has p s : bool` – whether `s` contains an element `x` that satisfies `p x`, where `p : 'a -> bool`
- o `all p s : bool` – whether all elements of `s` satisfy `p x`, where `p : 'a -> bool`
- o `index x s : int` – the index of the first occurrence of `x` in `s`, or `(size s)` if `x` is not in `s`.
The actual declaration is

```
op index ['a] (x : 'a) : 'a list -> int = find (pred1 x)
```

Think about how that works.

- o `drop n s : 'a list` – the list `s` with the first `n` elements removed, `[]` if `(size s) <= n`, or `s` if `n <= 0`
- o `take n s : 'a list` – the list containing the first `n` elements of `s`, `[]` if `n <= 0` or `s` if `(size s) <= n`
- o `rot n s : 'a list` – `s` “rotated” `n` positions to the left, where left rotation by 1 moves the first element to be the last element; has no effect if `n <= 0` or `(size s) <= n` (so rotating by `(size s) + 1` is not the same as rotating by 1)
- o `rotr n s : 'a list` – `s` “rotated” `n` positions to the right, where right rotation by 1 moves the last element to be the first element; has no effect if `n <= 0` or `(size s) <= n` (so rotating by `(size s) + 1` is not the same as rotating by 1)
- o `perm_eq s t : bool` – whether `s` and `t` contain the same elements, possibly in a different order
- o `rem x s : 'a list` – `s` without the first occurrence of `x` in `s`; has no effect if `x` is not in `s`
- o `insert x s n : 'a list` – `s` with `x` inserted at index `n`, at index 0 if `n < 0`, or at index `(size s)` if `(size s) < n`
- o `put s n x : 'a list` – `s` with `x` at index `n`; has no effect if `n < 0` or `(size s) <= n`
- o `trim s n : 'a list` – `s` without the element at index `n`; has no effect if `n < 0` or `(size s) <= n`
- o `rev s : 'a list` – `s` with its elements in reverse order
- o `catrev s t : 'a list` – the reverse of `s` concatenated with `t`
- o `uniq s : bool` – whether `s` has only one occurrence of each element
- o `undup s : 'a list` – `s` with duplicate elements removed (the last occurrence of each element is kept)

- `map f s : 'b list` – the list consisting of `f x` for each element `x` of `s`, where `f : 'a -> 'b`
- `pmap f s : 'b list` – the list consisting of `oget (f x)` for each element `x` of `s` where `f x <> None`, where `f : 'a -> 'b option`
- `mapi_rec f s i : 'b list` – the list consisting of `f j x` for each element `x` of `s`, where `f : int -> 'a -> 'b` and `j` is `i +` the index of `x`
- `mapi f s : 'b list` – the list consisting of `f i x` for each element `x` of `s`, where `f : int -> 'a -> 'b` and `i` is the index of `x` (i.e., `mapi_rec f s 0`)
- `iota_m n : int list` – the list of ints from `m` to `m + n - 1`, or `[]` if `n <= 0`
- `range m n : int list` – the list of ints from `m` to `n - 1`, or `[]` if `n <= m`
- `mkseq f n : 'a list` – the list consisting of `f i` for each `i` from `0` to `n - 1`, where `f : int -> 'a`.
- `foldr f z0 s : 'b` – for `f : 'a -> 'b -> 'b`, `z0 : 'b` and `s : 'a list = [x0; x1; ...; xn-1]`, return `f x0 (f x1 (... (f xn-1 z0)))`
 - E.g., for `'a = 'b = int`, `foldr (+) 0 s` returns the sum of the elements of `s`
 - `f` is applied right-to-left, meaning from the last element to the first, and `z0` is often (but not always) an identity element for `f`
 - `foldr f z0 [] = z0`
- `foldl f z0 s : 'b` – for `f : 'b -> 'a -> 'b`, `z0 : 'b` and `s : 'a list = [x0; x1; ...; xn-1]`, return `f (f (f (f z0 x0) x1) ...) xn-1`
 - E.g., for `'a = 'b = int`, `foldl (+) 0 s` returns the sum of the elements of `s`
 - `f` is applied left-to-right, meaning from the first element to the last, and `z0` is often (but not always) an identity element for `f`
 - `foldl f z0 [] = z0`
- `flatten (s : 'a list list) : 'a list` – flattens a list of lists into a list by concatenating each successive element
 - `flatten s = foldr (::) [] s`
- `subseq s t : bool` – whether the elements of `s` occur in the same order in `t`
- `zip s t : ('a * 'b) list` – for `s : 'a list` and `t : 'b list`, the list of ordered pairs consisting of corresponding elements of `s` and `t`. The size of the returned list is the size of the shorter of `s` and `t`
- `allpairs f s t : 'c list` – for `f : 'a -> 'b -> 'c`, `s : 'a list` and `t : 'b list`, the list of elements `f x y` where `x` is in `s` and `y` is in `t`
- `sort e s : 'a list` – a list that is `perm_eq` to `s` and ordered according to `e : 'a -> 'a -> bool`; `e` should be a total order, but this is not enforced
- `assoc s a : 'b option` – for `s : ('a * 'b) list` (an *association list*), Some `b` if `(a, b)` is the first pair in `s` with `a` as its first component, or `None` if there are no such pairs. I.e., it treats `s` as a map from `'a` values to `'b` values and does a look up on `a`, taking the first match.

and a few more obscure ones. It has over 500 lemmas describing their properties. Despite this richness, one occasionally needs lemmas that are not in the theory.

4.10.3 Finite

The `Finite` theory is in the `structure` folder. It defines the notion of *finiteness* for sets of a specified type. It again represents a set of `'a` values by its characteristic function, `p : 'a -> bool` (see *Logic, Part 2*) and declares

```

op is_finite_for ['a] (p : 'a -> bool) (s : 'a list) =
  uniq s /\ (forall x, x \in s <=> p x).
op is_finite ['a] (p : 'a -> bool) =
  exists s, is_finite_for p s.

```

which together state that a set (a predicate) is finite if all the elements in the set (or for which the predicate is true) can be put into a list that does not contain duplicates (is `uniq`), which is necessarily a finite structure. The list could, of course, be extremely large and impractical to form, but it only needs to exist. The size of the list is the size (cardinality) of the set. A set of 'a can be finite even if 'a is not.

The theory also defines

```

op to_seq ['a] (p : 'a -> bool) : 'a list =
  choiceb (fun (s : 'a list) => is_finite_for p s) [].

```

to return a list of values that satisfy `p`, if one exists, or the empty list otherwise. Note that the empty list is a valid response if `p x` is always false—it doesn't always mean `p` is infinite.

4.10.4 FSet

The `FSet` theory is in the `datatypes` folder. It defines an abstract polymorphic type and operators for *finite sets* of values of a specified type that are defined by enumerating their members rather than using predicates. Finite sets are abstractly represented as the quotient over `perm_eq` of lists without duplicates:

```

type 'a fset.
op elems ['a] : 'a fset -> 'a list.
op oflist ['a] : 'a list -> 'a fset.
axiom elemsK ['a] (s : 'a fset) : oflist (elems s) = s.
axiom oflistK ['a] (s : 'a list) :
  perm_eq (undup s) (elems (oflist s)).
axiom fset_eq ['a] (s1 s2 : 'a fset) :
  (perm_eq (elems s1) (elems s2)) => (s1 = s2).

```

The other operators are

- o `card s : int` – the size (cardinality) of `s`
- o `mem s x : bool` – whether `x` is a member of `s`
 - o `x \in s` is an abbreviation for `mem s x`
 - o `x \notin s` is an abbreviation for `! mem s x`
- o `fset0 : 'a fset` – the empty set
- o `fset1 x : 'a fset` – the singleton set containing only `x`
- o `(`|`) s1 s2 : 'a fset` – set union
- o `(`&`) s1 s2 : 'a fset` – set intersection
- o `(`\`) s1 s2 : 'a fset` – set difference
- o `pick s : 'a` – a member of `s`, or witness `<: 'a>` if `s` is `fset0`
- o `filter p s : 'a fset` – the set of members of `s` that satisfy `p x`, for `p : 'a -> bool`
- o `(\subset) s1 s2 : bool` – whether all members of `s1` are members of `s2`
- o `(\proper) s1 s2 : bool` – whether `s1 \subset s2` and `s1 <> s2`

- o `image f s : 'b fset` – the set containing `f x` for each `x` that is a member of `s`, where `f : 'a -> 'b`
- o `product s1 s2 : ('a * 'b) fset` – the Cartesian product of `s1` and `s2`
- o `fold f z0 s : 'b` – for `f : 'a -> 'b -> 'b`, `z0 : 'b` and `s : 'a fset = {x0, x1, ..., xn-1}`, return `f x0 (f x1 (... (f xn-1 z0)))`
- o `rangeset m n : 'a fset` – the set `{m, m + 1, ..., n - 1}`, or `fset0` if `n <= m`

Most of these are defined in terms of `elem` and `oflist`.

The theory also contains the `fset_ind` lemma for induction over a set of “constructors” for `fset`, namely `fset0` and the addition of a single element to a set:

```
lemma fset_ind (p : 'a fset -> bool) :
  p fset0 =>
    (forall x s, !mem s x => p s => p (s `|` (fset1 x))) =>
      (forall s, p s).
```

This states that any predicate `p` that is true of all sets that can be made starting with the empty set by adding one element at a time, in any order, is true of all sets. Again, this is analogous to the “no junk” axiom of an inductive type. The use of an inductive type as a basis for `fset` is incidental.

4.10.5 FMap

The `FMap` theory is in the `datatypes` folder. It defines a type and operators for *finite maps*, which are partial functions from one type to another. A finite map associates each “key” of type `'a` with a “value” of type `'b option`. A finite map can hold only a finite number of (key, value) pairs; all other keys are mapped to `None`. Finite maps are also known as lookup tables and dictionaries.

The `fmap` type is abstract. The three basic operations on finite maps are the empty map, adding a (key, value) pair to a map and retrieving the value to which a key is mapped:

```
type ('a, 'b) fmap.
(* The empty map, which maps every key to [None] *)
empty : ('a, 'b) fmap.
(* The map that maps [k] to [v] and otherwise equals [m] *)
op "_.[<-_]" (m: ('a, 'b) fmap) (k: 'a) (v: 'b) : ('a, 'b) fmap.
(* The value to which [m] maps [k]: either [Some v] or [None] *)
op "_.[_]" (m : ('a, 'b) fmap) (k : 'a) : 'b option.
```

The other operators are

- o `fsize m` – the number of (key, value) pairs in `m`, which is the number of keys not mapped to `None`
- o `dom m k : bool` – whether `m` has a mapping for `k : 'a` (i.e., `m.[k] <> None`)
 - o `k \in m` is an abbreviation for `dom m k`
 - o `k \notin m` is an abbreviation for `! dom m k`
- o `rng m v : bool` – whether `m` maps any key to `Some v`
- o `fdom m` – the domain of `m` as an `fset` (i.e., the set of keys `k` for which `dom m k` holds)
- o `frng m` – the domain of `m` as an `fset` (i.e., the set of values `v` for which `rng m v` holds)
- o `rem m k : ('a, 'b) fmap` – the `fmap` that maps `k` to `None` and otherwise equals `m`

- `map f m : ('a, 'c) fmap`—for `f : 'a -> 'b -> 'c`, the map that maps each `k` to `omap (f m.[k])`
- `filter p m : ('a, 'b) fmap`—for `p : 'a -> 'b -> bool`, the map that maps each `k` to `m.[k]` if `m.[k] = Some v` and `p k v` holds, and to `None` otherwise
- `(+) m1 m2 : ('a, 'b) fmap`—the union of the (key, value) pairs of `m1` and `m2`, giving precedence to `m2` (i.e., the map that maps each `k` to if `k \in m2` then `m2.[x]` else `m1.[x]`)
- `has p m : bool`—for `p : 'a -> 'b -> bool`, whether there is a key `k` such that `m.[k] = Some v` and `p k v` holds.
- `find p m : 'a option`—for `p : 'a -> 'b -> bool`, a key `k` such that `m.[k] = Some v` and `p k v` holds, or `None` if there is no such key.
- `fcoll f m : bool`—for `f : 'b -> 'c`, whether there exist keys `i` and `j` in `m` for which `f (oget m.[i]) = f (oget m.[j])` (`f` is said to *collide* in `m`)
- **Flagged maps**—a map of type `('a, 'b * 'f) fmap`
 - `noflags m : ('a, 'b) fmap`—removes the flags from `m`
 - `in_dom_with m k f : bool`—whether `k` is in `m` and has flag `f`
 - `restr f m : ('a, 'b) fmap`—a map containing the `(k, v)` pairs that, in `m`, are `(k, (v, f))`.
- `merge f m1 m2 : ('a, 'b3) fmap`—for `f : 'a -> 'b1 option -> 'b2 option -> 'b3 option`, `m1 : ('a, 'b1) fmap` and `m2 : ('a, 'b2) fmap`, the map that *merges* `f`, `m1` and `m2` by mapping each key `k` to `f k m1.[k] m2.[k]`
- `union_map m1 m2 : ('a, 'b) fmap`—the map that maps each `k` to `m1.[k]` if it is not `None` and to `m2.[k]` otherwise. This is just `m2 + m1`.
 - `o_union _ x y : 'b option`—a helper operator for `union_map`
- `pair_map m1 m2 : ('a, 'b1 * 'b2) fmap`—for `m1 : ('a, 'b1) fmap` and `m2 : ('a, 'b2) fmap`, the map that maps each `k` to `(m1.[k], m2.[k])` as long as neither is `None`, and to `None` otherwise
 - `o_pair`—a helper operator for `pair_map`
- `ofassoc s : ('a, 'b) fmap`—for `s : ('a * 'b) list` (an association list), the corresponding map (i.e., a map containing a key pair for each element of `s`, but keeping only the first occurrence of each key)
- `eq_except P m1 m2 : bool`—whether `m1` and `m2` are equal except possibly for keys `k` where `P k` holds

A note on representation. Finite maps are abstractly represented as functions that return a constant codomain value except for a finite set of domain values (analogous to a finite set represented as a predicate that is true for a finite set of domain values). This is useful to know when proving lemmas that aren't in `FMap`. The `CoreMap` theory in the `core` folder defines

```
type ('a, 'b) map.
op cst ['a 'b] : 'b -> ('a, 'b) map
op "_.[_]" ['a 'b] : ('a, 'b) map -> 'a -> 'b.
op "_.[_<-_]" ['a 'b] : ('a, 'b) map -> 'a -> 'b -> ('a, 'b) map.
```

with no axioms. The `FMap` theory starts with a `Map` sub-theory that adds

```
op tofun ['a 'b] (m : ('a, 'b) map) : 'a -> 'b = ("_.[_]") m.
op offun ['a 'b] : ('a -> 'b) -> ('a, 'b) map.
```

```
axiom offunE ['a 'b] (m : 'a -> 'b) :
  forall a, (offun m).[a] = m a.
```

for converting back and forth between maps and functions. It then proves various lemmas about `tofun` and `offun` that are used to prove lemmas that finally explain the intended behavior of the original three operators. Intuitively,

- o `cst b0` creates a map that maps every value of type `'a` to the default value `b0`
- o `m.[a<-b]` returns a map that maps `a` to `b` and otherwise equals `m`
- o `m.[a]` returns the value to which `m` maps `a`, which would be the most recent value set by `m.[a<-b]` or, if `a` was never assigned a value, `b0`

The `FMap` theory then defines

```
type ('a, 'b) fmap.
op tomap ['a 'b] : ('a, 'b) fmap -> ('a, 'b option) map.
op ofmap ['a 'b] : ('a, 'b option) map -> ('a, 'b) fmap.
axiom tomapK ['a 'b] : cancel tomap ofmap<:'a, 'b>.
axiom ofmapK ['a 'b] (m : ('a, 'b option) map) :
  is_finite (fun a => m.[a] <> None) => tomap (ofmap m) = m.
axiom isfmap_ofmap (m : ('a, 'b) fmap) :
  is_finite (fun a => (tomap m).[a] <> None).
```

for converting back and forth between `('a, 'b) fmap` and `('a, 'b option) map`. Most of the other `fmap` operators are defined in terms of them. While `fmap` is not an inductive type, the `empty` and `"_.[_<-_] "` operators are effectively its constructors in the sense that there is an induction axiom stating, in effect, that all maps can be built from `empty` by adding one key-value pair at a time with no duplicate keys (albeit not uniquely, since order doesn't matter),

```
lemma fmapW ['a 'b] (p : ('a, 'b) fmap -> bool) :
  p empty
  => (forall m k v, !k \in fdom m => p m => p m.[k <- v])
  => forall m, p m.
```

4.10.6 Xint

The `XInt` theory is in the `datatypes` folder. It defines an *extended integer* type as an inductive type extending `int` with a value representing infinity:

```
type xint = [N of int | Inf].
```

with operators

- o `xopp x` (abbreviated `[-]`) – if `x = N n` then `-n` else `Inf`
- o `xadd x y` (abbreviated `[+]`) – if `x = N n` and `y = N m` then `n + m` else `Inf`
- o `xmult x y` (abbreviated `[*]`) – if `x = N n` and `y = N m` then `n * m` else `Inf`
- o `x ** y` – abbreviation for `N x * y`
- o `is_int x` – whether `x = N _`
- o `is_inf x` – whether `x = Inf`

5 Ambient Logic Proofs

This section is about how to prove lemmas in the ambient logic. Although EasyCrypt can be used to prove interesting mathematical theorems, its main purpose is to prove judgements (lemmas) about programs. We saw in the introduction that modules (programs) make free use of types and operators declared in the ambient logic, so there is an obvious need to reason about them. Moreover, the program logics are extensions of the ambient logic, so proving lemmas in it is foundational to reasoning about the properties of programs. For those who know Coq/Rocq, the ambient logic proof system in EasyCrypt closely follows the SSReflect library, with some simplifications.

5.1 Goals

A **proof** demonstrates the truth or *necessity* of a lemma. A proof must immediately follow the lemma it proves. A proof is delimited like this:

```
proof.      (* optional, but good practice *)
<steps of the proof.>
qed.
```

The only kinds of steps that can occur inside a proof are *proof steps*, *print*, *search* and *locate*. Proof steps can occur only inside a proof. Pragmas may also occur inside a proof; see *Pragmas*.

There is a shorthand that can be used when the proof of a lemma is a single step:

```
lemma ... by <step>.
```

This omits the `proof` and `qed` steps and saves the lemma just as if `qed` were present.

During a proof, the proof engine maintains a list of **goals** that remain to be proven. A goal has two parts:

- A **context**, consisting of
 - a set of type variables, which can be empty
 - an ordered list of *assumptions*, of which there are three kinds:
 - A variable, consisting of a name and a type expression
 - A hypothesis, consisting of a name and an expression of type `bool`
 - A local definition, consisting of a name, a type and an expression (a value)
- A **conclusion**, consisting of an expression of type `bool`

In typed lambda calculus, the context serves to declare the names and types of any type variables and universally quantified regular or term variables that appear in the lemma. It is not considered part of the lemma. The context is said to *entail* the lemma, meaning if the context is satisfied then the lemma will be true. If this seems to be just like implication, that's because it is. A context is often denoted as Γ and entailment as the *judgement* $\Gamma \vdash \phi$, where ϕ is a formula (the conclusion of the lemma).

Type variables appear in the context only if the lemma is polymorphic. Regular variables appear in the context only if a lemma has parameters. In all other cases, the context will be empty. Type expressions in the context may refer to the type variables declared previously and an assumption may refer to the assumptions that precede it. Hypotheses and local definitions will be described shortly in *Using the Context*.

When a lemma is parsed, the proof engine starts with a single goal consisting of the lemma itself presented as a context and a conclusion. For example, the polymorphic lemma

```
lemma PairEq1 ['a, 'b] : forall (x x' : 'a, y y' : 'b),
  x = x' => y = y' => (x, y) = (x', y').
```

becomes the goal

```
Type variables: 'a, 'b
-----
forall (x x' : 'a) (y y' : 'b),
  x = x' => y = y' => (x, y) = (x', y')
```

This goal has two type variables in the context, a line, and the body of the lemma as the conclusion. Note the minor formatting changes it made, too. The same lemma in parameterized form,

```
lemma PairEq2 ['a, 'b] (x x' : 'a, y y' : 'b) :
  x = x' => y = y' => (x, y) = (x', y').
```

gives rise to the goal

```
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' => y = y' => (x, y) = (x', y')
```

with variable declarations for the parameters in the context.

A note on Proof General. Figure 6 shows a screen shot of Emacs running EasyCrypt interactively in Proof General. The left window shows the script file containing the proof being developed, the upper right shows the current goal (the **goals** buffer) and the lower right shows other EasyCrypt output (the **response** buffer): in this case, the initial startup message. One other buffer, **easycrypt**, is not shown by default but shows raw output matching what you would see running EasyCrypt interactively from the command line. The menu bar has selections for EasyCrypt and for Proof General for setting options and such, and below that is a toolbar with various actions. In case of a parse or other error, Emacs will highlight the location of the error in the script window. If the Emacs window is not wide enough for a side-by-side view, the three windows are arranged vertically.

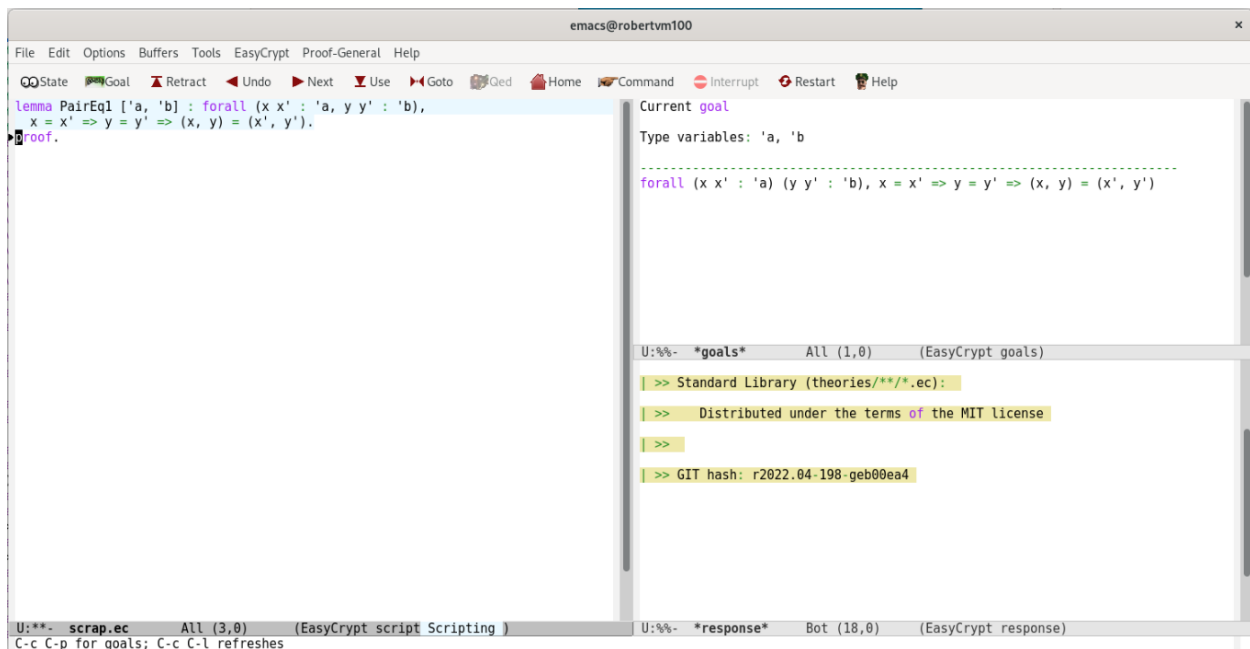


Figure 6 Emacs running EasyCrypt in Proof General

5.2 Proof Tactics and Steps

Proofs are carried out using **tactics**, which are logical rules that transform (or *reduce*) a goal into zero or more *subgoals*, which are sufficient conditions for the goal to be true. This form of reasoning, from conclusions to antecedents, is called *backward reasoning* (or *backward chaining*). It says, in effect, if I want to prove ϕ and I know $\psi \Rightarrow \phi$, then I can focus on proving ψ . A proof is typically read and understood in the forward direction: starting from these known facts or assumptions, the conclusion follows by a series of logical steps. But the proof is *constructed* in reverse. The trick lies in choosing ψ —it may imply ϕ , but if it isn't provable, it's of no use. After all, *false* implies anything, but *false* is also unprovable. At any point in a proof, there are typically many steps that might be taken, most of which lead nowhere. In general, the search for a proof explodes combinatorially. It is the user's job, therefore, to figure out steps that make progress toward eliminating goals, while it is the computer's job to perform each step without error. That's why EasyCrypt is a *proof assistant*, not a *prover*: the user is the brains and EasyCrypt is the brawn.

Conceptually, the proof engine maintains a list of goals as a stack, starting with a single goal. The top element is called the **current goal**. A *proof step* applies one or more tactics to the current goal. If the step fails, the current goal is unchanged and the step must be removed (i.e., from the script). If it succeeds, the proof engine pops the stack and pushes onto it any subgoals the step generated. If a step generates subgoals G_1 through G_n , the proof engine pushes them onto the stack from G_n to G_1 , so they are worked in the order they were generated. If a step generates no subgoals, that means the current goal is proved (a.k.a. *solved* or *closed*) and the new top goal becomes the current goal. When the stack is empty, the proof is complete. The `qed` step saves the lemma in memory, where it can be used to prove other lemmas. It fails if there are goals remaining to be proved. It also fails if the name of the lemma conflicts with that of another lemma or axiom, which is a lousy time to find that out.

Proof steps are replayed without output when EasyCrypt is used in batch mode. When a theory is loaded, proofs are parsed and type-checked but not otherwise executed or confirmed.

In interactive mode (e.g., from within Emacs using Proof General or from the command line), the default behavior is to display the current goal after each step. If you want to see all goals after each step, include the step

```
pragma printall.
```

in your script. If you don't want to see any goals, include

```
pragma silent.
```

These steps can be counteracted by

```
pragma printone.  
pragma verbose.
```

respectively. See also *Pragmas*. Regardless of these settings, you can print a goal by number using

```
print goal n.
```

where the current goal is 1.

The remainder of this section describes tactics for the ambient logic in an order that minimizes forward references. Sometimes different aspects of a tactic will be described separately. *Ambient Logic Proofs* in the appendix lists all the tactics and so-called *tacticals* in alphabetical order as a quick reference.

5.2.1 The Simplify Tactic

The various kinds of reduction described in *The Ambient Logic* provide basic computation in the typed lambda calculus. Proof tactics go far beyond that, but reduction is the core. The `simplify` tactic applies reduction rules exhaustively to *normalize* the conclusion of a goal, which often results in it being “simpler.” Normalization just means “reduced as far as possible.” Two formulas are called **convertible** when they can be reduced to the same form or have the same normalized form. Another way to say this is that the formulas are equal “up to computation.”

For example, here is a sample interactive session showing a proof of the `PairEq1` lemma, with user input in **boldface** and extra blank lines in the output omitted:

```
$ easycrypt
>> Copyright ...
[0|check]>
lemma PairEq1 ['a 'b] : forall (x x' : 'a, y y' : 'b),
  x = x' => y = y' => (x, y) = (x', y').
Current goal:
Type variables: 'a, 'b
-----
forall (x x' : 'a) (y y' : 'b),
  x = x' => y = y' => (x, y) = (x', y')
[1|check]>
proof. (* this displays the current goal again *)
Current goal:
...
[2|check]>
```

The `simplify` tactic reduces the conclusion of the goal like this:

```

simplify.
Type variables: 'a, 'b
-----
true
[2|check]>

```

This is the simplest formula there is, along with `false` (which cannot be proved, of course). The `simplify` tactic isn't bothered by the presence or absence of assumptions in the conclusion of a goal, so it works just as well on the initial goal of `PairEq2`:

```

[0|check]>
lemma PairEq2 ['a, 'b] (x x' : 'a, y y' : 'b) :
  x = x' => y = y' => (x, y) = (x', y') .
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' => y = y' => (x, y) = (x', y')
[1|check]>
simplify.
Type variables: 'a, 'b
-----
true
[2|check]>

```

The `done` tactic, among others, can close the reduced conclusion left by `simplify`:

```

done.
No more goals
qed.
added lemma: `PairEq1'
[3|check]>

```

For the remainder of this document, examples will show a proof step and the resulting goal(s) without showing EasyCrypt prompts and other output. The `qed` step will also be omitted.

The `simplify` tactic combines a set of lower-level tactics. For each form of reduction there is a corresponding tactic:

- `beta` – replaces all occurrences of `(fun (x : t) => φ) y` with `φ[x := y]`
- `zeta` – replaces all occurrences of `let x = y in φ` with `φ[x := y]`
 - It does not, however, reduce tuples, such as `let (x, y) = f a b in φ`, which would be `φ[x := (f a b).`1, y := (f a b).`2]`.
- `iota` – if `t` is an inductive type with a constructor `c` of `u` and `f` is an operator over `t` that defines `f (c x) => φ`, replaces all occurrences of `f (c y)` with `φ[x := y]` (for example, replace `size []` with `0` and `size (x :: s)` with `1 + size s`)
 - `iota` also reduces projections applied to literal tuples and records. For example, it reduces `(7, 15).`2` to `15`

- `iota` also reduces conditional expressions when the condition is known (see *Conditional Expressions*).

Several other tactics are also related to reduction:

- `delta` – if `f` is a concrete operator, `op f (x : t) = φ`, replaces all occurrences of `f y` with `φ[x := y]` (*delta* is for *definition*, where `φ` is the definition of `f`)
 - This is called “unfolding” `f`. There is a complementary operation called “folding” that `delta` does not do. The `rewrite` tactic does both folding and unfolding (see *Folding and Unfolding*).
 - `delta` also unfolds local definitions by replacing all occurrences of their names with their values
 - `delta` does not unfold operators defined inductively (i.e., operators over inductive types that are defined by cases), such as `size` (of a list); `iota` does that
 - `delta op1 ... opn` unfolds applications of `op1` through `opn` only, ignoring other concrete operators.
- `logic` – this tactic reduces expressions containing literal values of type `bool`, `int`, `real` or a tuple type. For example, it reduces `a /\ true` to `a`, `5 + 7` to `12` and `(x, y) = (x', y')` to `x = x' /\ y = y'`. It also reduces `a => a` to `true` but has no effect on `a /\ a`, so exactly what facts are in its arsenal is not clear.
- `eta` – this tactic does η -reduction and/or expansion. Never used in the library but see the `Core`, `Logic` and `Distr` theories.
- `user` – this is not a standalone tactic; it refers to the use of certain axioms or lemmas as *user reductions* during simplification. See *Hints*. Also see the `redlogic` pragma in *Pragmas*.
- `modpath` – this tactic relates to the module language, but what it does is unclear

The `simplify` tactic has three forms:

```
simplify.
simplify delta.
simplify op1 ... opn.
```

The first form reduces a goal’s conclusion to its $\beta\iota\zeta\Lambda$ -head normal form using `beta`, `eta`, `iota`, `zeta`, `logic`, `user` and `modpath`. The second does the same, but first does one step of parallel, strong δ -reduction using `delta` (i.e., it unfolds all concrete operators at the same time, but does not recursively unfold operators exposed by the first step). The third restricts the δ -reduction to the specified operators.

A fourth form is a sequence of one or more of the individual tactics (except `user`), such as

```
beta iota logic.
```

The `logic` tactic has the same effect on this goal as `simplify`, while `beta`, `zeta` and `iota` have no effect because the patterns they reduce are not present.

Here is an example of the individual tactics that comprise `simplify`. The `mem` operator and its abbreviation (`\in`) are from the `List` theory and `(==)` and `pred0` are from the `Logic` theory.

```
lemma in_nilE ['a] : mem[<:'a>] == pred0.
Type variables: 'a
```

```

-----
mem [] == pred0

```

We start by unfolding some operators.

```

delta pred0.
Type variables: 'a
-----
mem [] = (fun (_ : 'a) => false)
delta (==).
Type variables: 'a
-----
forall (x : 'a), (x \in []) = (fun (_ : 'a) => false) x

```

Note that `mem` cannot be unfolded because it is defined over an inductive type. We now have an anonymous function applied to an argument, so we can use the `beta` tactic:

```

beta.
Type variables: 'a
-----
forall (x : 'a), (x \in []) = false

```

We also have an operator (`mem`) applied to a list constructor (`[]`), so we can use the `iota` tactic:

```

iota.
Type variables: 'a
-----
forall (_ : 'a), false = false

```

The `logic` tactic reduces this to `true`, which we know the `done` tactic can solve:

```

logic.
true
done.
No more goals

```

The `simplify` tactic never fails and never solves the goal: it only normalizes its conclusion, which leaves it unchanged if it is already in normal form. Many proofs run `simplify` on a whole bunch of goals at once to reduce those it can.

5.2.2 The Done and Trivial Tactics

The `done` tactic uses “a mixture of low-level tactics” to solve a goal automatically. Exactly what these are is unclear, but one of them is `simplify delta`: to a first approximation, if the conclusion of a goal reduces to `true`, `done` closes it; otherwise, it fails. In the `PairEq1` and `in_nilE` examples above, therefore, it was not necessary to have explicit `simplify` or `delta` steps: a one-step proof consisting of `done` suffices. We will further dissect the `done` tactic as other tactics are presented.

The `trivial` tactic does the same thing as `done` except that it never fails: if it cannot solve the goal, it simply leaves it unchanged (it throws away any reductions that it made). As with `simplify`, many proofs run `trivial` on a whole bunch of goals at once to solve those it can.

5.2.3 The Reflexivity Tactic

The `reflexivity` tactic solves goals of the form $x = x$, where x is any expression. The two sides do not have to be identical, only equal “up to computation,” implying that the tactic uses `simplify` internally to normalize both sides. The tactic does not consider $\leq$ to be equality, meaning it cannot solve goals of the form $b \leq b$.

The `reflexivity` tactic is rarely seen in practice, probably because it is applied automatically by other tactics, including `done` (and hence `trivial`). Examples can be found in *The Clear, Subst and Assumption Tactics* and various other sections.

5.2.4 The Admit Tactic and Abort

The easiest proof, and one guaranteed to work, is hand-waving, also known as a “poof,” “magic happens here” or “because I said so”—just assume the current goal is true and move to the next one, if there is one. This is what the `admit` tactic does:

```
<any goal whatsoever>
admit.
No more goals
```

In practice, `admit` is used to postpone working on a subgoal so you can skip ahead to another one or to use a lemma without proof to make sure it is useful. It should never be used in a finished proof.

The `admitted` step assumes all open goals are true and saves the lemma. It’s an alternative to `qed`, not a tactic.

The `abort` step provides a convenient way to exit a proof and discard the lemma, whether to work on later or as an example or warning to others. It’s another alternative to `qed`.

5.3 Using the Context

Above we said that the context of a goal is not considered part of the lemma, that the context entails the conclusion and that entailment is the same thing as implication. We saw that the initial goal for `PairEq1` has all of its variable declarations in the conclusion part, whereas the initial goal for `PairEq2` has them in the context. This makes no logical difference, but it can affect which tactics can be applied, as some are looking for very specific patterns in the conclusion. This practical difference has led EasyCrypt (and other proof assistants) to use the context in a way not contemplated by lambda calculus: as a “free parking” area where parts of a lemma can be stored and later retrieved. Moreover, this new usage is not limited to variable declarations.

There are three kinds of **assumptions** that can be in either the context or the conclusion of a goal (where ϕ' represents the remainder of the conclusion) :

In the context	In the conclusion
Variable declaration: $x : t$	Quantifier: $\text{forall } (x : t), \phi'$
Hypothesis: $H : \phi$	Implication: $\phi \Rightarrow \phi'$
Local definition: $x : t := \phi$	Let expression: $\text{let } (x : t) = \phi \text{ in } \phi'$

Introducing an assumption means moving it from the conclusion to the context and **reverting** one means moving it from the context to the conclusion. Both are performed using a tactical rather than a tactic. In general, a **tactical** is a way to combine, compose or modify one or more tactics in a proof step.

A note on local definitions. Local definitions and let expressions are only honorary members of the assumption club: they can be moved into and out of the context but are ignored by most tactics. They simply give a name to a definition. See also *Specifying the Instantiation (Proof Terms)*.

5.3.1 The Introduction Tactical

The introduction tactical **introduces** the first, or **top**, assumption of the conclusion to the context immediately following the application of a proof tactic; it looks like this:

```
idtac => x.
```

The `idtac` tactic (short for *identity tactic*) does nothing at all, which is handy when all you want to do is introduce some assumptions. The introduction tactical is indicated by `=>` and `x` can be any identifier, which becomes the name of the assumption. From the initial goal for `PairEq1`, for example, the top assumption is the universal quantifier on the variable `x`, which we can introduce (and rename) like this:

```
Type variables: 'a, 'b
-----
forall (x x' : 'a) (y y' : 'b),
  x = x' => y = y' => (x, y) = (x', y')
idtac => a.
Type variables: 'a, 'b
a: 'a
-----
forall (x' : 'a) (y y' : 'b),
  a = x' => y = y' => (a, y) = (x', y')
```

The step above introduces `x` to the context and names it `a` both there and throughout the conclusion. One can proceed to introduce the other variables in the same way or instead do all four in one step, this time keeping their original names:

```
Type variables: 'a, 'b
-----
forall (x x' : 'a) (y y' : 'b),
  x = x' => y = y' => (x, y) = (x', y')
idtac => x x' y y'.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' => y = y' => (x, y) = (x', y')
```

This result is identical to the initial goal for `PairEq2`. The conclusion is now an implication, so the top assumption is its antecedent, which can be introduced as a hypothesis:

```
idtac => x_eq.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
```

```

x_eq: x = x'
-----
y = y' => (x, y) = (x', y')

```

The name of the hypothesis can again be any identifier. In this case, we can do it again:

```

idtac => y_eq.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
x_eq: x = x'
y_eq: y = y'
-----
(x, y) = (x', y')

```

This conclusion has no assumptions, so nothing more can be introduced. This proof will be continued below.

Here is an example of let expressions becoming local definitions:

```

lemma PairEq3 (x x' : 'a, y y' : 'b) :
  let p = (x, y) in
  let p' = (x', y') in
  x = x' => y = y' => p = p'.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
let p = (x, y) in let p' = (x', y') in x = x' => y = y' => p = p'
idtac => p p'.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
p: 'a * 'b := (x, y)
p': 'a * 'b := (x', y')
-----
x = x' => y = y' => p = p'

```

An identifier used by the introduction tactical is one kind of ***introduction pattern***. There are actually three introduction patterns that move the top assumption from the conclusion to the context, one that deletes it, one that duplicates it, a pair that play peek-a-boo and one that deletes assumptions that are already in the context but no longer needed. The first is the one we've been discussing:

<code>ident</code>	Introduces the top assumption with the specified name; the step fails if there is already an assumption with that name
--------------------	--

<code>n ?</code>	Introduces the top assumption <i>anonymously</i> (as <code>_</code> for a hypothesis or <code>`name</code> for a variable or local definition, which are not valid identifiers); the assumption cannot be used by the user but tactics such as <code>trivial</code> and <code>assumption</code> (see below) still can. <code>n</code> is an optional non-negative integer saying how many assumptions to introduce anonymously
<code>*</code>	Successively introduces all assumptions anonymously (see the <code>? pattern</code>)
<code>_</code> (underscore)	Clears (deletes) the top assumption, which must not be used by the rest of the goal (e.g., if it is a variable or let expression)
<code>^</code> (hat)	Duplicates the top assumption, which must be a hypothesis: the conclusion $\psi \Rightarrow \phi$ becomes $\psi \Rightarrow \psi \Rightarrow \phi$.
<code>+</code>	Clears the top assumption and then restores it after the last pattern or when the <code>-</code> pattern is encountered
<code>-</code> (hyphen)	Restores assumptions cleared earlier by the <code>+</code> pattern. If this pattern does not appear, such assumptions are restored after the last pattern. Multiple <code>+</code> patterns are restored in reverse order, from right to left.
<code>{a₁ ... a_n}</code>	Clears (deletes) the specified assumptions from the context

Here are some quick examples, including a few that show the error messages generated when patterns can't be applied. Each one starts with the initial goal for `PairEq1`. We will also switch to the more common `move` tactic, which also does nothing but has a few tricks we'll discuss (much) later.

```
Type variables: 'a, 'b
-----
forall (x x' : 'a, y y' : 'b),
  x = x' => y = y' => (x, y) = (x', y')
```

Duplicate name error:

```
move => a a.
an hypothesis or variable named `a` already exists
```

Move assumptions anonymously:

```
move => ? ? ? ? ? ?.
Type variables: 'a, 'b
`x: 'a
`x': 'a
`y: 'b
`y': 'b
_: `x = `x'
_: `y = `y'
-----
(`x, `y) = (`x', `y')
```

This could be accomplished more succinctly with

```
move => 6?.
```

Move all remaining assumptions anonymously:

```
move => x1 x2 *.
```

```

Type variables: 'a, 'b
x1: 'a
x2: 'a
`y: 'b
`y': 'b
_: x1 = x2
_: `y = `y'
-----
(x1, `y) = (x2, `y')

```

Clear the top assumption:

```

move => _.
Cannot clear _ that is/are used in the conclusion
move => x1 x2 y1 y2 _ _.
Type variables: 'a, 'b
x1: 'a
x2: 'a
y1: 'b
y2: 'b
-----
(x1, y1) = (x2, y2)

```

Duplicate the top assumption. This is almost always followed by a name pattern to move the duplicate to the context for safe keeping,

```

move => x1 x2 y1 y2 ^ H.
Type variables: 'a, 'b
x1: 'a
x2: 'a
y1: 'b
y2: 'b
H: x1 = x2
-----
x1 = x2 => y1 = y2 => (x1, y1) = (x2, y2)

```

but there is nothing stopping you from using it frivolously:

```

move => x1 x2 y1 y2 ^ ^ _ ^.
Type variables: 'a, 'b
x1: 'a
x2: 'a
y1: 'b
y2: 'b
-----
x1 = x2 => x1 = x2 => x1 = x2 => y1 = y2 => (x1, y1) = (x2, y2)

```

If you attempt to duplicate an assumption that isn't a hypothesis, the error message is unfortunately misleading:

```

move => ^.
no top-assumption to duplicate

```

Move and revert the top assumption in order to move the second one:

```
move => + x2.
Type variables: 'a, 'b
x2: 'a
-----
forall (x : 'a) (y y' : 'b),
  x = x2 => y = y' => (x, y) = (x2, y')
```

Clear an assumption from the context by name (starting from after they were moved up):

```
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
x_eq: x = x'
y_eq: y = y'
-----
(x, y) = (x', y')
move => {y}.
cannot clear y that is/are used in the conclusion
move => {x_eq y_eq}.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
(x, y) = (x', y')
```

The introduction tactical can do still more, which we will discuss in *The Introduction Tactical Revisited*.

5.3.2 Reverting an Assumption

Reversion moves an assumption from the context to become the top assumption of the conclusion: it reverts, or undoes, introduction. One way to revert an assumption is with the `move` tactic:

```
move : name.
```

where `name` is the name of an assumption. For example, the introductions done above can be reverted like this:

```
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
x_eq: x = x'
y_eq: y = y'
-----
(x, y) = (x', y')
move : x x' y y' x_eq y_eq.
```

```
Type variables: 'a, 'b
-----
forall (x x' : 'a) (y y' : 'b), x = x' => y = y' => (x, y) = (x',
y')
```

Several things are of note here.

- The `: ident` syntax is not allowed with the `idtac` or most other tactics; we'll stick with `move` for now
- The assumption to revert is selected by name, not position (it needn't be the first or last one)
- Assumptions are reverted from *right to left*, as befits the inverse of the introduction tactical, and then `move` is applied *last* (and does nothing, as usual)
- As each assumption is reverted, it becomes the top assumption of the conclusion
- An assumption cannot be reverted if a later assumption depends on it; for example, it is not possible to revert the `x` or `x'` variable declarations until `x_eq` is moved, but then they can be reverted in any order

For some reason, reversion changes local definitions to universally quantified variables without their definitions:

```
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
p: 'a * 'b := (x, y)
p': 'a * 'b := (x', y')
-----
x = x' => y = y' => p = p'
move : p.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
p': 'a * 'b := (x', y')
-----
forall (p: 'a * 'b), x = x' => y = y' => p = p'
```

However, if the assumption name is preceded by `@`, it is reverted as a `let` expression with its definition:

```
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
p: 'a * 'b := (x, y)
p': 'a * 'b := (x', y')
-----
x = x' => y = y' => p = p'
move : @p.
```

```

Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
p': 'a * 'b := (x', y')
-----
let p = (x, y) in x = x' => y = y' => p = p'

```

If you use `@` with an assumption that isn't a local definition, you get an error message that refers to a local definition as a "let-in":

```

move : @x.
symbol must reference let-in

```

Reversion is one facet of a broader technique called **generalization**. See *The Generalization Tactical*.

5.3.3 The Clear, Subst and Assumption Tactics

The `clear` tactic deletes one or more assumptions from the context.

```

Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
x_eq: x = x'
y_eq: y = y'
-----
(x, y) = (x', y')
clear x_eq y_eq.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
(x, y) = (x', y')

```

If the assumption is a variable declaration, it must not be used anywhere. The tactic is used to unclutter a goal when variables become unused or hypotheses are no longer needed.

The `subst` tactic searches the context for the first hypothesis of the form $x = t$ or $t = x$, where x is a variable and t is an expression, and uses it in the context and the conclusion as a **rewrite rule** to replace all occurrences of x with t . It then clears the hypothesis and the variable. It fails if there is no such hypothesis. If x is not specified, it repeatedly applies `subst` to all identifiers in the context. This form never fails, even if it finds nothing to substitute.

Here is a complete proof of `PairEq2` that uses several of the tactics discussed so far:

```

lemma PairEq2 (x x' : 'a, y y' : 'b) :
  x = x' => y = y' => (x, y) = (x', y').
Type variables: 'a, 'b

```

```

x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' => y = y' (x, y) = (x', y')
move=>x_eq y_eq.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
x_eq: x = x'
y_eq: y = y'
-----
(x, y) = (x', y')
subst x. (* Rewrites x, using x_eq, then clears it *)
Type variables: 'a, 'b
x': 'a
y: 'b
y': 'b
y_eq: y = y'
-----
(x', y) = (x', y')
subst y. (* Rewrites y, using y_eq, then clears it *)
Type variables: 'a, 'b
x: 'a
x': 'a
y': 'b
x_eq: x = x'
-----
(x', y') = (x', y')
reflexivity.
No more goals

```

The `assumption` tactic searches the context of the current goal for a hypothesis that is the same as (or convertible to, by reduction) the goal's conclusion. The tactic fails if it cannot solve the goal. An example can be found in *Rewriting in a Hypothesis*. This is another tactic that is rarely seen in practice, and I think it is again because `trivial`, `done` and other tactics use it.

5.3.4 The Pose Tactic

The `pose` tactic creates a local definition “out of thin air” and adds it to the context:

```

Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' /\ y = y' => (x, y) = (x', y')
pose p := (x, y).

```



```

Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
p: 'a * 'b := (x, y)
-----
x = x' /\ y = y' => p = (x', y')

```

Here the type of `p` was inferred to be `'a * 'b`. Since there is an occurrence of the expression `(x, y)` in the conclusion, it is replaced with an occurrence of the variable. If there are no occurrences, the conclusion is unchanged. For example,

```

Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' /\ y = y' => (x, y) = (x', y')
pose p' := (x, x').
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
p': 'a * 'a := (x, x')
-----
x = x' /\ y = y' => (x, y) = (x', y')

```

Some nuances of the `pose` tactic are explained in a note at the end of *Specifying the Match*. A proof that uses it can be found in *The Exists Tactic*.

5.4 Leveraging Lemmas I: The Rewrite Tactic

We are now almost ready to consider one of the workhorses of theorem proving: the `rewrite` tactic. Rewriting is based on the law of substitution of equals: it replaces one expression with another that is known to have the same value. We saw something similar with the `subst` tactic, which replaces a variable with an expression provided by a hypothesis in the context. The `rewrite` tactic replaces (rewrites) an arbitrary expression with an equal expression provided by a lemma. It is one of the most frequently used tactics. Before we start, we have one last preliminary topic to get out of the way.

Note. In this section, we shall use the term “lemma” to refer to both axioms and lemmas, and occasionally to hypotheses in the context of a goal as well.

5.4.1 Searching for and Locating Lemmas

When applying lemmas, it helps to know what is available. The `search` step displays a list of lemmas involving one or more operators, including constructors and predicates:

```

search f.      (* operator or predicate *)
search (+).    (* infix operator *)
search [-].    (* prefix operator *)

```

```
search "_.[_]" (* mixfix operator *)
search (+) (-). (* lemmas involving both operators *)
```

The `search` step only searches theories that are loaded (have been required, directly or indirectly) and you will need to use qualified names to search for operators or predicates defined in theories that have not been imported. A search can be done at any time, whether you are in a proof or not; Proof General also has a Command button on the toolbar to run a command without adding it to the script being edited.

If you search for an abbreviation, it may not find anything, depending on how the abbreviation is defined (see *Abbreviations*). For example, the `Int` theory defines `(+)` and `(-)` like this:

```
abbrev ( + ) = CoreInt.add.
abbrev ( - ) (x y : int) = x + (-y).
```

Doing

```
search (+).
```

produces a lot of output, whereas

```
search (-).
```

produces no output. When this happens, print the operator to see if it is an abbreviation and then search for an operator in its definition. The output will use the abbreviation where possible.

```
search (+) [-].
axiom subzz: forall (z : int), z - z = 0.
...
```

At the risk of getting ahead of ourselves, the arguments to the search command are not limited to operator names: they can be arbitrary *expression patterns*. These are fully explained in *Specifying the Instantiation* but here is a pattern that searches specifically for the definition of the `(-)` operator, yielding much more focused results:

```
search ( _ + (- _ ) ).
axiom subzz: forall (z : int), z - z = 0.
...
```

If you want to know where a type, operator or lemma is defined, you can ask EasyCrypt to locate it:

```
locate subzz.
+ In section [lemmas]
|
| - Int.subzz (shorten name: subzz)
```

Depending on which theories are loaded, you will get different answers; depending on which are imported, shortened names will be listed for those that don't require qualified names.

5.4.2 Basic Rewriting

The simplest form of the `rewrite` tactic is

```
rewrite L.
```

where `L` is the name of a lemma. The lemma's conclusion must be an equality, say `f1 = f2` (or `f1 <=> f2`) (with a caveat to be explained shortly). By default, the `rewrite` tactic searches for the first occurrence of `f1` in the conclusion of the current goal, thereby *instantiating* the lemma's variables, and rewrites all occurrences of this subexpression with the corresponding instantiation of `f2`. We say the lemma is used from left to right.

For example, the `Int` theory has these axioms about the comparison operators `<` and `<=`:

```
axiom lez_eqVlt :
  forall (x y : int), (x <= y) <=> ((x = y) \ / (x < y)).
axiom ltzNge (x y : int) : (x < y) <=> !(y <= x).
```

and the `List` theory has these lemmas that we shall use without proof:

```
lemma count_ge0 ['a] (p : 'a -> bool) (s : 'a list) :
  0 <= count p s.
lemma has_count ['a] (p : 'a -> bool) (s : 'a list) :
  has p s <=> (0 < count p s).
```

Now consider a proof for this lemma, also from the `List` theory:

```
lemma count_eq0 ['a] (p : 'a -> bool) (s : 'a list) :
  !(has p s) <=> count p s = 0.
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
! has p s <=> count p s = 0.
rewrite has_count.
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
! 0 < count p s <=> count p s = 0.
rewrite ltzNge.
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
!! count p s <= 0 <=> count p s = 0.
simplify.
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
count p s <= 0 <=> count p s = 0.
rewrite lez_eqVlt.
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
```

```

count p s = 0 \ / count p s < 0 <=> count p s = 0.
rewrite ltzNge.
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
count p s = 0 \ / ! 0 <= count p s <=> count p s = 0.
rewrite count_ge0.
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
count p s = 0 \ / ! true <=> count p s = 0.
done.
No more goals

```

In the first step, `f1` is `has p s` and `f2` is `0 < count p s` from the `has_count` lemma. The first (and only) match for `f1` in the conclusion of the goal is `has p s`, an exact fit, so the tactic rewrites `has p s` as `0 < count p s`. Now the conclusion has two occurrences of `count p s` and it already seems like we are nearly there, but it takes a fair amount of finagling to prove the final equality. In the second step, `f1` is `x < y` and `f2` is `!y <= x` from the `ltzNge` lemma (with extra parentheses removed) and there is no exact match, but `x` and `y` are variables in the lemma, so they can take on any values, namely `count p s` and `0`, respectively. In the third step we use the `simplify` tactic to get rid of the double negative.

The next two steps do more rewriting and then comes a step that shows something rather curious: the `count_ge0` lemma is not an equality, but it can still be used! The expression `0 <= count p s` has type `bool` and can therefore be `true` or `false`, but the lemma is asserting that it in fact equals `true` and the `rewrite` tactic is taking it at its word by rewriting `0 <= count p s` as `true`. From there, the `done` tactic takes us home.

The matching and instantiation process is more than a simple string match between the current goal and the lemma, so let's examine it more closely. To use the `lez_eqVlt` lemma from left to right, for example, the tactic is looking for a match for `x <= y`. The constant operator `<=` does indeed have to match exactly but `x` and `y` are variables. The lemma holds no matter what their values, so they have to be *instantiated* with subexpressions of the conclusion that occur as arguments of `<=`. During the match process, `x` and `y` are called **placeholders**. Following a pre-order traversal of the parse tree of the conclusion, an occurrence of `<=` is found with arguments `count p s` and `0`, so that is how `x` and `y`, respectively, are instantiated. Using this instantiation, `count p s <= 0` is rewritten as `count p s = 0 \ / count p s < 0`.

The expression used to instantiate a variable must refer only to variables defined in the context, not any bound in the conclusion. For example, if the `count_eq0` lemma is written slightly differently, the first step won't work until we move `p` and `s` to the context (we'll even change their names to be crystal clear):

```

lemma count_eq0 ['a] :
  forall (p : 'a -> bool) (s : 'a list),
    !(has p s) <=> count p s = 0.

```

```

Type variables: 'a
-----
forall (p : 'a -> bool) (s : 'a list),
  !(has p s) <=> count p s = 0
rewrite has_count.
[error-9-17]Nothing to rewrite
move => P t.
Type variables: 'a
P : 'a -> bool
t: 'a list
-----
!(has P t) <=> count P t = 0
rewrite has_count.
Type variables: 'a
P : 'a -> bool
t: 'a list
-----
! 0 < count P t <=> count P t = 0

```

(In Emacs, Proof General doesn't display [error-9-17] but uses it to highlight `has_count` to show what failed.)

Note that the placement of variables in the lemma used does not matter. If we had

```

lemma has_count ['a] :
  forall (p : 'a -> bool) (s : 'a list),
    has p s <=> (0 < count p s).

```

it would make absolutely no difference.

5.4.3 Generating Subgoals

If the lemma we are using to rewrite has assumptions other than variables, they become additional subgoals to be proved. For example, yet another lemma in the `List` theory is

```

lemma drop_oversize ['a] (n : int) (s : 'a list) :
  size s <= n => drop n s = [].

```

and another from the `Int` theory is

```

axiom lezz : forall (x : int), x <= x.

```

which we want to use to prove this lemma as a special case:

```

lemma drop_size ['a] (s : 'a list) : drop (size s) s = [].
Type variables: 'a
s: 'a list
-----
drop (size s) s = []
rewrite drop_oversize.
(* Subgoal 1 *)
Type variables: 'a
s: 'a list
-----

```

```

size s <= size s
rewrite lezz.
(* Subgoal 2 *)
Type variables: 'a
s: 'a list
-----
[] = []
reflexivity.
No more goals

```

This `rewrite` step matches `drop n s` in the lemma with `drop (size s) s` in the conclusion, instantiating `n` with `size s` and `s` with `s`, and generates two subgoals: the first is for the assumption `size s <= n` in the lemma (using the instantiation of `n` as `size s`) and the second is the original goal rewritten. The first subgoal “inherits” the context of the original goal. Both subgoals turn out to be very easy to solve.

5.4.4 Rewriting Using a Hypothesis

As we implied in the introduction to this section, hypotheses in the context can be used as lemmas. The next example proves `PairEq2` by rewriting with a hypothesis instead of using the `subst` tactic:

```

Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' => y = y' => (x, y) = (x', y')
move=>x_eq y_eq.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
x_eq: x = x'
y_eq: y = y'
-----
(x, y) = (x', y')
rewrite x_eq.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
x_eq: x = x'
y_eq: y = y'
-----
(x', y) = (x', y')
rewrite y_eq.
Type variables: 'a, 'b
x: 'a

```

```

x': 'a
y: 'b
y': 'b
x_eq: x = x'
y_eq: y = y'
-----
(x', y') = (x', y')
reflexivity.
No more goals

```

5.4.5 Rewriting from Right to Left

Sometimes the way `rewrite` works by default isn't what we need. For example, rewriting from left to right may not work. Here are some more lemmas from `List`:

```

lemma size_filter ['a] (p : 'a -> bool) (s : 'a list) :
  size (filter p s) = count p s.
lemma filter_pred0 ['a] (s : 'a list) : filter pred0 s = [].

```

and one to prove:

```

lemma count_pred0 ['a] (s : 'a list) : count pred0 s = 0.
Type variables: 'a
s: 'a list
-----
count pred0 s = 0

```

The `size_filter` lemma can clearly help prove this, but by default `rewrite` tries to match `f1`, which is `size (filter p s)`, to something in the conclusion, and since there's no match, the step fails

```

rewrite size_filter.
[error-9-19]Nothing to rewrite

```

In this case, we want to use the lemma the other way around: match `f2` and rewrite it as `f1`. To do that, we use the ***direction indicator***, which is `-`, before the name of the lemma:

```

rewrite -size_filter.
Type variables: 'a
s: 'a list
-----
size (filter pred0 s) = 0

```

Now we have a conclusion for which `filter_pred0` can help, and this time the left-to-right default is just what we want:

```

rewrite filter_pred0.
Type variables: 'a
s: 'a list
-----
size [] = 0

```

We finish using the inductive definition of `size` (courtesy of `simplify` as used by `done`):

done.

No more goals

5.4.6 Occurrence Selectors

Another default we can change is rewriting *all* occurrences of the matched expression in the conclusion. This is done using an **occurrence selector**. As a contrived example, consider using this axiom from the Int theory:

```
axiom addz0 : forall (x : int), x + 0 = x.
```

on a variant of PairEq2 where all the variables have type int

```
lemma PairEq2_int (x x' y y' : int) :  
  x = x' /\ y = y' => (x, y) = (x', y').  
Type variables: <none>  
x: int  
x': int  
y: int  
y': int  
-----  
x = x' /\ y = y' => (x, y) = (x', y')
```

Say we want to rewrite only the second occurrence of x . Occurrences are numbered starting from 1 using a pre-order traversal of the parse tree, so the first occurrence is in $x = x'$ and the second in (x, y) . The step is

```
rewrite -{2} addz0.  
Type variables: <none>  
x: int  
x': int  
y: int  
y': int  
-----  
x = x' /\ y = y' => (x + 0, y) = (x', y')
```

The second occurrence of x in the conclusion is the one rewritten as $x + 0$.

An occurrence selector has one of the following forms:

$\{i_1 \dots i_n\}$	Replace only the specified occurrences
$\{-i_1 \dots i_n\}$	Replace all but the specified occurrences
$\{+\}$	Replace all occurrences

The absence of an occurrence selector also means to rewrite all occurrences. Occurrences that don't exist (if, say, the selector is $\{9\}$ but there aren't at least 9 occurrences to choose from) produce the error message "invalid occurrence selector".

Since there are only two occurrences of x in the conclusion of PairEq2_int, this step is equivalent to the previous one:

```
rewrite -{-1} addz0.
```


5.4.7 Rewriting in a Hypothesis

The `rewrite` tactic can be used *on* (rather than *using*) a hypothesis instead of in the conclusion. For example, here is a lemma and its proof, which uses several lemmas from the `Logic` theory:

```
lemma impW (a b c : bool) : (a /\ b => c) => a => b => c.
a: bool
b: bool
c: bool
-----
(a /\ b => c) => a => b => c
move => H.
a: bool
b: bool
c: bool
H: (a /\ b => c)
-----
a => b => c
rewrite implybE in H.
a: bool
b: bool
c: bool
H: ! (a /\ b) \/ c
-----
a => b => c
rewrite negb_and in H.
a: bool
b: bool
c: bool
H: (!a \/ !b) \/ c
-----
a => b => c
rewrite -orbA in H.
a: bool
b: bool
c: bool
H: !a \/ !b \/ c
-----
a => b => c
rewrite -implybE in H.
a: bool
b: bool
c: bool
H: a => !b \/ c
-----
a => b => c
rewrite -implybE in H.
a: bool
b: bool
c: bool
H: a => b => c
```

```

-----
a => b => c
assumption.
No more goals

```

The first lemma used is

```
lemma implybE (a b : bool) : (a => b) <=> !a /\ b.
```

The default pre-order traversal of the conclusion of the goal seeking a match for $(a \Rightarrow b)$ would find $\Rightarrow$ right at the root of the tree and instantiate a with $a \wedge b \Rightarrow c$ and b with $a \Rightarrow b \Rightarrow c$.

What we want, however, is to rewrite the hypothesis by instantiating a with $a \wedge b$ and b with c . We can do that by moving the hypothesis to the context and rewriting there. In the next two sections, we'll see better ways to solve this particular problem.

Note that a hypothesis can be used to rewrite another hypothesis that comes after it in the context but not one that comes before it.

If the lemma used in H has hypotheses, these become subgoals in the usual way, except the context of the subgoal only inherits assumptions from the original goal up to but not including H . Here is an example excerpted from a rather complex lemma and proof in the `DynMatrix` theory in the `algebra` folder:

```

lemma dmatrix_uni (d : R distr) (r c : int)
  is_uniform d => is_uniform (dmatrix d r c).
d: R distr
r: int
c: int
-----
is_uniform d => is_uniform (dmatrix d r c)
... many steps omitted ...
d: R distr
r: int
c: int
uni_d: is_uniform d
xs: QuotientVec.Subtype.sT list
ys: QuotientVec.Subtype.sT list
xsin: xs \in dlist (dvector d r) (max 0 c)
ysin: ys \in dlist (dvector d r) (max 0 c)
eq_offunm: ofcols r c xs = ofcols r c ys
i: int
i_bound: 0 <= i && i < size xs
-----
size (nth witness xs i) = size (nth witness ys i) /\
forall (i0: int),
  0 <= i0 && i0 < size (nth witness xs i) =>
    (nth witness xs i).[i0] = (nth witness ys i).[i0]

```

We wish to rewrite the hypothesis `xsin` using this lemma from the `Distr` theory in the `distributions` folder:

```
lemma supp_dlist ['a] (d : 'a distr) (n : int) (xs : 'a list) :
```

```

0 <= n => xs \in dlist d n <=>
size xs = n /\ all (support d) xs.

```

Here is the proof step:

```

rewrite supp_dlist in xsin.
(* Subgoal 1 *)
d: R distr
r: int
c: int
uni_d: is_uniform d
xs: QuotientVec.Subtype.sT list
ys: QuotientVec.Subtype.sT list
-----
0 <= max 0 c
print goal 2.
(* Subgoal 2 *)
d: R distr
r: int
c: int
uni_d: is_uniform d
xs: QuotientVec.Subtype.sT list
ys: QuotientVec.Subtype.sT list
xsin: size xs = max 0 c /\ all (support (dvector d r)) xs
ysin: ys \in dlist (dvector d r) (max 0 c)
eq_offunm: ofcols r c xs = ofcols r c ys
i: int
i_bound: 0 <= i && i < size xs
-----
size (nth witness xs i) = size (nth witness ys i) /\
forall (i0: int),
  0 <= i0 && i0 < size (nth witness xs i) =>
    (nth witness xs i).[i0] = (nth witness ys i).[i0]

```

The first subgoal is the instantiation of the hypothesis $0 \leq n$ from the lemma, with only the first 6 assumptions from the original goal in its context. The second subgoal is the original goal with `xsin` rewritten as the instantiation of the right side of the lemma's equality. The proofs of these subgoals are omitted for brevity, though the first is simple enough.

5.4.8 Specifying the Instantiation (Proof Terms)

The next default we'll examine is taking the first match for `f1` (or `f2`). To prove `impW`, we moved a hypothesis to the context and worked on it there, mostly just to illustrate the technique, but we noted at the time that it also avoided a match we didn't want. Let's examine that more closely. In the `implybE` lemma, `f1` is $a \Rightarrow b$, where, again, a and b are variables of the lemma. Here are all the possible matches for $a \Rightarrow b$ in the conclusion of `impW` and the corresponding rewrite:

$(a \wedge b \Rightarrow c) \Rightarrow a \Rightarrow b \Rightarrow c$	$! (a \wedge b \Rightarrow c) \wedge (a \Rightarrow b \Rightarrow c)$
$a \wedge b \Rightarrow c$	$! (a \wedge b) \wedge c \Rightarrow a \Rightarrow b \Rightarrow c$
$a \Rightarrow b \Rightarrow c$	$(a \wedge b \Rightarrow c) \Rightarrow !a \wedge (b \Rightarrow c)$
$b \Rightarrow c$	$(a \wedge b \Rightarrow c) \Rightarrow a \Rightarrow !b \wedge c$

These are listed in the order they are encountered in a pre-order traversal of the conclusion's parse tree. There are two ways to control which potential match is used by the `rewrite` tactic:

- Provide part or all of the *instantiation* for the variables of the lemma
- Provide part or all of the *match* for `f1` (or `f2`) in the lemma (described in the next section)

The first way is to provide the `rewrite` tactic with not only the name of a lemma but also some or all of the instantiation you have in mind. The `implybE` lemma has parameters `a` and `b`, so we can provide two expressions to tell `rewrite` how to instantiate them:

```
lemma impW (a b c : bool) : (a /\ b => c) => a => b => c.
a: bool
b: bool
c: bool
-----
(a /\ b => c) => a => b => c
rewrite (implybE (a /\ b) c).
a: bool
b: bool
c: bool
-----
! (a /\ b) \ / c => a => b => c
```

The remaining steps of that proof make no erroneous matches when done in the conclusion, so the only other change needed to drop all the `in H` is to change the last step from `assumption` to `done`.

Sometimes you have to provide an instantiation because EasyCrypt can't find one on its own. Let's revisit the `drop_size` lemma. The proof above used the `drop_oversize` lemma in the default left-to-right direction, but it can be used from right to left with just a little more work. A straightforward attempt fails:

```
type variables: 'a
s: 'a list
-----
drop (size s) s = []
rewrite - drop_oversize.
cannot infer all placeholders
```

The `rewrite` tactic is referring to the variables of `drop_overside` as *placeholders* and saying it can't determine values for them. This is because it is looking for an occurrence of `[]` in the conclusion, which it finds, but this does not tell it how to instantiate `n` or `s`. We can tell it like this:

```
rewrite - (drop_oversize (size s) s).
(* Subgoal 1 *)
type variables: 'a
s: 'a list
-----
size s <= size s
rewrite lezz.
(* Subgoal 2 *)
type variables: 'a
```

```

s: 'a list
-----
drop (size s) s = drop (size s) s
reflexivity.
No more goals

```

The same proof steps as before finish the proof, even though subgoal 2 is different.

Rewrite arguments like `(implybE (a /\ b) c)` and `(drop_oversize (size s) s)` are called *proof terms*. That is, a **proof term** is the name of a lemma followed by an argument for each placeholder, all enclosed in parentheses. So far we've implied that each placeholder corresponds to a variable, but that isn't quite right: placeholders correspond to a lemma's *assumptions*, which includes both variables and hypotheses (but not local definitions/let expressions). Thus, a proof term for `drop_oversize` has *three* placeholders: one for `n`, one for `s` and one for `n <= size s`.

An argument for a variable placeholder tells how to instantiate the variable, and if none is provided, the tactic tries to find one on its own and fails if it can't. An argument for a hypothesis placeholder tells how to prove the hypothesis, and if none is provided, the tactic generates a subgoal for it. This is what has "really" been happening in the examples above.

If you don't want to provide an argument for a placeholder, you can use `_` (a single underscore, here called a **proof hole**). Trailing arguments can be omitted entirely. When we wrote

```
rewrite drop_oversize. (* what everyone writes *)
```

above, we could instead have written any of

```

rewrite (drop_oversize _ _ _). (* the "true" proof term *)
rewrite (drop_oversize _ _).  (* one trailing arg omitted *)
rewrite (drop_oversize _).    (* two trailing args omitted *)
rewrite (drop_oversize).      (* three trailing args omitted *)

```

If a proof term has too many arguments (holes or otherwise), the result is the error, "too many arguments".

Expressions in a proof term can also have holes, making them *expression patterns*. In the `impW` example, we can use the proof term `(implybE (_ /\ _))` to instantiate the `a` parameter of `implybE` with any application of the `(/\)` operator, which is sufficient information for it to match only `a /\ b` and thereby instantiate `b` with `c` as well. The expression pattern `(/\)` would not work, not because it is invalid, but because it has type `bool -> bool -> bool`, whereas `a` has type `bool`.

When using occurrence selectors with expression patterns, it is important to remember that occurrences of a subexpression are only counted *after* a match has been made. In the conclusion of lemma `PairEq2`,

```
x = x' => y = y' => (x, y) = (x', y')
```

the pattern `__ = __` matches three subexpressions, but that does not constitute three occurrences. It will be matched to the first one, `x = x'`, and there is only one occurrence of that. *Matching is not affected by an occurrence selector*, and counting occurrences happens after matching.

That leaves one burning question: what sort of argument do you provide for a hypothesis placeholder? Well, you provide a (sub) proof term! If we add a proof term for `lezz` as the third placeholder of `drop_oversize`, we prevent the subgoal from appearing at all:

```

type variables: 'a
s: 'a list
-----
drop (size s) s = []
rewrite - (drop_oversize (size s) s (lezz (size s))).
type variables: 'a
s: 'a list
-----
drop (size s) s = drop (size s) s
reflexivity.
No more goals

```

In summary, we fill variable placeholders of a proof term with (possibly holey) expression patterns and hypothesis placeholders with (possibly holey) sub-proof terms.

In principle, proof terms can be nested to an arbitrary depth, producing amazingly compact, unreadable proofs. In practice, variable placeholders are only provided when necessary to get the desired instantiation and hypothesis placeholders are left blank (by omission or `_`).

5.4.9 Specifying the Match

The second way to control the match is to specify it directly by providing an expression pattern for where in the conclusion (or `H`) you want the lemma to match. In other words, we provide a **match pattern** that the tactic will match to the conclusion (selecting the first match), and then match `f1` (or `f2`) of the lemma to the instantiated match pattern to instantiate the lemma's assumptions. The match pattern is specified in brackets before the name of the lemma (or other proof term—you can use both techniques together).

For example, in the proof of `impW`, we can specify the desired match for `a => b` in `implybE` like this:

```

a: bool
b: bool
c: bool
-----
(a /\ b => c) => a => b => c
rewrite [a /\ b => c] implybE.
a: bool
b: bool
c: bool
-----
! (a /\ b) /\ c => a => b => c

```

The default match without a match expression is the entire conclusion. The expression pattern `_` also matches the entire conclusion:

```

rewrite [_] implybE.
a: bool
b: bool

```

```

c: bool
-----
! (a /\ b => c) /\ a => b => c

```

If the expression pattern does not match anything in the conclusion, the dreaded “nothing to rewrite” error is issued. For example, and contrary to appearances, this pattern fails:

```

rewrite [a => b] implybE.
[error-7-24]nothing to rewrite

```

That’s because $a \Rightarrow b \Rightarrow c$ means $a \Rightarrow (b \Rightarrow c)$ rather than $(a \Rightarrow b) \Rightarrow c$. The following pattern works fine (and doesn’t match the antecedent, which is $(a /\ b) \Rightarrow c$):

```

rewrite [b => c] implybE.
a: bool
b: bool
c: bool
-----
(a /\ b => c) => a => !b /\ c

```

An expression pattern that matches in the conclusion but doesn’t match f_1 (or f_2) in the lemma produces the same error message:

```

rewrite [a /\ b] implybE.
[error-7-24]nothing to rewrite

```

A note on the pose tactic. The `pose` tactic’s argument is an expression pattern, so it, too, can have holes and an occurrence selector:

```

Type variables: <none>
x : int
y : int
-----
2 * ((x + 1 + y) * x) = (x + 1 + y) * x + (x + 1 + y) * x
pose {-2} z := (_ + y) * _. (* Don't replace 2nd occurrence *)
Type variables: <none>
x : int
y : int
z : int := (x + 1 + y) * x
-----
2 * z = (x + 1 + y) * x + z

```

5.4.10 Rewriting with an Anonymous Lemma

A proof term may also specify an *anonymous lemma*. The lemma is a `bool` expression preceded by a colon (and optional underscore, which doesn’t mean anything), all in parentheses. The expression may refer to any of the variables and local definitions in the context, as usual, but may not contain holes. When used with the `rewrite` tactic, the expression is typically an equation or equivalence.

An anonymous lemma is basically a lemma that isn’t worth proving on its own, so we just prove it inline. For example, we’ve seen lemmas for basic arithmetic, such as $x + 0 = x$ or $x + 1 < x$, but there obviously won’t be a ready-made lemma for every arithmetic fact you might need. Such lemmas are

typically easy to prove (especially for concrete numbers), so the only trick is deciding which one you need.

Moreover, an anonymous lemma is an unfounded assumption until it is proved. The `rewrite` tactic generates two subgoals from it: one to prove that the lemma is true, given the context, and one with the original goal rewritten using it. If the rewriting fails, the tactic fails.

For example, here is the first in a series of lemmas and proofs about specific powers of two that are useful when working with unsigned integers. The proofs use these lemmas from the `Ring.ComRing` theory:

```
lemma exp2 (x : int) : x ^ 2 = x * x.
lemma exprD_nneg (x m n : int) : 0 <= m => 0 <= n => x ^ (m + n)
= x ^ m * x ^ n.
```

The first one just gets us started:

```
lemma exp2 : 2 ^ 2 = 4.
-----
2 ^ 2 = 4
rewrite exp2.
-----
2 * 2 = 4
done.
No more goals
```

The next one uses an anonymous lemma to decompose 4 in just the right way to use the first lemma:

```
lemma exp4 : 2 ^ 4 = 16.
-----
2 ^ 4 = 16.
rewrite (: 4 = 2 + 2).
(* Subgoal 1 *)
-----
4 = 2 + 2
done.
(* Subgoal 2 *)
2 ^ (2 + 2) = 16
rewrite exprD_nneg.
(* Subgoal 1 *)
-----
0 <= 2
done.
(* Subgoal 2 *)
-----
0 <= 2
done.
(* Subgoal 3 *)
-----
2 ^ 2 * 2 ^ 2 = 16
rewrite exp2.
(* 4 * 4 = 16 *)
```



```
done.
No more goals
```

Occurrence selectors and the direction indicator work normally with this kind of proof term. Just to illustrate that, here's one more power of 2 with the anonymous lemma backwards:

```
lemma exp8 : 2 ^ 8 = 256.
-----
2 ^ 8 = 256.
rewrite - (: 4 + 4 = 8).
(* Subgoal 1 *)
-----
4 + 4 = 8
done.
(* Subgoal 2 *)
2 ^ (4 + 4) = 256
rewrite exprD_nneg.
(* Subgoal 1 *)
-----
0 <= 4
done.
(* Subgoal 2 *)
-----
0 <= 4
done.
(* Subgoal 3 *)
-----
2 ^ 4 * 2 ^ 4 = 256
rewrite exp2.
(* 16 * 16 = 256 *)
done.
No more goals
```

An anonymous lemma may have assumptions and the proof term may optionally instantiate them. For example,

```
lemma exp16 : 2 ^ 16 = 65536.
-----
2 ^ 16 = 65536.
rewrite ((: forall n k, n = (n - k) + k) 16 8).
(* Subgoal 1 *)
-----
forall (n k : int), n = n - k + k
move => n k.
n: int
k: int
-----
n = n - k + k.
rewrite -addzA. (* Int theory: x + (y + z) = (x + y) + z *)
n: int
k: int
```

```

-----
n = n + ((-k) + k).
rewrite subzz. (* Int theory: z - z = 0 *)
n: int
k: int
-----
n = n + 0.
rewrite addz0. (* Int theory: x + 0 = x *)
n: int
k: int
-----
n = n.
reflexivity.
(* Subgoal 2 *)
2 ^ (16 - 8 + 8) = 65536
rewrite exprD_nneg.
(* Subgoal 1 *)
-----
0 <= 16 - 8
done.
(* Subgoal 2 *)
-----
0 <= 8
done.
(* Subgoal 3 *)
-----
2 ^ (16 - 8) * 2 ^ 8 = 65536
simplify.
2 ^ 8 * 2 ^ 8 = 65536
rewrite exp2.
(* 256 * 256 = 65536 *)
done.
No more goals

```

5.4.11 Rewriting with Multiple Proof Terms

The `rewrite` tactic will accept multiple arguments to rewrite with a sequence of proof terms in one step. The general form is,

```

rewrite p1 ... pn.
rewrite p1 ... pn in H.

```

This applies p_1 to the current goal, then applies p_2 to *each* of the subgoals it generates in turn, replacing each one with a list of new subgoals (if any), and so on through p_n . Once a subgoal is solved, of course, the remaining proof terms won't know it was ever there. If all remaining subgoals are solved by, say, p_i , the remaining terms are effectively ignored because there is nothing left to do. The tactic fails without changing the current goal (i.e., discarding however far it got before failing) if any of the applications fail.

For example, in the proof of the `count_pred0` lemma above, the first rewrite replaced the current goal with one subgoal and the second rewrite replaced that subgoal with a different one. These steps can be combined:

```

Type variables: 'a
s: 'a list
-----
count pred0 s = 0
rewrite -size_filter filter_pred0.
Type variables: 'a
s: 'a list
-----
size [] = 0

```

In the proof of the `drop_size` lemma, however, we cannot combine steps because rewriting with `drop_oversize` generates two subgoals and the `lezz` lemma applies to the first but not the second:

```

type variables: 'a
s: 'a list
-----
drop (size s) s = []
rewrite drop_oversize lezz.
[error-76-81]nothing to rewrite

```

To address this problem, a proof term can be preceded by a *focus index*, which in its simplest form is an integer followed by a colon that specifies to which subgoal the proof term is applied. For example,

```

type variables: 'a
s: 'a list
-----
drop (size s) s = []
rewrite drop_oversize 1: lezz.
Type variables: 'a
s: 'a list
-----
[] = []

```

Subgoals are numbered starting at one. If an index is too high, the message “invalid focus index: *n*” is displayed; if an index is 0, the message “parse error: focus-index must be positive” is displayed, even though this statement is false. A negative focus index is relative to the position just past the last subgoal, so in the above example, `-1` refers to the second subgoal, `-2` to the first and `-3` and below are invalid. A focus index can have any of these forms:

<code>N</code>	apply the proof term to the subgoal <code>N</code>
<code>N .. M</code>	apply the proof term to subgoals <code>N</code> through <code>M</code> , inclusive; <code>N</code> does not have to be less than or equal to <code>M</code> or even have the same sign
<code>N ..</code>	apply the proof term to subgoals <code>N</code> through the last subgoal
<code>.. M</code>	apply the proof term to subgoals 1 through <code>M</code> , inclusive
<code>f₁, ..., f_n</code>	apply the proof term to each subgoal in the union of the indexes and ranges listed; overlaps and repetition don't matter
<code>~f₁, ..., f_n</code>	apply the proof term to all subgoals except those in the union of the indexes and ranges listed
<code>f₁, ..., f_n</code>	apply the proof term to each subgoal in the union of the indexes and ranges

$\sim f_1, \dots, f_n$ listed before $\sim$ except for those listed in the union of the indexes and ranges listed after $\sim$

To apply a proof term more than once without repeating it, it can be preceded by a **repetition marker** to indicate how many times to apply the term in succession to each subgoal. The markers are as follows:

- ! apply the proof term once to each subgoal and fail if it does not apply to all of them; then make repeated passes over the subgoals until it no longer applies to any of them
- ? same as previous, but do not fail even if the proof term does not apply to all subgoals in the first pass (i.e., applying it is optional)
- n! apply the proof term to a depth of exactly n (i.e., make n passes and fail if the proof term does not apply to any goal in any pass)
- n? apply the proof term to a depth of at most n (i.e., make up to n passes, continuing as long as the proof term applies to at least one subgoal)

For example, here is a proof of another `List` lemma:

```
lemma belast_rcons (x : 'a) (s : 'a list) (z : 'a) :
  belast x (rcons s z) = x :: s.
Type variables: 'a
x: 'a
s: 'a list
z: 'a
-----
belast x (rcons s z) = x :: s
rewrite lastI -cats1 -cats1 belast_cat.
Type variables: 'a
x: 'a
s: 'a list
z: 'a
-----
belast x s ++ belast (last x s) [z] = belast x s ++ [last x s]
reflexivity.
No more goals
```

Each lemma in the `rewrite` step replaces the current (sub)goal with one new subgoal. It explicitly applies the `cats1` lemma in the reverse direction two times in a row. This step can be written more compactly using a repetition marker like this:

```
rewrite lastI -2!cats1 belast_cat.
```

This step specifies exactly 2 applications of `cats1`, but we needn't be so specific:

```
rewrite lastI -!cats1 belast_cat.
```

For illustrative purposes, we can add repetition markers to the other proof terms, throw in some unnecessary focus indices and occurrence selectors and vary the spacing:

```
rewrite 1: 5? {1} lastI - 1 : - 2 ! { 1 } cats1 1:!!{1}belast_cat.
```

For the record, the correct order of the various options is

- Focus index
- Direction indicator
- Repetition marker
- Occurrence selector
- Match expression pattern
- Proof term or list of proof terms

The above process highlights the difference between a proof step and a tactic: a step is applied only to the current goal, but a tactic is free to range over all of the subgoals it generates repeatedly until it produces its final list, which is then pushed onto the goal stack. See *The Chain Tactical*, which works the same way, and Figure 8 for a visualization of the process.

5.4.12 Rewriting with a Choice of Proof Terms

Instead of specifying a single proof term, you can list several proof terms separated by commas and enclosed in parentheses. This kind of rewrite argument tries each proof term in the order provided until one of them succeeds; it only fails if all of the proof terms fail. The list can be preceded by the usual decorations (focus index, direction indicator, repetition marker, occurrence selector and/or match expression pattern) and each proof term can have its own direction indicator, which takes precedence.

This technique only makes sense if there are multiple subgoals in need of rewrite, such as when rewriting with multiple arguments or using the chain tactical (see *The Chain Tactical*). Even then, it can be hard to follow. For example, consider this lemma and proof from the `List` theory:

```
lemma onth_index ['a] (x : 'a) (xs : 'a list) :
  x \in xs => onth xs (index x xs) = Some x.
Type variables: 'a
x: 'a
xs: 'a list
-----
x \in xs => onth xs (index x xs) = Some x
move => x_in_xs.
Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
-----
onth xs (index x xs) = Some x
rewrite (onth_nth witness).
(* Subgoal 1 *)
Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
-----
0 <= index x xs < size xs
rewrite index_mem index_ge0.
Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
```

```

-----
true && (x \in xs)
done.
(* Subgoal 2 *)
Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
-----
Some (nth witness xs (index x xs)) = Some x
rewrite nth_index.
(* Subgoal 1 *)
Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
-----
x \in xs
done.
(* Subgoal 2 *)
Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
-----
Some x = Some x
done.
No more goals

```

With a bit of cleverness, we can combine all the rewrite steps into just one:

```

Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
-----
onth xs (index x xs) = Some x
rewrite (onth_nth witness) (index_mem, nth_index) 1:index_ge0.

```

The first proof term results in two subgoals. In the choice term, the first proof term rewrites the first subgoal, so the second proof term is ignored, while the first proof term fails on the second subgoal and the second proof term works, which results in two subgoals. The proof term with the focus index then rewrites the first subgoal with one more lemma. This leaves three subgoals, all of which can be solved by the `done` tactic.

```

(* Subgoal 1 *)
Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
-----

```

```

true && (x \in xs)
done.
(* Subgoal 2 *)
Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
-----
x \in xs
done.
(* Subgoal 3 *)
Type variables: 'a
x: 'a
xs: 'a list
x_in_xs: x \in xs
-----
Some x = Some x
done.
No more goals

```

This proof is shorter, but I think most readers will agree that the longer proof is clearer.

5.4.13 Subst versus Rewrite

The `rewrite` tactic is vastly more versatile than the `subst` tactic but the `subst` tactic does *more* than the `rewrite` tactic in two key ways:

1. The `subst` tactic rewrites `x` as `t` throughout the context *and* the conclusion
2. The `subst` tactic clears `x`, which is now superfluous, as well as the hypothesis it used

5.4.14 Hints

Hints are directives or suggestions to certain tactics that influence how they operate. Most or all of them relate to using lemmas for rewriting.

Simplify. Hints for the `simplify` tactic are also called *user reductions*. They allow you to define your own normal or canonical forms for selected expressions, beyond those of the lambda calculus. For example, you might decide that the normal form of an expression over an associative operator like `+` for the integers is when it is written in a left associative way, so it can be printed without parentheses. Normally, the `simplify` tactic has no effect on such expressions:

```

x: int
y: int
z: int
-----
x + (y + x + (z + (y + x)) + y) = 5
simplify.
(* has no effect *)

```

We can change the tactic by adding a hint (which cannot be done inside a proof):

```

hint simplify addzA.          (* somewhere before the proof *)
simplify.

```

```

x: int
y: int
z: int
-----
x + y + x + z + y + x + y = 5

```

Like the `rewrite` tactic, the `simplify` tactic first finds a match for the left side of `addzA` to instantiate `x`, `y` and `z`, and then replaces all occurrences of the match with the instantiation of the right side of `addzA`. Unlike the `rewrite` tactic, it then matches `addzA` again, and keeps on until there are no matches for the left side. Another way to do this is

```
rewrite ! addzA.
```

The `simplify` tactic can have many hints and the lemmas all go into one “database” or registry. Once there, every use of `simplify` (including by tactics such as `trivial` and `reflexivity`) attempts to match all the lemmas in turn. If one is found, the goal is reduced and the whole set is tried again, until none match.

Again like the `rewrite` tactic, a lemma whose conclusion is not an equality can be used to replace occurrences of its conclusion with `true`. Indeed, any lemma can be used this way, including `addzA`. If you don’t want a lemma used this way, you can annotate the hint like this:

```
hint simplify [reduce] addzA.
```

and if you want a lemma used *only* this way, you can annotate it like this:

```
hint simplify [eqtrue] addzA.
```

You can also explicitly list both. In practice, this seems to have little or no effect. In particular, using `eqtrue` with `addzA` does not prevent the first example above from working.

Unlike the `rewrite` tactic, the `simplify` tactic will never generate subgoals from a user reduction. That is, if a lemma has hypotheses, they must be provable without using user reductions (i.e., reducible to `true` using only `beta`, `iota`, `zeta` and `logic` reduction) or the match is ignored.

Also unlike with the `rewrite` tactic, there is no way to specify that an equality lemma should be used right-to-left and there are no repetition markers, occurrence selectors or focus indices. Since we are defining a canonical form, the intent of a user reduction is to reduce *every* occurrence *every* time.

Another example when a canonical form is useful is the `drop` operator in the `List` theory. When this operator is applied to a `list` expression involving the constructors `[]` and/or `(: :)` and simplified, the `iota` tactic applies the inductive definition of `drop` to reduce it. It would be nice if `simplify` would also reduce expressions where the argument for `n` is a literal. For example, it is hard to imagine *not* wanting to replace `drop 0 s` with `s`, and

```
hint simplify drop0.
```

will do exactly that. The `List` theory has dozens of such lemmas that you might want to apply automatically, including

```
hint simplify size_cat, size_nseq, take0, take_cat, take_size,
              nseq0, nseq1, cat_nseq, drop_cat, drop_size.
```


It would be nice to make a lemma like `nseqS` a user reduction to reduce, say, `(nseq 3 [])` to the literal list `[[[]; []; []]]`. Unfortunately, this doesn't work because `simplify` will not automatically change `3` to `2 + 1` to apply the lemma. However, a very similar lemma does work:

```
lemma nseqP ['a] (n : int) (x : 'a) :
  0 < n => nseq n x = x :: nseq (n - 1) x.
proof
  move => ng0.
  rewrite (nseqS (n - 1)).
  rewrite -(ltzS 0 (n - 1)).
  done.
  reflexivity.
hint simplify nseqP.
```

When `n` is a literal like `3`, the precondition `0 < 3` is reduced to `true` by the `logic` tactic.

User reductions can be equally useful in cases like the `FSet` and `FMap` theories, where a non-inductive type (`fset` and `fmap`, respectively) nevertheless behaves like one over a set of operators (*de facto* constructors). We can again use hints to make `simplify` work as if these were inductive types:

```
hint simplify emptyE, get_setE, set_setE.
```

Laws about idempotency, units (identity elements), inverses (a.k.a. cancellation) and involution are other good candidates for user reductions. If you want the `simplify` tactic to unfold all occurrences of an operator automatically, axiomatize its definition and add that as a hint. For example, in the `FSet` theory, the definition of the empty set is axiomatized and one could easily make it a user reduction:

```
op fset0 = oflist [<:'a>] axiomatized by set0E.
hint simplify set0E.
```

The downside of such a hint is that the intuitive meaning of `fset0` is less apparent when written as `oflist [<:'a>]`. In a case like the distributive law of multiplication over addition, it is not evident that `a * b + a * c` is always to be preferred over `a * (b + c)`, or vice versa: either form might be useful in a proof. Even a lemma such as `addz0 (x + 0 = x)` is occasionally applied right-to-left and making it a user reduction would make this a little more common. The `EasyCrypt` library contains thousands of lemmas but makes very few of them user reductions, leaving that to the user.

Equational logic and *term rewriting*, where rewriting continues until no rules apply, are their own fields of study, each with an extensive literature. Two aspects of interest here are *termination* and *confluence*.

The standard typed lambda calculus reductions are guaranteed to terminate after a finite number of rounds, but not so user reductions. A commutative law is the classic example of a non-terminating reduction: you can rewrite `a + b` as `b + a` and then back to `a + b` forever.

```
hint simplify addzC.
x: int
y: int
z: int
-----
x + (y + x + (z + (y + x)) + y) = 5
simplify.
anomaly: Stack overflow
```

Other reductions might terminate on some inputs and not on others. It is also possible (all too easy, really) to have a cycle of rewrite rules that never terminate, even though individually each one does. This is unfortunate when, for example, you want to use commutativity and associativity to get all the literal numbers or terms involving x in an expression together so they can be combined. It takes many proof steps to reduce $x + (y + x + (z + (y + x)) + y)$ to $3 * (x + y) + z$.

Confluence means that the result of reduction is the same regardless of the order in which the rules are applied. A lack of confluence means that an expression can have multiple normal forms, which is not an error but is generally undesirable. For example, consider the following lemmas about the `flatten` operator on lists, which takes a list of lists and concatenates them into one list.

```
lemma flatten_nil ['a] : flatten [<:'a list>] = [].
lemma flatten_cons ['a] (x : 'a list) s :
  flatten (x :: s) = x ++ flatten s.
lemma flatten_seq1 ['a] (s : 'a list) : flatten [s] = s.
```

If I create hints for the first two:

```
hint simplify flatten_nil, flatten_cons.
```

then `simplify` works as expected:

```
lemma flatten_two ['a] (a b : 'a list) :
  flatten [a; b] = a ++ b.
Type variables: 'a          | The context never changes,
a: 'a list                | so it will not be repeated
b: 'a list                |
-----                  |
flatten [a; b] = a ++ b
simplify.
a ++ (b ++ []) = a ++ b
```

The order of the hints doesn't matter because no list can match both `[]` and `x :: s`. It would be nice, though, to reduce `b ++ []` to `b`., which makes the goal `a ++ b = a ++ b`, which reduces to `true`. The third lemma looks like it could help, so let's add a hint for it:

```
hint simplify flatten_seq1, flatten_nil, flatten_cons.
```

Now we get

```
flatten [a; b] = a ++ b
simplify.
true
```

Perfect! But if we add the third hint at the end,

```
hint simplify flatten_nil, flatten_cons, flatten_seq1.
```

we're back where we were:

```
flatten [a; b] = a ++ b
simplify.
a ++ (b ++ []) = a ++ b
```

We can gain insight by dropping all hints and applying lemmas manually. For the first step, only `flatten_cons` matches the goal:

```
flatten [a; b] = a ++ b
rewrite flatten_cons.
a ++ flatten [b] = a ++ b
```

Now we have a choice, because `flatten [b]` matches both `flatten_cons` and `flatten_seq1`. If we choose the former, we can only choose `flatten_nil` as the third and last step:

```
rewrite flatten_cons.
a ++ (b ++ flatten []) = a ++ b
rewrite flatten_nil
a ++ (b ++ []) = a ++ b
```

If we choose the latter, we get

```
rewrite flatten_seq1.
a ++ b = a ++ b
```

and “vanilla” `simplify` (specifically the `logic` tactic) reduces this to `true`. So if `flatten_seq1` comes before `flatten_cons` in the hint, it will match singleton sequences and leave longer sequences for `flatten_cons` to match: together, they give the desired result; but if we list `flatten_cons` first it will prevent `flatten_seq1` from ever being used and not give the desired result.

An even better solution is to add a fourth lemma to the list:

```
lemma cats0 ['a] (s : 'a list): s ++ [] = s.
hint simplify flatten_nil, flatten_cons, flatten_seq1, cats0.
```

With all four lemmas in play, we get the desired result regardless of the order in which they are listed. We may notice that `flatten_seq1` is redundant and remove it, but it is doing no harm.

If we try to reduce a longer list, we run into another problem:

```
lemma flatten_three ['a] (a b c : 'a list) :
flatten [a; b; c] = a ++ b ++ c.
...
flatten [a; b; c] = a ++ b ++ c
simplify.
a ++ (b ++ c) = a ++ b ++ c
```

Now we’re stuck because of how the applications of `++` are ordered. A solution is to add yet another lemma to the hint:

```
lemma catA ['a] (s1 s2 s3 : 'a list):
s1 ++ (s2 ++ s3) = (s1 ++ s2) ++ s3.
hint simplify
flatten_nil, flatten_cons, flatten_seq1, cats0, catA.
```

Now we get the desired result:

```

flatten [a; b; c] = a ++ b ++ c
simplify.
true

```

As we add hints for more list operators, will we run into trouble again? It's very hard to predict, detect or remediate.

To provide a degree of control over the order in which reductions are applied, EasyCrypt lets you assign priority levels to lemmas. A lemma can be assigned a non-zero priority N by adding $@N$ after its name. Without $@N$, the priority is 0 (the highest). A group of lemmas can be assigned a priority by listing them in parentheses and adding $@N$ after the right parenthesis. For example, we could have fixed our 3-lemma hint above by specifying

```

hint simplify flatten_nil, flatten_cons@1, flatten_seq1.

```

Now the order no longer matters, but identifying additional lemmas to add is a far better solution because assigning priority levels is notoriously brittle.

To summarize, the full syntax for user reductions is

```

hint simplify lemma1, lemma2@N, (lemma3, ..., lemmak)@N, ..., lemman.
hint simplify []
      lemma1, lemma2@N, (lemma3, ..., lemmak)@N, ..., lemman.
hint simplify [reduce]
      lemma1, lemma2@N, (lemma3, ..., lemmak)@N, ..., lemman.
hint simplify [eqtrue]
      lemma1, lemma2@N, (lemma3, ..., lemmak)@N, ..., lemman.

```

Once added, there is no way to remove a lemma from the user reduction database. The step

```

print hint simplify.

```

lists the lemmas in the user reduction database.

Rewrite. Lemmas can be added to any number of user-defined rewrite databases and applied *en masse* by the `rewrite` tactic. To add one or more lemmas to a new or existing database, use

```

hint rewrite my_rw_db : lemma1 ... lemman.

```

You can create an empty database by omitting the list of lemmas. You can add lemmas incrementally to a database over time, from the same or other theories. Lemmas are maintained in a particular order, but the details are unclear. As of this writing, a bug in the parser requires the lemma names to start with a lower-case letter, even though lemmas themselves do not have this restriction.

The `rewrite` tactic accepts a database name as one of its arguments, to be mixed and matched with all the other kinds of arguments. It tries to match each lemma in turn to the current goal until one succeeds. It fails if none succeeds. The order in which the lemmas are tried may be significant: see the discussion of confluence for user reductions. Since the `rewrite` tactic generates subgoals, it can also happen that the first lemma that matches generates subgoals that cannot be proved, while another lemma might have made true progress. Rewrite databases can't replace good judgement.

Like the other kinds of pattern, a database name can be preceded by a focus index, direction indicator, repetition marker and/or occurrence selector. A repetition marker applies to the group as a whole—

after applying one lemma in the group, the tactic tries again to find a matching lemma, which need not be the same one each time. This raises termination concerns, also discussed above for user reductions. The other options apply in the usual way. In particular, the direction indicator means that *all* the lemmas in the database will be used right-to-left rather than left-to-right and an occurrence selector means that, no matter which lemma matches, only the indicated occurrences will be rewritten.

You can print a rewrite database to see what lemmas it contains. The category is `rewrite`.

```
print rewrite my_rw_db.
```

The step

```
print hint rewrite.
```

lists all rewrite databases and the lemmas they contain. Lemmas cannot be removed from a rewrite database.

Solve. Lemmas can be added to any number of user-defined solve databases, which might be used by the `solve` tactic. See *The Solve Tactic*. There is also a default solve database that the `trivial` tactic uses.

There are two forms of solve hint:

```
hint exact my_solve_db : lemma1 ... lemman.
hint solve N my_solve_db : lemma1 ... lemman.
```

The database is optional, as is the list of lemmas. The database is created if it doesn't exist and lemmas can be added incrementally over time, from the same or other theories. An `exact` hint is the same as a `solve` hint with `N = 0`. It doesn't mean the `exact` tactic. `N` might be a priority level, as it is for user reductions, or something like a limit on the depth of the proof tree. The `List` theory has

```
hint exact size_ge0.
```

immediately following the lemma's proof. As a result, the `trivial` tactic will try to apply this lemma. The `simplify` tactic does not, so the default database for solve hints is distinct from the one for user reductions.

You can print a solve database to see what lemmas it contains and their assigned priority level. The category is `solve`.

```
print solve my_solve_db.
```

The step

```
print hint solve.
```

lists all solve databases and the lemmas they contain. Lemmas cannot be removed from a solve database.

5.4.15 Folding and Unfolding

The `rewrite` tactic performs a second kind of rewriting that has nothing to do with lemmas. In its general forms,

```
rewrite  $\rho_1 \dots \rho_n$ .
```

```
rewrite  $\rho_1 \dots \rho_n$  in H.
```

each argument can be either a proof term, as we've been discussing, or an expression pattern prefixed by /. The expression pattern must have a **defined symbol** as its root, meaning the name of a concrete operator or predicate or a variable with a local definition. This excludes abstract operators and predicates, constructors, projections, and operators that are defined inductively (i.e., by cases).

The tactic finds the first subexpression of the goal's conclusion (or H), using a pre-order traversal, that matches and fully instantiates the pattern and then replaces all occurrences of this subexpression with the definition of the defined symbol, instantiating its parameters according to the match.

As an academic example, consider the definition

```
op g x = x * x.
```

and the lemma

```
lemma g_xx (x : int) :
  g (x + x) = g x + 2 * x * x + g x.
Type variables: <none>
x: int
-----
g (x + x) = g x + 2 * x * x + g x
```

Occurrences of (g x) can be replaced with its definition like this:

```
rewrite /(g x).
Type variables: <none>
x: int
-----
g (x + x) = x * x + 2 * x * x + x * x
```

This is called *unfolding* the symbol g and is pretty much what the `delta` tactic does (see *The Simplify Tactic*), only `rewrite` does it far more flexibly. For example, unfolding can be undone using the direction indicator:

```
rewrite -(g x).
Type variables: <none>
x: int
-----
g (x + x) = g x + 2 * x * x + g x
```

This is called *folding* the symbol g. Note that it only found two occurrences of $x * x$ to fold because $2 * x * x$ is parsed as $(2 * x) * x$.

The pattern can be preceded by an occurrence selector:

```
rewrite {2}/(g x).      (* unfold the second occurrence *)
Type variables: <none>
x: int
-----
g (x + x) = g x + 2 * x * x + x * x
rewrite /(g x).        (* unfold the remaining occurrences *)
```

```

Type variables: <none>
x: int
-----
g (x + x) = x * x + 2 * x * x + x * x
rewrite -{1}/(g x).    (* fold the first occurrence *)
Type variables: <none>
x: int
-----
g (x + x) = g x + 2 * x * x + x * x

```

If the pattern has one or more holes, it might have multiple matches with different instantiations. In this case, only occurrences of the first match are unfolded:

```

rewrite /(g _).      (* first match is (g (x + x)) *)
Type variables: <none>
x: int
-----
(x + x) * (x + x) = g x + 2 * x * x + x * x

```

or folded

```

rewrite -/(g _).
Type variables: <none>
x: int
-----
g (x + x) = g x + 2 * x * x + x * x

```

The pattern can be preceded by a repetition marker to fold or unfold multiple instantiations:

```

rewrite !/(g _).      (* unfold (g x) and (g (x + x)) *)
Type variables: <none>
x: int
-----
(x + x) * (x + x) = x * x + 2 * x * x + x * x
rewrite -!/(g _).
Type variables: <none>
x: int
-----
g (x + x) = g x + 2 * x * x + g x

```

The pattern can reference a local definition to unfold it:

```

Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
x_eq: bool := x = x'
-----
x_eq => y = y' => (x, y) = (x', y')
rewrite /x_eq.
Type variables: 'a, 'b
x: 'a

```

```

x': 'a
y: 'b
y': 'b
x_eq: bool := x = x'
-----
x = x' => y = y' => (x, y) = (x', y')

```

and fold it:

```

rewrite -/x_eq.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
x_eq: bool := x = x'
-----
x_eq => y = y' => (x, y) = (x', y')

```

If the pattern is just the symbol's name with no arguments, not even parentheses, `rewrite` behaves more like `delta` by unfolding all occurrences of all instantiations:

```

rewrite /g.          (* unfold (g x) and (g (x + x)) *)
Type variables: <none>
x: int
-----
(x + x) * (x + x) = x * x + 2 * x * x + x * x

```

but folding none of them:

```

rewrite -/g.          (* has no effect *)
Type variables: <none>
x: int
-----
2 * x * (2 * x) = x * x + x * x + 2 * (x * x)

```

If the symbol's name is undefined, the tactic has no effect: it doesn't fail or display an error message (the `delta` tactic displays "unknown operator").

As long as the pattern has parentheses, however, errors cause the tactic to fail with an appropriate error message, such as "unknown variable" or "no matching operator." If you try to fold or unfold a symbol that is inappropriate, such as an abstract operator or predicate, constructor or quantifier, these are the error messages displayed:

```

rewrite /(_ + _).      (* (+) is abstract *)
the operator cannot be unfolded
rewrite /(Some).       (* Some is a constructor *)
the operator cannot be unfolded
rewrite /(_ = _).      (* or rewrite /(2). *)
not headed by an operator/predicate

```

The last message is a little misleading, since the expression *is* headed by an operator, but equality is special.

Finally, if the expression pattern does not occur in the conclusion, the tactic has no effect.

Note. Most tactics will unfold symbols in the goal without being told, as part of their normal operation. These include `rewrite`, `case`, `elim`, `apply`, `exact`, `split`, `trivial`, `done`, `progress`, `assumption`, `reflexivity`, `exists` and even several introduction patterns (`name`, `?`, `+`, `*` and the `case` pattern, of course), but not `congr`. It wouldn't make sense for `smt`, `admit`, `clear`, `have`, `idtac`, `move`, `simplify` (without explicit `delta`) or `suff` to do it. I don't know about `change`, `solve`, `replace` or `algebra`. In one example, the `rewrite` tactic unfolds operators in the `negbK` lemma in order to find a match. Automatic unfolding means less work for the user but sure does make proofs even harder to follow sometimes.

5.5 Case Analysis: The Case Tactic

5.5.1 The Case Tactic with No Arguments

The `case` tactic with no arguments decomposes a goal into cases (subgoals) based on its top assumption. Some patterns apply to hypotheses, others to variables. We say the `case` tactic *destructs* the top assumption, by analogy with inductive types: *to destruct* is the opposite of *to construct*. It's a loose analogy and it's easier to remember that the `case` tactic solves a goal by cases.

If the top assumption is a hypothesis, the `case` tactic recognizes these patterns:

- `false` – if the assumption simplifies to `false`, the goal is solved
- `conjunction` – replaces a conjunction (`/\` or `&&`) with individual hypotheses: the goal $p \wedge q \Rightarrow \phi$ becomes $p \Rightarrow q \Rightarrow \phi$
 - `p` and `q` can now be moved independently to the context, for example
 - Only one conjunction is destructed: $p \wedge q \wedge r \Rightarrow \phi$ becomes $p \Rightarrow (q \wedge r) \Rightarrow \phi$
- `equivalence` – replaces an equivalence with individual hypotheses: the goal $(p \Leftrightarrow q) \Rightarrow \phi$ becomes $(p \Rightarrow q) \Rightarrow (q \Rightarrow p) \Rightarrow \phi$
 - This can be seen as a special case of conjunction
 - This does not work with any other inverse pair of partial orders and their equivalence relation, but lemmas can be applied to achieve the same effect
- `disjunction` – replaces a disjunction (`\|` or `||`) with individual subgoals:
 - the goal $p \vee q \Rightarrow \phi$ becomes two subgoals, $p \Rightarrow \phi$ and $q \Rightarrow \phi$, both of which must be solved
 - The goal $p \vee q \Rightarrow \phi$ becomes the subgoals $p \Rightarrow \phi$ and $!p \Rightarrow q \Rightarrow \phi$ (see *Pervasive* regarding `||`)
 - Only one disjunction is destructed: $p \vee q \vee r \Rightarrow \phi$ becomes $p \Rightarrow \phi$ and $q \vee r \Rightarrow \phi$
- `less than or equal` – replaces the goal $a \leq b \Rightarrow \phi$ with two subgoals, $a < b \Rightarrow \phi$ and $a = b \Rightarrow \phi$, both of which must be solved
 - This can be seen as a special case of disjunction
 - This works for `<=` over `int` and `real` but not for any other pair of a partial order and its strict partial order. Lemmas can be applied to achieve the same effect
- `conditional` - replaces $(\text{if } b \text{ then } bt \text{ else } bf) \Rightarrow \phi$ (or $b ? bt : bf \Rightarrow \phi$) with subgoals $b \Rightarrow bt \Rightarrow \phi$ and $!b \Rightarrow bf \Rightarrow \phi$
 - This can be seen as a special case of disjunction

- existential – replaces an existential quantifier with a universal quantifier: `exists (x : t),  $\psi$  x =>  $\phi$` becomes `forall (x : t),  $\psi$  x =>  $\phi$`
 - This is not an equivalent hypothesis and may make the goal unprovable, but if not, it is sound
 - See also *The Exists Tactic*

If the top assumption is a universally quantified variable, the `case` tactic destructs it based on its type:

- `unit` – replaces a variable of type `unit` with `tt` (the only value it can take):
`forall (x : unit),  $\phi$  x` becomes `$\phi$  tt`
- `bool` – replaces a variable of type `bool` by generating two subgoals, substituting `true` in one and `false` in the other: `forall (x : bool),  $\phi$  x` becomes `$\phi$  true` and `$\phi$  false`, both of which must be solved
- `int` – replaces a goal of the form `forall (x : int), 0 <= x =>  $\phi$  x` with two subgoals, both of which must be solved:

```
 $\phi$  0
forall (i : int), 0 <= i =>  $\phi$  i =>  $\phi$  (i + 1)
```

- This applies only to the natural numbers (`0 <= x`). Without this hypothesis as the second assumption, the tactic fails.
- `tuple type` – replaces a variable of a tuple type with a variable for each of its components:
`forall (x : bool * bool),  $\phi$  x` becomes `forall (x1 x2 : bool),  $\phi$  (x1, x2)`
- `record type` – replaces a variable of a record type with a variable for each of its components: using the `coordinates type` from *Record Types*, `forall (c : coordinates),  $\phi$  c` becomes `forall (x x0 x1 : real),  $\phi$  { | x = x; y = x0; z = x1 | }`
- `inductive type` – replaces a variable of an inductive type by generating a subgoal for each constructor of the inductive type and substituting an application of that constructor for the variable: for example, `forall (x : int list),  $\phi$  x` becomes two subgoals, both of which must be solved:

```
 $\phi$  []
forall (x1 : int, x2 : int list),  $\phi$  (x1 :: x2).
```

- This is a proof by induction using the “`_case`” axiom automatically generated for the inductive type

If the goal has no top assumption or if the top assumption has none of these forms, the tactic reports “don’t know what to eliminate”. In particular, it does not work if the relevant assumption is in the context:

```
lemma L (s : intlist) : 0 <= size s.
Type variables: <none>
s : intlist
-----
0 <= size s
case.
don't known [sic] what to eliminate
```

which is easily fixed by reverting it:

```

move : s. (* revert assumption 's' *)
Type variables: <none>
-----
forall (s : intlist), 0 <= size s
case.
Type variables: <none>
Current goal (remaining: 2)
-----
0 <= size nil

```

where the other goal is

```

Type variables: <none>
-----
forall (x0 : int, x1 : intlist), 0 <= size (cons x0 x1)

```

5.5.2 The Case Tactic with One Argument

The `case` tactic with one argument does excluded-middle analysis. The argument is an expression pattern that does not have holes and must have type `bool`. If the conclusion of the current goal is ϕ , the step

```
case  $\psi$ .
```

replaces the goal with two subgoals that have the context of the current goal and the conclusions $\psi \Rightarrow \phi$ and $!\psi \Rightarrow \phi$. If there are any occurrences of ψ in ϕ , they are replaced with `true` in the first subgoal and `false` in the second.

For example, to prove the lemma from the `List` theory that says replacing one element of a list with another does not change the size of the list,

```

lemma size_put (s : 'a list) (n : int) (x : 'a) :
  size (put s n x) = size s.
Type variables: 'a
s: 'a list
n: int
x: 'a
-----
size (put s n x) = size s

```

we start by unfolding `put`:

```

delta put.
Type variables: 'a
s: 'a list
n: int
x: 'a
-----
size
  (if 0 <= n && n < size s then take n s ++ x :: drop (n + 1) s
   else s) =

```

```
size s
```

The conditional statement defines two cases: where the index `i` is valid and where it isn't. The latter leaves `s` unchanged, so clearly its size is also unchanged. The conditional is buried as an argument to `size`, but we can extract it, so to speak, and make it the top assumption like this:

```
case (0 <= n && n < size s) .
(* Subgoal 1 *)
Type variables: 'a
s: 'a list
n: int
x: 'a
-----
0 <= n && n < size s =>
size (take n s ++ x :: drop (n + 1) s) = size s
```

The `case` tactic replaced the occurrence of the expression in the first subgoal with `true` and then simplified the goal automatically, eliminating the conditional expression. We will need the new top assumption soon, so we move it to the context. Since it is a conjunction of two hypotheses, however, we first use the `case` tactic again to separate them.

```
case.
Type variables: 'a
s: 'a list
n: int
x: 'a
-----
0 <= n => n < size s =>
size (take n s ++ x :: drop (n + 1) s) = size s
move => gt0_n lts_n .
Type variables: 'a
s: 'a list
n: int
x: 'a
ge0_n: 0 <= n
lts_n: n < size s
-----
size (take n s ++ x :: drop (n + 1) s) = size s
```

Now we use the `rewrite` tactic with some other lemmas of the `List` theory to work on this conclusion.

```
rewrite size_cat.
Type variables: 'a
s: 'a list
n: int
x: 'a
ge0_n: 0 <= n
lts_n: n < size s
-----
size (take n s) + size (x :: drop (n + 1) s) = size s
```

The next lemma, `size_take`, is conditional on $0 \leq n$, so let's nip that in the bud with a proof term argument:

```
rewrite (size_take _ _ ge0_n).
Type variables: 'a
s: 'a list
n: int
x: 'a
ge0_n: 0 <= n
lts_n: n < size s
-----
(if n < size s then n else size s) + size (x :: drop (n + 1) s) =
size s
```

Hypothesis `lts_n` matches the condition in the conditional expression, so we rewrite with it. The `rewrite` tactic doesn't simplify automatically, so we do that, too.

```
rewrite lts_n.
Type variables: 'a
s: 'a list
n: int
x: 'a
ge0_n: 0 <= n
lts_n: n < size s
-----
(if true then n else size s) + size (x :: drop (n + 1) s) =
size s
simplify.
Type variables: 'a
s: 'a list
n: int
x: 'a
ge0_n: 0 <= n
lts_n: n < size s
-----
n + size (x :: drop (n + 1) s) = size s
```

The `size_drop` lemma is again conditional, so rewriting adds a subgoal. Both subgoals are solved by simple algebra that is rather tedious to do in EasyCrypt, so we will use a tactic we haven't discussed yet; see *The Smt Tactic*.

```
rewrite size_drop.
(* Subgoal 1a *)
Type variables: 'a
s: 'a list
n: int
x: 'a
ge0_n: 0 <= n
lts_n: n < size s
-----
0 <= n + 1
```

```

smt () .
(* Subgoal 1b *)
Type variables: 'a
s: 'a list
n: int
x: 'a
ge0_n: 0 <= n
lts_n: n < size s
-----
n + (1 + max 0 (size s - (n + 1))) = size s
smt () .
(* Subgoal 2 *)
Type variables: 'a
s: 'a list
n: int
x: 'a
-----
! (0 <= n && n < size s) => size s = size s

```

The `case` tactic replaced the occurrence of the expression in the second subgoal with `false` and then simplified the goal automatically, again eliminating the conditional expression. The new top assumption is irrelevant: the conclusion holds regardless. Rather than clearing the hypothesis and using the `reflexivity` tactic, we'll let the `done` tactic handle those details:

```

done .
No more goals

```

Other forms of the `case` tactic are described in *The Full Syntax of the Case Tactic*.

5.6 Conjunctive Goals: The Split Tactic

The `split` tactic reduces a goal whose conclusion is “intrinsically conjunctive” into multiple goals with the same context:

- Replaces $\varphi_1 \wedge \varphi_2$ with subgoals φ_1 and φ_2
- Replaces $\varphi_1 \&\& \varphi_2$ with subgoals φ_1 and $! \varphi_1 \Rightarrow \varphi_2$ (see *Pervasive* regarding `&&`)
- Replaces $\varphi_1 \Leftrightarrow \varphi_2$ with subgoals $\varphi_1 \Rightarrow \varphi_2$ and $\varphi_2 \Rightarrow \varphi_1$
 - This does not work with any other pair of partial orders whose overlap is an equivalence relation, but lemmas can be applied to produce a conjunctive goal, then `split`. For example, the `Fset` theory has

```

lemma eqEsubset (A B : 'a fset) :
  (A = B) <=> (A \subset B) /\ (B \subset A)

```

- Replaces equality or inequality between n-tuples with n subgoals comparing their components
- Replaces equality or inequality between records with subgoals comparing their components
- Replaces `if b then  $\varphi_1$  else  $\varphi_2$` (or `b ?  $\varphi_1$  :  $\varphi_2$`) with subgoals that simplify to `b =>  $\varphi_1$` and `!b =>  $\varphi_2$`
- Closes any goal that is convertible to `true` or provable by the `reflexivity` tactic
 - This is why `split` can solve `PairEq1` and `PairEq2` in one step: they simplify to `true` (which is the definition of *convertible*) and `split` closes them.

For the first two forms only, if the goal consists of a conjunction (`/\` or `&&`) of more than two terms, `split` can be given an argument specifying which conjunction to split on. For example, if the current goal is `P /\ Q /\ R /\ S`, any of the following are legal:

```
split.  (* on first conjunction *)
(* Subgoal 1 *)
P
(* Subgoal 2 *)
Q /\ R /\ S
split 1. (* on second conjunction *)
(* Subgoal 1 *)
P /\ Q
(* Subgoal 2 *)
R /\ S
split 2. (* on third conjunction *)
(* Subgoal 1 *)
P /\ Q /\ R
(* Subgoal 2 *)
S
```

If the argument is 3 or more, the error message is “not enough conjunctions.”

If the goal has none of these forms, the tactic fails. The difference with `case` is that `case` is looking at the *top assumption* and `split` is looking at the *whole conclusion*. The presence of a hypothesis or universally quantified variable is enough to prevent `split` from working.

5.7 Leveraging Lemmas II: The Apply and Exact Tactics

Where the `rewrite` tactic uses lemmas to tweak parts of a conclusion or hypothesis, the `apply` tactic uses them to solve goals in their entirety. The `apply` tactic can reason backward from a conclusion or forward from a hypothesis. The `exact` tactic is a variant that reasons backwards like `apply` and then runs the `done` tactic, failing if it cannot solve all the subgoals generated. In this section, we shall again use the term “lemma” to refer to both axioms and lemmas, and often to hypotheses in the context of a goal as well.

5.7.1 Reasoning Backward

The `apply` tactic by default matches the body of a lemma to the conclusion of a goal. The entire conclusion must be included in the match or the tactic fails. If the entire lemma is also included in the match, the goal is solved. If the lemma has hypotheses that are not included in the match, they become a new subgoal. This exemplifies the overall approach of backward chaining, or reasoning from consequents to antecedents.

For example, here are two lemmas from `List` that state that `nth` returns a default value if `n` is not a valid index (too small or too large, respectively).

```
lemma nth_neg [] (x0 : 'a) (s : 'a list) (n : int) :
  n < 0 => nth x0 s n = x0.
lemma nth_default (z : 'a) (s : 'a list) (n : int) :
  size s <= n => nth z s n = z.
```

We will use these to prove the same thing in a single lemma. (In case you're wondering why there are so many lemmas that say nearly the same thing, the best I can offer is that machine-checked proofs are very sensitive to minor changes, so when a situation arises where the existing lemmas don't quite work, the solution is often a new, slightly different lemma.)

```
lemma nth_out ['a] (x0 : 'a) (s : 'a list) (n : int) :
  ! (0 <= n && n < size s) => nth x0 s n = x0.
Type variables: 'a
x0: 'a
s: 'a list
n: int
-----
! (0 <= n && n < size s) => nth x0 s n = x0
```

Neither of the lemmas apply because the conclusion has a hypothesis that doesn't match. The first few steps of the proof break up the troublesome hypothesis:

```
rewrite andaE. (* in theory Logic *)
Type variables: 'a
x0: 'a
s: 'a list
n: int
-----
! (0 <= n /\ n < size s) => nth x0 s n = x0
rewrite negb_and. (* in theory Logic *)
Type variables: 'a
x0: 'a
s: 'a list
n: int
-----
! 0 <= n /\ ! n < size s => nth x0 s n = x0
case.
(* Subgoal 1 *)
Type variables: 'a
x0: 'a
s: 'a list
n: int
-----
! 0 <= n => nth x0 s n = x0
```

Now we've isolated part of the original goal into a form very similar to the `nth_out` lemma. It needs a little more finagling:

```
rewrite lezNgt. (* in theory Int *)
Type variables: 'a
x0: 'a
s: 'a list
n: int
-----
! ! n < 0 => nth x0 s n = x0
simplify.
```



```

Type variables: 'a
x0: 'a
s: 'a list
n: int
-----
n < 0 => nth x0 s n = x0
apply nth_neg. (* Subgoal 1 solved *)

```

The other subgoal also needs a little finagling before we can apply `nth_default`:

```

(* Subgoal 2 *)
Type variables: 'a
x0: 'a
s: 'a list
n: int
-----
! n < size s => nth x0 s n = x0
rewrite ltzNge. (* in theory Int *)
Type variables: 'a
x0: 'a
s: 'a list
n: int
-----
! ! size s <= n => nth x0 s n = x0
simplify.
Type variables: 'a
x0: 'a
s: 'a list
n: int
-----
size s <= n => nth x0 s n = x0
apply nth_default.
No more goals

```

All the steps leading up to each use of `apply` serve to get the current goal “just right” for one of the lemmas. They do such a good job that the goal is an exact match—even most of the variable names are the same. The match needn’t be that perfect. If the lemma used has hypotheses that don’t match anything in the conclusion of the goal, they become a subgoal with the same context as the original goal. This is similar to the `rewrite` tactic, but `rewrite` puts each hypothesis in a separate subgoal.

For example, the law of contrapositive is a lemma in the `Logic` theory:

```

lemma contra (c b : bool) : (c => b) => !b => !c.

```

It can be used to prove this lemma from the `List` theory:

```

lemma neq_from_nth ['a] (x0 : 'a) (s1 s2 : 'a list) (i : int) :
  nth x0 s1 i <> nth x0 s2 i => s1 <> s2.
Type variables: 'a
x0: 'a
s1: 'a list
s2: 'a list

```

```

i: int
-----
nth x0 s1 i <> nth x0 s2 i => s1 <> s2
apply contra.
Type variables: 'a
x0: 'a
s1: 'a list
s2: 'a list
i: int
-----
s1 = s2 => nth x0 s1 i = nth x0 s2 i

```

The `apply` tactic matches the conclusion of the goal with `!b => !c` in the lemma, which instantiates `b` with `nth x0 s1 i = nth x0 s2 i` and `c` with `s1 = s2` (recalling that `<>` is an abbreviation). This leaves `c => b` in the lemma unmatched, so it becomes a new subgoal using the instantiation.

The next two steps move the top assumption to the context and then use `subst` to rewrite the conclusion—later we'll see a more direct way to do this.

```

move => H.
Type variables: 'a
x0: 'a
s1: 'a list
s2: 'a list
i: int
H: s1 = s2
-----
nth x0 s1 i = nth x0 s2 i
subst s1.
Type variables: 'a
x0: 'a
s2: 'a list
i: int
-----
nth x0 s2 i = nth x0 s2 i
reflexivity.
No more goals

```

When a lemma is an equivalence, it can be used to rewrite a goal in either direction automatically. For example, the `Logic` theory defines `f == g` to mean that two unary functions are *extensionally* equal, and then states an axiom that this is equivalent to regular equality:

```

pred (==) ['b, 'a] (f g : 'a -> 'b) = forall (x : 'a), f x = g x.
axiom fun_ext ['a, 'b] :
  forall (f g : 'a -> 'b), f = g <=> f == g.

```

The `List` theory uses these to prove this lemma:

```

lemma in_nilE ['a] : mem [<:'a] = pred0.
Type variables: 'a
-----
mem [] = pred0

```

The `apply` tactic matches this goal with the left side of `fun_ext` and generates a subgoal corresponding to the right side:

```
apply fun_ext.
Type variables: 'a
-----
mem [] == pred0
```

This is valid backward reasoning because $f == g \Rightarrow f = g$. Now we unfold the definitions of `pred0` and `(==)` and finish:

```
delta pred0 (==).
Type variables: 'a
-----
forall (x : 'a), (x \in []) = false
done.
No more goals
```

If the axiom happened to be written the other way around,

```
axiom fun_ext ['a, 'b] :
  forall (f g : 'a -> 'b), f == g <=> f = g.
```

the `apply` tactic would work just as well by matching the goal to the right side and generating the left as the subgoal. The `rewrite` tactic would work as well in the first case, but in the second it would require the direction indicator. Conceptually, `rewrite` is treating $a <=> b$ as $a = b$ and replacing one with the other, while the `apply` tactic is treating $a <=> b$ as $a \Rightarrow b \wedge b \Rightarrow a$ and choosing the direction that matches the conclusion.

Sometimes the `apply` tactic needs the same kind of help finding an instantiation of the lemma's assumptions as the `rewrite` tactic, and indeed, the `apply` tactic's argument is again a proof term. For example, a proof of the `List` lemma

```
lemma hasPn ['a] (p : 'a -> bool) (s : 'a list) :
  !has p s <=> (forall x, mem s x => !p x).
```

begins by splitting the equivalence into separate implications and moving assumptions out of the way:

```
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
! has p s <=> forall (x : 'a), x \in s => ! p x
split.
(* Subgoal 1 *)
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
! has p s => forall (x : 'a), x \in s => ! p x
move => h x sx.
Type variables: 'a
```

```

p: 'a -> bool
s: 'a list
h: ! has p s
x: 'a
sx: x \in s
-----
! p x

```

Now we apply the `contra` lemma we used before, this time with arguments in the proof term:

```

apply (contra _ _ h) .
Type variables: 'a
p: 'a -> bool
s: 'a list
h: ! has p s
x: 'a
sx: x \in s
-----
p x => has p s

```

The tactic matches the conclusion of the lemma, `!c`, with the conclusion of the goal, `! p x`, by instantiating `c` with `p x`, but what about `b`? The `contra` lemma has four assumptions: the variables `c` and `b` (in that order!) and the hypotheses `(c => b)` and `!b`. The proof term says to prove the lemma's fourth assumption, `!b`, with hypothesis `h`, which is `! has p s`; this instantiates `b` with `has p s` (clever, no?). The only subgoal generated is the instantiation of the third assumption, `c => b`. One could instead specify the instantiation of the variable `b` with `has p s` like this:

```

apply (contra _ (has p s) _ _).

```

and get the same result, except there would be a second subgoal corresponding to `!b`, which would be solved by applying `h` (using it as a lemma):

```

Type variables: 'a
p: 'a -> bool
s: 'a list
h: ! has p s
x: 'a
sx: x \in s
-----
! has p s
apply h.

```

One cannot, however, use

```

apply (contra _ _ _ (! has p s)). (* fails *)
expecting a `proof-term`, not a `formula`

```

This fails because `(! has p s)` is an expression pattern but proving `!b` needs a proof term.

The rest of the proof is provided without explication for brevity:

```

move => px. apply hasP. exists x. done. (* Subgoal 1 solved *)
rewrite hasP negb_exists. simplify. apply forall_eq.

```

```
simplify. move => x. rewrite negb_and -implybE. done.
```

The `exact` tactic with one proof term is the same as the `apply` tactic except that it applies `done` at the end, which fails if any part of the original goal remains unsolved (i.e., if one or more subgoals remain).

5.7.2 The Apply Tactic with No Arguments

The `apply` tactic can be used without a proof term. This applies the top assumption to the rest of the conclusion and clears the top assumption. It is equivalent to moving the assumption to the context, applying it by name and clearing it.

An example of this is found in the `FSet` theory. Recall that `fset1 x` is the singleton set containing `x` and the binary infix operator `\in` is set membership. The theory proves this lemma:

```
lemma in_fset1 ['a] (z : 'a) :
  forall (x : 'a), x \in fset1 z <=> x = z.
```

It goes on to define the subset relation,

```
op (\subset) (s1 s2 : 'a fset) = forall x, x \in s1 => x \in s2.
```

and states this lemma:

```
lemma subset x (A : 'a fset) : fset1 x \subset A <=> x \in A.
Type variables: 'a
x: 'a
A: 'a fset
-----
fset1 x \subset A <=> x \in A.
```

Our proof starts by unfolding `\subset`:

```
delta (\subset).
Type variables: 'a
x: 'a
A: 'a fset
-----
(forall (x0 : 'a), x0 \in fset1 x => x0 \in A) <=> x \in A
```

We again use the `split` tactic to reduce the equivalence to two implications:

```
split.
(* Subgoal 1 *)
Type variables: 'a
x: 'a
A: 'a fset
-----
(forall (x0 : 'a), x0 \in fset1 x => x0 \in A) => x \in A
```

The top assumption is a general statement whose conclusion, `x0 \in A`, matches the goal's conclusion, `x \in A`, with the instantiation of `x0` with `x`. Rather than introduce the top assumption to the context and `apply` it, we can play it where it lies:

```
apply.
```

```

Type variables: 'a
x: 'a
A: 'a fset
-----
x \in fset1 x

```

Now we can use the `in_fset1` lemma and finish off this subgoal:

```

rewrite in_fset1.
Type variables: 'a
x: 'a
A: 'a fset
-----
x = x
reflexivity.

```

We'll abbreviate the proof of the second implication, since it adds nothing new.

```

(* Subgoal 2 *)
Type variables: 'a
x: 'a
A: 'a fset
-----
x \in A => forall (x0 : 'a), x0 \in fset1 x => x0 \in A
move => x_A y.
rewrite in_fset1.
move => y_eq_x.
rewrite y_eq_x.
assumption.
No more goals

```

Notice steps 3 and 4 in this part: we `move` the top assumption (which is `y = x`) to the context just to use it to `rewrite` the rest of the conclusion. Why can't we play it where it lies, too? It turns out we can, but not with the `rewrite` tactic. We'll see how in *The Introduction Tactical Revisited*.

The `exact` tactic also has a zero-argument form.

5.7.3 Proof Terms Again (Optional)

The `apply` tactic is where proof terms can really shine. Consider these declarations and axioms:

```

require import Int.
pred P : int.
pred Q : int.
pred R : int.
axiom P0 (x : int) : P x.
axiom Q0 (x : int) : P x => Q x.
axiom R0 (x : int) : P (x + 1) => Q x => R x.

```

Also consider the lemma

```

lemma Q1 (y : int) : Q y.

```

which has a simple proof:

```

Type variables: <none>
y : int
-----
Q y
apply Q0.
Type variables: <none>
y : int
-----
P y
apply P0.
No more goals.

```

Using a more complex proof term, we can do this in one step:

```

Type variables: <none>
y : int
-----
Q y
apply (Q0 y (P0 y)).
No more goals.

```

This proof term says to apply lemma `Q0` by instantiating its variable `x` with `y` and proving its hypothesis, `P x`, using the lemma `P0`, with its variable also instantiated with `y`. This proof term actually provides more help than the tactic needs.

```

apply (Q0 _ (P0 y)).

```

also works, but not

```

apply (Q0 y P0).
this proof-term proves
  forall (x : int), P x
but is expected to prove:
  P y

```

The error message is saying the tactic couldn't figure out how to instantiate `x`.

Now consider this lemma and its initial goal:

```

lemma R1 (z : int) : R z.
Type variables: <none>
z : int
-----
R z

```

When we apply `R0`, we get two subgoals, one for each of `R0`'s assumptions:

```

Type variables: <none>
z : int
-----
P (z + 1)

```

and

```

Type variables: <none>
z : int
-----
Q z

```

The first subgoal can be solved by applying $P0$ and the second subgoal by applying the lemma $Q1$, which we just proved. Using a proof term, we can do this in one step and not even use $Q1$ (so maybe we don't need $Q1$ at all):

```

Type variables: <none>
z : int
-----
R z
apply (R0 z (P0 (z + 1)) (Q0 z (P0 z))) .
No more goals

```

The proof term literally encodes the entire proof.

One can allow selected subgoals to be generated by placing holes for hypotheses. For example,

```
apply (R0 z _ (Q0 z _)) .
```

results in two subgoals, one from $R0$ and one from $Q0$, both solvable by $P0$ with no help:

```

(* Subgoal 1 *)
Type variables: <none>
z : int
-----
P (z + 1)
apply P0.
(* Subgoal 2 *)
Type variables: <none>
z : int
-----
P z
apply P0.
No more goals.

```

Similarly,

```
apply (R0 z (P0 (z + 1)) _) .
```

generates the second $R0$ subgoal:

```

Type variables: <none>
z : int
-----
Q z
apply (Q0 z _) .
No more goals.

```


The need to work out the proof before encoding it in a proof term, and the consequent difficulty in understanding it, is probably why proof terms like this are rare. Applying multiple lemmas in sequence is easier to work out and just as compact.

A note on proof terms. Several library theories use proof terms beginning with ‘@’. The @ does not affect the meaning of the proof term and can be ignored. It appears only in files that contain the directive

```
pragma +implicits.
```

which tells EasyCrypt to be more aggressive at inferring arguments to fill holes in a proof term. The @ tells it to revert to its normal inference for the proof term containing it.

5.7.4 The Apply Tactic with Multiple Arguments

Like `rewrite`, `apply` will accept multiple arguments to apply a sequence of proof terms in one step. The general form for this is

```
apply /p1 ... /pn.
```

where / is used to mark where each proof term begins. Unlike `rewrite`, however, it applies `p1` to the current goal, then applies `p2` only to the *last* subgoal it generates and replaces that subgoal with its list of subgoals (if any) and so on through `pn`. If all remaining subgoals are solved by, say, `pi`, the remaining terms are ignored. The tactic fails without changing the current goal (i.e., discarding however far it got before failing) if any of the applications fail.

For example, a one-step proof of lemma `R1` is

```
apply /R0 /Q0 /P0 /P0.
```

This corresponds to a reverse pre-order traversal of the complex proof term we used earlier. In contrast, when you apply proof terms one at a time, each term is applied to the *first* subgoal generated by the previous one.

```
apply R0. apply P0. apply Q0. apply P0.
```

This corresponds to a pre-order traversal of the complex proof term.

The `exact` tactic also has a multiple-argument form, which applies `done` after the last proof term to all the subgoals generated.

5.7.5 Reasoning Forward

The `apply` tactic can also be used *on* (rather than *using*) a hypothesis, but the lemma is applied in a *forward* direction. That is, it reasons from antecedents to consequences by matching the hypothesis of the lemma with a hypothesis in the context and replacing the hypothesis with the lemma’s conclusion. Think *modus ponens*: $((\psi \Rightarrow \varphi) \wedge \psi) \Rightarrow \varphi$, where $\psi \Rightarrow \varphi$ is the lemma and ψ is the hypothesis, which the `apply` tactic replaces with φ . An example uses these lemmas in `List`:

```
lemma onth_some ['a] (xs : 'a list) (n : int) (x : 'a) :
  onth xs n = Some x =>
    (0 <= n && n < size xs) /\ nth witness xs n = x.
lemma mem_nth ['a] (x0 : 'a) (s : 'a list) (n : int) :
  0 <= n && n < size s => mem s (nth x0 s n).
```

to prove this one:

```
lemma onth_some_mem ['a] (xs : 'a list) (n : int) x:
  onth xs n = Some x => x \in xs.
Type variables: 'a
xs: 'a list
n: int
x: 'a
-----
onth xs n = Some x => x \in xs
```

The `onth_some` lemma's conclusion seems relevant to the goal's conclusion but certainly doesn't match it. The goal's hypothesis matches that of the lemma, so let's move the hypothesis to the context:

```
move => H.
Type variables: 'a
xs: 'a list
n: int
x: 'a
H: onth xs n = Some x
-----
x \in xs
```

Now we apply the lemma to the hypothesis in the forward direction:

```
apply onth_some in H.
Type variables: 'a
xs: 'a list
n: int
x: 'a
H: (0 <= n && n < size xs) /\ nth witness xs n = x
-----
x \in xs
```

If we move `H` back to the conclusion, we can use `case` to split it into two hypotheses and move them back:

```
move : H.
Type variables: 'a
xs: 'a list
n: int
x: 'a
-----
(0 <= n && n < size xs) /\ nth witness xs n = x => x \in xs
case.
Type variables: 'a
xs: 'a list
n: int
x: 'a
-----
0 <= n && n < size xs => nth witness xs n = x => x \in xs
move => H1 H2.
```

```

Type variables: 'a
xs: 'a list
n: int
x: 'a
H1: 0 <= n && n < size xs
H2: nth witness xs n = x
-----
x \in xs

```

Now we can substitute for `x` in the conclusion, which clears both `x` and `H2`:

```

subst x.
Type variables: 'a
xs: 'a list
n: int
H1: 0 <= n && n < size xs
-----
nth witness xs n \in xs

```

The goal now matches the conclusion of `mem_nth` (recalling `\in` abbreviates `mem`), so we apply it. The hypothesis of the lemma becomes the new goal and is quickly dispatched by `H1`:

```

apply mem_nth.
Type variables: 'a
xs: 'a list
n: int
H1: 0 <= n && n < size xs
-----
0 <= n && n < size xs
assumption.
No more goals

```

Instead of `assumption`, we could have used `apply H1`. For that matter, the last two steps could have been

```

move : H1.
Type variables: 'a
xs: 'a list
n: int
x: 'a
-----
0 <= n && n < size xs => nth witness xs n \in xs

```

so that the goal matches the entire lemma, followed by

```

apply mem_nth.
No more goals

```

If the tactic can't figure out how to instantiate all of the variables of the lemma, such as if its hypothesis doesn't reference them all, then the proof term must make up the difference, as usual. For example, we can use this lemma from the `List` theory

```

lemma before_find (x0 : 'a) (p : 'a -> bool) (s : 'a list) (i :
int) :
  0 <= i < find p s => ! p (nth x0 s i).

```

to prove this one:

```

lemma before_index (x0 x : 'a) (s : 'a list) (i : int) :
  0 <= i < index x s => nth x0 s i <> x.
Type variables: 'a
x0: 'a
x: 'a
s: 'a list
i: int
-----
0 <= i && i < index x s => nth x0 s i <> x.

```

To apply the `before_find` lemma to this goal, we have to reconcile `find` vs. `index`. It turns out that `index` is defined in terms of `find`, so we just have to unfold it.

```

rewrite /index.
Type variables: 'a
x0: 'a
x: 'a
s: 'a list
i: int
-----
0 <= i && i < find (pred1 x) s => nth x0 s i <> x.

```

Now the lemma and the goal have the same hypothesis and similar conclusions, so let's move the hypothesis to the context and apply the lemma to it using forward reasoning. The hypothesis provides instantiations for `p`, `s` and `i` but not `x0`:

```

move => i_bounds.
Type variables: 'a
x0: 'a
x: 'a
s: 'a list
i: int
i_bounds: 0 <= i && i < find (pred1 x) s
-----
nth x0 s i <> x.
apply (before_find x0) in i_bounds.
Type variables: 'a
x0: 'a
x: 'a
s: 'a list
i: int
i_bounds: ! pred1 x (nth x0 s i)
-----
nth x0 s i <> x.

```

In case it is not obvious that `i_bounds` and the conclusion are the same, we will unfold `pred1`:

```

rewrite /pred1 in i_bounds.
Type variables: 'a
x0: 'a
x: 'a
s: 'a list
i: int
i_bounds: nth x0 s i <> x
-----
nth x0 s i <> x.
assumption.
No more goals

```

Since both the `assumption` and `apply` tactics will unfold symbols automatically, neither of the unfold steps above is needed. Automatic unfolding can certainly make a proof hard to follow, though!

When reasoning forward, the proof term can prove one or more hypotheses of the lemma, as long as whatever is left of the lemma has a hypothesis to match with `H` and a conclusion to reason forward to. As another academic exercise, assume we have these definitions:

```

op a, b, c, d : bool.
axiom one : a => b => c.

```

and we want to play around with this:

```

lemma four : a => b => d.
-----
a => b => d

```

The conclusion has two assumptions, so let's move them to the context.

```

move => H1 H2.
H1: a
H2: b
-----
d

```

The `one` lemma has `b` as its second assumption, so we can apply it to `H2` to reason forward, as long as the proof term provides a match for `a`:

```

apply (one H1) in H2.
H1: a
H2: c
-----
d
abort.

```

The match for hypothesis `a` in `one` is `H1`, which proves it. The match for hypothesis `b` is `H2`, so it is replaced with `one`'s conclusion, `c`. The lemma isn't true, so we just abort the proof.

In fact, we don't even have to provide a real proof for `a` in the proof term: it is enough to promise (using a hole) to provide one later:

```

apply (one _) in H2.

```

```

(* Subgoal 1 *)
H1: a
H2: b
-----
a
assumption.
(* Subgoal 2 *)
H1: a
H2: c
-----
d
abort.

```

Subgoal 1 is the bill for the promise, while subgoal 2 is the one we got above.

A similar situation that I find confusing, and maybe it's just me, comes up if we try to prove this lemma:

```

lemma two : (a => b) => c.
-----
(a => b) => c
move => hab.
hab: a => b
-----
c
apply one in hab.
cannot apply (in hab) the given proof-term for
a => b => c

```

This might look like `a => b` in `one` and `hab` should match and can be replaced with `c`, but they don't match, because `one` is really `a => (b => c)`. It has two separate hypotheses, `a` and `b`, and that's not the same as `(a => b) => c`, which only has one.

Forward reasoning never solves the current goal; it only affects the context. For this reason, the `exact` tactic does not accept the `in H` option. The no-argument and multiple argument forms of `apply` also cannot be used with `in H`, although there's no technical reason the latter couldn't be.

5.7.6 The Apply Tactic with an Anonymous Lemma

The `apply` tactic accepts anonymous lemmas for reasoning forward or backward. For example, the proof of `fmapW`, the induction axiom for finite maps in the `FMap` theory, begins

```

lemma fmapW ['a 'b] (p: ('a, 'b) fmap -> bool) :
  p empty =>
    (forall (m : ('a, 'b) fmap) (k : 'a) (v : 'b),
      k \notin fdom m => p m => p m.[k <- v]) =>
      forall (m : ('a, 'b) fmap), p m.
Type variables: 'a, 'b
p: ('a, 'b) fmap -> bool
-----
p empty =>
  (forall (m : ('a, 'b) fmap) (k : 'a) (v : 'b),
    k \notin fdom m => p m => p m.[k <- v]) =>

```

```

forall (m : ('a, 'b) fmap), p m.
move => h0 hS m0.
Type variables: 'a, 'b
p: ('a, 'b) fmap -> bool
h0: p empty
hS: (forall (m : ('a, 'b) fmap) (k : 'a) (v : 'b),
      k \notin fdom m => p m => p m.[k <- v])
m0: ('a, 'b) fmap
-----
p m0.

```

It turns out we can use induction on `fsets` to prove induction on `fmaps`. Specifically, we can do induction on the domain of the map using an anonymous lemma. This lemma takes some effort to prove, but makes the goal trivial. Our focus, though, is on the mechanics of the anonymous lemma.

```

apply ((: forall s, forall m, fdom m = s => p m) (fdom m0)).
(* Subgoal 1 *)
Type variables: 'a, 'b
p: ('a, 'b) fmap -> bool
h0: p empty
hS: (forall (m : ('a, 'b) fmap) (k : 'a) (v : 'b),
      k \notin fdom m => p m => p m.[k <- v])
m0: ('a, 'b) fmap
-----
forall (s : 'a fset) (m : ('a, 'b) fmap), fdom m = s => p m)

```

The first subgoal is to prove the lemma, but we will just `admit` it. The second subgoal is to prove that the lemma, instantiated with the argument `(fdom m0)`, proves the original goal. Their conclusions match, so what we see is the hypothesis of the anonymous lemma:

```

admit.
(* Subgoal 2 *)
Type variables: 'a, 'b
p: ('a, 'b) fmap -> bool
h0: p empty
hS: (forall (m : ('a, 'b) fmap) (k : 'a) (v : 'b),
      k \notin fdom m => p m => p m.[k <- v])
m0: ('a, 'b) fmap
-----
fdom m0 = fdom m0
done.
No more goals

```

An anonymous lemma used this way is called a *cut* because it uses modus ponens backward to “cut” $H \Rightarrow \phi$ into $H \Rightarrow \psi$ and $H \Rightarrow \psi \Rightarrow \phi$. See also *The Have Tactic* and *The Suff Tactic*.

5.7.7 Proof by Induction: The `Elim` Tactic

The `elim` tactic with no argument is identical to `case` except for its handling of inductive types, where it uses the “`_ind`” axiom. For example, we can change `case` to `elim` in an earlier example:

```

Type variables: <none>

```

```

-----
forall (s : intlist), 0 <= size s
elim.
Type variables: <none>
Current goal (remaining: 2)
-----
0 <= size nil

```

where the other goal is now

```

Type variables: <none>
-----
forall (x0 : int, x1 : intlist),
  0 <= size x1 => 0 <= size (cons x0 x1)

```

The way `elim` (and `case`) destructs a variable of type `unit`, `bool`, `int` or an inductive type `t` into cases is essentially proof by induction, albeit somewhat degenerately for `unit`. The induction lemmas are `unitW`, `boolW`, `intind` and `t_ind` (or `t_case`), respectively. These lemmas have a case (a hypothesis) that the values returned by each constructor of the variable's type have a certain property and then conclude that the property, whatever it is, applies to all values of the type. Here we use "constructor" in the loose sense of a set of operators capable of generating all the values of the type, or even more generally as a set of predicates that are collectively exhaustive but not necessarily mutually exclusive. In the case of `intind`, it isn't even all values: it's just all non-negative values.

The `elim` tactic goes one step further by destructing a variable of *any* type using a specified induction lemma over any suitable set of operators that return values of that type, like this:

```
elim /L.
```

For example, the "official" constructors of the inductive type `'a list` are `[]` and `(::)` (a.k.a. `nil` and `cons`), but all lists can also be constructed using `[]` and `rcons`, and the corresponding induction lemma is

```

lemma last_ind (p : 'a list -> bool) :
  p [] =>
    (forall s x, p s => p (rcons s x)) =>
    forall s, p s.

```

We can prove the `size_ge0` lemma for lists using `last_ind`:

```

lemma size_ge0 ['a]: forall (s : 'a list), 0 <= size s.
Type variables: 'a
-----
forall (s: 'a list), 0 <= size s
elim /last_ind.
(* Subgoal 1 *)
Type variables: 'a
-----
0 <= size []
done.
(* Subgoal 2 *)
Type variables: 'a

```



```

-----
forall (s : 'a list) (x : 'a),
  0 <= size s => 0 <= size (rcons s x)
move => s x IHs.
Type variables: 'a
s: 'a list
x: 'a
IHs: 0 <= size s
-----
0 <= size (rcons s x)
rewrite size_rcons. (* See List theory *)
Type variables: 'a
s: 'a list
x: 'a
IHs: 0 <= size s
-----
0 <= size s + 1
apply addz_ge0. (* See Int theory *)
(* Subgoal 1 *)
Type variables: 'a
s: 'a list
x: 'a
IHs: 0 <= size s
-----
0 <= size s
apply IHs.
(* Subgoal 2 *)
Type variables: 'a
s: 'a list
x: 'a
IHs: 0 <= size s
-----
0 <= 1
done.
No more goals

```

The step `elim /L` is very similar to `apply L`, which illuminates what's going on a bit.

```

lemma size_ge0 ['a]: forall (s : 'a list), 0 <= size s.
Type variables: 'a
-----
forall (s: 'a list), 0 <= size s
apply last_ind.
(* Subgoal 1 *)
Type variables: 'a
-----
(fun (s : 'a list) => 0 <= size s) []
beta.
Type variables: 'a
-----
0 <= size []

```

```

admit.
(* Subgoal 2 *)
Type variables: 'a
-----
forall (s : 'a list) (x : 'a),
  (fun (s0 : 'a list) => 0 <= size s0) s =>
  (fun (s0 : 'a list) => 0 <= size s0) (rcons s x)
beta.
Type variables: 'a
-----
forall (s : 'a list) (x : 'a),
  0 <= size s => 0 <= size (rcons s x)

```

The generated subgoals make clear that `p` in the `last_ind` lemma, which is a function of type `'a list -> bool`, has been instantiated with `(fun (s0 : 'a list) => 0 <= size s0)`. This function is then applied in turn to `[], s` and `(rcons s x)` to generate the subgoals. As shown, the `beta` (or `simplify`) tactic reduces the function applications to the exact subgoals generated by `elim /last_ind`. This may be all that `elim /L` does: apply `L` and simplify each subgoal.

There is also an induction lemma for predicates over two lists of the same length:

```

lemma seq2_ind ['a 'b] (P : 'a list -> 'b list -> bool) :
  P [] [] =>
  (forall x1 x2 s1 s2, P s1 s2 => P (x1 :: s1) (x2 :: s2)) =>
  forall s1 s2, size s1 = size s2 => P s1 s2.

```

The `Int` theory contains no fewer than eight induction lemmas to choose from:

```

lemma intind (p : int -> bool) :
  (p 0) =>
  (forall i, 0 <= i => p i => p (i + 1)) =>
  (forall i, 0 <= i => p i).
lemma sintind (p : int -> bool) :
  (forall i, 0 <= i => (forall j, 0 <= j < i => p j) => p i) =>
  (forall i, 0 <= i => p i).
lemma natind (p : int -> bool) :
  (forall n, n <= 0 => p n) =>
  (forall n, 0 <= n => p n => p (n+1)) =>
  forall n, p n.
lemma natcase (p : int -> bool) :
  (forall n, n <= 0 => p n) =>
  (forall n, 0 <= n => p (n+1)) =>
  forall n, p n.
lemma ge0ind (p : int -> bool) :
  (forall n, n < 0 => p n) =>
  p 0 =>
  (forall n, 0 <= n => p n => p (n+1)) =>
  forall n, p n.
lemma ge0case (p : int -> bool) :
  (forall n, n < 0 => p n) =>
  p 0 =>

```

```

      (forall n, 0 <= n => p (n+1)) =>
      forall n, p n.
lemma intwlog (p : int -> bool) :
  (forall i, p (-i) => p i) =>
  p 0 =>
  (forall i, 0 <= i => p i => p (i + 1)) =>
  (forall i, p i).
lemma intswlog (p : int -> bool) :
  ((forall i, 0 <= i => p i) => (forall i, i < 0 => p i)) =>
  p 0 =>
  (forall i, 0 <= i => p i => p (i + 1)) =>
  (forall i, p i).

```

The constructors used by `intind` are `0` and `(+) 1`, the classic constructors for the natural numbers, and it only applies to natural (i.e., non-negative) ints. The `sintind` lemma is more subtle but uses the same constructors (`0 <= j < 0` is false, so it implies `p 0`; `0 <= j < n + 1` ends with `p n` and implies `p (n + 1)`). The `natind` and `natcase` lemmas divide the ints into `i <= 0` and `0 <= i`, which cleverly overlap at `0`, using `(+) 1` only for the latter. The difference is that `natcase` has a simpler induction hypothesis. The `ge0ind` and `ge0case` lemmas are similar but partition the ints into `i < 0`, `0` and `0 <= i`. The `intwlog` lemma uses the constructors `0`, `(+) 1` and `[-]` (opposite, or negation) and the `intswlog` lemma is in a class of its own.

As one last example, the `FSet` theory has an induction lemma for the type `fset`, which, like `int`, is not an inductive type:

```

lemma fset_ind ['a] (p : 'a fset -> bool) :
  p fset0 =>
  (forall (x : 'a, s : 'a fset),
    x \notin s => p s => p (s `|` (fset1 x))) =>
  (forall s, p s).

```

Recall that `fset0` is the empty set, `fset1` is a singleton set, `\notin` is set non-membership and ``|`` is set union. This lemma is often more convenient to use than working with the operators `elem` and `oflist`, in terms of which most `fset` operators are defined but which also expose how `fset` is represented (implemented, you might say) using `list`. The `fset_ind` lemma is used, for example, to prove this lemma:

```

lemma fcard_image_leq ['b 'a] (f : 'a -> 'b) :
  forall (A : 'a fset), card (image f A) <= card A.
Type variables: 'b, 'a
f: 'a -> 'b
-----
forall (A: 'a fset), card (image f A) <= card A.

```

The `card` operator returns the size (cardinality) of the `fset` and `image` forms a new `fset` by applying a function `f` to each element of `A`. Since `f` may map several elements to the same value, the resulting `fset` may be smaller but cannot be larger. The `fset_ind` lemma is used as follows (proofs of the subgoals are omitted for brevity):

```

elim /fset_ind.
(* Subgoal 1 *)

```

```

Type variables: 'b, 'a
f: 'a -> 'b
-----
card (image f fset0) <= card fset0.
admit.
(* Subgoal 2 *),
forall (x : 'a) (s : 'a fset),
  x \notin s =>
    card (image f s) <= card s =>
      card (image f (s `|` fset1 x)) <= card (s `|` fset1 x))

```

5.8 The Smt Tactic

SMT stands for *satisfiability modulo theories*, which is a generalization of Boolean satisfiability (i.e., propositional logic) to handle formulas over particular types, such as integers, lists and maps. SMT solvers typically have specialized algorithms for proving satisfiability over the types they support and are fully automated.

The `smt` tactic packages up the current goal and selected definitions, axioms and lemmas and sends it to an external SMT theorem prover. EasyCrypt uses Why3, which in turn can use a variety of SMT provers, including Alt-Ergo, CVC4 and Z3, each with different strengths and weaknesses. See *Running EasyCrypt* for some details on configuring EasyCrypt and Why3.

For example, the `smt` tactic proves the `hasPn` lemma from *Reasoning Backward* in one step.

```

Type variables: 'a
p: 'a -> bool
s: 'a list
-----
! has p s <=> forall (x : 'a), x \in s => ! p x
smt.
No more goals

```

SMT provers incorporate a lot of knowledge about the data types they support, but EasyCrypt can augment that knowledge with its own axioms and lemmas. It is unclear what the tactic selects by default. Operators can be tagged as `smt_opaque` to prevent lemmas involving them from being selected by default but they can still be selected by the `wantedlemmas` option.

The `smt` tactic has two general forms and the first one is

```
smt <smt-options>.
```

This form takes a list of options specifying various aspects of its operation. The available options are

- `[prover-selector] or prover=[prover-selector]`: a list of provers to run, as a list of strings in square brackets and separated by spaces
 - Recall `easycrypt config` lists known provers, among other things
 - Prover names can omit the version or specify all or part of it: "Alt-Ergo", "Alt-Ergo2" and "Alt-Ergo2.4.2" are all accepted, as of this writing
 - To modify the global selection (see below), a string can be prefixed with + to add the prover to the list or - to remove it
- `quorum=n`: the minimum number of provers that must return success for the tactic to succeed

- `timeout=n`: the maximum time (in integer seconds) each prover has to respond
- `maxprovers=n`: the maximum number of provers to run in parallel
- `verbose`: set the verbosity level; also `verbose=n`, but `n` seems to have no effect
 - The tactic reports how long it ran
- `debug`: print debug output
 - The tactic reports each solver that succeeds, ignoring those that fail
- `selected`: list the EasyCrypt axioms and lemmas that were sent to the SMT prover
- `dump in="file"`: writes a copy of the data sent to Why3 to a file named `0000-file` in the current folder. The prefix is incremented each time to insure a unique file name. Including a relative or absolute file path such as `"../out.txt"` results in an error message like `"anomaly: Sys_error(\"0001-../out.txt: no such file or directory\")."`
- `wantedlemmas=(dbhint)`: select only the specified axioms and lemmas
 - `dbhint` is a list of axioms and lemmas separated by spaces
 - An axiom or lemma can be excluded by prefixing it with `-` (or use `unwantedlemmas`)
 - All axioms and lemmas of theory `T` can be included (or excluded) using `@T` (or `-@T`)
- `unwantedlemmas=(dbhint)`: exclude the specified axioms, lemmas and theories
- `all`: select all available lemmas except `unwantedlemmas` and those with operators marked `smt_opaque`
 - This option causes the `wantedlemmas` option to be ignored
- `n` or `maxlemmas=n`: the maximum number of lemmas that may be selected
 - This option may be broken in the current version
- `iterate`: incrementally augment the number of selected lemmas
- `lazy`: how this affects the `smt` tactic and/or SMT provers is unclear
- `full`: how this affects the `smt` tactic and/or SMT provers is unclear

Many of these options are used only when developing a proof, to troubleshoot why the tactic is failing or limit how long each attempt takes. For example, if `smt all.` doesn't work, try a different tactic. Once you get it working (or give up), the final proof won't need options like `debug`, `selected` and so on.

These options can also be set globally and overridden as needed in individual uses of the `smt` tactic. This is done with a `prover` step. For example, near the top of a theory file you might state

```
prover prover=["Alt-Ergo", "Z3"] quorum=2 timeout=5.
```

To set just the `timeout` globally, you can use

```
timeout 5.
timeout * 5.
```

The first limits each solver to no more than 5 seconds of CPU time, while the second limits it to no more than 5% of a CPU's capacity.

Finally, many options can be set from the command line. Run `easycrypt -help` and consult the `[provers]` group of options.

The second general form of the `smt` tactic is

```
smt (dbhint).
```

which is synonymous with

```
smt wantedlemmas=(dbhint) .
```

The most useful form of this is perhaps `smt ()`, which minimizes the amount of information sent to the theorem prover and therefore the time it takes to return a result, though it also maximizes the likelihood of failure.

Using the `smt` tactic with the default lemma selection is generally frowned upon, because the lemmas sent to the SMT prover may slow it down significantly. In the example above, we can see if the tactic succeeds without help:

```
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
! has p s <=> forall (x : 'a), x \in s => ! p x
smt () .
cannot prove goal (strict)
```

Turns out it doesn't. The `smt` tactic operates in an all or nothing fashion: it either solves the current goal completely or it fails, which leaves it unchanged. It is also another black box, as it provides no feedback on how it solved the goal or what went wrong when it fails. Since in this case we know it works with the default selection of lemmas but not with none, we can try to narrow down what it needs and send only that. If we add the `selected` option,

```
smt selected.
+ selected lemmas: mema_iota mem_iota mem_range_subr
...
```

we get a list of about 100 lemmas, all from the `List` theory. SMT provers are pretty good with lists, so we probably don't need to send nearly that much. Since we already have a proof of this lemma that we did ourselves, we see that the only `List` lemma it uses is `hasPn`. The rest were from the `Logic` theory, and apparently our SMT prover already knows that stuff. Sure enough,

```
smt (hasPn) .
No more goals
```

works, and (on my machine) noticeably faster.

A note on Proof General. When the tactic fails, the error message overwrites any output generated by the `debug`, `selected` or `verbose` options in the `*response*` buffer. This output can still be seen in the `*easycrypt*` buffer.

5.9 Combining Steps with Tacticals

We saw earlier how the `rewrite` and `apply` tactics let you combine multiple lemma applications into one step. **Tacticals** provide a more general mechanism for combining multiple tactics into one step. Introduction and generalization are also tacticals, and we will have more to say about them shortly.

5.9.1 The Try Tactical

The `try` tactical applies a tactic tentatively or provisionally. It has the general form

`try  $\tau$ .`

which means to apply τ but don't produce an error if it fails. It is typically used in situations where it applies to a list of subgoals, to let τ solve those it can without failing on those it can't. One such situation is the chain tactical and its variations, which are discussed below.

The use of `try smt` is strongly discouraged because `smt` can be quite time consuming and should only be used where it definitely will work, not like a shotgun.

5.9.2 The Do Tactical

The `do` tactical brings the repetition markers of the `rewrite` tactic to all tactics. Its basic form is

`do !  $\tau$`

Conceptually, the `do` tactical generates a tree with the current goal as the root; every goal to which τ applies has the subgoals it generates as its children; if τ solves a subgoal, it is deleted. At the end, τ does not apply to any of the leaf nodes, which constitute the list of subgoals going forward.

Figure 7 illustrates the tree that `do !  $\tau$` generates from goal G_0 if it proceeds as follows:

- τ applies to G_0 and generates subgoals G_1 and G_2
- τ applies to G_1 and solves it

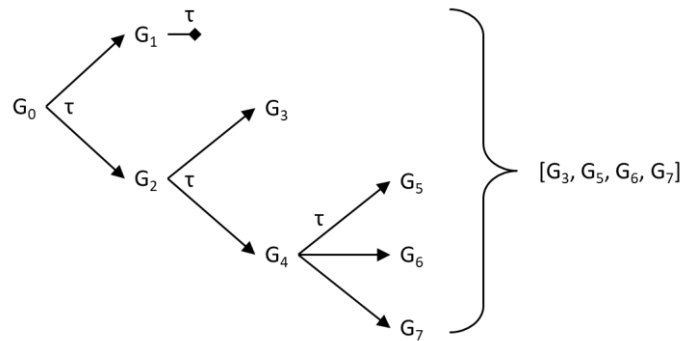


Figure 7 The Do Tactical

- τ applies to G_2 and generates subgoals G_3 and G_4
- τ does not apply to G_3
- τ applies to G_4 and generates subgoals G_5 , G_6 and G_7 but does not apply to any of them

The final result is the list of leaf subgoals $[G_3, G_5, G_6, G_7]$.

The `do` tactical has the following additional forms, which are variations of the basic process:

- | | |
|--------------------------------------|--|
| <code>do ? τ</code> | same as <code>do ! τ</code> , but do not fail if τ does not apply to the current goal |
| <code>do n! τ</code> | same as <code>do ! τ</code> , but stop at depth n even if τ still applies and fail as soon as τ fails to apply to any subgoal at a depth less than n (counting the root as depth 0). |
| <code>do n? τ</code> | same as <code>do ? τ</code> , but stop at depth n even if τ still applies |

For this example, the step `do ?  $\tau$` would have the same final result; the only difference is that if τ did not apply to G_0 , `do !  $\tau$` would fail and `do ?  $\tau$` would not. The steps `do 2!  $\tau$` and `do 2?  $\tau$` would

both succeed but not generate children for G_4 : the final result would be $[G_3, G_4]$. The step `do 3!  $\tau$` would fail because τ does not apply to G_3 , and this is true for any count higher than 3. The step `do 3?  $\tau$` would succeed and produce the result shown, and this is true for any count higher than 3.

The step `try  $\tau$` is equivalent to `do 1?  $\tau$` . Conversely, `do ?  $\tau$` is equivalent to `try do !  $\tau$` .

5.9.3 The Alternative Tactical

The alternative tactical has the form

```
 $\tau_1$  ||  $\tau_2$  || ... ||  $\tau_n$ 
```

It runs τ_1 and if it fails, it runs τ_2 and so on until a tactic succeeds or the last one fails, in which case the tactical fails.

5.9.4 The Chain Tactical

The next several tacticals link a series of tactics into an assembly line or pipeline, applying each one to the subgoals generated by the previous one, as a single proof step. The simplest has the form

```
 $\tau_1$ ;  $\tau_2$ ; ...;  $\tau_n$ 
```

which means to apply τ_1 to the current goal and then apply τ_2 to *each* of the subgoals generated by τ_1 and so on until τ_n is applied to each subgoal generated by τ_{n-1} . The chain fails if τ_1 fails on the current goal or if any τ_{i+1} fails on any of the subgoals generated by τ_i .

A basic use of the tactical is when a series of steps each produce one subgoal. For example, the proof of `count_eq0` earlier had 7 steps. Many were rewrites and we've already seen how sequences of rewrites can be combined. Using a chain, we can combine the other steps, too:

```
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
! has p s <=> count p s = 0.
rewrite has_count ltzNge; logic;
rewrite lez_eqVlt ltzNge count_ge0; done.
No more goals
```

By convention, chains are written on as few lines as possible, filling each line. The proof engine treats the entire sequence as one step: the intermediate subgoals are not shown. Other than that, using the chain tactical hasn't gained us much yet.

A more powerful use is when a step produces multiple subgoals that all use the same next step(s). For example, instead of doing this

```
lemma head_ohed ['a] (z0 : 'a) (s : 'a list) :
  head z0 s = odflt z0 (ohed s).
Type variables: 'a
z0: 'a
s: ' list
-----
head z0 s = odflt z0 (ohed s)
case: s.
```



```

(* Subgoal 1 *)
Type variables: 'a
z0: 'a
-----
head z0 [] = odflt z0 (ohead [])
done.
(* Subgoal 2 *)
Type variables: 'a
z0: 'a
-----
forall (x : 'a) (l : ' list),
  head z0 (x :: l) = odflt z0 (ohead (x :: l))
done.
No more goals

```

we can do this:

```

Type variables: 'a
z0: 'a
s: ' list
-----
head z0 s = odflt z0 (ohead s)
case: s; done.
No more goals

```

This saves us typing `done` twice.

A chain can be visualized as a tree, as we did with the `do` tactical. Let's call the current goal G_0 and apply the step $\tau_1; \tau_2; \tau_3.$ to it. If τ_1 generates two subgoals, G_1 and G_2 , then the root has two branches and τ_2 is applied to both. If τ_2 solves G_1 , that branch ends and τ_3 won't have to deal with it. If τ_2 applied to G_2 generates two subgoals, G_3 and G_4 , then G_2 has two branches and τ_3 will be applied to each in turn. If τ_3 applied to G_3 generates one subgoal, G_5 , and applied to G_4 it generates three, G_6 , G_7 and G_8 , then the final result is that G_0 is replaced by the leaf goals of the tree, subgoals G_5 through G_8 . See Figure 8. The stack of subgoals is shown with the head on the right.

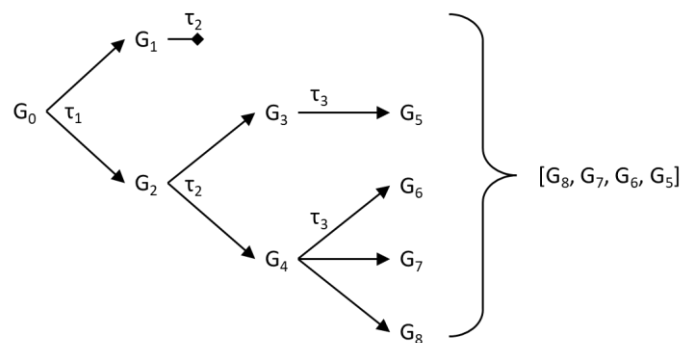


Figure 8 Execution of $\tau_1; \tau_2; \tau_3.$

This is different from applying the tactics in separate steps, as $\tau_1 . \tau_2 . \tau_3$. This sequence applies τ_1 to G_0 to generate G_1 and G_2 , then applies τ_2 only to G_1 , which solves it. Then it applies τ_3 to G_2 , which was not done in the chain, so we don't know if it will work or not. See Figure 9.

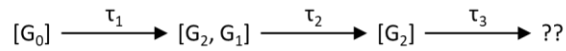


Figure 9 Execution of $\tau_1 . \tau_2 . \tau_3$.

The `rewrite` tactic follows the same process when it has multiple arguments:

```
rewrite  $\rho_1 \dots \rho_n$  (in  $H$ ).
```

means exactly the same thing as

```
rewrite  $\rho_1$  (in  $H$ ); ...; rewrite  $\rho_n$  (in  $H$ ).
```

5.9.5 Variations on a Theme

A variant of the chain tactical is

```
 $\tau_1$ ; first  $\tau_2$ 
```

which means to apply τ_1 to the current goal and then apply τ_2 *only* to the first subgoal generated by τ_1 , leaving the rest untouched. The step fails if τ_1 generates no subgoals or if τ_2 fails on the first one. For example,

```
lemma drop_size ['a] (s : 'a list) : drop (size s) s = [].
type variables: 'a
s: 'a list
-----
drop (size s) s = []
rewrite drop_oversize; first rewrite lezz.
Type variables: 'a
s: 'a list
-----
[] = []
```

More variations on this theme include the following, where n is a positive integer:

```
 $\tau_1$ ; first n  $\tau_2$  (*apply  $\tau_2$  to the first n subgoals generated by  $\tau_1$ *)
 $\tau_1$ ; last  $\tau_2$       (*apply  $\tau_2$  to the last subgoal generated by  $\tau_1$ *)
 $\tau_1$ ; last n  $\tau_2$    (*apply  $\tau_2$  to the last n subgoals generated by  $\tau_1$ *)
```

One can state how many subgoals are expected with

```
 $\tau_1$ ; expect n      (*fail if  $\tau_1$  does not generate exactly n subgoals*)
```

A more flexible variant applies τ_2 to specific subgoals generated by τ_1 by preceding it with a focus index f (see *Rewriting with Multiple Proof Terms* for all the forms f can take):

```
 $\tau_1$ ; f:  $\tau_2$ 
```

Thus, τ_1 ; 1: τ_2 is synonymous with τ_1 ; first τ_2 and τ_1 ; -1: τ_2 with τ_1 ; last τ_2 .

An even more flexible variant applies different tactics to different subgoals, working them in parallel, as it were. It has the form

```
 $\tau; [\tau_1 \mid \dots \mid \tau_n]$ 
```

and it applies τ to the current goal, which must generate exactly n subgoals, then applies each τ_i only to subgoal i , collecting all of their results in a new list of subgoals. If subgoal i doesn't need to have a tactic applied, τ_i can be omitted, which is the same as τ_i being `idtac`.

If some sets of subgoals need the same tactic applied next, there is

```
 $\tau; [f_1: \tau_1 \mid \dots \mid f_k: \tau_k]$ 
```

where each τ_i has a focus index, f_i and $k \leq n$. Each focus index implicitly excludes any previous ones, so overlapping indices only apply the first tactic. For example,

```
 $\tau; [1..2: \tau_1 \mid 1..3: \tau_2]$ 
```

applies τ_2 only to the third subgoal.

Any subgoals not included in any focus index are carried through unchanged, as if by `idtac`. If you prefer, you can apply a tactic to all such subgoals with

```
 $\tau; [f_1: \tau_1 \mid \dots \mid f_{k-1}: \tau_{k-1} \text{ else } \tau_k]$ 
```

Finally, subgoals can be reordered like this:

```
 $\tau; \text{first last}$       (* move the first subgoal to the end *)  
 $\tau; \text{first } n \text{ last}$   (* move the first  $n$  subgoals to the end *)  
 $\tau; \text{last first}$       (* move the last subgoal to the front *)  
 $\tau; \text{last } n \text{ first}$   (* move the last  $n$  subgoals to the front *)
```

5.9.6 The By Tactical

The `by` tactical has the general form

```
by  $\tau$ .
```

which means to apply tactic τ and then try to close all the generated subgoals using `done`, which applies `trivial` and fails (i.e., leaves the current goal unchanged) if any subgoal cannot be closed. That is, it is equivalent to `$\tau; \text{done}$` .

For example, a condensed proof of `head_ohhead` using the `by` tactical is

```
Type variables: 'a  
z0: 'a  
s: ' list  
-----  
head z0 s = odflt z0 (ohhead s)  
by case: s.  
No more goals.
```

The shorthand for a one-step proof has the same meaning:

```
lemma drop_size (s : 'a list) : drop (size s) s = []  
by rewrite drop_overside 1: lezz.
```

```
+ added lemma: `drop-size`
```

Each time we've proved this lemma, the `rewrite` tactic left the subgoal `[] = []` for the `reflexivity` tactic to finish. Here this happens automatically thanks to the `done` tactic added by the `by` tactical.

The shorthand `by []` is synonymous with `by idtac`, `by move`, or simply `done`. This, too, can be used in a single-step proof, but with a slight shift in meaning:

```
lemma ... by [].
```

means

```
lemma ... by (trivial; smt()).
```

5.9.7 Tactical Precedence

Tacticals can be combined into arbitrarily complex, nested structures, which, for the sake of sanity, you probably don't want to do. This section explains the rules, however, in case you want to take a few steps down that road. We will introduce four grammatical terms: *core tactic*, *tactic*, *subtactic* and *tactic chain*. We will italicize these terms to indicate when we intend the grammatical meaning.

The simplest *core tactic* consists of the name of a proof tactic and whatever arguments it takes. We will denote these as τ_i . This can be combined with the `do` or `try` tactical (or both, but the `by` tactical does things a little differently, as we'll see shortly). A *core tactic* can then be followed by a list of introduction and generalization tacticals. Examples of *core tactics* include

```
 $\tau_1 \Rightarrow l_{1a} \ l_{1b}$ 
 $\tau_2$ 
do 3? do 2!  $\tau_3 \Rightarrow l_{3a}$ 
```

The last of these will attempt to do τ_3 twice up to three times, meaning it will do τ_3 either 0, 2, 4 or 6 times, and then it will do l_{3a} once at the end, regardless. For example,

```
lemma phoo (x y z : int) : x < y => z + x < z + y.
x: int
y: int
z: int
-----
x < y => z + x < z + y
do 3? do 2? rewrite addz0 => H.
x: int
y: int
z: int
H: x < y
-----
z + x < z + y
```

does not rewrite anything because `x + 0` does not occur in the goal, but does move the hypothesis to the context. Contrast this with

```
do 3? do 2? rewrite -addz0 => H.
x: int
y: int
```

```

z: int
H: x + 0 + 0 + 0 + 0 + 0 + 0 < y
-----
z + (x + 0 + 0 + 0 + 0 + 0 + 0) < z + y

```

which rewrites x as $x + 0$ six times and then moves the hypothesis to the context. We know the hypothesis was moved last because if it was moved earlier its x would not have been rewritten all 6 times.

A *tactic* is a list of *core tactics* separated by `||`. For example,

```

τ1 => l1a l1b || τ2 || do 3? do 2! τ3 => l3a.

```

is one *tactic*. Each *core tactic* will be applied from left to right, with its introduction and generalization tacticals, until one succeeds. If they all fail, then the *tactic* fails.

The simplest *tactic chain* is a single *tactic*. A *tactic chain* can also be a *tactic* (called the head) followed by a semi-colon and a list of *subtactics* separated by semi-colons. This is the basic chain tactical where the head is applied to the current goal and each *subtactic* is applied to the list of subgoals generated by the head or *subtactic* to its left. A *subtactic* can be a *tactic*, or it can be any of the special forms we called variations on a theme above. For example

```

τ1 => l1a l1b || τ2 || do 2! τ3 => l3a ; τ4 ; first last ; (τ5 ; τ6).

```

is a *tactic chain* of length 4. The last *tactic* in this chain illustrates another feature: a *core tactic* can be a *tactic chain* enclosed in parentheses. Further, the `first` (N) τ and `last` (N) τ tacticals take a *tactic chain* as their argument, and the parallel tacticals (the ones with brackets) take a list of *tactic chains* separated by `|`, one for each subgoal. Finally, the `by` tactical takes a *tactic chain* as its argument, so if we add `by` in front of the *tactic chain* above, it becomes a single *core tactic* that executes the chain as a unit and then applies `done` to each subgoal and fails if it can't close every one. We can achieve the same effect without the `by` tactical by adding parentheses around the *tactic chain* and chaining it with `done`:

```

(τ1 => l1a l1b || τ2 || do 2! τ3 => l3a ; τ4 ; first last ; (τ5 ;
τ6)) ; done.

```

This is a *tactic chain* with two *tactics*, the first of which is a nested *tactic chain*.

5.9.8 Bullets

Steps of a proof can optionally be prefixed by one of the symbols `+`, `-` or `*`. The general form is

```

+ τ.

```

Bullets are used informally, along with line breaks and indentation, to indicate the structure of a proof, such as to indicate a sequence of steps that prove a subgoal. Unlike other proof assistants (e.g., Rocq), they have no formal meaning and are ignored, like comments. Using more than one (e.g., `++` or `***`) produces a parse error.

5.10 The Introduction Tactical Revisited

When it comes to combining steps in proofs, the introduction tactical is the Big Dog. In *The Introduction Tactical*, we saw how to use it to move the top assumption(s) from the conclusion of a goal to the context. That was just the beginning. The tactical can also apply selected tactics to the top assumption

or apply the top assumption itself to the rest of the conclusion in various ways, all without moving it. Moreover, it can do a series of such manipulations in a single proof step. It is the “everything to do with the top assumption” tactical and more. Unfortunately, perhaps, it uses an incredibly terse syntax that can make following a proof very difficult.

Pro tip. Everything the tactical does other than moving assumptions to the context can be done in other ways, at the cost of a certain amount of extra or repeated steps, so it may be helpful when creating or studying a proof to use the individual steps first. Once the proof works or you understand it, you can condense it.

The introduction tactical has the general form

$$\tau \Rightarrow l_1 \dots l_n$$

where τ is a tactic and the l_i are called **introduction patterns**. The tactical first applies τ to the current goal and then applies *each* pattern, from left to right, to *each* of the resulting subgoals, like the chain tactical but with introduction patterns as the links (see *The Chain Tactical* for a discussion of proof trees). Each pattern may, in general, solve a subgoal (i.e., delete it) or replace it with one or more subgoals (such as by removing its top assumption). After each pattern has been applied to all subgoals, the next pattern is applied to the revised list of subgoals.

So far, we have used `idtac` or `move` as τ , which do nothing, but any tactic can be used. For example, the hypothesis introductions in the proof of `PairEq2_conj` can be tacked onto the preceding `case` step:

```
case => x_eq y_eq.
```

We’ve already met the patterns that copy, delete and move the top assumption to the context. Another set of patterns applies a tactic to each goal:

<code>//</code>	Apply the <code>trivial</code> tactic to each goal
<code>/=</code>	Apply the <code>simplify</code> tactic to each goal
<code>/~=</code>	Apply a variant of the <code>simplify</code> tactic to each goal
<code>/#</code>	Apply the <code>smt()</code> tactic to each goal; if it fails to solve the goal, the proof step fails
<code>//=</code>	Apply the <code>simplify</code> and <code>trivial</code> tactics to each goal (shorthand for <code>/= //</code>)
<code>//~=</code>	Apply the variant <code>simplify</code> and <code>trivial</code> tactics to each goal (shorthand for <code>/~= //</code>)
<code>//#</code>	Apply the <code>trivial</code> tactic to each goal and, if not solved, apply the <code>smt()</code> tactic; if it fails to solve the goal, the proof step fails (shorthand for <code>// /#</code>)
<code>@ dir occ /op</code>	Apply the <code>rewrite</code> tactic to each goal to unfold occurrences of the top operator in the expression pattern <code>op</code> . If the direction indicator <code>dir</code> is present, occurrences are folded rather than unfolded. If an occurrence selector <code>occ</code> is present, only selected occurrences are unfolded or folded. See <i>Folding and Unfolding</i> .

The difference between the default and variant `simplify` tactics is minimal, amounting to a few reductions that the `logic` subtactic uses that the variant version doesn’t.

There are four patterns that use the top assumption of a goal's conclusion as an argument to a tactic. The word `rep` indicates an optional repetition marker using the syntax described in *The Do Tactical*. The word `occ` indicates an optional occurrence selector (see *Occurrence Selectors*).

<code>rep ->></code>	Apply the <code>subst</code> tactic to each goal using its top assumption (which must be an equality with a variable on the left) left-to-right in both the context and the conclusion, then <code>clear</code> the assumption and the variable
<code>rep <<-</code>	Same as previous, except it uses the top assumption (which must be an equality with a variable on the right) right-to-left
<code>rep occ -></code>	Apply the <code>rewrite</code> tactic to each goal using its top assumption (which must be an equality, equivalence or <code>bool</code> expression that occurs in the rest of the conclusion) left-to-right on selected occurrences of the left side in the rest of the conclusion, then <code>clear</code> the assumption
<code>rep occ <-</code>	Same as previous, except it uses the top assumption right-to-left

The next three patterns are called ***subst top*** patterns. Each may optionally be preceded by a non-negative integer followed by `!`. These use as many hypotheses from the conclusion as possible as substitutions, up to `n`, if it is specified.

<code>n! <*></code>	Apply each hypothesis to each goal in either direction
<code>n! <*>></code>	Apply each hypothesis to each goal favoring left-to-right
<code>n! <<*></code>	Apply each hypothesis to each goal favoring right-to-left

The next four patterns collectively define ***crush mode***. This mode tries many introduction patterns repeatedly to solve or make progress on as many subgoals as possible. These include `//`, `/=`, `[]` (the case pattern, below) and `->`. It uses `+` to move hypotheses out of the way temporarily to uncover other hypotheses it can use. It tries some tactics specific to program logic, such as `conseq` and `auto`. Crush mode has two Boolean parameters, *unfold* and *solve*, that are encoded in the patterns. *Unfold* indicates whether to unfold symbols (`delta`) when simplifying. *Solve* indicates whether to use a program-logic-specific version of `trivial`.

<code> ></code>	<code>unfold = false, solve = false</code>
<code>/></code>	<code>unfold = true, solve = false</code>
<code> ></code>	<code>unfold = false, solve = true</code>
<code>//></code>	<code>unfold = true, solve = true</code>

The remaining two patterns have their own subsections.

<code>/p</code>	Apply the proof term <code>p</code> to the top assumption of each goal using forward reasoning
<code>[l₁₁ ... l_{1m1} ... l_{r1} ... l_{rmr}]</code>	This is called a <i>case pattern</i> (not to be confused with the <code>case</code> tactic)

Here are some quick examples of most of the introduction patterns.

A condensed proof of `PairEq2_conj` using `rewrite` patterns:

```
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
```

```

y': 'b
-----
x = x' /\ y = y' => (x, y) = (x', y')
case => -> -> //.
No more goals

```

A condensed proof of `PairEq2_conj` using `subst` patterns:

```

Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' /\ y = y' => (x, y) = (x', y')
case => ->> ->> //.
No more goals

```

A condensed proof from the `List` theory:

```

lemma drop_le0 ['a] (n : int) (s : 'a list) :
  n <= 0 => drop n s = s.
Type variables: 'a
n: int
s: 'a list
-----
n <= 0 => drop n s = s
by elim s => // = x s _ -> /=.
No more goals

```

Let's unpack this proof. We ignore `by` until the end. The first thing the proof does is move `s` down from the context and eliminate it, generating two subgoals:

```

elim s.
(* Subgoal 1 *)
Type variables: 'a
n: int
-----
n <= 0 => drop n [] = []

```

The `// =` pattern does both `simplify` and `trivial`, which is sufficient to close the first subgoal, so we can ignore the remaining introduction patterns for it:

```

simplify.
Type variables: 'a
n: int
-----
true
trivial.
(* Subgoal 2 *)
Type variables: 'a
n: int

```



```

-----
forall (x : 'a) (l : 'a list),
  (n <= 0 => drop n l = l) => n <= 0 => drop n (x :: l) = x :: l

```

Note that `simplify` is redundant when `trivial` succeeds. We do `//=` to the second subgoal, too, but `trivial` has no effect, so `simplify` was useful:

```

simplify.
Type variables: 'a
n: int
-----
forall (x : 'a) (l : 'a list),
  (n <= 0 => drop n l = l) =>
    n <= 0 => (if n <= 0 then x :: l else drop (n - 1) l) = x :: l
trivial.
Type variables: 'a
n: int
-----
forall (x : 'a) (l : 'a list),
  (n <= 0 => drop n l = l) =>
    n <= 0 => (if n <= 0 then x :: l else drop (n - 1) l) = x :: l

```

Now we move two assumptions to the context and throw away the third, which is the induction hypothesis (suggesting we could have used `case` instead of `elim`):

```

move => x s _.
Type variables: 'a
n: int
x: 'a
s: 'a list
-----
n <= 0 => (if n <= 0 then x :: s else drop (n - 1) s) = x :: s

```

Now we want to rewrite with the top assumption, but to use the `rewrite` tactic (rather than the introduction tactical) we need to move the top assumption to the context:

```

move => H.
Type variables: 'a
n: int
x: 'a
s: 'a list
H: n <= 0
-----
(if n <= 0 then x :: s else drop (n - 1) s) = x :: s
rewrite H.
Type variables: 'a
n: int
x: 'a
s: 'a list
H: n <= 0
-----
(if true then x :: s else drop (n - 1) s) = x :: s

```

```

clear H.
Type variables: 'a
n: int
x: 'a
s: 'a list
-----
(if true then x :: s else drop (n - 1) s) = x :: s

```

Note that `H` is not an equality but is of type `bool` and occurs in the conclusion, so the occurrence of `n < 0` in the conclusion is rewritten as `true`. Now we do `simplify` for the `/=` pattern and finish with `done` for the `by` tactical:

```

simplify.
Type variables: 'a
n: int
x: 'a
s: 'a list
-----
true
done.
No more goals

```

Now that we understand the proof, we can improve it a little:

```

Type variables: 'a
n: int
s: 'a list
-----
n <= 0 => drop n s = s
by case s => // x s /= ->.
No more goals

```

The availability of `trivial` as an introduction pattern contributes to the low incidence of the `reflexivity` and `assumption` tactics in proofs, in my opinion.

5.10.1 Applying a Lemma to the Top Assumption

The `/p` introduction pattern applies the proof term `p` to the top assumption of each goal using forward reasoning. For example, the proof of `onth_some_mem` given above can be streamlined by *not* using the `apply` tactic explicitly:

```

lemma onth_some_mem ['a] (xs : 'a list) (n : int) x:
  onth xs n = Some x => x \in xs.
Type variables: 'a
xs: 'a list
n: int
x: 'a
-----
onth xs n = Some x => x \in xs
move => /onth_some.
Type variables: 'a
xs: 'a list

```

```

n: int
x: 'a
-----
(0 <= n && n < size xs) /\ nth witness xs n = x => x \in xs

```

This step takes the place of 3 steps used previously:

```

move => H.
apply onth_some in H.
move : H.

```

There are some subtle differences between how the introduction tactical applies a lemma to the top assumption and how the `apply` tactic applies one `in H`.

- Successive `/p` patterns can be applied to the top assumption, whereas `apply in H` does not allow multiple arguments
- The lemma can be an equivalence and will be applied in whichever direction matches the top assumption, whereas `apply in H` fails if the lemma is an equivalence
- It is not necessary to instantiate all of the lemma's variables: any that aren't just remain universally quantified

Continuing the proof of `onth_some_mem` provides an example of the third difference. If we use the `case` tactic to destruct the top assumption and then move the first part to the context, we cannot apply the `mem_nth` variable to it there:

```

case => H1.
Type variables: 'a
xs: 'a list
n: int
x: 'a
H1: 0 <= n && n < size xs
-----
nth witness xs n = x => x \in xs
apply mem_nth in H1.
cannot infer all variables

```

The variable it can't figure out is `x0`, which only appears in the conclusion of `mem_nth`. We could provide a match, of course, but we don't have to if we leave the top assumption where it is and use the introduction pattern instead:

```

case => /mem_nth.
Type variables: 'a
xs: 'a list
n: int
x: 'a
-----
(forall (x0 : 'a), nth x0 xs n \in xs) =>
nth witness xs n = x => x \in xs

```

The unmatched `x0` is part of the new top assumption. The second assumption can be used to rewrite `x`. We can move the top assumption out of the way or let crush mode do it for us. Either method tacks on some more introduction patterns:

```

case => /mem_nth |>.
Type variables: 'a
xs: 'a list
n: int
x: 'a
-----
(forall (x0 : 'a), nth x0 xs n \in xs) => nth witness xs n \in xs

```

Now the `apply` tactic with no arguments can solve the goal with no help concerning `x0`:

```

apply.
No more goals

```

What if the `apply` tactic couldn't figure out `x0`? With no argument, there is no way to provide it an instantiation. You could move the top assumption to the context and then apply it by name with a proof term that provides the needed argument, but the `/p` introduction pattern has an alternative solution. If the pattern has the special form `/(_ ρ1 ... ρk)`, with an underscore in place of a lemma name, then instead of applying a lemma to the top assumption, it instantiates the top assumption's assumptions with the arguments $\rho_1 \dots \rho_k$. For example, in the proof above we could have done

```

Type variables: 'a
xs: 'a list
n: int
x: 'a
-----
(forall (x0 : 'a), nth x0 xs n \in xs) => nth witness xs n \in xs
move => /(_ witness).
xs: 'a list
n: int
x: 'a
-----
nth witness xs n \in xs => nth witness xs n \in xs

```

We don't even need the `apply` tactic now:

```

done.
No more goals

```

Instantiating the top assumption is very similar to instantiating a lemma, which is a form of generalization. See *Instantiating a Lemma*.

5.10.2 The Case Introduction Pattern

The following introduction pattern is called the *case pattern* (not to be confused with the `case` tactic):

$$[l_{i1} \dots l_{im1} \mid \dots \mid l_{r1} \dots l_{rmr}]$$

It is used to apply different sets of introduction patterns to a list of subgoals. This is similar in syntax and meaning to the parallel tactical described in *Variations on a Theme* but uses introduction patterns instead of tactics. If there are exactly r subgoals, then the pattern applies to each subgoal G_i the introduction patterns in the corresponding sequence $l_{i1} \dots l_{imi}$; otherwise the step fails with the message, "not the right number of intro-patterns (got n , expecting r)," with n and r filled in appropriately.

For example, a proof of the `seq2_ind` induction lemma in the `List` theory (and mentioned in *Proof by Induction: The Elim Tactic*) begins

```

lemma seq2_ind ['a 'b] (P : 'a list -> 'b list -> bool) :
  P [] [] =>
    (forall x1 x2 s1 s2, P s1 s2 => P (x1 :: s1) (x2 :: s2)) =>
      forall s1 s2, size s1 = size s2 => P s1 s2.
Type variables: 'a, 'b
P: 'a list -> 'b list -> bool
-----
P [] [] =>
  (forall (x1 : 'a) (x2 : 'b) (s1 : 'a list) (s2 : 'b list),
    P s1 s2 => P (x1 :: s1) (x2 :: s2)) =>
    forall (s1 : 'a list) (s2 : 'b list),
      size s1 = size s2 => P s1 s2.
move => Pnil Pcons.
Type variables: 'a, 'b
P: 'a list -> 'b list -> bool
Pnil: P [] []
Pcons: (forall (x1 : 'a) (x2 : 'b) (s1 : 'a list) (s2 : 'b list),
        P s1 s2 => P (x1 :: s1) (x2 :: s2))
-----
forall (s1 : 'a list) (s2 : 'b list),
  size s1 = size s2 => P s1 s2.
elim => [ | x s ih ].

```

This case pattern has two lists of introduction patterns (`r = 2`) and the `elim` tactic generates two subgoals, so all is well. The first list (before the `|`) is empty and does nothing, while the second introduces three assumptions, naming them `x`, `s` and `ih`. This makes the two subgoals

```

Type variables: 'a, 'b
P: 'a list -> 'b list -> bool
Pnil: P [] []
Pcons: (forall (x1 : 'a) (x2 : 'b) (s1 : 'a list) (s2 : 'b list),
        P s1 s2 => P (x1 :: s1) (x2 :: s2))
-----
forall (s2 : 'b list), size [] = size s2 => P [] s2

```

and

```

Type variables: 'a, 'b
P: 'a list -> 'b list -> bool
Pnil: P [] []
Pcons: (forall (x1 : 'a) (x2 : 'b) (s1 : 'a list) (s2 : 'b list),
        P s1 s2 => P (x1 :: s1) (x2 :: s2))
x: 'a
s: 'a list
ih: forall (s2 : 'b list), size s = size s2 => P s s2
-----
forall (s2 : 'b list), size (x :: s) = size s2 => P (x :: s) s2

```

Now note that the top assumption of both subgoals is the `list` variable `s2`. We will continue by using induction on both, but with the `case` tactic this time. Each subgoal will yield two new subgoals, for a total of four subgoals. The first subgoal of each pair will be for the empty list and will have nothing to introduce, while the second subgoal will be for a non-empty list with two new variables we would like to introduce as `y` and `s'`.

One way to do this is to add a second case pattern after the one we used above. This will hide the subgoals generated by the first case pattern, but we know what they are from above and so can follow what the second one does. The `admit` tactic is used for brevity, to omit the proofs of each subgoal.

```
elim => [ | x s ih ] [ | y s' ].
(* Subgoal 1 *)
Type variables: 'a, 'b
P: 'a list -> 'b list -> bool
Pnil: P [] []
Pcons: (forall (x1 : 'a) (x2 : 'b) (s1 : 'a list) (s2 : 'b list),
        P s1 s2 => P (x1 :: s1) (x2 :: s2))
-----
size [] = size [] => P [] []
admit.
(* Subgoal 2 *)
Type variables: 'a, 'b
P: 'a list -> 'b list -> bool
Pnil: P [] []
Pcons: (forall (x1 : 'a) (x2 : 'b) (s1 : 'a list) (s2 : 'b list),
        P s1 s2 => P (x1 :: s1) (x2 :: s2))
y: 'b
s': 'b list
-----
size [] = size (y :: s') => P [] (y :: s')
admit.
(* Subgoal 3 *)
Type variables: 'a, 'b
P: 'a list -> 'b list -> bool
Pnil: P [] []
Pcons: (forall (x1 : 'a) (x2 : 'b) (s1 : 'a list) (s2 : 'b list),
        P s1 s2 => P (x1 :: s1) (x2 :: s2))
x: 'a
s: 'a list
ih: forall (s2 : 'b list), size s = size s2 => P s s2
-----
size (x :: s) = size [] => P (x :: s) []
admit.
(* Subgoal 4 *)
Type variables: 'a, 'b
P: 'a list -> 'b list -> bool
Pnil: P [] []
Pcons: (forall (x1 : 'a) (x2 : 'b) (s1 : 'a list) (s2 : 'b list),
        P s1 s2 => P (x1 :: s1) (x2 :: s2))
x: 'a
```

```

s: 'a list
ih: forall (s2 : 'b list), size s = size s2 => P s s2
y: 'b
s': 'b list
-----
size (x :: s) = size (y :: s') => P (x :: s) (y :: s')

```

You may be wondering why the second case pattern runs the `case` tactic on each subgoal before applying the pattern when the first one didn't. This is because the `case` tactic is the only tactic the case pattern can run, and whether it does depends on what was done just before the pattern, namely the identity of τ and whether it is the first introduction pattern, not counting any of the `//`, `/=`, `//=`, `/#` and `//#` patterns. If $\tau <> \text{idtac}$ or `move` (i.e., τ does something) and the case pattern is first, then it is applied to the list of subgoals generated by τ . Otherwise, the `case` tactic is run on each subgoal and the case pattern is applied to the list of subgoals generated by `case`. If the `case` tactic cannot be applied, the step fails with the message, "invalid intro-pattern: nothing to eliminate".

Pro tip. To force the introduction tactical to run the `case` tactic when the case pattern is the first pattern, place the `-` pattern just before it. There won't be any assumptions to restore, but it isn't `//` and so on, so it makes the case pattern second.

Pro tip. To run the `case` tactic without providing introduction patterns for each subgoal, use the case pattern with `r = 0` (i.e., the pattern `[]`) and the tactical will immediately move on to the next pattern.

In the `seq_ind` example, if we had needed to use `elim` (or any tactic other than `case`), we could have used a different tactical before the second case pattern like this:

```
elim => [ | x s ih ]; elim => [ | y s' ].
```

Since case patterns contain lists of introduction patterns, they can be nested. The proof of the `subseqsP` lemma in the `List` theory contains an impressive stack of 5 case patterns! As a bonus, the innermost one contains a crush mode pattern. The first part of the proof is not relevant (though it does use several case patterns), so we will skip over its details:

```

lemma subseqsP ['a] (xs ys : 'a list) :
  subseq xs ys <=> xs \in subseqs ys.
Type variables: 'a
xs: 'a list
-----
subseq xs ys <=> xs \in subseqs ys
elim: ys xs => [|y ys ih] [|x xs] //=.
- by rewrite mem_cat -ih // sub0seq.
rewrite mem_cat; case: (y = x) => [<-|ne_xy|.
- rewrite (mem_map ((::) y)) 1:inj_cons -!ih -eq_iff.
  by rewrite eq_sym orb_idr // &(subseq_trans) subseq_cons.
Type variables: 'a
y: 'a
ys: 'a list
ih: forall (xs : 'a list), subseq xs ys <=> xs \in subseqs ys
x: 'a
xs: 'a list
ne_xy: y <> x

```

```

-----
subseq (x :: xs) ys <=>
(x :: xs \in map ((::) y) (subseqs ys)) \ /
(x :: xs \in subseqs ys)

```

The next step uses the `split` tactic to break the equivalence into two implications.

```

split.
(* Subgoal 1 *)
Type variables: 'a
y: 'a
ys: 'a list
ih: forall (xs : 'a list), subseq xs ys <=> xs \in subseqs ys
x: 'a
xs: 'a list
ne_xy: y <> x
-----
subseq (x :: xs) ys =>
(x :: xs \in map ((::) y) (subseqs ys)) \ /
(x :: xs \in subseqs ys)

```

Three introduction patterns solve the first subgoal by applying the `ih` hypothesis from left to right on the top assumption and then using the top assumption as a rewrite rule, finishing with the `trivial` tactic, taking us to the second subgoal:

```

move => /ih -> //.
(* Subgoal 2 *)
Type variables: 'a
y: 'a
ys: 'a list
ih: forall (xs : 'a list), subseq xs ys <=> xs \in subseqs ys
x: 'a
xs: 'a list
ne_xy: y <> x
-----
(x :: xs \in map ((::) y) (subseqs ys)) \ /
(x :: xs \in subseqs ys) =>
subseq (x :: xs) ys

```

The top assumption is a disjunction, so we will use the `case` tactic to set up a subgoal for each disjunct:

```

case.
(* Subgoal 2a *)
Type variables: 'a
y: 'a
ys: 'a list
ih: forall (xs : 'a list), subseq xs ys <=> xs \in subseqs ys
x: 'a
xs: 'a list
ne_xy: y <> x
-----
x :: xs \in map ((::) y) (subseqs ys) => subseq (x :: xs) ys

```


The `mapP` lemma from the `List` theory (recall, `x \in s` is an abbreviation for `mem s x`),

```
lemma mapP ['a 'b] (f : 'a -> 'b) s y:
  mem (map f s) y <=> (exists x, mem s x /\ y = f x).
```

can be applied to the top assumption using forward reasoning:

```
move => /mapP.
Type variables: 'a
y: 'a
ys: 'a list
ih: forall (xs : 'a list), subseq xs ys <=> xs \in subseqs ys
x: 'a
xs: 'a list
ne_xy: y <> x
-----
(exists (x0 : 'a list), (x0 \in subseqs ys) /\ x :: xs = y :: x0)
=> subseq (x :: xs) ys
```

We will use the `case` tactic to change `exists` to `forall` and then introduce `x0` anonymously:

```
case => ?.
Type variables: 'a
y: 'a
ys: 'a list
ih: forall (xs : 'a list), subseq xs ys <=> xs \in subseqs ys
x: 'a
xs: 'a list
ne_xy: y <> x
`x: 'a list
-----
(`x \in subseqs ys) /\ x :: xs = y :: `x => subseq (x :: xs) ys
```

We will use the `case` tactic yet again to break the conjunctive top assumption into two assumptions and discard the first one:

```
case => _.
Type variables: 'a
y: 'a
ys: 'a list
ih: forall (xs : 'a list), subseq xs ys <=> xs \in subseqs ys
x: 'a
xs: 'a list
ne_xy: y <> x
`x: 'a list
-----
x :: xs = y :: `x => subseq (x :: xs) ys
```

We will use the `case` pattern one last time to destruct the top assumption into two separate equalities, because constructors of inductive types are known to be injective:

```
case.
Type variables: 'a
```

```

y: 'a
ys: 'a list
ih: forall (xs : 'a list), subseq xs ys <=> xs \in subseqs ys
x: 'a
xs: 'a list
ne_xy: y <> x
`x: 'a list
-----
x = y => xs = `x => subseq (x :: xs) ys

```

Now we use crush mode to solve this goal:

```
move => |>.
```

The final subgoal is nearly the converse of the first subgoal after the `split` tactic and can be solved almost the same way, applying `ih` from right to left this time:

```

(* Subgoal 2b *)
Type variables: 'a
y: 'a
ys: 'a list
ih: forall (xs : 'a list), subseq xs ys <=> xs \in subseqs ys
x: 'a
xs: 'a list
ne_xy: y <> x
-----
x :: xs \in subseqs ys => subseq (x :: xs) ys
move => /ih //.
No more goals

```

Everything we did starting from the `split` can be combined into one step. You can even dispense with all spaces, if you like (I personally don't):

```

split=>[/ih->///|[/mapP[?[_[/>]]]]|/ih//]].
No more goals

```

A special-purpose but useful feature of the case pattern lets you repeat certain case destructions on one or more top assumptions and apply an introduction pattern after each one. This is done by adding a **case mode** just after the left bracket. The presence of a case mode restricts the `case` tactic to just two destruction rules: the ones for conjunction and disjunction. It also forces the pattern to run the `case` tactic, regardless of whether it is first or what `τ` is. To keep the examples simple, let's declare the following operators as generic stand-ins for hypotheses and a conclusion:

```
const a, b, c, d, concl : bool.
```

All case modes contain the `#` symbol and the simplest case mode is `#`. In this mode, the `case` tactic applies only the conjunction rule to change all occurrences of `/\` or `&&` to `=>` in the top assumption (normally, recall, it only does the first one). For example,

```

lemma and_elim : a /\ b /\ c /\ d => concl.
move => [#].
-----

```

```
a => b => c => d => concl
```

We can add a list of names after the mode to introduce the newly isolated top assumption after each application of the conjunction rule like this:

```
move => [# ha hb hc hd] .
ha: a
hb: b
hc: c
hd: d
-----
concl
```

If we apply the # mode to a top assumption that is a mix of conjunction and disjunction, not much happens:

```
lemma and_or_elim : (a \/ b) /\ (c \/ d) => concl.
move => [#].
-----
a \/ b => c \/ d => concl
```

Adding the | symbol to the end of a mode tells the case tactic to use both the conjunction and disjunction rules. Because | is also used to separate lists of introduction patterns in a case pattern, there must not be a space between # and |.

```
move => [#|].
-----
a => c => concl
admit.
-----
a => d => concl
admit.
-----
b => c => concl
admit.
-----
b => d => concl
```

The modes used so far only destruct the top assumption, as usual for the case tactic and case pattern. There are two modes that destruct multiple top assumptions. Adding the ! symbol to the front of the mode will destruct as many top assumptions as possible, while adding n! will destruct up to n of them:

```
lemma or_imp_elim : (a \/ b) => (c \/ d) => concl.
move => [#!|].
-----
a => c => concl
admit.
-----
a => d => concl
admit.
-----
b => c => concl
```

```

admit.
-----
b => d => concl

```

By default, the `case` tactic will unfold symbols in the top assumption if necessary to find something to destruct, but with a case mode, it does not. Adding the `/` symbol mode before `#` (and after `!`, if also present) restores the default behavior. Because `/#` is the introduction pattern for running the `smt` tactic, you must put a space between `/` and `#`. For example, in the `Ideal` theory (in the `algebra` folder), the `ideal` operator has a conjunctive definition that can be unfolded and destructed twice. A case pattern without a mode unfolds but only destructs once:

```

lemma ideal0 (I : t -> bool) : Ideal I => I zeror.
I: t -> bool
-----
ideal I => I zeror
move => [].
I: t -> bool
-----
I zeror =>
  ((forall (x y : t), I x => I y => I (x - y)) /\
   forall (a x : t), I x => I (a * x)) =>
  I zeror

```

A simple case mode has no effect because it doesn't unfold:

```

move => [#].
I: t -> bool
-----
ideal I => I zeror

```

This case mode unfolds and destructs twice:

```

move => [/ #].
I: t -> bool
-----
I zeror =>
  (forall (x y : t), I x => I y => I (x - y)) =>
  (forall (a x : t), I x => I (a * x)) =>
  I zeror

```

By adding other introduction patterns, we can complete the proof:

```

move => [/ #] //.
No more goals

```

There is one other mode symbol, `~`, which relates to destructing inductive predicates. Since we minimized our discussion of predicates, this mode will not be described.

5.10.3 Introduction Patterns in Odd Places

Selected introduction patterns can be used in a couple of places outside of the introduction tactical:

- In the `rewrite` tactic, the patterns `//`, `/=`, `/~`, `//=`, `//~`, `/#` and `//#` can be used to simplify and/or solve subgoals between proof terms or fold/unfold operations

- In a proof term, the patterns `//`, `/#` and `//#` can be used as an argument for a hypothesis

For example, the original proof of `count_eq0` in *Basic Rewriting* had 7 steps. Using a chain, we reduced this to one step in *The Chain Tactical*. It turns out the rewrite tactic can handle all the steps by itself after all:

```
lemma count_eq0 ['a] (p : 'a -> bool) (s : 'a list) :
  ! (has p s) <=> count p s = 0.
Type variables: 'a
p: 'a -> bool
s: 'a list
-----
! has p s <=> count p s = 0.
rewrite has_count ltzNge // lez_eqVlt ltzNge count_ge0 //.
No more goals
```

As we'll see in *Generalization Tactics*, introduction patterns can also be used with the generalization tactics `have`, `suff`, `wlog` and `gen have`.

5.11 Specialized Proof Tactics

If `rewrite` and `apply` are the acme of generality, the remaining tactics are the nadir. Many are based on or justified by particular lemmas, and interestingly, many of these lemmas are gathered into the `Tactics` theory in the `prelude` folder.

5.11.1 The `congr`, `Left` and `Right` Tactics

These tactics are in the same spirit as `split`. The `congr` tactic (for congruence) replaces a conclusion that is an equality (or equivalence) between two applications of an operator or predicate (such as `f x = f y`) with equalities between their corresponding arguments (`x = y`), then applies the `reflexivity` tactic to each subgoal to close those it can. For example, if the current goal is

```
Type variables: <none>
x: int
x': int
y: int
y': int
-----
x + y = x' + y'
```

then running

```
congr.
```

produces the subgoals

```
Type variables: <none>
x: int
x': int
y: int
y': int
-----
x = x'
```

and

```
Type variables: <none>
x: int
x': int
y: int
y': int
-----
y = y'
```

Note that proving $x = y$ is not the *only* way to prove $f\ x = f\ y$: it is merely *a* way. In other words, it is *sufficient* but not *necessary*. Only use the `congr` tactic if you want to try proving it that way.

The `congr` tactic can be used in place of `split` in the proof of `PairEq2_conj` and it immediately solves both subgoals (no need for `assumption`). It cannot, however, solve the entire lemma at once because the assumptions get in the way:

```
Type variables: 'a, 'b
-----
forall (x x' : 'a) (y y' : 'b), x = x' /\ y = y' => (x, y) = (x',
y')
congr.
goal must be an equality or an equivalence
```

The `congr` tactic can also solve the record form of our lemma, which `split` cannot:

```
type point = { x : int; y : int }.
lemma PointEq : forall (a a' b b' : int),
  a = a' => b = b' => mk_point a b = mk_point a' b'.
Type variables: <none>
-----
forall (a a' b b' : int),
  a = a' => b = b' => {| x = a; y = b; |} = {| x = a'; y = b'; |}
move => a a' b b' a_eq b_eq.
Type variables: <none>
a: int
a': int
b: int
b': int
a_eq: a = a'
b_eq: b = b'
-----
{| x = a; y = b; |} = {| x = a'; y = b'; |}
congr.
No more goals
```

The `left` and `right` tactics reduce a goal whose conclusion is a disjunction:

- The `left` tactic replaces $\phi_1 \ \backslash / \ \phi_2$ or $\phi_1 \ || \ \phi_2$ with ϕ_1
- The `right` tactic replaces $\phi_1 \ \backslash / \ \phi_2$ with ϕ_2
- The `right` tactic replaces $\phi_1 \ || \ \phi_2$ with $!\phi_1 \Rightarrow \phi_2$ (again, see *Pervasive* regarding `||`)

This is analogous to what `split` does to conjunctive goals, except for a disjunction it is sufficient for just one of the disjuncts to work; the user decides which to pursue.

5.11.2 The Exists Tactic

The `exists` tactic replaces an existentially quantified variable with an expression of the same type. In other words, it provides a witness with which to prove the claim.

For example, this lemma from the `List` theory says that a list of size one and a singleton list are the same thing. It is easy to show that a singleton list has size one, but the other direction is non-trivial.

```
lemma size_eq1 ['a] (xs : 'a list) :
  size xs = 1 <=> exists (x : 'a), xs = [x].
Type variables: 'a
xs: 'a list
-----
size xs = 1 <=> exists (x : 'a), xs = [x].
split.
(* => direction *)
Type variables: 'a
xs: 'a list
-----
size xs = 1 => exists (x : 'a), xs = [x].
```

For this direction, we will use induction twice (the `case` tactic variant, using case patterns) to break this into three cases: the empty list, singleton lists and longer lists. Induction will introduce the variable we need to provide a witness for the existential and the other two cases will be vacuously true because their size isn't 1. The first induction yields two cases: the empty list and non-empty lists. The empty list case is trivial:

```
move : xs => [ | x].
(* empty list *)
Type variables: 'a
-----
size [] = 1 => exists (x : 'a), [] = [x].
done.
(* non-empty list *)
Type variables: 'a
x: 'a
-----
forall (l : 'a list),
  size (x :: l) = 1 => exists (x0 : 'a), x :: l = [x0].
```

For non-empty lists we introduce `x` but keep `l` because we want to do induction again. The second induction (case pattern) breaks non-empty lists into singleton lists and longer lists. For the former, we clear the hypothesis `size [x] = 1` and for the latter, we introduce two new variables as `y` and `xs`.

```
move => [_ | y xs].
(* Singleton list *)
Type variables: 'a
x: 'a
-----
```

```
exists (x0 : 'a), [x] = [x0].
```

The obvious solution here is to use `x` as the value or witness for `x0`.

```
exists x.
Type variables: 'a
x: 'a
-----
[x] = [x].
reflexivity.
(* longer lists *)
Type variables: 'a
xs: 'a list
x: 'a
y: 'a
-----
size (x :: y :: xs) = 1 => exists (x0 : 'a), x :: y :: xs = [x0].
```

The `smt` tactic will handle the tedious details of showing `size (x :: y :: xs) <> 1`, with a small assist:

```
smt (size_ge0).
(* <= direction *)
Type variables: 'a
xs: 'a list
-----
exists (x : 'a), xs = [x] => size xs = 1
```

This direction is easier: replace the existential quantifier with a universal quantifier, introduce the variable it binds and then use the top assumption as a rewrite:

```
by case => x ->.
No more goals
```

5.11.3 The Algebra Tactics

The `algebra` tactic solves goals involving ring and field operators. It doesn't have any arguments. It can save you a lot of fiddling with lemmas about commutativity, associativity, cancellation, how multiplication distributes over addition and all that sort of thing. It works on `int`, `real`, `bool` and user-defined rings and fields. It fails if it cannot solve the goal, so it works more like `smt` and `done` than like `simplify`—it's all or nothing, and it doesn't tell you why it fails. The `smt` tactic can almost certainly do everything `algebra` can do, but at the expense of a call to an external SMT prover.

A simple example of a goal it can solve is

```
lemma a3 (x y z : int) : x - (y + z) = x - y - z.
x: int
y: int
z: int
-----
x - (y + z) = x - y - z
algebra.
No more goals
```


To use the `algebra` tactic on `bool` or `int`, you must require the `StdRing` theory. To use it on `real`, you must require `Real`. To use it on your own type, you must require the `Ring` theory and declare your type to be a ring, a Boolean ring or a field. See *Ring and Field Instances*. EasyCrypt treats `ring`, `bring` and `field` as **type classes** and the parser supports defining additional ones, but the EasyCrypt library has no examples of this (not even for these three). Type classes appear to be a new feature that is not finished yet.

The `Ring` theory has two hint databases that are used by the `algebra` tactic:

```
hint rewrite rw_algebra : .
hint rewrite inj_algebra : .
```

You may be able to add hints to these databases to extend the tactic’s capabilities. See *Hints*.

The `ring`, `ringeq`, `field` and `fieldeq` tactics are further specialized to just one type class. Each of them takes an optional list of rewrite lemmas and/or databases to use in addition to whatever they use by default. The `_eq` variants reason exclusively via equation rewriting.

5.11.4 The Change Tactic

The `change` tactic is never used in the library. It takes an expression as an argument. It has to have type `bool` and be convertible to the current goal’s conclusion. This could be used to manually “unreduce” a goal (e.g., undo what the `simplify` tactic does or did) to get a desired form. The `logic` tactic and user reductions both have the potential to make a proof harder, not simpler.

For example, this lemma can be changed and then `simplify` can put it back, demonstrating the equivalence.

```
lemma change_me (i n : int) : n + 1 + (i - n - 1) = i.
i: int
n: int
-----
n + 1 + (i - n - 1) = i
change (n + 1 + size [<:int>] + (i - n - 1) = i).
i: int
n: int
-----
n + 1 + size [] + (i - n - 1) = i
simplify.
i: int
n: int
-----
n + 1 + (i - n - 1) = i
```

Unfortunately, this change is not progress toward a proof. You can’t change the lemma like this

```
change (n - n + 1 - 1 + i = i).      (* Doesn't work *)
```

because `simplify` isn’t strong enough to show the equivalence. (The `smt` tactic can prove the lemma.)

5.11.5 The Progress Tactic

The `progress` tactic repeatedly applies a small set of tactics to the current goal until it is solved or stops changing (“making progress”). It never fails, but may leave the goal unchanged. Tactics that are always applied are `simplify`, parts of `case` and introducing assumptions. The parts of `case` are the “conjunctive” patterns: `false`, `conjunction`, `existential` and `tuple equality`. Additional tactics are applied inside program logic proofs.

Its general form is

```
progress @[options] tactic.
```

Both arguments are optional, but if brackets are present, there must be at least one option. The `@` is also optional and doesn’t seem to mean anything.

The options indicate additional tactics to apply at each step. An option preceded by a hyphen is not applied. The options and their meanings are

<code>solve</code>	apply the <code>assumption</code> tactic
<code>split</code>	apply the <code>split</code> tactic
<code>subst</code>	apply the <code>subst</code> tactic to the top assumption
<code>disjunct</code>	when applying <code>case</code> , include the disjunction pattern
<code>delta : case</code>	when applying <code>case</code> , also try unfolding
<code>delta : split</code>	when applying <code>split</code> , also try unfolding
<code>delta</code>	a shorthand for <code>delta : case</code> and <code>delta : split</code>

By default, `solve`, `split`, `subst` and `delta : split` are selected and the rest are not, so for example, `disjunct` is false and `split` is true unless `disjunct` and/or `-split` are specified, respectively.

The tactic argument may be any tactic or the `by`, `do` or `try` tactical, but not a chain of tactics (i.e., any *core tactic*). This tactic is tentatively applied after each step the `progress` tactic performs, meaning it is ok if the tactic fails (like the `try` tactical).

The `progress` tactic proves `PairEq1` and `PairEq2` in one step. Other examples are in `crypto/DiffieHellman`, `looping/IterProc` and `query_counting/OracleBounds`; none has any arguments.

To illustrate the tactic, let’s revisit the proof of the `size_eq1` lemma.

```
lemma size_eq1 ['a] (xs : 'a list) :
  size xs = 1 <=> exists (x : 'a), xs = [x].
Type variables: 'a
xs: 'a list
-----
size xs = 1 <=> exists (x : 'a), xs = [x].
```

For some reason, inclusion of the `subst` tactic prevents the `split` tactic from working:

```
progress [-subst].
(* => direction *)
Type variables: 'a
xs: 'a list
H: size xs = 1
```

```

-----
exists (x : 'a), xs = [x].

```

We need to undo the introductions made by `progress` to destruct the list with `case`:

```

move : xs H.
Type variables: 'a
-----
forall (xs: 'a list), size xs = 1 => exists (x : 'a), xs = [x].
case.
(* empty list *)
Type variables: 'a
-----
size [] = 1 => exists (x : 'a), [] = [x].
progress. (* goal solved *)
(* non-empty list *)
Type variables: 'a
-----
forall (x : 'a) (l: 'a list),
  size (x :: l) = 1 => exists (x0 : 'a), x :: l = [x0].
progress.
Type variables: 'a
l: 'a list
H: 1 + size l = 1
-----
exists (x0 : 'a), x = x0 /\ l = [].

```

Here the proof has taken a different turn: instead of using `case` a second time, it has deduced that the non-empty list starts with `x` and the rest is empty. We again use `x` as the witness for the existential:

```

exists x.
Type variables: 'a
l: 'a list
H: 1 + size l = 1
-----
x = x /\ l = [].
progress.
Type variables: 'a
l: 'a list
H: 1 + size l = 1
-----
l = [].
smt (size_ge0). (* goal solved *)
(* <= direction *)
Type variables: 'a
xs: 'a list
x: 'a
H: xs = [x]
-----
size xs = 1

```

In the `<=` direction, `progress` converted the existential quantifier to a universal one automatically, but it also introduced its body as `H`, where it can't do `subst` on it, so we move it out of the context:

```
move : H.
Type variables: 'a
xs: 'a list
-----
xs = [x] => size xs = 1
progress.
No more goals
```

Overall, `progress` doesn't seem to combine a very powerful set of tactics, at least in the ambient logic.

5.11.6 The Solve Tactic

There are no examples of the `solve` tactic in the library. It might try combinations of other tactics automatically, as `trivial` and `progress` do. It might make use of hints in solve databases (see *Hints*).

Its general form is

```
solve N (db1, ..., dbn).
```

where `N` is a positive integer and `dbi` is a database name. Both arguments are optional. The default value for `N` is 1.

The tactic is recursive and `N` is the maximum depth of recursion.

5.12 The Generalization Tactical

Generalization in EasyCrypt refers to adding a top assumption to the conclusion of the current goal. An example is reversion, which moves an assumption from the goal's context to its conclusion. We've covered a lot of ground since discussing reversion and are now able to describe the broader concept.

There are three ways to add an assumption:

- **Reverting** an assumption in the context
- **Instantiating** a named or anonymous lemma (including an axiom or hypothesis)
- **Abstracting** an expression that occurs in the conclusion

Reversion undoes introduction, and together they let us move an assumption as needed to apply a proof tactic: out of the way for tactics that manipulate the entire conclusion, such as `split` and `congr`, or `apply` or `exact` with one or more arguments; or into the way for tactics and tacticals that manipulate the top assumption, such as `case`, `elim` and some introduction patterns.

Instantiating a named lemma optionally instantiates some or all of the assumptions of the lemma with the arguments of a proof term, then adds the result to the conclusion as the new top assumption. Instantiating an anonymous lemma (an arbitrary expression of type `bool`) generates two subgoals: the lemma on its own with the current goal's context, to prove it's a lemma, and the current goal with the assumption added as the top assumption (we called this a *cut* in *The Apply Tactic with an Anonymous Lemma*). Instantiation is used when a lemma needs some manipulation before it can be used, for example by the `rewrite` or `apply` tactic.

Abstracting an expression matches an expression pattern to a subexpression of the conclusion of the goal, adds a variable of the appropriate type as the new top assumption, and replaces all (or selected) occurrences of the matched pattern with the new variable. Abstraction is frequently followed by the `case` or `elim` tactic, to destruct the variable just created.

There are two ways to perform generalization: using the generalization tactical and using one of four generalization tactics. This section is about the former, and a later section is about the latter.

5.12.1 Generalization Patterns

Generalization is performed using the **generalization tactical** by listing one or more **generalization patterns**. Certain tactics, namely `move`, `case`, `elim`, `apply` and `exact`, have incorporated generalization patterns into their own syntax so successfully that the generalization tactical itself is rarely used in practice. While much is common to all of them, these specialized forms also have their quirks and are described individually in following sections.

The generalization tactical is used in much the same way as the introduction tactical. In particular, any tactic τ can be followed by any number of introduction and generalization tacticals. Where the introduction tactical has the general form

$$\tau \Rightarrow l_1 \dots l_n$$

where $l_1 \dots l_n$ is a list of introduction patterns, the generalization tactical has the general form

$$\tau <@ \{a_1 \dots a_m\} \Pi_1 \dots \Pi_n$$

where $\{a_1 \dots a_m\}$ is a list of assumption names in the context and $\Pi_1 \dots \Pi_n$ is list of generalization patterns. Both parts are optional. We shall use the `idtac` tactic as τ to focus on how the generalization patterns work.

Tactical execution is as follows. The tactic τ is done first, then each tactical from left to right is applied to the proof tree. The introduction tactical processes its patterns from left to right. The generalization tactical clears the listed assumptions first and then processes its patterns from right to left. Recall that assumptions can also be cleared by the introduction tactical using the same syntax and by the `clear` tactic (without the braces).

As a silly example using reversion,

```
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' => y = y' => (x, y) = (x', y')
idtac => x_eq <@ y <@ x x_eq => + + z yz_eq.
x': 'a
y': 'b
z: 'b
yz_eq: z = y'
-----
forall (x : 'a), x = x' => (x, z) = (x', y')
```

applies the `idtac` tactic to the current goal (i.e., does nothing), moves assumption `x = x'` up (naming it `x_eq`), moves `y` down, moves `x_eq` and `x` down, in that order, moves `x`, `x = x'`, `y` and `y = y'` up (naming the last two `z` and `yz_eq`, respectively) and finally moves `x_eq` and `x` down, in that order. Recall the `+` introduction pattern clears the top assumption temporarily and then restores it after the last pattern in the list (or when the `-` pattern is encountered). Each of these introductions and reversions replaced one subgoal with one new subgoal, so the proof tree is linear.

Generalization patterns have the following forms:

<code>name</code>	Revert the assumption named <code>name</code> . Fail if that assumption is used by other assumptions in the context. If <code>name</code> is a local definition, revert it as a universally quantified variable and omit its definition
<code>@name</code>	Revert the assumption named <code>name</code> (which must be a local definition, also called a “let in”) as a let expression
<code>p</code>	Instantiate a lemma using the proof term <code>p</code>
<code>(_: b)</code>	Instantiate the Boolean expression <code>b</code> as an anonymous lemma
<code>((_: b) args)</code>	Instantiate the Boolean expression <code>b</code> with the specified arguments as an anonymous lemma
<code>occ e</code>	Abstract a subexpression in the conclusion that matches expression pattern <code>e</code> ; <code>occ</code> is an optional occurrence selector

The first two were covered in *Reverting an Assumption*.

Note. The name of a hypothesis is technically a proof term and the name of a variable is technically an expression pattern. The way generalization handles these small overlaps can be confusing at first.

5.12.2 Instantiating a Lemma

A named lemma is instantiated by providing a proof term. As usual, “lemma” here includes axioms and hypotheses in the context. Instantiation does not change the meaning of the goal: it is like assuming `true`, which has no effect (`φ` and `true => φ` are equivalent). Its purpose is to introduce additional facts that can be used to make progress on the current goal, possibly after some manipulation.

Unlike the `apply` or `rewrite` tactics, instantiating a lemma does not try to match the lemma with any part of the current goal, so the only information the tactical has to guide the instantiation is what the proof term provides. Instead of failing when there’s not enough information for a full instantiation, however, it simply retains any assumptions that aren’t instantiated. If the proof term is just a name, the new top assumption will be an exact copy of the lemma.

For example, there is a point in the proof of the `natind` lemma in the `Int` theory when the current goal is

```
p: int -> bool
ihn: (forall n, n <= 0 => p n)
ihp: forall n, 0 <= n => p n => p (n+1)
n: int
-----
p n
```

We can generalize this by instantiating the `lezWP` lemma, which says `<=` is total,

```
lemma lezWP : forall (z1 z2 : int), z1 <= z2 || z2 <= z1.
```

like this:

```
idtac <@ lezWP.
p: int -> bool
ihn: (forall n, n <= 0 => p n)
ihp: forall n, 0 <= n => p n => p (n+1))
n: int
-----
forall (z1 z2 : int), z1 <= z2 || z2 <= z1) => p n
```

While this goal is equivalent to the previous one, the new top assumption has no connection with the conclusion (e.g., no variables in common), so it isn't very useful. If instead we craft arguments for the lemma's variables, we get

```
idtac <@ (lezWP 0 n).
p: int -> bool
ihn: (forall n, n <= 0 => p n)
ihp: forall n, 0 <= n => p n => p (n+1))
n: int
-----
0 <= n || n <= 0 => p n
```

It's no coincidence that with this instantiation, the top assumption mirrors `ihp` and `ihn`. The proof continues as follows, omitting some intermediate subgoals and using "`idtac <@`" instead of "`move :>`" for reversion:

```
case.
(* Subgoal 1 *)
p: int -> bool
ihn: (forall n, n <= 0 => p n)
ihp: forall n, 0 <= n => p n => p (n+1))
n: int
-----
0 <= n => p n
idtac <@ n. elim /intind. by apply ihn. apply ihp.
(* Subgoal 2 *)
p: int -> bool
ihn: (forall n, n <= 0 => p n)
ihp: forall n, 0 <= n => p n => p (n+1))
n: int
-----
! 0 <= n => n <= 0 => p n
move => _. apply ihn.
No more goals
```

A similar example involving a hypothesis comes from the proof of the `pmap_inj_in_uniq` lemma in the `List` theory, again joined in progress:

```
Type variables: 'a, 'b
f: 'a -> 'b option
```

```

x: 'a
s: 'a list
inj_f: forall (x0 y : 'a) (v : 'b),
  x0 = x \/\ (x0 \in s) =>
  y = x \/\ (y \in s) =>
  f x0 = Some v => f y = Some v => x0 = y
...
-----
x = y
idtac <@ inj_f.
Type variables: 'a, 'b
f: 'a -> 'b option
x: 'a
s: 'a list
...
-----
(forall (x0 y0 : 'a) (v0 : 'b),
  x0 = x \/\ (x0 \in s) =>
  y0 = x \/\ (y0 \in s) =>
  f x0 = Some v0 => f y0 = Some v0 => x0 = y0) =>
x = y

```

Again, the new top assumption has very little connection with the conclusion (e.g., no variables in common except x and f). If we add arguments to the proof term, we get

```

idtac <@ (inj_f x y v).
Type variables: 'a, 'b
f: 'a -> 'b option
x: 'a
s: 'a list
inj_f: forall (x0 y : 'a) (v : 'b),
  x0 = x \/\ (x0 \in s) =>
  y = x \/\ (y \in s) =>
  f x0 = Some v => f y = Some v => x0 = y
...
-----
(x = x \/\ (x \in s) =>
  y = x \/\ (y \in s) =>
  f x = Some v => f y = Some v => x = y) =>
x = y

```

which seems eminently more promising. Note that the hypothesis `inj_f` is still in the context: the new top assumption instantiates it, but the general version remains true.

As something of an edge case, if H is a hypothesis with no assumptions, the generalization pattern H reverts it but the pattern (H) instantiates it and retains the declaration of H in the context. Either way, the new top assumption is the same. If H is not a hypothesis, the generalization patterns H and (H) both instantiate H .

A final example illustrates a subtlety regarding proof terms with holes. Consider this lemma from the `List` theory and the start of its proof:


```

lemma take_nseq ['a] (k n : int) (x : 'a) :
  0 <= k <= n => take k (nseq n x) = nseq k x.
Type variables: 'a
k: int
n: int
x: 'a
-----
0 <= k && k <= n => take k (nseq n x) = nseq k x
case => le0_k lek_n.
Type variables: 'a
k: int
n: int
x: 'a
le0_k: 0 <= k
lek_n: k <= n
-----
take k (nseq n x) = nseq k x

```

Assume we notice that we can decompose n into k and $(n - k)$ and both are non-negative. The `cat_nseq` lemma sort of does that, but it will require some manipulation in order to use it, so first we just add it as a new assumption, instantiating its variables with the goal's:

```

idtac <@ (cat_nseq k (n - k) x) .
Type variables: 'a
k: int
n: int
x: 'a
le0_k: 0 <= k
lek_n: k <= n
-----
(0 <= k => 0 <= n - k =>
  nseq k x ++ nseq (n - k) x = nseq (k + (n - k)) x) =>
take k (nseq n x) = nseq k x

```

This top assumption has two hypotheses we will have to deal with later. In the meantime, we can simplify $k + (n - k)$ with some algebra lemmas from `Ring.ZModule`:

```

rewrite addrA addrAC /=.
Type variables: 'a
k: int
n: int
x: 'a
le0_k: 0 <= k
lek_n: k <= n
-----
(0 <= k => 0 <= n - k =>
  nseq k x ++ nseq (n - k) x = nseq n x) =>
take k (nseq n x) = nseq k x

```

Now the top assumption is ready to use as a rewrite rule and those hypotheses become subgoals:

```

move => <- .

```

```

(* Subgoal 1 *)
Type variables: 'a
k: int
n: int
x: 'a
le0_k: 0 <= k
lek_n: k <= n
-----
0 <= k
assumption.
(* Subgoal 2 *)
Type variables: 'a
k: int
n: int
x: 'a
le0_k: 0 <= k
lek_n: k <= n
-----
0 <= n - k
smt ().
(* Subgoal 3 *)
take k (nseq k x ++ nseq (n - k) x) = nseq k x

```

If we go back to the generalization step and use explicit holes for the hypotheses, they become subgoals right away and have to be solved first:

```

idtac <@ (cat_nseq k (n - k) x _ _).
(* Subgoal 1 *)
Type variables: 'a
k: int
n: int
x: 'a
le0_k: 0 <= k
lek_n: k <= n
-----
0 <= k
assumption.
(* Subgoal 2 *)
Type variables: 'a
k: int
n: int
x: 'a
le0_k: 0 <= k
lek_n: k <= n
-----
0 <= n - k
smt ().
(* Subgoal 3 *)
nseq k x ++ nseq (n - k) x = nseq (k + (n - k)) x =>
take k (nseq n x) = nseq k x

```

Now we can do the `algebra` and `rewrite` steps and be exactly where we were before. The rest of the proof is omitted for brevity.

We saw something similar in *Reasoning Forward*, where providing a proof term for a hypothesis excluded it from the match that `apply` was trying to make, but if the proof term was just a hole, the hypothesis became a subgoal. Here, we said any hypotheses without proof terms are just copied into the top assumption, but a hole is enough to trigger making it a subgoal instead. The `rewrite` tactic, on the other hand, always turns hypotheses into subgoals because they are of no use when rewriting.

Instantiating an anonymous lemma is similar to instantiating a named one. In place of the name of the lemma, we use a term of the form

```
(_: b)
```

where `b` is any `bool` expression and the underscore and colon are both required. The underscore may optionally be preceded by `@`, which undoes the effect of the `+implicit` pragma, as explained in *The Apply Tactic with Multiple Arguments*.

As with the `rewrite` and `apply` tactics, a subgoal is created with the same context as the current goal and `b` as the conclusion, and the current goal is replaced with one in which `b` is the new top assumption.

If the anonymous lemma has assumptions, these can be instantiated using a term of the form

```
((_: b) args)
```

(with optional `@`). Everything said above about instantiating proof terms with omitted arguments and holes applies when instantiating anonymous lemmas.

5.12.3 Abstracting an Expression

An expression is abstracted by providing an expression pattern, optionally preceded by an occurrence selector. The proof engine looks for a subexpression in the conclusion of the goal that matches the pattern and instantiates any holes in it. If there are no matches, it displays the message “cannot find an occurrence” and the whole tactic fails. If there are multiple matches, it chooses the first one encountered in a pre-order traversal of the conclusion. It then selects a fresh variable name, makes its type the type of the subexpression, and adds a universal quantifier over it as the new top assumption. Finally, it replaces each occurrence of the subexpression with a reference to the variable; if an occurrence selector is present, it replaces only selected occurrences.

For example, in the following goal, the expression $(x + 1 + y) * x$ has type `int`, so it can be abstracted as an `int` variable:

```
Type variables: <none>
x: int
y: int
-----
2 * ((x + 1 + y) * x) = (x + 1 + y) * x + (x + 1 + y) * x
idtac <@ ((x + 1 + y) * x).
Type variables: <none>
x: int
y: int
-----
```

```
forall (x0 : int), 2 * x0 = x0 + x0
```

Extra parentheses are used to make the expression pattern a single argument. The proof engine chose `x0` as the name of the new variable. All (three) occurrences of the formula are replaced, making it more obvious (to a human, anyway) that the conclusion is true.

Here is an example with holes in the expression pattern:

```
Type variables: <none>
x: int
y: int
-----
2 * ((x + 1 + y) * x) = (x + 1 + y) * x + (x + 1 + y) * x
idtac <@ ((  +   + y) *  ) .
Type variables: <none>
x: int
y: int
-----
forall (x0 : int), 2 * x0 = x0 + x0
```

and one with an occurrence selector:

```
Type variables: <none>
x: int
y: int
-----
2 * ((x + 1 + y) * x) = (x + 1 + y) * x + (x + 1 + y) * x
idtac <@ {1 3} ((x + 1 + y) * x) .
Type variables: <none>
x : int
y : int
-----
forall (x0 : int), 2 * x0 = (x + 1 + y) * x + x0
```

In this case, the result is false (unprovable) because there is no connection between the old variables `x` and `y`, which still occur, and the new one, `x0`. The step must be undone and a new approach tried.

As an edge case, an expression pattern of the form `(x)` where `x` is a variable replaces occurrences of `x` with a new universally quantified variable, say `x0`, but retains the declaration of `x` in the context. If all occurrences of `x` in the conclusion are replaced, the declaration is superfluous. The generalization pattern `x` is treated as a name and reverted, rather than as an expression pattern, unless an occurrence selector forces the latter interpretation.

Abstraction is the only form of generalization that can change the meaning of a goal. As with reverting a local definition without `@`, occurrences of the abstracted expression are discarded, potentially losing information. This says, in essence, “this expression could have any value; its exact form doesn’t matter to this proof; only its type does.” In the above examples, `x` and `y` could take any value, so `(x + 1 + y) * x` could, as well. In the first two examples, the abstracted goal was equivalent to the original, but in the third, where one occurrence of the match was not replaced, the abstracted goal was false. In the extreme, a single underscore always matches the entire conclusion:

```
Type variables: 'a, 'b
```

```

x: 'a
x': 'a
y: 'b
y': 'b
-----
x = x' /\ y = y' => (x, y) = (x', y')
idtac <@ _.
Type variables: 'a, 'b
x: 'a
x': 'a
y: 'b
y': 'b
-----
forall (x0 : bool), x0

```

This conclusion is obviously false. A less extreme example is

```

a: int
b: int
-----
(a + b) * (a + b) = a * a + 2ab + b * b

```

which is true, but if we abstract $a + b$ as a new variable, c , the result is not:

```

a: int
b: int
-----
forall (c : int), c * c = a * a + 2ab + b * b.

```

This is logically sound, since if we *could* prove this statement, the original statement would follow: it is standard backward reasoning. In the extreme, $\text{false} \Rightarrow \phi$ is equivalent to true), but false cannot be proved, so it isn't useful. In less extreme cases, abstraction "strengthens" the goal without making it unprovable, or doesn't change its meaning at all.

Abstracting an expression is often followed by a `case` or `elim` step to destruct the new variable. For that matter, variables in the context are often reverted for the same purpose.

Pro tip: combining instantiation and abstraction. If you want to abstract a subexpression without losing it, here is a way to require the new variable to have the value of the expression. Using the example above, do this:

```

x: int
y: int
-----
2 * ((x + 1 + y) * x) = (x + 1 + y) * x + (x + 1 + y) * x
idtac <@ {-1} ((x + 1 + y) * x) (eq_refl ((x + 1 + y) * x)).

```

Working from the right, the proof term instantiates a copy of the `eq_refl` lemma in the `Logic` theory (which states `forall (x : 'a), x = x`) with $(x + 1 + y) * x$, giving

```

(x + 1 + y) * x = (x + 1 + y) * x =>
2 * ((x + 1 + y) * x) = (x + 1 + y) * x + (x + 1 + y) * x

```

and the expression pattern generalizes $(x + 1 + y) * x$ as the new variable $x0$ *except for its first occurrence*, giving

```
x: int
y: int
-----
forall (x0 : int), (x + 1 + y) * x = x0 => 2 * x0 = x0 + x0
```

Neat! For a “pro pro” tip, see the second special situation in *The Full Syntax of the Case Tactic*.

5.13 The Full Syntax of the Move Tactic

The fundamental purpose of the `move` and `idtac` tactics is to do nothing. These tactics are very useful when all you want to do is apply introduction and/or generalization patterns, which are tacticals rather than tactics. For some reason, however, generalization patterns and one particular introduction pattern have become an integral part of the syntax of the `move` tactic (the `idtac` tactic has remained true to itself).

The full syntax of the `move` tactic is

```
move /p1 ... /pr : {a1 ... am} π1 ... πn
```

where each p_i is a proof term to apply to the top assumption using forward reasoning, exactly like the introduction pattern; each a_i is an assumption in the context to clear; and each π_i is a generalization pattern. All of the arguments are optional but must be in the order shown. If there are no assumptions to clear and no generalization patterns, the colon may also be omitted; otherwise, it is required.

Additional introduction and/or generalization patterns can be appended using the `=>` and `<@` tacticals as usual.

Why do this? The reason lies in the order of execution, which is as follows:

1. Clear $a_1 \dots a_m$ from left to right
2. Generalize $\pi_1 \dots \pi_n$ from right to left
3. Apply $/p_1 \dots /p_r$ from left to right to the top assumption using forward reasoning
4. Do nothing
5. If the tactic is followed by introduction or generalization tacticals, they are applied last

For example, if p is a proof term and π is a generalization pattern, the step

```
move /p : π => ->.
```

is done in this order: (1) generalize π ; (2) apply p to the new top assumption added by π using forward reasoning; (3) use the top assumption (as modified by p) as a left-to-right rewrite rule on the rest of the conclusion and then clear it.

When we first discussed introducing and reverting assumptions way back in *Using the Context*, we didn’t say anything about whether it happened before or after doing nothing, for the excellent reason that one cannot tell the difference anyway. Introduction and generalization patterns have accreted onto other tactics, however, and the order becomes apparent. The execution order above enhances our ability to combine proof steps and manage the proof tree efficiently. It is, however, an acquired taste.

5.14 The Full Syntax of the Case Tactic

The fundamental purpose of the `case` tactic is to destruct the top assumption, as explained in *Case Analysis: The Case Tactic*. For the reasons explained in the previous section, parts of the introduction and generalization tacticals have been incorporated, along with a few idiosyncrasies all its own.

The full syntax of the `case` tactic is

```
case /p1 ... /pr _ : {a1 ... am} π1 ... πn
```

All of the arguments are optional but must be in the order shown. Unlike with `move`, the colon is always optional unless preceded by an underscore, which is itself optional.

Caveat. Note that `case /p` is not at all the same as `elim /L`, in spite of their syntactic similarity. The former applies proof term `p` to the top assumption using forward logic and then does `case` to whatever the new top assumption is, which could have any form `case` can destruct. The latter does `elim` to the top assumption, which must be a universally quantified variable, by applying lemma (not proof term) `L` to the entire goal and simplifying.

The order of execution is as follows:

1. Clear `a1 ... am` from left to right
2. Generalize `π1 ... πn` from right to left
3. Apply `/p1 ... /pr` from left to right to the top assumption using forward reasoning
4. Apply the `case` tactic to the top assumption on each branch of the proof tree
5. If the tactic is followed by introduction or generalization tacticals, they are applied last

So far, so normal, but the `case` tactic also treats expression patterns differently from the generalization tactical in two special situations. First, if the following conditions are met:

1. There are no proof terms (`r = 0`)
2. `π1` is the only generalization pattern (`n = 1`)
3. `π1` is an expression pattern
4. `π1` has no occurrence selector
5. `π1` has no holes
6. `π1` is of type `bool`

then it performs excluded-middle analysis using `π1` and ignores the underscore if it is present. This process was described in *The Case Tactic with One Argument*, but not as an exception to normal generalization pattern processing.

```
y' : int
-----
! x < 0 => x = x' => y = y' => (x, y) = (x', y')
```

Here is an example that illustrates the overlap between a generalization pattern that is a variable name and one that is an expression pattern. Consider this lemma from the `Logic` theory:

```
lemma forall_orl (P : bool) (Q : 'a -> bool) :
  (P \/ (forall (x : 'a), Q x)) <=> forall (x : 'a), (P \/ Q x).
Type variables: 'a
P: bool
```

```

Q: 'a -> bool
-----
(P \/\ forall (x : 'a), Q x <=> forall (x : 'a), P \/\ Q x

```

If we revert P using the move tactic and then apply case, we get

```

move : P.
Type variables: 'a
Q: 'a -> bool
-----
forall (P bool),
  (P \/\ forall (x : 'a), Q x) <=> forall (x : 'a), P \/\ Q x
case.
(* Subgoal 1 *)
Type variables: 'a
Q: 'a -> bool
-----
(true \/\ forall (x : 'a), Q x) <=> forall (x : 'a), true \/\ Q x
admit.
(* Subgoal 2 *)
Type variables: 'a
Q: 'a -> bool
-----
(false \/\ forall (x : 'a), Q x) <=> forall (x : 'a), false \/\ Q x

```

If we combine the steps, we get a different result:

```

case : P.
(* Subgoal 1 *)
Type variables: 'a
P: bool
Q: 'a -> bool
-----
P => (true \/\ forall (x : 'a), Q x) <=>
forall (x : 'a), true \/\ Q x
admit.
(* Subgoal 2 *)
Type variables: 'a
P: bool
Q: 'a -> bool
-----
!P => (false \/\ forall (x : 'a), Q x) <=>
forall (x : 'a), false \/\ Q x

```

The generalization pattern `P` satisfies all the requirements for excluded-middle analysis, so that is what happens: `P` is treated as an expression pattern and is not reverted. Occurrences of `P` in the conclusions of the subgoals have replaced with `true` and `false`, respectively, and top assumptions `P` and `!P` have been added.

But what if we want to revert `P`? If we add an occurrence selector, excluded-middle analysis is prevented but `P` is still treated as an expression pattern and we get a third result:


```

case : {1 2} P.
(* Subgoal 1 *)
Type variables: 'a
P: bool
Q: 'a -> bool
-----
(true \/\ forall (x : 'a), Q x) <=> forall (x : 'a), true \/\ Q x
admit.
(* Subgoal 2 *)
Type variables: 'a
P: bool
Q: 'a -> bool
-----
(false \/\ forall (x : 'a), Q x) <=> forall (x : 'a), false \/\ Q x

```

For this lemma, we can revert `P` if we revert `Q` along with it. This prevents excluded-middle analysis and treats `P` as an assumption name:

```

case : P Q.
(* Subgoal 1 *)
Type variables: 'a
-----
forall (Q : 'a -> bool),
  (true \/\ forall (x : 'a), Q x) <=> forall (x : 'a), true \/\ Q x
admit.
(* Subgoal 2 *)
Type variables: 'a
-----
forall (Q : 'a -> bool),
!P => (false \/\ forall (x : 'a), Q x) <=>
forall (x : 'a), false \/\ Q x

```

In general, though, the best way to force a variable of type `bool` to be reverted is to use the `move` tactic and do `case` after.

The second special situation arises when the conditions of excluded-middle analysis are not met and an underscore precedes the colon: the effect is to replace Π_1 (only) with two generalization patterns: $\{-1\} \Pi_1$ and $(eq_refl \Pi_1)$. This is the pro tip given in *Abstracting an Expression*: it's so great, especially when using the `case` tactic, there's a special syntax for it! Additional generalization patterns may follow Π_1 , but the underscore only affects Π_1 .

The underscore is only effective if Π_1 is an expression pattern. If it is a proof term, the underscore is silently ignored and Π_1 is processed normally. If Π_1 has an occurrence selector, then the selector $\{-1\}$ in the replacement pattern is adjusted accordingly.

5.15 The Full Syntax of the `Elim` tactic

The fundamental purpose of the `elim` tactic is to destruct the top assumption, as explained in *Proof by Induction: The Elim Tactic*. It has a lot in common with the `case` tactic and just enough differences to be confusing. Unlike the `case` tactic, it has accreted no part of the introduction tactical and the generalization tactical has been incorporated with no changes.

The full syntax of the `elim` tactic is

`elim /L : {a1 ... am}  $\Pi_1$  ...  $\Pi_n$`

All of the arguments are optional but must be in the order shown. Like the `case` tactic, the colon is always optional.

The order of execution is as follows:

1. Clear `a1 ... am` from left to right
2. Generalize `$\Pi_1$  ...  $\Pi_n$` from right to left
3. Apply the `elim` tactic to the current goal
4. If the tactic is followed by introduction or generalization tacticals, they are applied last

5.16 The Full Syntax of the Apply and Exact Tactics

The fundamental purpose of the `apply` tactic is two-fold, as explained in *Reasoning Backward*:

- to match the conclusion of a lemma with the conclusion of the current goal and replace it with the assumptions of the lemma (backward reasoning); or
- to match the hypotheses of a lemma with hypotheses in the context or top assumptions of the current goal and replace them with the conclusion of the lemma (forward reasoning)

As we also said, the `exact` tactic is equivalent to `by apply`, so the fundamental purpose of the `exact` tactic is to `apply` a lemma using backward reasoning and then apply the `done` tactic to each subgoal and fail if any remain unsolved. Forward reasoning never solves the goal, so the `exact` tactic doesn't do that.

The generalization tactical, but not the introduction tactical, has been incorporated into both tactics. The `apply` tactic has the following general forms:

```
apply.  
apply p.  
apply p in H.  
apply : p {a1 ... am}  $\Pi_1$  ...  $\Pi_n$ .  
apply : p {a1 ... am}  $\Pi_1$  ...  $\Pi_n$  in H.  
apply /p1 ... /pr.
```

The `exact` tactic has the same forms except for those with the `in H` option:

```
exact.  
exact p.  
exact : p {a1 ... am}  $\Pi_1$  ...  $\Pi_n$ .  
exact /p1 ... /pr.
```

If exactly one proof term, `p`, is to be applied, then axioms to clear and generalization patterns, and `in H` for the `apply` tactic, are all allowed, whereas if no proof term or multiple proof terms are to be applied, then they are not allowed. The colon is required when axioms to clear and/or generalization patterns are present but is idiosyncratically placed immediately following `apply` or `exact`, inviting confusion that `p` is a generalization pattern. Axioms to clear and generalization patterns are both optional even when the colon is present, so the following forms are legal:

```
apply : p
```

```

apply : p in H
exact : p

```

Speaking of confusion, keep in mind that the forms with multiple proof terms have nothing to do with the introduction pattern they so closely resemble.

The order of execution for the `apply` tactic is as follows:

1. Clear $a_1 \dots a_m$ from left to right
2. Generalize $\Pi_1 \dots \Pi_n$ from right to left
3. Apply the `apply` tactic to the current goal
4. If the tactic is followed by introduction or generalization tacticals, they are applied last

When we say the `exact` tactic is equivalent to `by apply`, that includes the lower precedence of the `by` tactical, so for example,

```
exact : p  $\Pi_1 \Rightarrow$   $\iota$ ;  $\tau_2 <@ \Pi_2$ ;  $\tau_3$ 
```

is equivalent to

```
by (apply : p  $\Pi_1 \Rightarrow$   $\iota$ ;  $\tau_2 <@ \Pi_2$ ;  $\tau_3$ )
```

and therefore to

```
(( (apply : p  $\Pi_1 \Rightarrow$   $\iota$ ); ( $\tau_2 <@ \Pi_2$ ));  $\tau_3$ ); done
```

5.17 The Full Syntax of the Rewrite Tactic

The fundamental purpose of the `rewrite` tactic is two-fold

- To rewrite expressions
- To fold and unfold operators and local definitions

Even more than other tactics, however, the `rewrite` tactic has incorporated a variety of introduction patterns and other features in an attempt to “keep the party going” and cram as much into one proof step as possible. A few of these are mentioned here for the first time.

The full syntax of the `rewrite` tactic consists of the general forms,

```

rewrite  $\rho_1 \dots \rho_n$ .
rewrite  $\rho_1 \dots \rho_n$  in H.

```

where each ρ_i is a **rewrite argument**. An argument can have the following forms:

- p – a proof term, or a list of proof terms separated by commas and enclosed in parentheses
 - Optionally preceded by $[e]$, where e is a match expression pattern
 - If a list is provided, proof terms are tried in order until one works
 - p can also be the name of a database of rewrite hints, which is a list of lemmas that are tried in order until one works
- $\& p$ – a proof term to be applied (i.e., with the `apply` tactic) rather than used to rewrite. If p is an identifier, then $\&p$ is a memory identifier (see *Reasoning about Memories and Modules*) and a space between them is required for this usage. Otherwise, it is optional. Or use $\&(p)$
- $/e$ – an expression pattern, to fold or unfold its head operator

- Any of these introduction patterns `//`, `/=`, `/~=`, `//=`, `//~=`, `/#` or `//#`
- Any of these smt patterns: `#smt`, `#smt:[smt_info]`, `#smt:(dbhint)`, where `dbhint` is a list of wanted lemmas
- Any of these algebra patterns: `#ring`, `#field`

Any pattern can be preceded by a focus index. The forms `p` and `/e` may be preceded by one or more of the following options, in the order listed and following the focus index, if present:

- Direction indicator
- Repetition marker
- Occurrence selector

Pro tip. The fact that any pattern can be applied in `H` means you can apply `simplify` to a hypothesis:

```
rewrite /= in H.
```

It doesn't make sense to apply the other introduction patterns (e.g., `//` or `/#`) this way, because what happens in `H` stays in `H`—it doesn't affect the conclusion, so it can't solve the goal, so at best it has no effect (`trivial`) and at worst it makes the whole step fail (`smt`).

5.18 Generalization Tactics

Generalization tactics are used to add an assumption (a.k.a. a *cut*) to the current goal. They add support for various special cases that make them more convenient than the generalization tactical in some circumstances.

5.18.1 The Have Tactic

The `have` tactic is used to add a lemma to the current goal and immediately introduce it. It has two modes. The first mode adds an anonymous lemma and is intended for when the lemma is easy to prove, minimizing the interruption to the main proof. The second mode adds an existing lemma using a proof term.

The general forms for the first mode are

```
have : φ.
have l1 ... ln : φ.
have : φ by τ.
have l1 ... ln : φ by τ.
```

where ϕ is the anonymous lemma, the l_i are introduction patterns and τ is a tactic chain. The lemma is a `bool` expression without holes and it can refer to any of the assumptions in the context. This form generates two subgoals: one to prove that the lemma is true, given the context, and one to prove the original goal with the new assumption added as its top assumption. Additionally, the tactic lets you specify introduction patterns to apply to the second subgoal (e.g., to introduce the new assumption) and a chain of tactics to apply to the first subgoal. This chain, if present, must solve the first subgoal or the tactic fails.

As an example, we can revisit the proof of the `exp4` lemma.

```
lemma exp4 : 2 ^ 4 = 16.
-----
```

```

2 ^ 4 = 16
have : 4 = 2 + 2.
(* Subgoal 1 *)
-----
4 = 2 + 2
done.
(* Subgoal 2 *)
-----
4 = 2 + 2 => 2 ^ 4 = 16

```

This shows that the `have` tactic in its simplest form is equivalent to using the generalization tactic:

```

idtac <@ (_ : 4 = 2 + 2).
(* Same two subgoals *)

```

Instead, we can use the optional parts of the `have` tactic to prove the anonymous lemma right away and get a head start on the second:

```

lemma exp4 : 2 ^ 4 = 16.
-----
2 ^ 4 = 16
have -> : 4 = 2 + 2 by trivial.
-----
2 ^ (4 + 4) = 256

```

Here `trivial` solved the first subgoal and `->` introduced the top assumption in the second subgoal by using it as a rewrite rule in the rest of the conclusion and then clearing it. The rest of the proof can be shortened by using the full syntax of the `rewrite` tactic:

```

by rewrite exprD_nneg // exp2.
No more goals

```

Here is a proof that provides a nice example of the `pose`, `have` and `exists` tactics.

```

lemma splitPr (xs : 'a list) (x : 'a) :
  mem xs x => exists s1, exists s2, xs = s1 ++ x :: s2.
Type variables: 'a
xs: 'a list
x: 'a
-----
x \in s => exists (s1 s2 : 'a list), xs = s1 ++ x :: s2
move => x_in_xs.
Type variables: 'a
xs: 'a list
x: 'a
x_in_s: x \in s
-----
exists (s1 s2 : 'a list), xs = s1 ++ x :: s2

```

The lemma claims that if `x` is a member of `xs`, then one can decompose `xs` into the elements before `x`, which we'll call `s1`; `x` itself; and the elements after `x`, which we'll call `s2`. If `x` occurs in `xs` many times, there will be many ways to do this, but certainly one way is to break `xs` at its first occurrence,

`index s x`. Let's pose a local definition for that index and then prove a useful fact about it as an assumption:

```

pose i := index x xs.
Type variables: 'a
xs: 'a list
x: 'a
x_in_s: x \in s
i: int := index x xs
-----
exists (s1 s2 : 'a list), xs = s1 ++ x :: s2
have lt_ip : i < size xs by rewrite /i index_mem.
Type variables: 'a
xs: 'a list
x: 'a
x_in_s: x \in s
i: int := index x xs
lt_ip: i < size xs
-----
exists (s1 s2 : 'a list), xs = s1 ++ x :: s2

```

The `have` step added the assumption `i < size xs`, proved it by unfolding `i` and rewriting it with the `index_mem` lemma (with a silent assist by `x_in_s`), and then moved it to the context as `lt_ip`. Now we are ready to assert our witnesses to the existential quantifiers. The tactic will accept both witnesses in one step, but let's do it one at a time this once:

```

exists (take i xs).
Type variables: 'a
xs: 'a list
x: 'a
x_in_s: x \in s
i: int := index x xs
lt_ip: i < size xs
-----
exists (s2 : 'a list), xs = take i xs ++ x :: s2
exists (drop (i+1) xs).
Type variables: 'a
xs: 'a list
x: 'a
x_in_s: x \in s
i: int := index x xs
lt_ip: i < size xs
-----
xs = take i xs ++ x :: drop (i+1) xs

```

All that remains is to prove that our witnesses do in fact satisfy the conditions of the lemma. If you want to understand the details, you can break down these steps and look up the lemmas they use.

```

rewrite -{1} (cat_take_drop i) - (nth_index x x xs) //.
Type variables: 'a
xs: 'a list

```

```

x: 'a
x_in_s: x \in s
i: int := index x xs
lt_ip: i < size xs
-----
take i xs ++ drop i xs =
take i xs ++ nth x xs (index xs) :: drop (i+1) xs
by rewrite -drop_nth // index_ge0 lt_ip.
No more goals

```

As a final example of the first mode, here is a proof where the anonymous lemma has variables of its own. This is one of the many induction lemmas for integers:

```

lemma intwlog (p : int -> bool) :
  (forall (i : int), p (-i) => p i) =>
  p 0 =>
  (forall (i : int), 0 <= i => p i => p (i + 1)) =>
  forall (i : int), p i.
p: int -> bool
-----
(forall (i : int), p (-i) => p i) =>
p 0 =>
(forall (i : int), 0 <= i => p i => p (i + 1)) =>
forall (i : int), p i
move => ihn ih0 ihS.
p: int -> bool
ihn: (forall (i : int), p (-i) => p i) =>
ih0: p 0 =>
ihS: (forall (i : int), 0 <= i => p i => p (i + 1)) =>
-----
forall (i : int), p i

```

The hypotheses `ih0` and `ihS` match the hypotheses of the usual induction lemma for natural numbers, `intind`, which will be useful in applying the `ihn` hypothesis. We can exploit this fact by supposing the conclusion, that `p i` holds for all natural numbers, and then eliminating the variable `i` using `intind`:

```

have : forall i, 0 <= i => p i.
(* Subgoal 1 *)
p: int -> bool
ihn: (forall (i : int), p (-i) => p i) =>
ih0: p 0 =>
ihS: (forall (i : int), 0 <= i => p i => p (i + 1)) =>
-----
forall (i : int), 0 <= i => p i
by elim /intind.
(* Subgoal 2 *)
p: int -> bool
ihn: (forall (i : int), p (-i) => p i) =>
ih0: p 0 =>
ihS: (forall (i : int), 0 <= i => p i => p (i + 1)) =>
-----

```

```
(forall (i : int), 0 <= i => p i) => forall (i : int), p i
```

From here, SMT can finish the proof:

```
smt ().
No more goals
```

We can combine the last three steps into one:

```
p: int -> bool
ihn: (forall (i : int), p (-i) => p i) =>
ih0: p 0 =>
ihS: (forall (i : int), 0 <= i => p i => p (i + 1)) =>
-----
forall (i : int), p i
have /# : forall i, 0 <= i => p i by elim /intind.
No more goals
```

The general forms for the second mode are

```
have := p.
have l1 ... ln := p.
```

where the l_i are again introduction patterns and p is a proof term for a named lemma (not an anonymous one). The proof term need not have its outer parentheses. This mode generates only one subgoal, that the lemma implies the goal's conclusion. If the lemma has hypotheses, these will be part of the new subgoal unless the proof term either proves them or has placeholders (holes) for them; in the latter case, they become additional subgoals (see *Instantiating a Lemma*).

The first form is synonymous with using the generalization tactical with the proof term, i.e., with

```
idtac <@ (p).
```

The second form does the same thing and then applies the introduction patterns, but only to the last subgoal, that the lemma (or its conclusion) implies the original goal's conclusion, and not to any other subgoals generated by p . It is thus nearly synonymous with

```
idtac <@ (p) => l1 ... ln.
```

except that this applies $l_1 \dots l_n$ to all the subgoals generated, which probably wouldn't work.

For example, here is a lemma from the `List` theory and the first part of its proof:

```
lemma index_nth ['a] (x0 : 'a) (s : 'a list) (i : int) :
  0 <= i < size s => index (nth x0 s i) s <= i.
Type variables: 'a
x0: 'a
s: 'a list
i: int
-----
0 <= i < size s => index (nth x0 s i) s <= i
case => ge0_i; apply contraLR; rewrite -lezNgt -ltzNge => lt.
Type variables: 'a
x0: 'a
```



```

s: 'a list
i: int
ge0_i: 0 <= i
lt: i < index (nth x0 s i) s
-----
size s <= i

```

The next step uses a proof term for the `before_index` lemma (which was also used in *Reasoning Forward*). The first subgoal is the hypothesis of `before_index` because the proof term provides a hole for it:

```

have := before_index x0 (nth x0 s i) s i _.
(* Subgoal 1 *)
Type variables: 'a
x0: 'a
s: 'a list
i: int
ge0_i: 0 <= i
lt: i < index (nth x0 s i) s
-----
0 <= i < index (nth x0 s i) s

```

It is proved by the assumptions `ge0_i` and `lt`, but, unfortunately, we can't pack both into the placeholder in the proof term, so we use `done`. The second subgoal is that the lemma's conclusion implies the original goal's conclusion:

```

done.
(* Subgoal 2 *)
Type variables: 'a
x0: 'a
s: 'a list
i: int
ge0_i: 0 <= i
lt: i < index (nth x0 s i) s
-----
nth x0 s i <> nth x0 s i => size s <= i

```

The antecedent is false, so this is true

```

done.
No more goals

```

It turns out that we *can* solve the first subgoal in the proof term, namely by using `//` as its argument! Moreover, we can solve the second by introducing it with `//`:

```

have // := before_index x0 (nth x0 s i) s i //.
No more goals

```

Pro tip. The `have` tactic can isolate an expression that `simplify` can't handle and ship it to the `smt` tactic to simplify:

```

have -> : complex_expr = simple_expr by smt ().

```

This can also be done by the `rewrite` tactic using the introduction pattern for `smt`:

```
rewrite ... (: complex_expr = simple_expr) 1:/# ....
```

5.18.2 The Suff Tactic

The `suff` tactic is used to add an anonymous lemma to the current goal and is intended for when the lemma makes proving the current goal easy. It is nearly the same as the first mode of the `have` tactic except it generates the subgoals in the opposite order, so the sub-proof is the hopefully easy one:

```
lemma exp4 : 2 ^ 4 = 16.
-----
2 ^ 4 = 16
suff : 4 = 2 + 2.
(* Subgoal 1 *)
-----
4 = 2 + 2 => 2 ^ 4 = 16
admit.
(* Subgoal 2 *)
-----
4 = 2 + 2
```

Introduction patterns before the colon, if present, are applied to the first subgoal (that the lemma implies the conclusion). The `τ` tactic chain, if present, is also applied to the first subgoal. If `τ` does not solve the subgoal, the tactic fails. In this example, the first subgoal is the harder of the two but we can still solve it in one step:

```
lemma exp32 : 2 ^ 32 = 4294967296.
-----
2 ^ 32 = 4294967296
suff -> : 32 = 16 + 16 by rewrite exprD_nneg // expr16.
-----
32 = 16 + 16
done.
No more goals
```

The `suff` tactic does not support the `:=` forms of the `have` tactic because its proof term is for the wrong subgoal.

5.18.3 The Gen Have Tactic

The `gen have` tactic is like `have` but adds the option of generalizing the anonymous lemma by reverting assumptions in the context into it. It also assumes that you want to introduce the generalized lemma into the context of the second subgoal. Its general forms are

```
gen have name : / φ.
gen have name , l1 ... lm : / φ.
gen have name : name1 ... namen / φ.
gen have name , l1 ... lm : name1 ... namen / φ.
```

where `name` is a name not appearing in the context, `l1 ... lm` are introduction patterns, `name1 ... namen` are names of assumptions in the context and `φ` is a `bool` expression without holes. As with any

reversion, you can prefix the name of a local definition with @ to preserve its definition. Only the third and fourth forms generalize φ , so they are the most common.

In all forms, the first subgoal is

```
context
-----
 $\varphi$ 
```

where `context` is the context of the current goal.

The second subgoal depends on the form. In the first form, it is

```
context
name:  $\varphi$ 
-----
G
```

In the second form, the second subgoal is

```
context
name:  $\varphi$ 
-----
 $\varphi \Rightarrow G$ 
```

as modified by applying the introduction patterns $\iota_1 \dots \iota_m$. Note that φ appears twice, as a hypothesis and as the top assumption, where the introduction patterns can act on it. This happens even if there are no introduction patterns, as long as the comma is present.

In the third form, the second subgoal is

```
context
name: generalized  $\varphi$ 
-----
G
```

where φ in the context has been generalized by taking the first subgoal and reverting `name1 ... namen`.

In the fourth form, the second subgoal is

```
context
name: generalized  $\varphi$ 
-----
 $\varphi \Rightarrow G$ 
```

as modified by applying the introduction patterns $\iota_1 \dots \iota_m$, where φ in the context has been generalized but φ in the conclusion has not.

For example, consider this abstract predicate and a lemma about it:

```
const P : int -> bool.
lemma simple (n : int) : 0 < n => P n.
n: int
-----
```

```

0 < n => P n
move => ng0.
n: int
ng0: 0 < n
-----
P n

```

Let's add `0 <= n && n <> 0` as an anonymous lemma to observe the effects of each form on the second subgoal. The first subgoal is always this:

```

n: int
ng0: 0 < n
-----
0 <= n && n <> 0

```

Form 1 second subgoal:

```

gen have ltnV : / 0 <= n && n <> 0.
n: int
ng0: 0 < n
ltnV: 0 <= n && n <> 0
-----
P n

```

Form 2 second subgoal:

```

gen have ltnV, /andaE [nge0 neq0] : / 0 <= n && n <> 0.
n: int
ng0: 0 < n
ltnV: 0 <= n && n <> 0
nge0: 0 <= n
neq0: n <> 0
-----
P n

```

Here the `andaE` lemma was applied to the top assumption of the conclusion to replace `&&` with `/\`, then the `case` tactic was applied to destruct `/\` and the two resulting assumptions were moved to the context.

Form 3 second subgoal:

```

gen have ltnV : n ng0 / 0 <= n && n <> 0.
n: int
ng0: 0 < n
ltnV: forall (n0 : int), 0 < n0 => 0 <= n0 && n0 <> 0
-----
P n

```

Here the anonymous lemma was generalized by `n` and `ng0`. Note that it cannot be generalized only by `n` because `ng0` refers to it. Also note that `n` and `ng0` remain in the context of the second subgoal.

Form 4 second subgoal:

```

gen have ltnV, /andaE [nge0 neq0] : n ngto / 0 <= n && n <> 0.
n: int
ngto: 0 < n
ltnV: forall (n0 : int), 0 < n0 => 0 <= n0 && n0 <> 0
nge0: 0 <= n
neq0: n <> 0
-----
P n

```

Here we see clearly that the introduction patterns were applied to the ungeneralized lemma.

5.18.4 The Wlog Tactic

The `wlog` tactic reduces a goal to a special case that is sufficient to prove the more general case. The name comes from the phrase, “Without loss of generality, suppose ...” Given an assumption ϕ , `wlog` performs a cut on $\phi \Rightarrow G$ rather than just on ϕ . That is, the subgoals it generates are $\phi \Rightarrow G$ (does the added assumption imply the goal) and $(\phi \Rightarrow G) \Rightarrow G$ (does the special case (the first subgoal) imply the general case). In addition, it is possible to generalize $\phi \Rightarrow G$ over one or more of the assumptions in the context when proving $(\phi \Rightarrow G) \Rightarrow G$. Its general forms are

```

wlog : /  $\phi$ .
wlog : name1 ... namek /  $\phi$ .

```

where `namei` is the name of an assumption in the context and ϕ is a `bool` expression without holes. As always, prefix the name of a local definition with `@` to preserve its definition.

As an example, consider the `mulz_gcdr` lemma from the `IntDiv` theory:

```

lemma mulz_gcdr a b c : `|a| * gcd b c = gcd (a * b) (a * c).
a: int
b: int
c: int
-----
`|a| * gcd b c = gcd (a * b) (a * c)

```

We can assume, without loss of generality, that $0 \leq a$. That is, we can prove the theorem with this added assumption and show how to relax the assumption to prove the general case. The proof starts like this:

```

wlog : a / (0 <= a).
(* Subgoal 1 *)
a: int
b: int
c: int
-----
(forall (a0 : int),
  0 <= a0 => `|a0| * gcd b c = gcd (a0 * b) (a0 * c)) =>
`|a| * gcd b c = gcd (a * b) (a * c)

```

This subgoal corresponds to $(\phi \Rightarrow G) \Rightarrow G$, that the special case implies the general case (that no generality was in fact lost). The special case has been generalized over `a`, giving `(forall a0,  $\phi(a0) \Rightarrow G(a0)$ )  $\Rightarrow G$` . The next thing we want to do is move the top assumption to the context

and then do a case analysis on the exact condition we are claiming does not lose generality. The first case is trivial, since it is what we are assuming.

```

move => H.
a: int
b: int
c: int
H: (forall (a0 : int),
    0 <= a0 => `|a0| * gcd b c = gcd (a0 * b) (a0 * c))
-----
`|a| * gcd b c = gcd (a * b) (a * c)
case (0 <= a) .
a: int
b: int
c: int
H: (forall (a0 : int),
    0 <= a0 => `|a0| * gcd b c = gcd (a0 * b) (a0 * c))
-----
0 <= a => `|a| * gcd b c = gcd (a * b) (a * c)
apply H.
a: int
b: int
c: int
H: (forall (a0 : int),
    0 <= a0 => `|a0| * gcd b c = gcd (a0 * b) (a0 * c))
-----
! 0 <= a => `|a| * gcd b c = gcd (a * b) (a * c)

```

For the `! 0 <= a` case, we want to apply `H` instantiated with `-a`. One can mess about with lemmas about integers or just let the `smt` tactic do it. The lemmas it needs can be found by first using `smt` without arguments, then `smt (@IntDiv)` and narrowing down from there.

```

smt (gcdNz gcdzN) .
(* Subgoal 2 *)
a: int
b: int
c: int
-----
0 <= a => `|a| * gcd b c = gcd (a * b) (a * c)

```

This is the second subgoal of the `wlog` tactic, namely the special case we want to prove. This is a complex proof but does not further our discussion of `wlog`, so it is omitted.

The `wlog suff` tactic treats ϕ as an anonymous lemma to be proved rather than as the added assumption of a special case. It generates the same subgoals as `gen have` in reverse order and with no introduction patterns or even a name for the generalized lemma.

6 Programs and Program Logics

6.1 Modules

A **module** is a unit of computational capability. The most important thing for a programmer to remember about modules is that they are *not* types (or classes): they don't have instances or values, and a variable, including a parameter of an operator (above) or a procedure (below), cannot have a module as its type (there's an exception for formal variables; see *Reasoning about Memories and Modules*). A module is an island where global variables and procedures live, surrounded by a sea of logic. Think of C, or even Fortran.

Module names must begin with an uppercase letter. Here is an empty module:

```
module M0 = {}.
```

A module consists of a sequence of zero or more **global variable declarations**, **procedures** and **sub-modules**, with no punctuation to separate or terminate them. A global variable refers to a storage location in memory that holds a value and the value held can be changed (see the assignment statements, below). A procedure has a sequence of statements that may manipulate (change) the values stored by global variables as side effects. This may seem obvious to a programmer but needs to be said in a proof system *about* programs. A sub-module is simply a nested module.

Global variable and procedure names must begin with a lowercase letter or an underscore. Here is a module with one global variable declaration and one procedure declaration:

```
module M1 = {  
  var x : int  
  proc one() : unit = {}  
}.
```

A global variable declaration must specify a type and cannot specify an initial value—it is initialized to the *witness* value of the type. A global variable declaration declares exactly one variable; there is no shorthand for declaring multiple global variables of the same type.

A procedure has a function type that is specified using the abbreviated syntax used for operators:

```
proc g(n : int, m : int, b : bool) : bool = { ... }  
proc g(n m : int, b : bool) : bool = { ... }
```

Unlike an operator declaration, a procedure must have a parameter list, even if it is empty, as illustrated by `one()` in module `M1`. As pointed out earlier, an empty parameter list is implicitly a single parameter of type `unit`. Similarly, a procedure cannot return “nothing”—the closest is to return `unit` (akin to `void` in other languages but more correct), also as illustrated by `one()` in module `M1`. The parameter and result types can often be inferred from the procedure's body and may then be omitted.

The body of a procedure consists of a list of **local variable declarations** followed by a list of **statements**, both of which are terminated by semicolons. Local variable names must begin with a lowercase letter or an underscore. Multiple local variable declarations with the same initial value can be combined, as in:

```
var x, y, z : int;  
var x, y, z : int <- 10; (* initializing all 3 to 10 *)  
var x, y, z <- 10;      (* ditto *)
```

Types are optional and will be inferred from how the variables are used or remain underspecified. Uninitialized variables have unknown values. A procedure's parameters and local variables must have distinct variable names (they share a namespace). Like global variables, local variables and procedure parameters are *mutable*: they hold values and may be modified during the execution of procedures. These changes are not visible outside the procedure but can be reasoned about while inside it.

A module may be nested in another module and may appear anywhere in the sequence of global variables and procedures. A nested module won't have a period after its closing brace, as it is not a step. The contents of a nested module are globally accessible by using a nested qualified name.

A module can be declared simply as an alias for another module. Why might you do this? Let's say a theory `T` defines a module `M`, a theory `U` also defines a module `M`, and you want to avoid using the qualified name `T.M` in `U`. Importing won't help because then `M` would be ambiguous. Instead, you can give `T.M` a local name in `U`:

```
require T.  
module N = T.M.
```

Similarly, a procedure can be declared as a local alias for another procedure.

```
module M2 = {  
  proc f = M.init;  
}
```

An alias can also provide a name for a parameterized module with an argument, as will be shown later.

6.1.1 Statements

The kinds of **statements** are:

- Assignment of a value (i.e., a typed expression) to a variable:

```
x <- 0;
```

Pro tip. Multiple parallel assignments can be combined using tuples:

```
(x, y) <- (y, x);
```

Note that you cannot assign a value to a field of a tuple or record:

```
x.`1 <- 15; (* Not legal *)
```

This is because `.`` is an operator, so `x.`1` is an expression rather than a variable.

On the other hand, a special exception is made for the mixfix operators `"_.[<-_]"` and `"_.[_]"`. These are typically used to associate a value with a key in a map or an index in an array. For example, finite maps are defined in the library theory `FMap`, where the operator `m.[x]` returns the value that `m` associates with the key `x` and the operator `m.[x <- y]` returns the map that is equal to `m` except `y` is associated with `x`. As a shorthand for

```
m <- m.[x <- y];
```

the EasyCrypt parser also accepts

```
m.[x] <- y;
```


This is the only case where an expression can be the left side of an assignment.

- Assignment of a value chosen at random from a (sub-)distribution to a variable, tuple or map key:

```
require import DInterval. (* somewhere previously *)
x <$ [5..10];             (* interval from 5 to 10, inclusive *)
```

See *Probability Distributions* for more information on distributions provided in the EasyCrypt library.

- Assignment of a procedure call return value to a variable, tuple or map key:

```
n <@ get();
```

Procedure call syntax is different from function or operator application in that parentheses are required, even for zero arguments.

A procedure call cannot be recursive, directly or indirectly. A procedure can only call procedures defined previously, either in the same module or an earlier module.

If the return value is not needed, it does not have to be assigned to a variable:

```
get();
```

This is typical when the procedure returns `unit`.

If a procedure returns a tuple, it can be assigned to a variable of tuple type or to a tuple of variables:

```
(* coordinates & pair are defined in Types, Functions *)
proc rect2polar(c : coordinates) : real pair = { ... }
proc f(c : coordinates) : real = {
  var p : real pair;
  p <@ rect2polar(c);           (* assign a tuple var *)
  var r, theta : real;
  (r, theta) <@ rect2polar(c); (* assign a tuple of vars *)
}
```

VERY IMPORTANT! A procedure call cannot be used *in any other way*! In particular, it cannot be part of an expression. The following are FORBIDDEN:

```
n <@ get() + 1;           (* not allowed *)
return get();             (* not allowed *)
```

Programmers, you have been warned! *When you forget*, you will see an error message like this:

```
[critical] [...] no matching operator, named 'M.get', for the
following parameters' type:
[1]: unit
```

The key point is that EasyCrypt is looking for an *operator* from the ambient logic (and not finding one) because you tried to call a *procedure* in an *expression*. Bad programmer! The only thing

worse would be if there actually *was* a matching operator. (Also note that the seemingly parameterless `get()` does in fact have a parameter, of type `unit`. Told you!)

- Conditional (if-then-else) statement:

```
if (b) { <statements> } [ else { <statements> } ]
```

If a branch consists of a single statement, the braces may be omitted from that branch. If the `else` branch consists of a conditional statement, “`else if`” may be abbreviated “`elif`”. Note, however, that the `print` command does not use `elif` and omits unnecessary braces somewhat conservatively.

An alternative syntax for the condition is

```
if (e is pattern) ...
```

where `e` is an expression of an inductive type and the pattern is for one of its constructors. For example, we can declare a global variable

```
var age : int
```

and define a procedure with a parameter of type `int option`:

```
proc set_age (age_opt : int option) = {  
  if (age_opt is Some n) {  
    age <- n;  
  } else {  
    age <- 0;  
  }  
}
```

This syntax is converted to a `match` statement (see next bullet).

- Match statement:

```
match e with  
  pattern1 => { statements1 }  
  | ...  
  | patternN => { statementsN }  
end;
```

where `e` is an expression of an inductive type and each pattern covers one of its constructors. The first pattern may optionally be preceded by a `|`. If a statement block consists of a single statement, the braces around it may be omitted. This is analogous to a match expression in the ambient logic.

For example, we can redo `set_age` from above as

```
proc set_age (age_opt : int option) = {  
  match age_opt with  
  | None => {  
    age <- 0;  
  }  
  | Some n => age <- n;  
}
```

```

    end;
}

```

As it happens, this is exactly how EasyCrypt interprets the `if` version anyway.

- While loop:

```
while (b) { <statements> }
```

If the body consists of a single statement, the braces may be omitted.

- Return statement:

```
return x > 0 /\ y > 0;
```

The return value can be any typed expression (but *not* a procedure call!). If a procedure returns `unit`, a return statement is not required—everyone knows it returns `tt`.

A return statement may only appear at the end of a procedure—multiple return statements (e.g., in loops and conditionals) and dead code following a return statement are not allowed.

- Assertion:

```
assert x > 0 /\ y > 0;
```

The argument is an expression of type `bool`. In a proof about the program, the expression must be shown to be true.

Note that structured programming is strictly enforced: there are no `goto`, `break` or `continue` statements.

Here is an example that illustrates most of the module language:

```

require import Int DInterval.
module M = {
  var x : int                (* global variable declaration *)
  proc init(bnd : int) : unit = {
    x <$ [-bnd .. bnd];      (* random assignment; see Dinterval *)
  }
  proc incr(n : int) : unit = {
    x <- x + n;              (* assignment *)
  }
  proc get() : int = {
    return x;                (* return statement *)
  }
  proc loop() : int = {
    var y, z : int;          (* local variable declarations *)
    y <- 0;
    while (y < 10) {         (* while loop *)
      z <$ [1 .. 10];
      if (z <= 3) {          (* conditional statement *)
        y <- y - z;
      } elif (z <= 6) {      (* shorthand for [else if] *)
        y <- y + 1;
      }
    }
  }
}

```

```

        } else {
            y <- y + (z - 5);
        }
    }
    return y;
}
proc main() : bool = {
    var n : int;
    init(100);                (* procedure call w/no return value *)
    incr(10);
    incr(-50);
    n <@ get();                (* procedure call assignment *)
    return n < 0;              (* NOT: return get() < 0; *)
}
}.

```

The variables and procedures of a module are accessed using qualified names consisting of the module name, a period, and the variable or procedure name (all preceded by one or more theory names, if needed).

```

module N = {
    var x : int
    var b : bool
    proc g(n m : int, b : bool) : bool = {
        M.init(n);
        M.x <- M.x + x + n - m;
        x <- x + 1;
        N.b <@ M.main();
        return b = N.b;
    }
}
}.

```

A module's procedures may refer to global variables and procedures in the same module using qualified names (e.g., if a local variable has the same name as a global one) but need not.

6.1.2 Code Positions

Each statement in a procedure is identified by its **code position** in a hierarchical fashion, reflecting the nested block structure of the program:

- Local variables are ignored
- Top-level statements are numbered sequentially starting at 1
- A `return` statement does not have a code position
- Statements in the `then` part of an `if` statement start at $n.1$, where n is the code position of the parent statement; statements in the `else` part of an `if` statement start at $n?1$
- Statements in the body of a `while` statement start at $n.1$, where n is the code position of the parent statement
- Each branch of a `match` statement is identified by its constructor, starting at $\#c_i.1$.

For example, in the `M.loop` procedure above, code position 2 is the entire `while` loop, while code position 2.1 assigns a randomly sampled value to `z`. Code position is independent of code layout, such as how many lines a statement spans or how many statements are on one line.

When code is displayed by a proof tactic, it displays a code position for each line and the code is stylized this way:

- Each statement starts on a new line
- Semicolons and `end` are omitted
- The `return` statement is omitted
- `else` appears on a line by itself
- The keyword `elif` is changed to `else if` (shown on separate lines)
- Each branch of a `match` statement starts on a new line
- If statements that use `is` and a pattern are changed to `match` statements

Procedure modifications (below) and many program logic proof tactics take code positions as arguments. A user has more ways to specify them at his or her disposal:

- In place of a positive integer, you can use a negative integer to count from the end of a block of statements
 - `-5` is the fifth statement from the end of the top-level block
 - `3?-2` is the second statement from the end of the `else` branch of code position 3 from the start of the procedure
- Instead of a number, you can specify a kind of statement
 - `^if`, `^while`, `^match` matches the first `if`, `while` or `match` statement in the block
 - `^<-`, `^<$`, `^<@`, `^()<-` matches the first assignment statement in the block (to a tuple in the last case)
 - `^x<-`, `^x<$`, `^x<@`, `^(x)<-` matches the first assignment statement to variable `x` (to a tuple containing `x` in the last case)
- A code position can be followed by a signed offset within the same code block (the space after `&` is required)
 - `5 & +2` is the second statement after the fifth one, which is the same as 7
 - `^while & -1` is the statement before the first `while` loop

All of these define one level of a code position and can be followed by `.`, `?` or `#Ci.` and another level.

For example, in the `M.loop` procedure above, code position `2.1` can also be written `-1.-2`, `^while.1`, `^while.^if & -1` and so on.

6.1.3 Glob

In reasoning about programs, it may be necessary to know all the global variables that a module might read or modify. The set of **global variables of a module *M*** is the union of

- the set of global variables that are declared in `M`; and
- the set of global variables declared in other modules that could be read or written by a series of procedure calls beginning with a call to one of `M`'s procedures. By "could" we mean the read/write analysis includes the execution of all branches of conditionals, all branches of a `match` statement, the execution of `while` loop bodies, and the termination of `while` loops.

For example, to print the global variables of module `N` above, run

```
print glob N.  
Globals [# = 0]:  
Prog. variables {# = 3]:  
  N.b : bool  
  N.x : int  
  M.x : int
```

The last line shows that module `N` can read or write `M.x` (via `N.g` calling `M.main`, for example).

For every module `M`, there is a corresponding type, `glob M`, of tuples consisting of a value for each of the global variables of `M`. To illustrate, we can declare

```
op gN : glob N.  
+ added operator gN : bool * int * int.
```

Nothing can be done with these values other than compare them for equality.

6.1.4 Defining Modules by Exception

A module can be defined as a modification of an existing module. The general syntax for this is

```
module Mod1 = M with { <modifications> }.
```

If there are no modifications, `Mod1` is identical to `M`. To modify the global variables of a module, do

```
module Mod2 = M with {  
  -var x, y      (* Mod2 will not have x *)  
  var r, s : int (* Mod2 will have r, s *)  
  var t : int    (* ... and t *)  
}.
```

There can be at most one list of variables to delete before any number of added variable declarations. Deleting a variable that doesn't exist (such as `y`) is silently ignored. Of course, deleting a variable that is accessed (such as `x`) is not a good idea. This step fails:

```
print glob Mod2.  
unknown symbol: Top.Mod2./x
```

You cannot add a variable you just deleted, such as to change its type.

After global variable adjustments, you can optionally modify one or more procedures. You can make at most one modification to each procedure, but the modification can be extensive and must always include at least one statement modification. You add, delete or replace statements from procedures like this:

```
module Mod3 = M with {  
  proc incr [ 1 + ^ { if (n < 0) { n <- -n; } } ]  
  proc main [ 3 - ]  
}.
```

This inserts the statement in braces before code position 1 of `incr` and deletes the statement at code position 3 of `main`. Note that the space between `+` and `^` is mandatory. To insert after a code position,

use `+`. To replace a statement at a code position, use `~`. The procedures of `Mod3` are `init`, `get` and `loop` the same as in `M` and

```
proc incr(n : int) : unit = {
  if (n < 0) {
    n <- -n;
  }
  x <- x + n;
}
proc main() : bool = {
  var n : int;
  init(100);
  incr(10);
  n <@ get();
  return n < 0;
}
```

You can add local variables to a procedure like this

```
module Mod4 = M with {
  proc loop [ var s : int 1 + ^ { s <- 0; } ]
}.
```

You cannot initialize the new variables and there is no semicolon after the type (if present). This also inserts a statement to initialize `s` before code position 1. The procedure `Mod4.loop` begins

```
proc loop() : int = {
  var y, z : int;
  var s : int;
  s <- 0;
  y <- 0;
```

You cannot modify a `return` statement directly because it has no code position, but you can change the value returned by a procedure. This example changes the return value of `loop` and inserts an empty statement after code position 1, to show how to sidestep the requirement to modify a statement:

```
module Mod5 = M with {
  proc loop [ 1 + { } ] res ~ (y * 2 + 1)
}.
```

`Mod5.loop` ends

```
return y * 2 + 1;
```

Finally, you can modify the condition of an `if` or `while` statement. You can also subordinate a statement by adding an `if` statement before it: the selected statement becomes its `then` branch. Here is an example that does both of those things and also inserts a statement:

```
module Mod6 = M with {
  proc loop [
    2 + ^ (0 < y)      (* Insert an if to contain the while *)
    2.1 + { y <- 2; } (* Insert an assignment statement *)
    2 ~ (y < 20)      (* Change the condition of the while *)
```

```
}.
```

The resulting procedure is

```
proc loop() : int = {
  var y, z : int;
  y <- 0;
  if (0 < y) {
    while (y < 20) {
      z <$ [1 .. 10];
      y <- 2;
      if (z <= 3) {
        y <- y - z;
      } elif (z <= 6) {
        y <- y + 1;
      } else {
        y <- y + (z - 5);
      }
    }
  }
  return y;
}
```

Notice that all of the modifications refer to the `while` loop using its original code position, not its new one. Also, there is no way to add an `else` branch to the new `if`.

Note. You cannot modify the statements of the procedure `M.get` because it has no valid code positions. This may be a bug.

A procedure modification also accepts a range of code positions in two forms:

- `[c1 .. c2]` refers to the statements from the one matching `c1` to the one matching `c2`, inclusive; both statements must be in the same block
- `[c1 - c2]` means the same thing, but `c2` must consist of a single-level pattern. For example, `[^while.1 - 3]` is a shorthand for `[^while.1 .. ^while.3]`

6.2 Module Types and Functors

A **module type** consists of a sequence of **signatures**, which are abstract procedure declarations without bodies. For example,

```
module type OR = {
  proc init(secret : int, max_tries : int) : unit
  proc guess(guess : int) : unit
  proc guessed() : bool
}.
```

specifies minimum expectations for a “guessing oracle” that holds a secret and lets callers guess its value up to a maximum number of tries (this declaration actually says far less than that). Here is an implementation that meets these expectations:

```
module Or = {
  var secret, tries_remaining : int
```



```

var guessed : bool
proc init(secret : int, max_tries : int) : unit = {
  Or.secret <- secret;
  tries_remaining <- max_tries;
  guessed <- false;
}
proc guess(guess : int) : unit = {
  if (0 < tries_remaining) {
    guessed <- guessed \/ guess = secret;
    tries_remaining <- tries_remaining - 1;
  }
}
proc guessed() : bool = { return guessed; }
}.

```

The module `Or` *satisfies* the type `OR` because it defines a procedure matching each of its signatures (name, return type and parameter types must match; parameter names don't matter). A module may have more procedures than the types it satisfies, and any global variables and sub-modules whatsoever.

A module may be explicitly declared as satisfying a module type and the type checker will verify that the required procedures are in fact present (possibly among others). For example,

```
module Or : OR = { ... }.
```

This is not a requirement, however: `Or` satisfies `OR` whether it is declared to or not.

Module types are not types of values: you cannot declare global or local variables or parameters of a procedure, function, operator or predicate to be of a module type and there are no typed expressions that are of a module type. In particular, all you OO programmers, they are not classes, though they are a bit like Java interfaces with respect to modules and even provide some information hiding. We will see later that logic formulas may quantify over modules by the module types they satisfy.

A module can be parameterized on one or more module types, so that when it is provided with modules satisfying each type, it can access the procedures defined by the types but not any global variables, sub-modules or other procedures the modules may have. For example, an adversary that tries to guess the oracle's secret by making 10 random guesses between 1 and 100 might be defined as

```

module SimpleAdv(O : OR) = {
  var range : int * int
  var tries : int
  proc chooseRange() : int * int = {
    range <- (1, 100);
    tries <- 10;
    return range;
  }
  proc doGuessing() : unit = {
    var x : int;
    while (0 < tries) {
      x <$ [range.`1 .. range.`2];
      O.guess(x);
      tries <- tries - 1;
    }
  }
}

```

```

    }
  }
}.

```

To refer to a global variable of a parameterized module, parameters may be ignored: `SimpleAdv.tries`, for example. To call one of its procedures, however, one needs to specify the arguments it will use, as in `SimpleAdv(Or).doGuessing()`. The caller is free to use different arguments from call to call.

Next we define a “game” that lets the adversary try its luck against `Or`:

```

module Game = {
  module A = SimpleAdv(Or)    (* aliased for convenience *)
  proc main() : bool = {
    var advWon : bool;
    var low, high, secret : int;
    (low, high) <@ A.chooseRange();
    secret <$ [low..high];
    Or.init(secret, SimpleAdv.tries);
    A.doGuessing();
    advWon <@ Or.guessed();
    return advWon;
  }
}.

```

The type `OR` characterizes what an oracle must do, but the adversary really shouldn’t be able to initialize it. One way to prevent this is to define a module type with more limited access to the oracle:

```

module type ORA = { proc guess(guess : int) : unit }

```

and modifying the adversary’s parameter to this type:

```

module SimpleAdv(O : ORA) = { ... }

```

This works because any module that satisfies `OR` also satisfies `ORA`, but there is a better way.

Module types may have parameters. A module type with parameters is called a **functor**. Functors have the option of restricting what each of its signatures can do with each parameter. For example, a more flexible way to limit the access `SimpleAdv` has to `O` is to define a functor with these restrictions:

```

module type ADV(O : OR) = {
  proc chooseRange() : int * int {}
  proc doGuessing() : unit {O.guess}
}.

```

Normally, each procedure in a module may access any of the procedures in its parameters. The `{}` following `chooseRange` means that, for a module to satisfy `ADV`, its implementation of `chooseRange` must not access anything in `O`, while `{O.guess}` on `doGuessing` means its implementation may access that procedure (only). Multiple procedures can be listed separated by commas.

Now we define `SimpleAdv` as satisfying the `ADV` functor. The syntax for a module that both satisfies a module type or functor and has parameters is

```
module (SimpleAdv : ADV) (O : OR) = { ... }.
```

Note that these restrictions apply only to parameters: `SimpleAdv` is still free to access any module's global variables and procedures directly, such as `Or.secret` or `Or.init()`, rather than via the parameter, as `O.init()`, which is restricted. In the next section we will see how to prevent this, too.

A module type or functor may **include** another module type (but not a functor), as module `B` does in

```
module type A = {
  proc get() : a
  proc put(x : a) : unit
}.
module type B = {
  include A
  proc init() : unit
}.
```

Module type `B` is now the same as if it had been defined as

```
module type B = {
  proc get() : a
  proc put(x : a) : unit
  proc init() : unit
}.
```

A module type can be selective by listing the signatures it wants to include

```
module type B = {
  include A [+put]
  proc init() : unit
}.
```

which is equivalent to

```
module type B = {
  proc put(x : a) : unit
  proc init() : unit
}.
```

or those it wants to exclude

```
module type B = {
  include A [-put]
  proc init() : unit
}.
```

which is equivalent to

```
module type B = {
  proc get() : a
  proc init() : unit
}.
```

To include or exclude more than one signature, list the names separated by commas after the initial – or +. The + is also optional. Signatures and include directives can be freely intermixed.

A functor may place restrictions on the signatures it gets by `include`. By default, full access is granted (but note that `init` is restricted in this example):

```
module type B(O : OR) = {
  include A
  proc init() : unit {}
}.
```

is equivalent to

```
module type B(O : OR) = {
  proc get() : a {O.init, O.guess, O.guessed}
  proc put(x : a) : unit {O.init, O.guess, O.guessed}
  proc init() : unit {}
}.
```

whereas

```
module type B(O : OR) = {
  include A {O.guess O.guessed}
  proc init() : unit {}
}.
```

is equivalent to

```
module type B(O : OR) = {
  proc get() : a {O.guess, O.guessed}
  proc put(x : a) : unit {O.guess, O.guessed}
  proc init() : unit {}
}.
```

Note that multiple signatures for `include` are separated by white space, not commas. A functor can, of course, be selective and restrictive at the same time:

```
module type B(O : OR) = {
  include A [get] {O.guess}
  proc init() : unit {}
}.
```

is equivalent to

```
module type B(O : OR) = {
  proc get() : a {O.guess}
  proc init() : unit {}
}.
```

6.3 Reasoning about Memories and Modules

The expression language of the ambient logic is extended to enable reasoning about computation: that is, about memories and modules. This section focuses on syntax, while the meaning of all this will become clearer in the sections on program logic proofs.

A **memory** records the values of all global variables of all declared modules at a point in time. In a proof, a memory may also include a procedure's parameters, local variables and/or result (denoted `res`). A **memory identifier** consists of `&` followed by either a sequence of digits or an identifier that begins with a lowercase letter or an underscore. If `&m` is a memory and `x` is a global variable in the domain of `&m`, then `x{m}` denotes the value of `x` in `&m`. If `M` is a module and `&m` is a memory, then `(glob M){m}` is a value of type `glob M` that is the "sub-memory" of `&m` restricted to the global variables of `M`. Finally, if ϕ is an expression, $\phi\{m\}$ means the expression in which every subexpression `u` that is a variable in `&m`, `res` or `glob M` is replaced by `u{m}`. For example,

$$(Y1.y = Y1.z \Rightarrow Y1.z = Y1.y)\{m\}$$

expands to

$$Y1.y\{m\} = Y1.z\{m\} \Rightarrow Y1.z\{m\} = Y1.y\{m\}$$

The parentheses are necessary, because `_ {m}` has higher precedence than function application.

Quantification over memories and modules is allowed. For example, suppose we have declared:

```
module X = { var x : int }.
module Y = { var y : int }.
```

Then, this is a (true) formula:

$$\text{forall } \&m, X.x\{m\} < Y.y\{m\} \Rightarrow X.x\{m\} + 1 \leq Y.y\{m\}$$

(Note that `&m` has no type, because there is no type for memories.) Similarly, if `T` is a module type, and `M` is a module name, then

$$\text{forall } (M <: T), \phi$$

means for all modules `M` satisfying `T`, the formula ϕ holds. The additional syntax

$$\text{forall } (M <: T\{-N_1, \dots, -N_k\}), \phi$$

means for all modules `M` satisfying `T` and whose sets of global variables are disjoint from those of the `Ni`, ϕ holds. Alternatively, the syntax

$$\text{forall } (M <: T\{+N_1, \dots, +N_k\}), \phi$$

means for all modules `M` satisfying `T` and whose sets of global variables are a subset of the union of those of the `Ni`, ϕ holds. One can also specify $\pm N_i.x$ to allow or forbid access to an individual variable and mix `+` and `-` to fine tune access, such as

$$\text{forall } (M <: T\{+N, -N.x, +O\}), \phi$$

The ability to restrict which global variables a module may access, including by calling procedures that access them, augments functor restrictions, as promised in the previous section.

By a quirk of the parser, quantified variables or parameters over modules must stand alone. That is,

$$\text{forall } (X <: T\{-G\}) (n : \text{int}), \dots \quad (* \text{ ok } *)$$

with `X` and `n` separated is acceptable but listing `X` and `n` together in parentheses like

```
forall (X <: T{-G}, n : int), ...      (* not ok *)
```

produces a parse error.

You cannot do anything with memories other than look up the values of variables in them; in particular, formulas cannot test or assert the equality of memories. Likewise, formulas cannot do anything with modules except access their elements; they cannot test or assert module equality.

Memories and modules constitute new kinds of assumptions that can appear in the context of a goal:

In the context	In the conclusion
Memory declaration: $\&m : \{\}$	Quantifier: <code>forall $\&m, \phi'$</code>
Module declaration: $M <: T\{+N, -N.x, +O\}$	Quantifier: <code>forall $(M <: T\{+N, -N.x, +O\}), \phi'$</code>

When a module or memory in the conclusion of a goal is moved to the context by the introduction tactical, the name provided must be legal for the kind of assumption moved: that is, it must start with $\&$ for a memory and a capital letter for a module.

A probability expression has the form

$$\text{Pr}[M.p(e_1, \dots, e_n) \ @ \ \&m : Q]$$

and denotes the (real-valued) probability that running procedure p of module M with the indicated arguments and initial memory $\&m$ will terminate with a memory that satisfies Q .

EasyCrypt has three sub-logics specific to reasoning about computation embedded in the so-called “ambient” logic:

- Hoare logic (HL) for stating pre- and postcondition pairs of executing a procedure.
- A probabilistic Hoare logic (pHL) for stating the probability of a procedure’s execution resulting in a postcondition holding
- A probabilistic, relational Hoare logic (pRHL) for relating pairs of procedures

Formulas in these logics are called **program logic judgements**.

A **HL judgement** is EasyCrypt’s way of writing a Hoare triple. It has the form

$$\text{hoare } [M.p : P ==> Q]$$

where M is the name of a module, p is the name of a procedure in M and P and Q are predicates. This judgement asserts that if $M.p$ is run with an initial memory satisfying P and it terminates, the final memory will satisfy Q . Note that it does not assert termination.

A **pHL judgement** is a probabilistic variant of Hoare logic. Its judgements have the form:

$$\text{phoare } [M.p : P ==> Q] \ \diamond \ e$$

where M is the name of a module, p is the name of a procedure in M , P and Q are predicates, $\diamond$ is a comparison operator ($<$, $\leq$, $=$, $\geq$ or $>$), and e is a real-valued expression. This judgement asserts that if $M.p$ is run with an initial memory satisfying P , then the probability that it terminates *and* the final memory satisfies Q has the indicated relationship to e .

For example, given the declarations

```
module type T = {
  proc f() : unit
}.
module G(X : T) = {
  var x : int
  proc g() : unit = {
    X.f();
  }
}.
```

the formula

```
forall (X <: T{-G}) (n : int),
  phoare [G(X).g : G.x = n ==> G.x = n] = 1%r.
```

says that for any module X of type T that has no global variables in common with G , calling $G(X).g()$ terminates with the value of $G.x$ unchanged with probability 1. This claim is false, because if the call $X.f()$ fails to terminate, so does $G.g()$.

A **lossless assertion** has the form

```
islossless M.p
```

and is shorthand for

```
phaore [M.p : true ==> true] = 1%r
```

which simply asserts that $M.p$ always terminates, no matter its arguments or initial memory. For example, if we add the condition that $X.f$ always terminates to the previous example, the formula is true:

```
forall (X <: T{G}) (n : int),
  islossless X.f => phoare [G(X).g : G.x = n ==> G.x = n] = 1%r.
```

NOTE: the `Distr` theory defines a predicate over distributions that has a very similar name:

```
pred is_lossless (d : 'a distr) = weight d = 1%r.
```

Don't confuse the two.

A **prhl judgement** relates the behavior of two procedures executed in separate memories, $\&1$ and $\&2$. It has the form

```
equiv [M.p ~ N.q : P ==> Q]
```

where $M.p$ and $N.q$ are names of procedures and P and Q are predicates. This judgement asserts that if $M.p$ is run with an initial memory $\&1$, $N.q$ is run with an initial memory $\&2$ and $\&1$ and $\&2$ satisfy P , then the sub-distributions of the final memories satisfy Q in the following sense: well, it's complicated. For example, if Q is $\text{res}\{1\} = \text{res}\{2\}$ then the return values of $M.p$ and $N.q$ have the same sub-distribution: they may not return the same value, but will both return, say, 0, 10% of the time; 1, 50% of the time; and 2, 40% of the time.

Lemmas consisting of program logic judgements can be abbreviated:

```
hoare phoo (x : t) : [M.p :  $\varphi \Rightarrow \psi$ ].
ehoare phoo (x : t) : [M.p :  $\varphi \Rightarrow \psi$ ].
phoare phoo (x : t) : [M.p :  $\varphi \Rightarrow \psi$ ]  $\diamond$  e
equiv phoo (x : t) : [M.p ~ N.q :  $\varphi \Rightarrow \psi$ ]
```

are shorthand for

```
lemma phoo (x : t) : hoare [M.p :  $\varphi \Rightarrow \psi$ ]
lemma phoo (x : t) : ehoare [M.p :  $\varphi \Rightarrow \psi$ ]
lemma phoo (x : t) : phoare [M.p :  $\varphi \Rightarrow \psi$ ]  $\diamond$  e
lemma phoo (x : t) : equiv [M.p ~ N.q :  $\varphi \Rightarrow \psi$ ]
```

respectively. Any mention of “lemmas” in this document includes these shorthand forms.

During a proof, a list of statements may appear in place of `M.p` in a judgement. Such are called **statement judgements**. Statement judgements cannot be directly input by the user.

6.4 Probability Distributions

Probability distributions and random selection from them feature heavily in modules and proofs about cryptography. The `Pervasive` theory in the `prelude` folder has

```
type 'a distr.
op mu : 'a distr -> ('a -> bool) -> real.
```

This is the beginning of EasyCrypt’s support for reasoning about discrete probability distributions. The rest is in the `distributions` folder, starting with the `Distr` theory.

For any type `t`, the type `t distr` is the type of **sub-distributions** over `t`. A sub-distribution is a real-valued function that is never negative and sums to no more than 1, whereas a (proper) distribution sums to exactly 1. From here on, we shall use the term *distribution* for both proper and sub-distributions. Note that selecting a value from a distribution is not part of the ambient logic, but of the module language (see *Modules*).

The following descriptions are adapted from comments in the `Distr` script, where `d : 'a distr` and `x : 'a`:

- o `mul d x : real` – the probability of the value `x` in the distribution `d` (the probability mass function)
- o `mu d E : real` – for `E : t -> bool`, the sum of `mul d x` for all `x` where `E x` holds. `E` is referred to as an **event**
- o `weight d : real` – the sum of `mul d x` for all `x` (i.e., `mu d predT`)
- o `x \in d : bool` – whether `x` is in the **support** of `d`, meaning `mul d x` is non-zero
- o `x \notin d : bool` – whether `x` isn’t in the support of `d`
- o `is_lossless d : bool` – whether `d` is **lossless**, meaning the weight of `d` is 1
- o `is_full d : bool` – whether `d` is **full**, meaning non-zero for all `t` (its support is all of `t`)
- o `is_uniform d : bool` – whether `d` is **uniform**, meaning all values in the support of `d` have the same probability
- o `is_funiform d : bool` – whether `d` is both full and uniform (i.e., all values of `t` have the same probability)

- o `p_max d : real` – the probability of the most likely value in `d`
- o `mode d : 'a` – one of the elements with the highest probability

You probably won't need to roll your own distributions very often, as the library defines many kinds. The `Distr` theory has many operators that return distributions, including, where `d : 'a distr` and `x : 'a`,

- o `dnull : 'a distr` – the distribution that assigns `0%` to every `x : 'a`
- o `dunit x : 'a distr` – the lossless uniform distribution that assigns probability `1%` to `x` and `0%` to all other values of type `'a`
- o `drat (s : 'a list) : 'a distr` – a lossless distribution whose support is the elements of `s` and the probability of each `x : 'a` is proportional to the number of times it occurs in `s`
 - o For example, `drat [1; 2]` is a distribution where 1 and 2 each have a probability of `1% / 2%`, while `drat [1; 1; 2]` is a distribution where the probability of 1 is `2% / 3%` and that of 2 is `1% / 3%`.
- o `duniform (s : 'a list) : 'a distr` – a lossless distribution whose support is the elements of `s` and where the probability of each `x \in s` is the same regardless of how many times it occurs in `s`
 - o `duniform s = drat (undup s)`
- o `drange (m n : int) : int distr` – a lossless uniform distribution with support `{m, m + 1, ..., n - 1}`, as long as `m < n`
- o `dlet d (f : 'a -> 'b distr) : 'b distr` – a distribution where the probability of each `y : 'b` is the sum over `x : 'a` of `mul d x * mul (f x) y`.
- o `dfold (f : int -> 'a -> 'a distr) x i : 'a distr` – a family of distributions where `dfold f x 0` is `dunit x` and `dfold f x (n + 1)` is `dlet (dfold f x i) (f i)`.
- o `dmap d (f : 'a -> 'b) : 'b distr` – `dlet d (dunit \o f)`
 - o if `x` is in the support of `d`, then `f x` is in the support of `dmap d f`
 - o `dmap d f` is lossless if `d` is
 - o `dmap d f` is uniform if `d` is and `f` is injective over the support of `d`
- o `dscalar k d : 'a distr` – the distribution that assigns `k * mul d x` to each `x`
 - o The result will only be a valid distribution if `0 <= k <= inv (weight d)`
 - o `k \cdot d` is an abbreviation for `dscalar k d`
- o `dscale d : 'a distr` – the lossless distribution `dscalar (inv (weight d)) d`
 - o Exception: `dscale dnull = dnull` is not lossless
- o `dopt d : 'a option distr` – for `d : 'a distr`, a lossless distribution that assigns `mul d x` to `Some x` and `1 - weight d` to `None` (i.e., `None` gets the “left over” probability not assigned by `d`, if there is any)
- o `drestrict d (p : 'a -> bool) : 'a distr` – the distribution equal to `d` for all `x` such that `p x` and 0 otherwise
- o `dcond d (p : 'a -> bool) : 'a distr` – `dscale (drestrict d p)`
- o `d1 `*` d2 : ('a * 'b) distr` – the joint distribution of `d1` and `d2`
- o `djoin (ds : 'a distr list) : 'a list distr` – the joint distribution of a list of distributions

- o `d1 + d2` : 'a `distr` – the distribution that is the pointwise sum of `d1` and `d2`, which is only a distribution if `weight d1 + weight d2 <= 1`
- o `dbin (p : real) (n : int) : int distr` – the binomial distribution for `p` and `n`
 - o if `0 <= n` and `0 <= p <= 1`, then `dbin p n` is lossless and its support is all `k` such that `0 <= k <= n`

In addition, the following theories define more kinds of distributions:

- o `DBool` – distributions over the `bool` type
 - o `dbiased (p : real) : bool distr`
- o `Dexcepted`
 - o `d \ P = dscale (drestrict d (predC P))`
- o `Dfilter`
 - o `dfilter d (P : 'a -> bool)` – the distribution equal to `d` for `x` such that `! P x` and zero otherwise
- o `DInterval`
 - o `dinter = duniform (range i (j + 1))`
 - o `[a..b]` is an abbreviation for `dinter a b`
- o `DList`
 - o `dlist d (n : int)` – the joint distribution of `n` copies of `d`

There is a small amount of parser support for probability distributions. We already mentioned that

`[a..b]`

is an abbreviation for

`dinter a b`

Another is that

`{0,1}`

is the lossless, uniform distribution on `bool`. It must be typed exactly as shown, with no spaces. Finally, the parser recognizes the following tags for operators that are distributions:

`op [lossless uniform full] d1 : int distr.`

means

```
op d1 : int distr.
axiom d1_ll : is_lossless d1.
axiom d1_uni : is_uniform d1.
axiom d1_fu : is_full d1.
```

where the axioms' names are generated automatically. Only one library theory, `SDist`, uses this feature.

7 Hoare Logic Proofs

An HL judgement (or *Hoare triple*) is a statement about how the execution of a program changes the state of a computation (i.e., the values contained in the memory in which the program runs). The state before the program is run is described by a predicate `P`, called the precondition, and the state after the

program is run (assuming it terminates) is described by a predicate Q , called the postcondition. Termination, in general, requires a separate proof, specifically whenever `while` statements are involved.

The traditional notation for a Hoare triple is $\{P\} \ C \ \{Q\}$, where C is the program (or *command*). The EasyCrypt syntax, as we've seen, is

```
hoare [M.p : P ==> Q]
```

where M is a module and p is a procedure in M . When a Hoare triple is the conclusion of a goal, it is rendered like this:

```
Type variables: <none>
-----
pre = P
    M.p
post = Q
```

The memory in which $M.p$ is run is implicit. P and Q are ambient logic formulas that may refer to global variables (which predicates outside the scope of a memory cannot). In addition, P may refer to the parameters of $M.p$ and Q may refer to the result returned by $M.p$, as `res`.

This section shall use the following program for searching a list for an element as a running example:

```
require import Int Real List.
module M = {
  proc indexOf(x : int, l : int list) : int = {
    var i <- 0;
    var found <- false;
    var h : int option;
    while (l <> [] /\ !found) {
      h <- ohead l;
      if (Some x = h) {
        found <- true;
      } else {
        l <- behead l;
        i <- i + 1;
      }
    }
    if (!found) {
      i <- -1;
    }
    return i;
  }
}.
```

7.1 Symbolic Execution

For a given program, there is an infinite number of pre- and postcondition pairs that result in a true judgement. The following lemma corresponds to a particular test case:

```
lemma HL1 : hoare [M.indexOf : x = 4
                  /\ l = 3 :: 1 :: 4 :: 1 :: 6 :: []
                  ==> res = 2 ].
```

whose proof begins with the corresponding initial goal

```
Type variables: <none>
-----
pre = x = 4 /\ l = [3; 1; 4; 1; 6]

      M.indexOf

post = res = 2
```

EasyCrypt's version of Hoare logic embeds inference rules for each statement type in the tactics. To reason about a procedure, it is typically necessary to reason about the statements that comprise it. The `proc*` tactic reduces a procedure judgement to a corresponding statement judgement that calls the procedure. For example,

```
proc*.
Type variables: <none>
-----
Context : {x, r : int, l : int list}
pre = (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1)  r <@ M.indexOf(x,
( )    l)

post = r = 2
```

A statement judgement has a “context” (a memory) that lists variables local to the program. This example has local variables `x` and `l` to hold the arguments to the call and `r` to hold its return value. This is not to be confused with the *proof context* above the line, which in this example is empty.

The (1) shown is no mere line number: each statement in a program is identified by its **code position**. Top-level statements of the program are numbered starting at 1. Statements in the `then` part of an `if` statement or the body of a `while` statement use a dot notation starting at `n.1`, where `n` is the code position of the parent statement. Statements in the `else` part of an `if` statement start at `n?1`. When a statement spans more than one line, as (1) does, only the first has a code position.

This statement judgement is not much better than the original HL judgement, in terms of being able to reason about `M.p`. The `inline` tactic modifies a statement judgement by replacing a call to a concrete procedure with its body, instantiated with the call’s arguments and with the `return` statement replaced by an assignment:

```
inline M.indexOf.
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : list,
          h : int option }
pre = (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1----)  x0 <- x
(2----)  l0 <- l
(3----)  i <- 0;
```

```

(4----) found <- false;
(5----) while (l0 <> [] /\
(  ---)      !found) {
(5.1--)   h <- ohead l0;
(5.2--)   if (Some x0 = h) {
(5.2.1)   found <- true;
(5.2--)   } else {
(5.2?1)   l0 <- behead l0;
(5.2?2)   i <- i + 1;
(5.2--)   }
(5----) }
(6----) if (!found) {
(6.1--)   i <- -1;
(6----) }
(7----) r <- i;

```

post = r = 2

This step added the parameters and local variables of `M.indexOf` to the program context and statements assigning the arguments of the call to the parameters.

The proof of this lemma uses a mix of “program reasoning” and “program transformation” tactics. Program reasoning tactics manipulate a program’s specification and statements at the same time by “consuming” statements and making corresponding changes to the pre- and postconditions based on the statements’ meaning, until the program is empty. Program transformation statements modify the statements of a program in semantics-preserving ways, so the pre- and post-conditions are unchanged. This is as close to executing the code as EasyCrypt gets.

To start, the `wp` tactic consumes ordinary assignment statements (`<-`) and conditionals at the end of a program and updates the postcondition according to the statements’ *weakest precondition*. It also consumes assertions by adding the asserted expression to the postcondition.

```

wp.
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre = (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1----) x0 <- x
(2----) l0 <- l
(3----) i <- 0
(4----) found <- false
(5----) while (l0 <> [] /\
(  ---)      !found) {
(5.1--)   h <- ohead l0
(5.2--)   if (Some x0 = h) {
(5.2.1)   found <- true
(5.2--)   } else {
(5.2?1)   l0 <- behead l0
(5.2?2)   i <- i + 1

```

```
(5.2--)    }
(5----)    }
```

```
post = if !found then -1 = 2 else i = 2
```

Similarly, the `sp` tactic consumes ordinary assignment statements (`<-`) and conditionals at the start of a program and updates the precondition according to the statements' *strongest postcondition*. Weakest precondition and strongest postcondition are known as *predicate transformers* in language semantics.

```
sp.
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1----) while (l0 <> [] /\
(  ---)      !found) {
(1.1--)      h <- ohead l0
(1.2--)      if (Some x0 = h) {
(1.2.1)        found <- true
(1.2--)      } else {
(1.2?1)        l0 <- behead l0
(1.2?2)        i <- i + 1
(1.2--)      }
(1----)    }
```

```
post = if !found then -1 = 2 else i = 2
```

Loops are more difficult because one generally doesn't know how many times the body will be executed, if any, or if the loop will terminate. In this case, however, we do. The `rcondt` and `rcondf` tactics transform a `while` statement whose `while` condition is `true` or `false`, respectively, into two subgoals. The first one verifies that the program up to the `while` statement makes its condition known and the second subgoal transforms the program accordingly: for `rcondt`, it inserts a copy of the `while` body before the `while` statement, while for `rcondf` it deletes the `while` statement. For this lemma, we know the `while` condition will be true the first three times it is encountered and false the fourth time, so we do this next:

```
rcondt 1.
Current goal (remaining: 2)

Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
```

```

x0 = x /\
l0 = l /\
i = 0 /\
found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

post = l0 <> [] /\ !found

```

The argument to `rcondt` (or `rcondf`) is the code position of the `while` statement. The first subgoal's program is all the statements before the `while` statement, which is none, and its postcondition is the `while` condition. It is trivial to solve, showing the second subgoal, which has a copy of the `while` body before the `while` loop:

```

done.
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1----) h <- ohead l0
(2----) if (Some x0 = h) {
(2.1--)   found <- true
(2----) } else {
(2?1--)   l0 <- behead l0
(2?2--)   i <- i + 1
(2----) }
(3----) while (l0 <> [] /\
(  ---)       !found) {
(3.1--)   h <- ohead l0
(3.2--)   if (Some x0 = h) {
(3.2.1)   found <- true
(3.2--)   } else {
(3.2?1)   l0 <- behead l0
(3.2?2)   i <- i + 1
(3.2--)   }
(3----) }

post = if !found then -1 = 2 else i = 2

```

This program starts with a regular assignment and an `if` statement, so we could do `sp`, but we also know in code position 2 that the `if` condition is false, because `x0 = 4` and `h = Some 3`. The `rcondt` and `rcondf` tactics also work on `if` statements, again generating two subgoals. The first `rcondf` subgoal verifies that the program up to the `if` statement makes its condition false:

```

rcondf 2.
Current goal (remaining: 2)
Type variables: <none>

```

```

-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1-- ) h <- ohead l0

post = Some x0 <> h

```

The assignment at code position 1 is equally at the start of the program and at the end; I prefer `wp` over `sp`. The resulting subgoal is trivial to solve:

```

wp.
Current goal (remaining: 2)
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

post = Some x0 <> ohead l0
done. (* first subgoal solved *)

```

The second `rcondf` subgoal is the original program with the `if` statement replaced by its `else` part

```

Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1----) h <- ohead l0
(2----) l0 <- behead l0
(3----) i <- i + 1
(4----) while (l0 <> [] /\
(  ---)      !found) {
(4.1-- )   h <- ohead l0
(4.2-- )   if (Some x0 = h) {
(4.2.1)    found <- true

```



```

(4.2--)      } else {
(4.2?1)      10 <- behead 10
(4.2?2)      i <- i + 1
(4.2--)      }
(4----)     }

```

```
post = if !found then -1 = 2 else i = 2
```

At this point we've symbolically executed the while loop once. The while condition at code position 4 is true because `10 = [1; 4; 1; 6]` and `found = false`, so we execute the loop again:

rcondt 4.

Current goal (remaining: 2)

Type variables: <none>

Context : {found : bool, x, r, x0, i, n : int, l, 10 : int list,
 h : int option}

pre =

 x0 = x /\

 10 = l /\

 i = 0 /\

 found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1) h <- ohead 10

(2) 10 <- behead 10

(3) i <- i + 1

post = 10 <> [] /\ !found

wp.

Current goal (remaining: 2)

Type variables: <none>

Context : {found : bool, x, r, x0, i, n : int, l, 10 : int list,
 h : int option}

pre =

 x0 = x /\

 10 = l /\

 i = 0 /\

 found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

post = behead 10 <> [] /\ !found

done. (* first subgoal solved *)

Type variables: <none>

Context : {found : bool, x, r, x0, i, n : int, l, 10 : int list,
 h : int option}

pre =

 x0 = x /\

 10 = l /\

 i = 0 /\

 found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

```

(1----) h <- ohead 10
(2----) 10 <- behead 10
(3----) i <- i + 1
(4----) h <- ohead 10
(5----) if (Some x0 = h) {
(5.1--)   found <- true
(5----) } else {
(5?1--)   10 <- behead 10
(5?2--)   i <- i + 1
(5----) }
(6----) while (10 <> [] /\
( ---)   !found) {
(6.1--)   h <- ohead 10
(6.2--)   if (Some x0 = h) {
(6.2.1)   found <- true
(6.2--)   } else {
(6.2?1)   10 <- behead 10
(6.2?2)   i <- i + 1
(6.2--)   }
(6----) }

```

post = if !found then -1 = 2 else i = 2

rcondf 5.

Current goal (remaining: 2)

Type variables: <none>

Context : {found : bool, x, r, x0, i, n : int, l, 10 : int list,
 h : int option}

pre =

```

x0 = x /\
10 = 1 /\
i = 0 /\
found = false /\ (x, 1).`1 = 4 /\ (x, 1).`2 = [3; 1; 4; 1; 6]

```

```

(1----) h <- ohead 10
(2----) 10 <- behead 10
(3----) i <- i + 1
(4----) h <- ohead 10

```

post = Some x0 <> h

wp.

Current goal (remaining: 2)

Type variables: <none>

Context : {found : bool, x, r, x0, i, n : int, l, 10 : int list,
 h : int option}

pre =

```

x0 = x /\
10 = 1 /\

```

```

i = 0 /\
found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

post = Some x0 <> ohead (behead l0)
done. (* first subgoal solved *)
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1----) h <- ohead l0
(2----) l0 <- behead l0
(3----) i <- i + 1
(4----) h <- ohead l0
(5----) l0 <- behead l0
(6----) i <- i + 1
(7----) while (l0 <> [] /\
(  ---)      !found) {
(7.1--)      h <- ohead l0
(7.2--)      if (Some x0 = h) {
(7.2.1)      found <- true
(7.2--)      } else {
(7.2?1)      l0 <- behead l0
(7.2?2)      i <- i + 1
(7.2--)      }
(7----)    }

post = if !found then -1 = 2 else i = 2

```

The next three steps start executing the loop a third time.

```

rcondt 7.
Current goal (remaining: 2)
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1) h <- ohead l0
(2) l0 <- behead l0
(3) i <- i + 1

```

```

(4)  h <- ohead 10
(5)  10 <- behead 10
(6)  i <- i + 1

post = 10 <> [] /\ !found
wp.
Current goal (remaining: 2)
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, 10 : int list,
          h : int option}
pre =
  x0 = x /\
  10 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

post = let 10_0 = behead 10 in behead 10_0 <> [] /\ !found
done. (* first subgoal solved *)
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, 10 : int list,
          h : int option}
pre =
  x0 = x /\
  10 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1----) h <- ohead 10
(2----) 10 <- behead 10
(3----) i <- i + 1
(4----) h <- ohead 10
(5----) 10 <- behead 10
(6----) i <- i + 1
(7----) h <- ohead 10
(8----) if (Some x0 = h) {
(8.1--)   found <- true
(8----) } else {
(8?1--)   10 <- behead 10
(8?2--)   i <- i + 1
(8----) }
(9----) while (10 <> [] /\
(  ---)   !found) {
(9.1--)   h <- ohead 10
(9.2--)   if (Some x0 = h) {
(9.2.1)   found <- true
(9.2--)   } else {
(9.2?1)   10 <- behead 10
(9.2?2)   i <- i + 1

```

```
(9.2--)    }
(9----)    }
```

```
post = if !found then -1 = 2 else i = 2
```

This time through the while body, however, something is different: `h = Some 4`, making the if condition at code position 8 true:

rcondt 8.

Current goal (remaining: 2)

Type variables: <none>

Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
 h : int option}

pre =

 x0 = x /\

 l0 = l /\

 i = 0 /\

 found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1----) h <- ohead l0

(2----) l0 <- behead l0

(3----) i <- i + 1

(4----) h <- ohead l0

(5----) l0 <- behead l0

(6----) i <- i + 1

(7----) h <- ohead l0

post = Some x0 = h

wp.

Current goal (remaining: 2)

Type variables: <none>

Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
 h : int option}

pre =

 x0 = x /\

 l0 = l /\

 i = 0 /\

 found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

post = let l0_0 = behead l0 in Some x0 = ohead (behead l0_0)

done. (* first subgoal solved *)

Type variables: <none>

Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
 h : int option}

pre =

 x0 = x /\

 l0 = l /\

 i = 0 /\

```

found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1----) h <- ohead l0
(2----) l0 <- behead l0
(3----) i <- i + 1
(4----) h <- ohead l0
(5----) l0 <- behead l0
(6----) i <- i + 1
(7----) h <- ohead l0
(8----) found <- true
(9----) while (l0 <> [] /\
(  ---)      !found) {
(9.1--)      h <- ohead l0
(9.2--)      if (Some x0 = h) {
(9.2.1)      found <- true
(9.2--)      } else {
(9.2?1)      l0 <- behead l0
(9.2?2)      i <- i + 1
(9.2--)      }
(9----)    }

post = if !found then -1 = 2 else i = 2

```

Now that found is true, the while condition is false. Switching tactics,

rcondf 9.

Current goal (remaining: 2)

Type variables: <none>

Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
 h : int option}

pre =

```

x0 = x /\
l0 = l /\
i = 0 /\
found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

```

```

(1) h <- ohead l0
(2) l0 <- behead l0
(3) i <- i + 1
(4) h <- ohead l0
(5) l0 <- behead l0
(6) i <- i + 1
(7) h <- ohead l0
(8) found <- true

```

```

post = ! (l0 <> [] /\ !found)

```

Note that the postcondition of the first subgoal is the negation of the while condition. We proceed as before:

```

wp.
Current goal (remaining: 2)
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

post = let l0_0 = behead l0 in let l0_1 = behead l0_0 in
  ! (l0_1 <> [] /\ !true)
done. (* first subgoal solved *)
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

(1)  h <- ohead l0
(2)  l0 <- behead l0
(3)  i <- i + 1
(4)  h <- ohead l0
(5)  l0 <- behead l0
(6)  i <- i + 1
(7)  h <- ohead l0
(8)  found <- true

post = if !found then -1 = 2 else i = 2

```

What we have left is a version of the original program specialized to the arguments provided in the lemma. The rest is easy:

```

wp.
Type variables: <none>
-----
Context : {found : bool, x, r, x0, i, n : int, l, l0 : int list,
          h : int option}
pre =
  x0 = x /\
  l0 = l /\
  i = 0 /\
  found = false /\ (x, l).`1 = 4 /\ (x, l).`2 = [3; 1; 4; 1; 6]

post = let _ = behead l0 in

```

```

    (if !true then -1 = 2 else i + 1 + 1 = 2)
done.
No more goals.

```

7.2 Loop Invariants

Symbolic execution and program specialization are powerful capabilities, but proving a program works for one test case is just as useful as executing one test case and verifying the result: not very. Fortunately, we are not limited to that. The following lemmas correspond to more general claims of correctness. They characterize the two cases of finding the element and not finding it:

```

lemma HL2 l0 x0 : hoare [M.indexOf :
  x0 = x /\ l0 = 1 ==> 0 <= res ==> onth res l0 = Some x0].
lemma HL3 : hoare [M.indexOf :
  x0 = x ==> -1 = res ==> ! x0 \in l0].

```

Note the idiom of using lemma parameters to “capture” the arguments passed to `M.indexOf` in the precondition so they can be used in the postcondition, which can’t refer to them directly.

The `proc` tactic (without the `*`) combines `proc*` and `inline`: it reduces a procedure judgement to a statement judgement consisting of the body of the procedure. It also avoids some redundant local variables.

```

lemma HL2 l0 x0 : hoare [M.indexOf :
    x0 = x /\ l0 = 1 ==> 0 <= res ==> onth res l0 = Some x0].
l0: int list
x0: int
-----
pre = x0 = x /\ l0 = 1

  M.indexOf

post = 0 <= res ==> onth l0 res = Some x0
proc.
Type variables: <none>

l0: int list
x0: int
-----
Context : {found : bool, x, i : int, l : int list, h : int
option}

pre = x0 = (x, l).`1 /\ l0 = (x, l).`2

(1----) i <- 0
(2----) found <- false
(3----) while (l <> [] /\
(  ---)      !found) {
(3.1--)   h <- ohead l
(3.2--)   if (Some x = h) {
(3.2.1)   found <- true
(3.2--)   } else {

```



```

(3.2?1)      l <- behead l
(3.2?2)      i <- i + 1
(3.2-- )     }
(3---- )    }
(4---- )    if (!found) {
(4.1-- )     i <- -1
(4---- )     }

post = 0 <= i => onth l0 i = Some x0

```

We start as before, using predicate transformers to consume the statements before and after the `while` statement:

```

wp. sp.
l0: int list
x0: int
-----
Context : {found : bool, x, i : int, l : int list, h : int
option}

pre = i = 0 /\ found = false /\ x0 = (x, l).`1 /\ l0 = (x, l).`2

(1---- ) while (l <> [] /\
(  --- )      !found) {
(1.1-- )      h <- ohead l
(1.2-- )      if (Some x = h) {
(1.2.1)        found <- true
(1.2-- )      } else {
(1.2?1)        l <- behead l
(1.2?2)        i <- i + 1
(1.2-- )      }
(1---- )      }

post =
  if !found then let i0 = -1 in (0 <= i0 => onth l0 i0 = Some x0)
  else (0 <= i => onth l0 i = Some x0)

```

Since nothing is asserted about `x` and `l` in the precondition, symbolic execution is out of the question. We need to express the meaning of the `while` statement abstractly. This is done by defining relationships among the variables modified by the body of the `while` that are always true, or *invariant*, no matter how many times the body is executed (including none). The invariant needs to capture enough information about the body of the `while` statement so that when the `while` condition is false (and the body no longer matters), the postcondition is true. In short, the invariant and the `while` condition take the place of the `while` statement, for proof purposes.

Deriving a loop invariant from code is hard in general, so EasyCrypt requires the user to provide it. The `while` tactic works on a statement judgement that ends with a `while` statement. It takes a formula, `I`, as an argument and generates two subgoals. The first one asserts that the formula is in fact invariant:

```
hoare [body: I /\ b ==> I]
```

where `b` is the `while` condition. The second one asserts that the program minus the `while` statement makes the invariant true (i.e., before the `while` statement is reached), and that no matter how the `while` statement changes the variables, if the `while` condition is false, the invariant makes the postcondition true. Formally, let $\mathcal{V}$ be the set of variables that might be modified by the `while` body and let $b(\mathcal{V})$, $I(\mathcal{V})$ and $Q(\mathcal{V})$ denote b , I and Q , respectively, written over those variables. The second subgoal is

```
hoare [prog\while: P ==> (I /\ forall v, ! b(v) => I(v) => Q(v))]
```

The quantification expresses our ignorance of the exact values of $\mathcal{V}$, which depend on how many times the `while` body was executed, if any: it is sufficient to know that $I(\mathcal{V})$ is true and $b(\mathcal{V})$ is false.

As is typical, the invariant of the `while` statement above is a conjunction of individual facts or relationships about the variables. An easy one is $0 \leq i$, because i is initially 0, per the precondition, and its value only increases. Slightly less obviously, every time i increases, l gets shorter: $i + \text{size } l = \text{size } l0$. This implies $i \leq \text{size } l0$.

Additional facts are more easily expressed if we use other operators defined in the `List` theory. These play no role in the code but help describe its effects logically and can be reasoned about because of the abundance of lemmas about them. The repeated beheadings can be expressed as $l = \text{drop } i \ l0$, where the `drop` operator removes the specified number of elements from the start of a list. This makes the second invariant redundant, since $\text{size } l = \text{size } (\text{drop } i \ l0) = \text{size } l0 - i$ (using the lemma `size_drop` in `List`, although it's not that simple).

The variable `h` is effectively local to the loop: in other languages, it probably would have been declared inside the body. It can safely be ignored.

The variable `found` is initialized to `false` and becomes `true` when `Some x = ohead l`; the body completes without updating l , preserving the relationship. This can be expressed as $\text{found} \Rightarrow \text{Some } x = \text{ohead } l$. The converse is not true: before the body is executed, `Some x` might equal `ohead l` but `found` is still `false`.

One might be tempted to think ahead and consider

```
found => Some x = onth l 0
found => Some x = onth l0 i
```

but it's better to stay close to the code and let the proof handle any inferences needed.

Two other invariants are more subtle. The `while` statement exits when `x` is found or the list is exhausted. These two conditions can never happen simultaneously: the body either sets `found` or it sets i and l . This can be expressed as $!\text{found} \ \wedge \ l \neq []$, or equivalently as $\text{found} \Rightarrow l \neq []$ or $l = [] \Rightarrow !\text{found}$.

Finally, as long as `found` is `false`, we know `x` hasn't been encountered. The `take` operator retains the front part of a list, just as the `drop` operator retains the back part, and the `(\in)` operator tests for list membership, so we can say $! x \in \text{take } i \ l0$. (Again, we aren't begging the question by using `\in`: the code doesn't use it but it is an accurate description of what the code does.) If the body finds `x`, i is not updated and the relationship is preserved. If it doesn't, i is updated and the relationship is preserved. Thus, it is invariant.

In summary, the invariants of this `while` statement appear to be

```
I1: 0 <= i
I2: l = drop i l0
I3: found => Some x = ohead l
I4: found => l <> []
I5: ! x \in take i l0
```

and others that these imply. This all sounds neat and logical, but in practice my brain, at least, doesn't actually work this way. Have we found all the invariants? Do we need them all? Have we found enough? Did we get any of them wrong? The proof is in the proof. Considering that lemma `HL2` concerns finding `x`, we probably don't need `I5`.

With all that said, the next step of the proof is

```
while (0 <= i /\ l = drop i l0 /\ (found => Some x = ohead l) /\
      (found => l <> [])).
Current goal (remaining: 2)
l0: int list
x0: int
-----
Context : {found : bool, x, i : int, l : int list, h : int
option}

pre =
  (0 <= i /\ l = drop i l0 /\ (found => Some x = ohead l)
   /\ (found => l <> [])) /\ l <> [] /\ !found

(1--) h <- ohead l
(2--) if (Some x = h) {
(2.1)   found <- true
(2--) } else {
(2?1)   l <- behead l
(2?2)   i <- i + 1
(2--) }

post =
  0 <= i /\ l = drop i l0 /\ (found => Some x = ohead l) /\
  (found => l <> [])
```

The body of the `while` statement is easily consumed:

```
wp.
Current goal (remaining: 2)
l0: int list
x0: int
-----
Context : {found : bool, x, i : int, l : int list, h : int
option}

pre =
  (0 <= i /\ l = drop i l0 /\ (found => Some x = ohead l) /\
```

```

(found => l <> [])) /\ l <> [] /\ !found

post =
  if Some x = ohead l then
    0 <= i /\ l = drop i 10 /\
      (true => Some x = ohead l) /\ (true => l <> [])
  else
    let l1 = behead l in
    let i0 = i + 1 in
    0 <= i0 /\ l1 = drop i0 10 /\
      (found => Some x = ohead l1) /\ (found => l1 <> [])

```

This goal has no program, as we've seen before, but `done` isn't going to solve this one. To bring other tactics to bear, we use the `skip` tactic to convert it to an ambient logic formula, which introduces a quantifier over an explicit memory in the process:

```

skip.
Current goal (remaining: 2)
l0: int list
x0: int
-----
forall &hr,
  (0 <= i{hr} /\ l{hr} = drop i{hr} 10 /\
    (found{hr} => Some x{hr} = ohead l{hr})) /\
    (found{hr} => l{hr} <> [])) /\
  l{hr} <> [] /\ !found{hr} =>
  if Some x{hr} = ohead l{hr} then
    0 <= i{hr} /\ l{hr} = drop i{hr} 10 /\
      (true => Some x{hr} = ohead l{hr})) /\ (true => l{hr} <> [])
  else
    let l1 = behead l{hr} in
    let i0 = i{hr} + 1 in
    0 <= i0 /\ l1 = drop i0 10 /\
      (found{hr} => Some x{hr} = ohead l1) /\
      (found{hr} => l1 <> [])

```

It turns out the `smt` tactic can handle this with just one lemma passed to it:

```

smt (behead_drop) .
l0: int list
x0: int
-----
Context : {found : bool, x, i : int, l : int list, h : int
option}

pre = i = 0 /\ found = false /\ x0 = (x, l).`1 /\ l0 = (x, l).`2

post =
  (0 <= i /\ l = drop i 10 /\ (found => Some x = ohead l) /\
    (found => l <> [])) /\
  forall (found0 : bool) (i0 : int) (l1 : int list),

```

```

! (l1 <> [] /\ !found0) =>
0 <= i0 /\ l1 = drop i0 l0 /\ (found0 => Some x = ohead l1)
/\ (found0 => l1 <> []) =>
if !found0 then let i1 = -1 in
  (0 <= i1 => onth l0 i1 = Some x0)
else (0 <= i0 => onth l0 i0 = Some x0)

```

If we try `skip` and `smt` again, however, the latter fails (as of this writing; in the previous version of this Guide, it failed on the goal above, too). It turns out `smt` can solve *most* of this, so let's just pull out all the stops and get to the part it can't as quickly as possible.

```

skip; move => &hr [# P1 P2 P3 P4]; do ! (split; first smt()).
move => found i l /negb_and /= [_ /# | _] [# I1 I2 I3 I4 Q1 Q2]
l0: int list
x0: int
&hr: {found : bool, x : int, l : int list, h : int option}
P1: i{hr} = 0
P2: found{hr} = false
P3: x0 = {x{hr}, l{hr}}.`1
P4: l0 = {x{hr}, l{hr}}.`2
found: bool
i: int
l: int list
I1: 0 <= i
I2: l = drop i l0
I3: found => Some x{hr} = ohead l
I4: found => l <> []
Q1: found
Q2: 0 <= i
-----
onth l0 i = Some x0

```

Examining all those assumptions, we can rewrite the conclusion:

```

rewrite P3 (I3 Q1) I2.
l0 ... Q2
-----
onth l0 i = ohead (drop i l0)

```

With the addition of one more lemma,

```

lemma onth_ohed (s`: 'a list) n:
  0 <= n /\ n < size s => onth s n = ohead (drop n s).
proof.
  case => ge0_n lt_size_n.
  by rewrite (drop_nth witness n s) // (onth_nth witness s n).
qed.

```

we can go back to the start of the second subgoal and solve it all with `smt`:

```

skip. smt (drop_oversize onth_ohed).
No more goals.

```

7.3 Random and Procedure Call Assignments

HL judgements don't deal directly with probabilities and distributions, so what you can say about random processes is somewhat limited. The `rnd` tactic reduces a program that ends with a random assignment statement (`<$`) by consuming the assignment and replacing the goal's postcondition with the assignment's "possibilistic" weakest precondition, which is that the postcondition holds no matter what value the distribution returns (and ignoring the probabilities thereof). For example, given the definitions

```
require import Int Dinterval.
module Dice = {
  proc roll() : int = {
    var v;
    v <$ [1..6];
    return v;
  }
  proc roll2() : int = {
    var sum;
    var d;
    d <@ roll();
    sum <@ roll();
    sum <- sum + d;
    return sum;
  }
}.
```

we can prove a lemma about the sum of two dice rolls:

```
lemma roll2_bnds : hoare [Dice.roll2 : true ==> 2 <= res <= 12].
-----
pre = true

    Dice.roll2

post = 2 <= res <= 12
proc.
-----
Context : {sum, d : int}
pre = true

(1)    d <@ roll()
(2)    sum <@ roll()
(3)    sum <- sum + d

post = 2 <= sum && sum <= 12
inline Dice.roll. (* both occurrences *)
-----
Context : {sum, d, v, v0 : int}
pre = true

(1)    v <$ [1..6]
```

```

(2)    d <- v
(3)    v0 <$ [1..6]
(4)    sum <- v0
(5)    sum <- sum + d

post = 2 <= sum <= 12
wp. (* consume (4) and (5) *)
-----
Context : {sum, d, v, v0 : int}
pre = true

(1)    v <$ [1..6]
(2)    d <- v
(3)    v0 <$ [1..6]

post = let sum0 = v0 + d in 2 <= sum0 <= 12
rnd. (* consume (3) random assignment *)
-----
Context : {sum, d, v, v0 : int}
pre = true

(1)    v <$ [1..6]
(2)    d <- v

post =
  forall (v0_0 : int), (v0_0 \in [1..6]) =>
    let sum0 = v0_0 + d in 2 <= sum0 <= 12
wp. rnd. (* consume (2). consume (1) random assignment *)
Context : {sum, d, v, v0 : int}
pre = true

post =
  forall (v1 : int), v1 \in [1..6] =>
    forall (v0_0 : int), v0_0 \in [1..6] =>
      let sum0 = v0_0 + v1 in 2 <= sum0 <= 12
-----
skip. smt(dinter1E).
No more goals

```

Note that we dealt with the two calls to `Dice.roll` by inlining them. There are more abstract ways of dealing with procedure calls and assignments (`<@`). One is to take advantage of a lemma about the called procedure (where *lemma* again includes axioms and hypotheses). If a goal's conclusion is a statement judgement with precondition P_0 , program C and postcondition Q_0 and ending with a procedure assignment or call to $N.q$, the step

```
call p.
```

uses a proof term for a Hoare lemma about $N.q$. If the lemma has precondition P and postcondition Q , it reduces the goal to the subgoal

```
hoare [C' : P0 ==> P' /\ forall v, Q(v) => Q0(v)]
```

where C' represents C with its last statement removed, P' represents P instantiated with the arguments of the call and $\forall$ stands for all variables modified by the procedure call and its return value. Informally, the subgoal verifies that (1) the calling program satisfies $N.q$ and (2) $N.q$ satisfies the calling program. Any preconditions of the lemma will also be subgoals.

For example, here is a `Dice.roll` lemma:

```
lemma roll_bounds : hoare [Dice.roll : true ==> 1 <= res <= 6].
proof. proc. rnd. skip. smt (). qed.
```

We can modify the proof of `roll2_bnds` to use this lemma in place of the `inline` step as follows:

```
-----
pre = true

    Dice.roll2

post = 2 <= res <= 12
proc.
-----
Context : {sum, d : int}
pre = true

(1)    d <@ roll()
(2)    sum <@ roll()
(3)    sum <- sum + d

post = 2 <= sum <= 12
wp.
-----
Context : {sum, d : int}
pre = true

(1)    d <@ roll()
(2)    sum <@ roll()

post = let sum0 = sum + d in 2 <= sum0 <= 12
call roll_bounds.
-----
Context : {sum, d : int}
pre = true

(1)    d <@ roll()

post =
  forall (result : int),
    1 <= result <= 6 =>
      let sum0 = result + d in 2 <= sum0 <= 12
```

The call tactic consumes only one procedure call, so we do the same thing for the remaining one and finish as before:


```

call roll_bounds.
-----
Context : {sum, d : int}
pre = true

post =
  forall (result : int),
    1 <= result <= 6 =>
      forall (result0 : int),
        1 <= result0 <= 6 =>
          let sum0 = result0 + result in 2 <= sum0 <= 12
skip. smt (dinter1E).
No more goals

```

Another way to abstract a procedure call is to use an ad hoc (or anonymous) Hoare triple, $\{P\} \ N.q \ \{Q\}$:

```

call ( _: P ==> Q ).

```

This form of the `call` tactic generates two subgoals. The first is to prove the triple is valid:

```

hoare [N.q : P ==> Q]

```

Note that this is a procedure judgement, not a statement judgement. For example, we can replace the second `call` step in the `roll2_bnds` proof with a slightly different Hoare triple and prove the first subgoal:

```

call ( _: true ==> res <= 6 /\ 1 <= res ).
Current goal (remaining: 2)
-----
pre = true

Dice.roll

post = res <= 6 /\ 1 <= res
proc. rnd. skip. smt (dinter1E).

```

The second subgoal is the subgoal of the `call p` form:

```

-----
Context : {sum, d : int}
pre = true

post =
  forall (result : int),
    result <= 6 /\ 1 <= result =>
      forall (result0 : int),
        1 <= result0 <= 6 =>
          let sum0 = result0 + result in 2 <= sum0 <= 12
skip. smt (dinter1E).
No more goals

```

In effect, you prove a lemma about the called procedure and then immediately use it.

It's not a coincidence that the `call` tactic subgoals resemble those of the `while` tactic, as both replace a body of code with a formula. If $P = Q$, the resemblance is even stronger.

There is a third form of the `call` tactic,

```
call (_: I).
```

that is just shorthand for

```
call (_: I ==> I); first proc.
```

to get started on the first subgoal. I is an invariant of the called procedure because it is both precondition and postcondition. I don't know why this form of `call` requires that.

7.4 Abstract Procedure Call Assignments

All of the above is for *concrete* procedures, where the body of the procedure is known. Variations of `call` and `proc` are used for *abstract* procedures, where the body is not known. This happens in a formula, axiom, lemma or section declaration when a formal variable for a module is quantified over a module type or functor, such as $M <: T$.

To prove a lemma about an abstract procedure, the `proc` tactic is again the starting point. The form for an abstract procedure call is

```
proc I.
```

where I is an invariant satisfied by the called procedure. It's not clear to me why there is no $P ==> Q$ form for abstract procedures, except that lemmas about them tend to emphasize what they cannot do rather than what they can. The tactic reduces the goal to the following subgoals

```
P ==> I
I ==> Q
{I} N.q {I}
```

where $N.q$ stands for any procedure that the type of M allows $M.p$ to call and P and Q are the pre- and post-condition, respectively, of the current goal.

For example, given declarations

```
require import Int IntDiv.
module type OR = {
  proc f1() : unit
  proc f2() : unit
  proc f3() : unit
}.

module Or : OR = {
  var x : int
  proc f1() : unit = {
    x <- x + 2;
  }
  proc f2() : unit = {
    x <- x - 2;
  }
}
```

```

proc f3() : unit = {
  x <= x + 1
}
}.

module type T (O : OR) = {
  proc g() : unit {O.f1, O.f2}
}.

module (Tx : T) (O : OR) = {
  var x : int
  proc g() : unit {
    x <- 0;
    O.f1(); O.f2(); O.f1();
    x <- x + 3;
  }
}.

```

suppose we want to prove the following lemma:

```

lemma g_parity (M <: T{-Or}) :
  hoare [M(Or).g : 0 = Or.x %% 2 ==> Or.x %% 2 = 0] .
M(O : OR) : T{-Or}
-----
pre = 0 = Or.x %% 2

    M(Or).g

post = Or.x %% 2 = 0

```

Note how abstract module `M` is displayed in the goal's context with its type's memory and procedure restrictions. Also note that `P` and `Q` (and `I` in the proof below) are logically equivalent but are written differently so we can tell them apart in the subgoals. The `proc` tactic generates four subgoals because `N.g` generically can be either `Or.f1` or `Or.f2`:

```

proc (Or.x %% 6 %% 2 = 0) .
(* Subgoal 1: P => I *)
M(O : OR) : T{-Or}
-----
forall &hr, 0 = Or.x{hr} %% 2 => Or.x{hr} %% 6 %% 2 = 0
smt () .
(* Subgoal 2: I => Q *)
M(O : OR) : T{-Or}
-----
forall &hr, Or.x{hr} %% 6 %% 2 = 0 => Or.x{hr} %% 2 = 0
smt () .
(* Subgoal 3: {I} Or.f1 {I} *)
M(O : OR) : T{-Or}
-----
pre = Or.x %% 6 %% 2 = 0
    Or.f1

```

```

post = Or.x %% 6 %% 2 = 0]
proc. wp. skip. smt ().
(* Subgoal 4: {I} Or.f1 {I} *)
M(O : OR) : T{-Or}
-----
pre = Or.x %% 6 %% 2 = 0
      Or.f2
post = Or.x %% 6 %% 2 = 0]
proc. wp. skip. smt ().

```

If a procedure called in a statement judgement is abstract, you can apply `call p` or `call (_ : P ==> Q)` to it, as for a concrete procedure call. The procedure judgement subgoal generated by the `P ==> Q` form will be abstract, so the only choice then is `proc I`. The third form, `call (_ : I)`, is mostly just shorthand for `call (_ : I ==> I); first proc I`, except it prunes the ambient logic subgoals generated by `proc` because they are redundant: the second `call` subgoal subsumes them. For example, in the proof above we can start by converting the procedure judgement to a statement judgement:

```

M(O : OR) : T{-Or}
-----
pre = 0 = Or.x %% 2

      M(Or).g

post = Or.x %% 2 = 0
proc*.
M(O : OR) : T{-Or}
-----
pre = 0 = Or.x %% 2

(1)  r <@ M(Or).g()

post = Or.x %% 2 = 0
call (: Or.x %% 6 %% 2 = 0).
M(O : OR) : T{-Or}
-----
pre = Or.x %% 6 %% 2 = 0
      Or.f1
post = Or.x %% 6 %% 2 = 0]
proc. wp. skip. smt ().
M(O : OR) : T{-Or}
-----
pre = Or.x %% 6 %% 2 = 0
      Or.f2
post = Or.x %% 6 %% 2 = 0]
proc. wp. skip. smt ().
M(O : OR) : T{-Or}
-----
pre = 0 = Or.x %% 2

```

```

post = Or.x %% 6 %% 2 = 0 &&
  forall (x : int), x %% 6 %% 2 = 0 => x %% 2 = 0
skip. smt ().

```

Here is an example where a concrete module is parameterized on an abstract one.

```

module type T = {
  proc f() : unit
}.
module G(X : T) = {
  var x : int
  proc g() : unit = {
    X.f();
  }
}.
lemma Invar (X <: T{-G}) (n : int) :
  hoare [G(X).g : G.x = n ==> G.x = n].
X : T{-G}
n: int
-----
pre = G.x = n
      G(X).g
post = G.x = n
proc.
X : T{-G}
n: int
-----
Context : {}
pre = G.x = n

(1)  X.f()

post = G.x = n
call (: n = G.x).
X : T{-G}
n: int
-----
pre = G.x = n

post = n = G.x && (n = G.x => G.x = n)
done.

```

Module `G` and procedure `G.g` are concrete. In the lemma, `G` is passed an abstract module `X` of type `T`. Nothing is known about the abstract procedure `X.f` except that `X`'s global variables do not include `G.x`. The claim that calling `X.f` will not affect the value of `G.x` is both obvious and trivial to prove. The `call` tactic generates only the `I && (I => Q)` subgoal.

7.5 Other Tactics

See *Hoare Logic Tactics* in the appendix for more Hoare logic tactics.

See `crypto/PRP`, `distributions/{DList, SDist}`, `looping/{Fold, Loop}Proc` and `modules/RndProc`.

8 Probabilistic Hoare Logic Proofs

See *Probabilistic Hoare Logic Tactics* in the appendix.

A potential example is

```
lemma LL1 islossless M.indexOf.
```

Another is to flip a coin with $\{0,1\}$ and claim the result is 0 with probability 0.5. *Dice* theory could also be used. The first code example in the RM has two, but they rely on a pRHL lemma.

9 Probabilistic Relational Hoare Logic Proofs

See *Probabilistic Relational Hoare Logic Tactics* in the appendix.

A potential example is the code in the Intro comparing one coin flip with the XOR of two flips.

The formula P may refer to the parameters of M . p in memory $\&1$ (e.g., as $x\{1\}$) and those of N . q in memory $\&2$ and Q may refer to $res\{1\}$ and $res\{2\}$.

10 Other Program Logic Tactics

Probability assertions may not justify their own section, or they may be all that remains to cover.

11 More on Theories

Theories do more than provide a way to organize material—they are a powerful means for large-scale reuse and abstraction. This section describes aspects of theories not yet considered, including nested theories (sub-theories), abstract theories, theory cloning and polymorphic theories. It then describes two kinds of declaration that implicitly involve theories. Finally, it describes sections of theories, which provide temporary namespaces for declarations that are used to prove global lemmas and then deleted.

11.1 Sub-Theories

A top-level theory T is a script file named $T.ec$ containing steps. A theory can also be nested in another theory by delimiting its boundaries:

```
theory Xyz.  
  <steps of the theory.>  
end Xyz.
```

Once the `end` step is reached, the theory can be imported or exported.

For example, the `List` theory has a sub-theory named `Iota` that defines an operator, `iota_`, that takes two integers, m and n , and returns the list of consecutive integers from m to $m + n - 1$ and proves 13 lemmas about it. Another sub-theory, `Range`, defines a similar operator, `range`, that takes two integers, m and n , and returns the list of consecutive integers from m to $n - 1$ and proves 24 lemmas about it. `List` exports both theories, so importing `List` gives unqualified access to both. These sub-theories simply bundle together a set of related facts, much as `List` itself does.

A sub-theory defines its own set of namespaces, so it can reuse identifiers used in higher-level theories. This is another circumstance where qualified names become necessary. Consider this deliberately confusing file `T.ec`:

```

type t.
op (+) : t -> t -> t.
axiom addA : associative (+).
(* A *)
theory T.
  type t.
  op (+) : t -> t -> t.
  axiom addA : associative T.(+).
  (* B *)
  theory T.
    type t.
    op (+) : t -> t -> t.
    axiom addA : associative T.(+).
    (* C *)
  end T.
  (* D *)
end T.
(* E *)

```

The outermost `t` has the qualified name `T.t` based on the file name, but this name cannot be used within `T.ec` itself—it can only be used in other theory files that `require T`. The top-level theory `T` does not exist until end-of-file is reached, which serves as its `end` step. Instead, the theory being defined can refer to itself generically as `Self`. Thus, the outermost `t` is uniquely identified by the qualified name `Self.t` anywhere in the file following its declaration.

At the point labeled `B`, even though sub-theory `T` is not ended, the code can refer to it, so its elements can be referred to unambiguously as `T.t`, `T.(+)` and `T.addA` or with the fully qualified names `Self.T.t`, `Self.T.(+)` and `Self.T.addA`. When `T.(+)` is declared, EasyCrypt assumes `t` refers to the type `T.t` but when `T.addA` is declared, EasyCrypt complains that multiple `(+)` operators are defined unless a qualified name is used, as shown. In the inner sub-theory `T`, `addA` refers to `T.(+)`, which is now ambiguous, but EasyCrypt assumes it refers to the inner one, meaning `T.T.(+)`, a.k.a. `Self.T.T.(+)`. At the point labeled `C`, the only way to refer to the “middle” `(+)` is `Self.T.(+)`, “from the top,” as it were. At the point labeled `D`, `T.(+)` now refers to the innermost definition and you again need `Self.T.(+)` for the middle one. From the point labeled `E`, `T.(+)` refers to the middle definition and `T.T.(+)` refers to the innermost one.

A theory can refer to an external theory (one in another file) or its elements using an “absolute” or fully qualified name that starts at a notional “root” theory `Top` that contains all top-level theories. Fully qualified names for elements of `T` therefore include `Top.T` for the whole theory, `Top.T.t`, `Top.T.(+)`, `Top.T.addA`, `Top.T.T`, `Top.T.T.t`, `Top.T.T.T` and so on.

Warning. EasyCrypt treats `Top` and `Self` virtually as synonyms, so `Top.X` (or `Self.X`) refers to both a top-level theory `X` and an element `X` in the current file. This is fine for an element `X` that is not a theory (e.g., a type or operator, in a different namespace)—no confusion can result; however, a file (`T.ec`, say) cannot both `require X` and have a sub-theory named `X`, even though `Top.X` (in file `X.ec`) and `Top.T.X` (in the file `T.ec`) are obviously distinct. This “feature” feels like a bug to me. As a best practice, you should never use `Top` to refer to something in the same file: when the file is required by another file, such references can become problematic. Unfortunately, when you print an identifier, EasyCrypt always uses `Top` and never `Self`, so beware.

11.2 Abstract Theories

An abstract theory is a theory whose elements cannot be referenced. It therefore makes no sense to import or export an abstract theory, either, so this is not allowed.

The file extension “.eca” indicates that the top-level theory the file defines is abstract. An abstract sub-theory is indicated by adding the keyword `abstract`:

```
abstract theory Xyz.  
  <steps of the theory.>  
end Xyz.
```

Abstract theories are made useful by cloning (copying) them. They are also typically polymorphic (parameterized). Neither cloning nor polymorphism, however, is limited to abstract theories. These two concepts are the topics of the next two sections.

11.3 Theory Cloning

Cloning a theory makes a copy of it, adding its text to the current theory, something like the `#include` directive in C and C++. In its simplest form, that’s all it does. For example,

```
clone T.
```

creates a new sub-theory named `T` in the current theory that is an exact copy of the existing theory `T`. If you clone a sub-theory, such as `A.B.C`, the new sub-theory is named `C`. If `T` and the current theory are both top-level theories, trying to clone `T` will result in a name clash (see Warning in *Sub-Theories*). To give the sub-theory a different name, do

```
clone T as T'.
```

Once created, a clone can be imported or exported just as any other theory can. This can be combined with the clone step by using

```
clone import T ...  
clone export T ...
```

Instead of creating a sub-theory, cloning can instead copy the elements of a theory into the current theory as its own elements.

```
clone include T.
```

Cloning with inclusion is very useful in defining a theory as an extension of another. For example, the algebraic concept of a magma, a set with a binary operation, can be represented by the theory

```
theory Magma.  
  type S.  
  op (\o) : S -> S -> S.  
end Magma.
```

One can then succinctly define successively richer algebraic structures with cloning:

```
theory SemiGroup.  
  clone include Magma.  
  axiom oA : associative (\o).  
end SemiGroup.
```



```

theory Monoid.
  clone include SemiGroup.
  op id : S.
  axiom left_id : left_id id (\o).
  axiom right_id : right_id id (\o).
  lemma iteropE : forall (n : int) (x : M),
    iterop n plus x id = iter n (plus x) id.
end Monoid.

```

The lemma states that `iterop` and `iter` (in the `Int` theory) are closely related when an operator has an identity element.

Elements within a theory can be renamed when it is cloned using one or more `rename` clauses. For example, we might prefer the type in `Monoid` to be named `M`.

```

theory Monoid
  clone include SemiGroup rename "S" as "M".
  op id : M.
  axiom left_id : left_id id (\o).
  axiom right_id : right_id id (\o).
end Monoid.

```

Renaming is a lexical operation: every occurrence of "S" in the name of a type, operator, predicate, lemma, module, module type or theory is changed to "M," so "abScd" becomes "abMcd," for example. A renaming that matches nothing is not an error: it just has no effect.

A renaming can specify one or more categories to limit the scope of renaming:

```
rename [type, op] "oldname" as "newname"
```

Multiple renamings are listed without punctuation:

```

rename [type] "oldname1" as "newname1"
      [op] "oldname2" as "newname2"

```

which is the same as using multiple `rename` clauses. Renamings are sequential, so if a later one matches the result of a previous one, that name will be rewritten again.

The first string of a renaming is, in fact, a regular expression that allegedly implements Perl-Compatible Regular Expressions (PCRE). Table 1 summarizes the main syntax. The second string is the value to substitute for each match of the first string. There does not seem to be any provision for capturing part of a pattern from the regular expression and referring to it in the substitution.

Table 1 Regular Expressions for Renaming

Metacharacter	Meaning
.	Any character
^	Start of string
\$	End of string
?	The previous pattern is optional
+	One or more occurrences of the previous pattern

<code>*</code>	Zero or more occurrences of the previous pattern
<code>{n}</code>	Exactly n occurrences of the previous pattern
<code>{m, n}</code>	From m to n occurrences of the previous pattern
<code>re₁ re₂</code>	One occurrence of either re ₁ or re ₂
<code>(...)</code>	Grouping to override operator precedence

Named and symbolic operators are tricky to rename because many of their characters have meaning in a regular expression and must be escaped. For example, to rename `(\o)` in Magma, one might try

```

rename "o" as "plus".          (* New name is (\plus) *)
rename "\o" as "plus".         (* New name is (\plus) *)
rename "\\o" as "plus".        (* illegal regular expression: \o *)
rename "\\o" as "plus".        (* illegal regular expression: \o *)
rename "\\o" as "plus".        (* New name is plus *)
rename "(\\o)" as "plus".       (* New name is plus *)
rename "\\(\\o)" as "plus".     (* New name is plus *)
rename "\\(\\o)" as "plus".     (* No match *)

```

EasyCrypt does not seem to let you rename an operator to a symbolic name, such as `(+)`. This might be a bug. A workaround is to use normal prefix names to keep renaming simple and then define abbreviations with symbolic names.

Speaking of abbreviations, if the theory being cloned has them, such as

```

theory Group.
  clone include Monoid
  rename [type] "M" as "G" [op] "\\o" as "plus".
  op inv : G -> G.
  axiom left_inv : left_inverse id inv plus.
  axiom right_inv : right_inverse id inv plus.
  abbrev minus (a b : G) : G = plus a (inv b).
end Group.

```

they can be selectively removed like this:

```

theory MyGroup.
  clone include Group
  rename "plus" as "times"
  remove abbrev minus.
  abbrev div (a b : G) : G = times a (inv b).
end MyGroup.

```

Abbreviations can also be renamed (recall they belong to the `op` category), which in this case would have been simpler. No other kind of element can be removed, however.

A cloned theory is concrete by default. It can be declared abstract like this:

```
clone [abstract] Monoid ... .
```

This option cannot be combined with `import`, `export` or `include`. A cloned theory can be declared concrete like this

```
clone [-abstract] Monoid ... .
```

but since this is the default, it is rarely done.

For some reason, when a theory contains a module with a declared type, the type is stripped when the theory is cloned. As we’ve said, a module need not declare any of the types it satisfies.

11.4 Polymorphic (ish) Theories

Theories are not polymorphic in the traditional sense of having parameters that can take on arbitrary values, but when a theory is cloned, its elements can be **overridden** to achieve a similar effect. For example, our `Monoid` theory describes an abstract structure that manifests itself in many ways across mathematics and computer science. That’s the point of `Monoid`—it describes what is common to a diverse set of instances. One such instance is the integers with addition and 0. We can state (and prove!) that this is a monoid with the following step:

```
clone Monoid as IntAsMonoid with
  type M = int,
  op id = 0,
  op plus = CoreInt.add,
  axiom oA = addzA,
  axiom left_id = add0z,
  axiom right_id = addz0.
```

The `with` clause selectively **overrides** elements of a theory with new definitions. The lemma comes along “for free,” since it is implied by the axioms. If we print `IntAsMonoid`, we get

```
theory IntAsMonoid.
  type M = int.
  op plus : int -> int -> int = (+).
  lemma oA: associative plus.
  op id : int = 0.
  lemma left_id: left_id id plus.
  lemma right_id: right_id id plus.
  lemma iteropE: forall (n : int) (x : M),
    iterop n plus x id = iter n (plus x) id.
end IntAsMonoid.
```

The type `M` is no longer abstract, although its definition, `int`, is itself abstract. Similarly, `plus` and `id` are no longer abstract. (Recall that `(+)` is an abbreviation in the `Int` theory, which `print` uses in place of `CoreInt.add`.) More importantly, the monoid axioms are now lemmas—they follow from the axioms and lemmas governing addition, specifically `addzA`, `add0z` and `addz0`. The clone step used the proof engine to prove that the lemmas provided prove the axioms using the `exact` tactic. We say that the `Monoid` axioms have been **realized** or **discharged** in the clone.

One can argue that we haven’t learned much that is new about `int`, other than recognizing that it, 0 and `plus` have the monoid structure. The lemma `iteropE` is new, and certainly with a more complex theory, say of groups or rings, we might gain true insight by recognizing a familiar structure. One could even argue that `M`, `plus` and `id` have outlived their usefulness in `IntAsMonoid`. We can act on this by varying the override operator that we use. If we use `<=` in place of `=`,

```
clone Monoid as IntAsMonoid' with
```

```

type M <= int,
op id <= 0,
op plus <= CoreInt.add,
axiom oA <= addzA,
axiom left_id <= add0z,
axiom right_id <= addz0.

```

the new sub-theory is

```

theory IntAsMonoid'.
  type M = int.
  op plus : int -> int -> int = (+).
  lemma (* hidden *) oA: associative (+).
  op id : int = 0.
  lemma (* hidden *) left_id: left_id 0 (+).
  lemma (* hidden *) right_id: right_id 0 (+).
  lemma iteropE: forall (n x : int),
    iterop n (+) x 0 = iter n ((+) x) 0.
end IntAsMonoid'.

```

Note that the lemmas now refer to 0 and (+) rather than id and plus. The <= operator retains the type and operator names of Monoid but in-lines their definitions each time they are used. It also marks the former-axiom lemmas as “hidden,” which they aren’t.

We can go further by using the <- override operator:

```

clone Monoid as IntAsMonoid'' with
  type M <- int,
  op id <- 0,
  op plus <- CoreInt.add,
  axiom oA <- addzA,
  axiom left_id <- add0z,
  axiom right_id <- addz0.

```

which gives

```

theory IntAsMonoid''.
  lemma iteropE: forall (n x : int),
    iterop n (+) x 0 = iter n ((+) x) 0.
end IntAsMonoid''.

```

The <- operator again in-lines the definitions for M, id and plus but then it deletes them. It also deletes the axioms once they are proved as lemmas. The result is a theory with a single lemma. The clone step still proved that int, plus and 0 are a monoid, but without leaving any clutter.

The choice of =, <= or <- is independent for each override; they may be intermixed freely in one with clause. For example, we can retain the axioms-as-lemmas by doing

```

clone Monoid as IntAsMonoid''' with
  type M <- int,
  op id <- 0,
  op plus <- CoreInt.add,
  axiom oA = addzA,

```

```

axiom left_id = add0z,
axiom right_id = addz0.

```

to get

```

theory IntAsMonoid''.
  lemma oA: associative (+).
  lemma left_id: left_id 0 (+).
  lemma right_id: right_id 0 (+).
  lemma iteropE: forall (n x : int),
    iterop n (+) x 0 = iter n ((+) x) 0.
end IntAsMonoid''.

```

While on the subject of deleting what is no longer useful, **clearing** a theory deletes all of its lemmas, and if the theory is then empty, deletes it as well. A list of theories to clear can be given in an explicit `clear` step or as part of a theory's end step:

```

clear [IntAsMonoid.*]. (* Clear sub-theory *)
end [* T.*].           (* Clear current theory & sub-theory T *)

```

Critically, clearing only happens when the current theory is closed, even with the `clear` step—if you print `IntAsMonoid` right after clearing it, it will be intact. If you omit the brackets from `clear`, the error “cannot process [proof script] outside a proof script” is produced because `clear` is also a proof tactic.

The library theory `Monoid` (in `algebra/Monoid.eca`) illustrates an interesting technique. This theory puts its axioms in a sub-theory named `Axioms`. It proves each axiom as a lemma in `Monoid` and a number of other lemmas as well. `Ring.ZModule` clones `Monoid` as `AddMonoid`, realizes the axioms in `AddMonoid.Axioms` and clears `AddMonoid.Axioms.*`, which deletes it.

It was highly convenient, above, that the `Int` theory had lemmas `addz0` and so on that so closely aligned with the axioms of `Monoid`. When this is not the case, one can treat them like any other lemma to prove using the full power of the proof engine and all its tactics. The most flexible way is to identify which ones will be proved and then provide the proofs immediately after the `clone` step:

```

clone Monoid as IntAsMonoid
  with type M = int,
    op id = 0,
    op plus = CoreInt.add
  proof oA, left_id, right_id.

```

After this step, EasyCrypt displays

```

Remaining lemmas to prove:
oA: associative plus
left_id: left_id id plus
right_id: right_id id plus

```

EasyCrypt is now technically in proof mode (you can't declare a new type or what-not), and yet there is no active proof. The `realize` step is used to select a lemma to work on:

```

realize left_id.
proof.

```

```

    split.
qed.

```

As each proof is finished, EasyCrypt again lists the remaining lemmas to prove. If you print the theory, it is displayed assuming all the proofs will succeed, with the axioms as lemmas. Since the axioms were not overridden, they will always be shown. Short proofs can be combined with the `realize` step like this:

```

realize oA by apply addzA.
realize right_id by [].

```

If you intend to prove all of the axioms in the theory you are cloning, you can use `proof *` instead of listing them:

```

clone Monoid as IntAsMonoid
  with ...
  proof *.

```

A third way to provide proofs of axioms, intermediate in terseness, is as part of the `proof` clause. Each axiom name or `*` can be followed by a proof consisting of a single *core tactic* (see *Tactical Precedence*; note that `by []` is not allowed in a `proof` clause):

```

clone Monoid as IntAsMonoid
  with ...
  proof oA by apply addzA, left_id by split, right_id by split.

```

The two proofs that are the same can be combined, such as

```

clone Monoid as IntAsMonoid
  with ...
  proof oA by apply addzA, * by split.

```

where `*` means “every axiom not listed explicitly.” You can also defer some proofs and provide others, such as

```

clone Monoid as IntAsMonoid
  with ...
  proof oA, * by split.
  realize oA by apply addzA.

```

Finally, you can have more than one `proof` clause and intermix them with `remove` and `rename` clauses; all three go after `with`. To illustrate this, let’s show that `int`, `0`, `(+)` and `[-]` form a group:

```

clone Group as IntAsGroup
  with type G = int,
    op id = 0,
    op inv = [-],
    op plus = CoreInt.add,
    axiom oA = addzA
  proof * by split
  remove abbrev minus
  proof left_inv, right_inv.
  realize left_inv by apply addNz.
  realize right_inv by apply addzN.

```

Note that `proof *` refers only to `right_id` and `left_id`, as the other axioms are mentioned by name in a later `proof` clause or proved in the `with` clause.

Cloning with type constructors introduces just a little more syntax for the type variables:

```
theory TCMagma.
  type 'a S.
  op (\o) : 'a S -> 'a S -> 'a S.
end TCMagma.

theory TCSEmiGroup.
  clone include TCMagma.
  axiom oA ['a] : associative (\o)<:'a>.
end TCSEmiGroup.

theory TCMonoid.
  clone include TCSEmiGroup
  rename [type] "S" as "M"
  [op] "\\o" as "plus".
  op id : 'a M.
  axiom left_id ['a] : left_id id plus<:'a>.
  axiom right_id ['a] : right_id id<:'a> plus.
end TCMonoid.

clone TCMonoid as ListAsTCMonoid
  with type 'a M <- 'a list,
  op id ['a] <= "[]",
  op plus ['a] = (++)
  proof *.
  realize oA by apply catA.
  realize left_id by apply cat0s.
  realize right_id by apply cats0.
```

In all of the examples so far, we have only overridden abstract types and operators and axioms. Abstract predicates work the same way as operators. Overriding concrete types, operators and predicates is legal, but the new value must equal the existing one (technically, the two definitions must *unify*). A `with` clause does not have to override all abstract types, operators and predicates, in which case they remain abstract. For that matter, the values provided may themselves be abstract. Additionally, not all axioms need to be proved, in which case they remain axioms. Note that this carries the risk of introducing a contradiction: nothing will stop me from doing

```
clone Monoid as IntAsMonoid
  with type M = int,
  op id = 1,
  op plus = CoreInt.add.
```

but since 1 is not the identity for addition, the theory is not sound because it contains `left_id` and `right_id` as axioms. As long as I try to prove the axioms, I will quickly realize my mistake.

A module type or functor can be overridden by naming another module type or functor that is compatible, meaning it has the same parameter types (the names don't matter) and the same signatures—no more and no less.

A sub-theory can also be overridden by naming a theory that is compatible. This means that for each element of the first theory, except lemmas, the second theory has a corresponding element with the same name that is compatible with it; the resulting element in the clone is the unification of the two elements. Extra elements are allowed in the overriding theory and are ignored.

A module cannot be overridden; consequently, a theory that contains a module cannot be overridden, either.

A lemma does not need to be overridden, but it turns out you can override it (using the `axiom` or `lemma` keyword) with another lemma. There is no proof obligation—literally any lemma will do.

Using the `<=` override operator on a module type, functor, sub-theory or lemma is the same as using `=`. Using the `<-` operator deletes the element from the clone following the compatibility check.

Here is a final example with a little bit of everything.

```
module type OR = {
  proc guess (x : int) : unit
  proc guessed () : bool
}.

theory P.
  type t.
  op f : t -> t.
  pred p : t.
  axiom f_inj (x y : t) : f x = f y => x = y.
  axiom p_f_mono (x : t) p x => p (f x).
  module type M (Or : OR) {
    proc get () : t
    proc put (x : t) : unit
  }.
  module (N : M) (Or : OR) = {
    var x : t
    proc get () : t = { return x; }
    proc put (y : t) : unit = { x <- y; }
  }.
theory Q.
  type u.
  op g : u -> u.
  pred q : u.
  axiom g_inj (x y : u) : g x = g y => x = y.
  axiom q_g_mono (x : u) : q x => q (g x).
  module type O (Or : OR) {
    proc get () : t
    proc put (x : t) : unit
  }.
theory R.
```



```

    type v.
    op join : v -> v -> v.
    pred comp : v & v.
    axiom joinC (x y : v) : join x y = join y x.
    axiom comp_refl (x : v) : comp x x.
    lemma comp_joinC (x y : v) : comp (join x y) (join y x).
        proof. rewrite joinC. apply comp_refl. qed.
    end R.
end Q.
end P.

(* Using [<-] means R' contains only [comp_joinC] *)
clone P.Q.R as R' with
    type v <= int,
    op join <= (+),
    op comp <= (<=),
    axiom joinC <= addzC,
    axiom comp_refl <= lezz.

clone P.Q as Q' with
    (* If R' uses [<-], missing elements make this fail: *)
    theory R = R'.

op idz (x : int) : int = x.
lemma idz_inj (x y : int) : idz x = idz y => x = y by smt ().
pred posp (x : int) = 0 < x.
lemma posp_idz_mono (x : int) : posp x => posp (idz x).
module type Z (Or : OR) {
    proc get () : t
    proc put (x : t) : unit
}.

clone P as S with
    type t = int,
    op f = fun x => x,
    pred p = fun x => 0 < x,
    axiom f_inj = idz_inj,
    axiom p_f_mono = posp_idz_mono,
    (* Using [<-] deletes M from S *)
    module type M = Z,
    theory Q = Q'.

print theory R'.
theory R'.
    type v = int.
    op join = (+).
    pred comp = (<=).
    lemma (* hidden *) joinC : forall (x y : int), x + y = y + x.
    lemma (* hidden *) comp_refl : forall (x : int), x <= x.
    lemma comp_joinC : forall (x y : int), x + y <= y + x.

```

```

end R'.

print theory Q'.
theory Q'.
  type u.
  op g : u -> u.
  pred q : u.
  axiom g_inj (x y : u) : g x = g y => x = y.
  axiom q_g_mono (x : u) : q x => q (g x).
  module type O (Or : OR) {
    proc get () : t
    proc put (x : t) : unit
  }.
  theory R.
    type v = R'.v.
    op join : int -> int -> int = (+).
    pred comp = (<=).
    lemma joinC (x y : v) : join x y = join y x.
    lemma comp_refl (x : v) : comp x x.
    lemma comp_joinC (x y : v) : comp (join x y) (join y x).
  end R.
end Q'.

print theory S.
theory S.
  type t = int.
  op f (x : int) : int = x.
  pred p (x : int) = 0 < x.
  lemma (* hidden *) f_inj (x y : int) : x = y => x = y.
  lemma (* hidden *) p_f_mono (x : t) 0 < x => 0 < x.
  module type M(Or : OR) {
    proc get() : t {Or.guess, Or.guessed}
    proc put(x : t) : unit {Or.guess, Or.guessed}
  }.
  module N(Or : OR) = {
    var x : int
    proc get () : int = { return N.x; }
    proc put (y : int) : unit = { N.x <- y; }
  }.
  theory Q.
    type u = Q'.u.
    op g : Q'.u -> Q'.u = Q'.g.
    pred q = Q'.q.
    lemma (* hidden *) g_inj :
      forall (x y : Q'.u), Q'.g x = Q'.g y => x = y.
    lemma (* hidden *) q_g_mono :
      forall (x : Q'.u), Q'.q x => Q'.q (Q'.g x).
    module type O (Or : OR) {
      proc get () : Q'.u {Or.guess, Or.guessed}
      proc put (x : Q'.u) : unit {Or.guess, Or.guessed}
    }

```

```

}.
theory R.
  type Q'.R.v.
  op join : int -> int -> int = Q'.R.join.
  pred comp = Q'.R.comp.
  axiom joinC : forall (x y : Q'.R.v),
    Q'.R.join x y = Q'.R.join y x.
  axiom comp_refl : forall (x : Q'.R.v), Q'.R.comp x x.
  lemma comp_joinC : forall (x y : Q'.R.v),
    Q'.R.comp (Q'.R.join x y) (Q'.R.join y x).
end R.
end Q.
end S.

```

11.5 Subtype Declarations

A **subtype** is a type that is related to another type by inclusion: every element of the subtype is also an element of the so-called **supertype**. In a strongly typed system, inclusion is represented by two operators: an injection from the subtype to the supertype and a partial projection from the supertype to the subtype. The abstract library theory `Subtype` (in the `structures` folder) formalizes these elements as follows:

```

type T, sT.
op P : T -> bool.
op insub : T -> sT option.    (* Partial projection *)
op val    : sT -> T.          (* Injection *)
axiom inhabited : exists (x : T), P x.
axiom insubN (x : T) : !P x => insub x = None.
axiom insubT (x : T) : P x => omap val (insub x) = Some x.
axiom valP (x : sT) : P (val x).
axiom valK : pcancel val insub.
op insubd (x : T) = odflt witness (insub x).

```

along with proving various lemmas. When cloning this theory, the intent is for `T`, `sT` and `P` to be overridden and `inhabited` to be proved. The last requirement is because all types in `EasyCrypt` must be non-empty. The remaining axioms define the operators.

A **subtype declaration** is syntactic sugar for cloning the `Subtype` theory. To use it, the file must require `Subtype`.

The library theory `ZModP` (in the `algebra` folder) provides an example. It contains an abstract sub-theory, `ZModRing`, parameterized on an integer `p` that must be at least 2, that constructs the ring $\mathbb{Z}/p\mathbb{Z}$, which is the ring over the set $\{0, 1, \dots, p-1\}$ (or integers modulo p). It defines the elements of the ring like this:

```

const p : { int | 2 <= p } as ge2_p.
subtype zmod as Sub = { x : int | 0 <= x < p }
  rename "to_zmod", "from_zmod".
realize inhabited by exists 0; smt (ge2_p).

```

The form of declaration used for `p` was described in *Shorthand Notations for Axioms*. The part in braces is essentially an anonymous subtype of `int` with no operators. The same notation is incorporated into

the declaration of the subtype `zmod`. The `rename` clause renames the operators `insub` and `insubd` using the first string and `val` using the second; it is optional. The subtype declaration is exactly equivalent to the following:

```
type zmod.
clone Subtype as Sub
  with type sT <- zmod,
        type T   <- int,
        op   P   <- fun x => 0 <= x < p
  rename "val" as "from_zmod"
        "insub" as "to_zmod"
proof inhabited.
```

The last line explains why the subtype declaration is followed by `realize inhabited`. Sub-theory `Sub` can be imported or exported, and its elements can be accessed using qualified names, as usual.

11.6 Ring and Field Instances

As mentioned in *Type Classes and Instances*, instances of certain algebraic types can be declared and are used by the `algebra`, `field` and `ring` tactics. There are indications that instances will be expanded to encompass user-defined type classes in general.

The syntax of an **instance declaration** is similar to that of a `clone` step, but there is no library theory that is being cloned and no sub-theories are created (but see the `Requires` sub-theory in `tactics/AlgTactic.ec`). For example, the library theory `StdRing` (in the `algebra` folder) defines the commutative ring over `int` as

```
instance ring with int
  op rzero = Coreint.zero
  op rone  = CoreInt.one
  op add   = CoreInt.add
  op opp   = CoreInt.opp
  op mul   = CoreInt.mul
  op expr  = Ring.IntID.exp

  proof oner_neq0 by smt()
  proof addr0     by smt()
  proof addrA     by smt()
  proof addrC     by smt()
  proof addrN     by smt()
  proof mulr1     by smt()
  proof mulrA     by smt()
  proof mulrC     by smt()
  proof mulrD1    by smt()
  proof expr0     by smt(Ring.IntID.expr0)
  proof exprS     by smt(Ring.IntID.exprS).
```

and the commutative ring over `bool` as

```
op bid (b : bool) = b.

instance bring with bool
```

```

op rzero = false
op rone  = true
op add   = Bool.( ^^ )
op mul   = (/ \)
op opp   = bid

proof oner_neq0 by smt ()
proof addr0     by smt ()
proof addrA     by smt ()
proof addrC     by smt ()
proof addrK     by smt ()
proof mulr1     by smt ()
proof mulrA     by smt ()
proof mulrC     by smt ()
proof mulrDl    by smt ()
proof mulrK     by smt ()
proof oppr_id   by smt ().

```

There are slight differences in the axioms for Boolean rings (`addrN` vs. `addrK` and `mulrK`). The `expr` (exponentiation by an `int`) operator is optional for both kinds; as seen with the `bring` example, if it isn't overridden, its axioms aren't generated, either. Another optional operator is `sub` (subtraction), with an axiom, `subrE`. When the type underlying the ring is not `int`, the `ofint` operator is also optional; it has four axioms. A `field` instance adds one required operator, `inv` (the inverse of `mul`, or reciprocal), and one optional one, `div` (division). The optional operators are optional because they have standard definitions in terms of the other operators (viz., their axioms constitute definitions) but may have more “natural” definitions in particular rings or fields. Note that for `sub` and `div`, abbreviations will not do—they have to be overridden by genuine operators. Like with a `clone`, any axioms without a `proof` clause can be proved following the `instance` step with `realize` steps. To see the full list of axioms with their definitions, omit all the `proof` clauses.

Rings and fields both have so-called modular forms parameterized on a coefficient modulus, `n`, and/or a power modulus, `p`. These are provided like this:

```
instance ring with t 3 5 ...
```

where `n = 3` and `p = 5`; both are `int` literals greater than or equal to 2 such that the ring (or field) satisfies one or both of these axioms:

```

axiom Cn_eq0 : ofint n = rzero.
axiom Cp_idp : forall (x : t), expr x p = x.

```

If `n = p = 2`, the ring or field is Boolean. One or both moduli can be an underscore.

Note the differences from a `clone` step. The overrides in the `with` clause are separated by white space, not commas. They also must use `=` and trying to use `<=` or `<-` gives a parse error. In the optional `proof` clauses, `by τ` is mandatory, `by []` is forbidden and they cannot prove `*` or multiple axioms. `Rename` and `remove` clauses are not allowed. Finally, since no sub-theory is created, instances do not have names (other than the name of the type underlying the ring or field).

The theory `Real` defines an instance of `ring` and one of `field`, both over `real`. The library theory `Ring` (in the `algebra` folder) defines an instance of `ring` over the abstract type `ComRing.t`, `bring`

over `BoolRing.t` and `field` over `Field.t`, so any theories derived from them (such as `Poly.Poly` for polynomials over a coefficient field) can be reasoned about using the algebra tactics. The library theory `ZModP` (in the algebra folder) defines an instance of `ring` over `ZModP.ZModRing.zmod` for integers modulo $p \geq 2$ and an instance of `field` over `ZModP.ZModField.zmod` for integers modulo $p \geq 2$ where p is prime.

11.7 Sections

A **section** textually delimits part of a theory. Sections are opened by the `section` step and closed by the `end section` step. Section names are optional and must begin with an uppercase letter. Sections may be nested. For example, here are a bunch of empty sections:

```
section.
  section.
  end section.
  section A.
    section A.
    end section A.
  end section A.
end section.
```

Sections are naming scopes that permit creating local declarations that can be used by other declarations in the section. Declarations are made local using the `declare` keyword. The following kinds of declarations made within sections can be declared local to the section: abstract types, abstract operators (including constants and predicates) and axioms.

In proofs done within a section, local declarations are included in the local context of the initial goal of the proof. When a section is closed, its local declarations are no longer available. Global declarations that refer to them will be generalized with respect to them so they're still usable outside the section. This example, adapted from the `List` theory, defines an operator `listmax` that returns the maximum value in a list relative to an arbitrary transitive and strongly connected (a.k.a. linear) order (note the embedded `print` steps, with their output following):

```
section ListMax.
  declare type t.
  declare axiom rel_trans : transitive rel.
  declare axiom rel_conn (a b : t) : rel a b \ / rel b a.
  lemma rel_refl : reflexive rel
    by move => x; elim (rel_conn x x).
  print lemma rel_refl.
lemma rel_refl: reflexive rel.
op listmax_bounded (x : t) (xs : t list) : t =
  with xs = [] => x
  with xs = x' :: xs' =>
    listmax_bounded (if rel x x' then x' else x) xs'.
op listmax (dfl : t) (xs : t list) : t =]
  with xs = [] => dfl
  with xs = x :: xs' => listmax_bounded x xs'.
lemma listmax_in (dfl : t) (xs : t list) :
  size xs <> 0 => listmax dfl xs \in xs.
proof. <omitted for brevity> qed.
```

```

    print lemma listmax_in.
lemma listmax_in: forall (dfl : t) (xs : t list),
  size xs <> 0 => listmax dfl xs \in xs.
end section.
print type t.
no such object in the category [type declarations]
print lemma rel_refl.
lemma rel_refl ['t] :
  forall (rel : 't -> 't -> bool),
    transitive rel =>
      (forall (a b : 't), rel a b \/ rel b a) => reflexive rel.
print lemma listmax_in.
lemma listmax_in ['t] :
  forall (rel : 't -> 't -> bool),
    transitive rel =>
      (forall (a b : 't), rel a b \/ rel b a) =>
        forall (dfl : 't) (xs : 't list),
          size xs <> 0 => Top.listmax rel dfl xs \in xs.

```

Comparing the definitions within the section to those following it, we see that the type `t` has been deleted and the lemmas `rel_refl` and `listmax_in` now have a type variable `'t` and hypotheses `transitive rel` and `forall (a b : 't), rel a b \/ rel b a`, the bodies of the `rel_trans` and `rel_conn` axioms, respectively, which were also deleted. The claim is that the operators and lemmas are easier to state and prove within the section and no harder to use after it.

Declarations can also be made local using the `local` keyword. Such declarations are deleted at the end of the section but are not used to generalize anything. They may not be referenced by global or declared declarations but can be referenced in proofs. The following kinds of declarations can use the `local` keyword: types, operators (including constants and predicates), lemmas (but not axioms), type classes, type instances, theories, cloned theories, rewrite, exact and solve hints (but not simplify hints), notations, abbreviations, modules, module types and proc ops. Given the restrictions, `local` declarations are much less common.

12 Pragmas

Pragmas are meta-steps that configure how EasyCrypt operates or, in some cases, take an immediate action. Some pragmas have simple names consisting of a single identifier, such as

```

pragma silent.
pragma verbose.

```

which are mutually exclusive settings for whether the proof engine displays the current goal(s) after each proof step. Others have compound names consisting of two or more identifiers separated by colons, such as

```

pragma Goals:printone.
pragma Goals:printall.

```

which are mutually exclusive settings for whether the proof engine displays the current goal only or all goals after each proof step. Some pragmas are Boolean options that can be turned on with `+` and turned off with `-`, such as

```
pragma +implicit.
pragma -implicit.
```

If you omit the + or –, the error “unknown pragma” is produced, which is misleading. Finally, there is a provision for integer-valued pragmas, but none are defined. Table 2 lists all currently defined pragmas.

Pragmas can also be set from the command line:

```
$ easycrypt -pragma +implicit, Goals:printall.
```

Pragmas are not like regular steps in two ways: they do not increment the current step number (as seen in the interactive command-line prompt; see *Running EasyCrypt*) and they cannot be undone (e.g., in Proof General—if you Undo one or Goto or edit a line above it in the script, its effect remains). You are better off retracting the file or exiting EasyCrypt and restarting it.

Fun Fact. The interactive command-line prompt, such as

```
[18|check]>
```

displays the current step number and which Proofs pragma is in effect (*check*, *weakcheck* or *report*). Proof General hides this prompt, but you can see it in the **easycrypt** buffer.

Table 2 List of Pragmas

Name	Kind	Meaning
noop	command	N/A
compact	command	runs the garbage collector
reset	command	resets EasyCrypt to its initial state, as if no input has been processed, but preserves pragma settings
restart	command	resets EasyCrypt to its initial state (see reset) and sets the verbose, Goals, Proofs and PrPo pragmas to their defaults
silent	one of a set	in a proof, do not display goal(s) after each command
verbose	one of a set	in a proof, display goal(s) after each command (default)
Goals:printall	one of a set	in a proof, always display all goals
Goals:printone	one of a set	in a proof, only display the current goal (default)
Proofs:check	one of a set	unknown (default)
Proofs:weak	one of a set	unknown
Proofs:report	one of a set	unknown
PrPo:pr:raw	one of a set	unknown (default)
PrPo:pr:ands	one of a set	unknown
PrPo:po:raw	one of a set	unknown (default)
PrPo:po:ands	one of a set	unknown
implicit	Boolean	be more aggressive in inferring arguments to fill holes in proof terms (default false)
redlogic	Boolean	apply user-defined reduction rules (simplify hints) in the <i>logic</i> tactic (default true)
und_delta	Boolean	report when an unfolding operation has no effect in the <i>rewrite</i> tactic or <i>@ intro</i> pattern (default false)

oldip	Boolean	for backward compatibility (default false)
old_mem_restr	Boolean	for backward compatibility (default false)

13 Appendix: Quick Reference

13.1 Lexical Details

A **comment** is arbitrary text delimited by “(” and “)”. Comments are treated as white space.

Comments may be nested:

```
(* Commented out:
op D(a b c : real) : real = b * b - 4%r * a * c. (* verify *)
*)
```

Note. Literal strings are not recognized in comments, so a comment like

```
(* rename "times" as "(*)" *)
```

is interpreted as an unterminated comment: “(”)” opens a nested comment, so the outer one is not closed.

An **identifier** is a sequence of letters, digits, underscores (`_`) and apostrophes (`'`) that begins with a letter or underscore. An identifier cannot be equal to a single underscore or most (but not all) keywords. In addition, EasyCrypt enforces the following restrictions on how identifiers are used:

- Names of modules, module types, (abstract) theories and sections are identifiers that begin with an uppercase letter. This applies to script files, as their names are theory names.
- Names of variables (both global and local), type variables (after an initial apostrophe) and procedures are identifiers that do not begin with an uppercase letter (this subtle wording allows `'_'` as the first character, which must otherwise be lowercase).
- Names of types, type constructors, record field projections, parameters (of anonymous functions, operators, constructors, predicates, quantifiers, axioms, lemmas, procedures, modules and module types), axioms and lemmas are identifiers.
- Names of operators (including constructors) and predicates are identifiers or symbolic operator names (see below).
- A **memory identifier** consists of an ampersand followed by either a nonempty sequence of digits or an identifier that does not begin with an uppercase letter.

Identifiers must be unique within the following namespaces:

- A namespace per theory or section for types and type constructors.
- A namespace per theory or section for operators, constructors, predicates, field projections and abbreviations.
- A namespace per theory or section for modules, module types and theories, except if a module type is declared first, a module or a theory (not both) may have the same name. This exception may be a bug.
- A namespace per theory or section for axioms and lemmas.
- A namespace per module for global variables.
- A namespace per module for procedures.

- A namespace per anonymous function (including let expressions), operator and predicate for parameters.
- A namespace per quantifier for formal variables.
- A namespace per procedure for parameters and local variables.
- A namespace per type, operator or predicate declaration, axiom or lemma for type variables.

This means, for example, that a type and an operator in the same theory can have the same name, since they are in different namespaces, but an operator and a predicate cannot.

In addition to using prefix notation, operators (including constructors and abbreviations but not predicates) can be applied using infix or mixfix notation by naming them with certain reserved names and following additional rules when declaring them and using them as values. Specifically, a **symbolic operator name** is one of a fixed set of strings and patterns (mostly punctuation) broken out by how they can be used: unary, binary or mixfix. See *Symbolic Operator Names*.

13.1.1 Reserved Words

Most keywords are reserved (may not be used as identifiers). They are

abbrev, abstract, admit, algebra, alias, apply, as, assert, assumption, auto, axiom, axiomatized, beta, by, byequiv, byphoare, bypr, byupto, call, case, cbv, cfold, change, class, clear, clone, congr, conseq, const, coq, declare, delta, do, done, eager, ehoare, elif, elim, else, end, equiv, eta, exact, exfalse, exists, export, fail, fel, fission, for, forall, fun, fusion, glob, goal, have, hint, hoare, idtac, if, import, in, include, inductive, inline, instance, iota, is, islossless, kill, lemma, let, local, locate, logic, match, modpath, module, move, notation, of, op, outline, phoare, pose, Pr, pr_bounded, pragma, pred, print, proc, progress, proof, prover, qed, rcondf, rcondt, realize, reflexivity, remove, rename, replace, require, res, return, rewrite, rnd, rndsem, rwnormal, search, section, Self, seq, sim, simplify, skip, smt, sp, split, splitwhile, subst, subtype, suff, swap, symmetry, then, theory, time, timeout, Top, transitivity, trivial, try, type, undo, unroll, var, weakmem, while, why3, with, wp, zeta

The keywords that are not reserved are

abort, admitted, async, check, debug, dump, ecall, edit, exit, exlim, expect, field, fieldeq, first, fix, from, gen, global, interleave, last, left, right, ring, ringeq, solve, wlog

EasyCrypt is case-sensitive, so `aBBrev` and so on are valid identifiers.

13.1.2 Symbolic Operator Names

Binary operators

- A backward slash (`\`) followed by a sequence of letters, digits, underscores and apostrophes can be used as a binary infix operator, such as `x \in m` or `S1 \subset S2`. These are called **named operators**.
- A sequence of the symbols `=`, `<`, `>`, `/`, `\`, `+`, `-`, `*`, `|`, `:`, `&`, `^` and `%` surrounded by backticks (```) can be used as a binary infix operator, such as `a`<`b` or `a`==//&=`b`.
- *Selected* sequences of these symbols without backticks can be used as binary infix operators. Almost any sequence of these symbols can be declared as an operator, but many will give parse errors when you try to use them infix.

- These symbols are expressly allowed: +, -, *, /, \, ^, &, &&, /\, ||, \/, <=>, =, <>, <, <=, > and >=
- These symbols are expressly reserved: :, #, #|, //, //#, /=, /#, //=#, />, //>, |>, ||>, |, :=, <-, ->, <<-, ->>, %, <<, >>, <:, ==>, <*>, <<*> and <*>>
- These substrings are recognized as tokens even when embedded in larger symbols: #, #|, //, //#, /=, /#, //=#, />, //>, |>, ||>, |, :=, <-, ->, <<-, ->>. For example, `a ==//&= b` is recognized as 5 tokens: `a`, `==`, `//`, `&=` and `b`, which is a syntax error.

Unary operators

- A named operator can be used as a unary prefix operator, such as `\rev s` or `\conj z`.
- The symbol `!`, sequences of `+` and sequences of `-` can be used as unary prefix operators, such as `!b` (used for Boolean not) or `-5` (used for int and real negation).
- Since prefix notation is the default, the benefit is that these names are also assigned a precedence level.

There are exactly four mixfix operators. Their names contain underscores to indicate where arguments go when they are applied. Typical usage is as follows:

- `[]` is a nullary operator (a constant) for *empty* or *null* values, such as an empty list.
- ``|_|` is a unary operator for *magnitudes* or *norms*, such as the absolute value of an integer (``|x - y|`), length of a vector or determinant of a matrix.
- `_. [_]` is a binary operator for accessing part of an object by an *index*, such as an element of a list by position (`a. [i]`) or of a map by a key (`map. [key]`).
- `_. [_<-_]` is a ternary operator for adding or overriding part of an object with an *index* and a *value*, such as `a. [i<-x]` or `map. [key<-value]`. These return a new object with one element changed—they do not modify the object. In the module language, however, `m. [k<-v]`; is treated as a shortcut for the assignment `m <- m. [k<-v]`;
- White space is allowed around and within arguments but not between symbols: ``| x |` and `map. [ key <- value ]` are legal, but ``|x|` and `a. [i]` are not.

Symbolic and named operators have assigned precedence levels and associativities to reduce the need for parentheses. All named operators have the same precedence and are left-associative. Symbols enclosed in backticks are assigned a precedence level based on their first character and are left-associative. The precedence hierarchy from lowest to highest is

- `=>` and `<@` (right-associative)
- `<=>` (non-associative)
- `||` and `\/` (right-associative)
- `&&` and `/\` (right-associative)
- `!` (non-associative)
- `=` and `<>` (non-associative)
- named operators (left-associative)
- `<`, `>`, `<=` and `>=` (left-associative)
- backtick operators starting with '`<`', '`>`' or '`=`' (left-associative)
- conditional expressions (right-associative)
- `-`, `+` and backtick operators starting with '`-`', '`+`' or '`|`' (left-associative)

- $\rightarrow$ (right-associative)
- $*$, $/$, any nonempty combination of $\&$, $/$ and $\%$ (other than $//$, which is illegal) and backtick operators starting with $'*$, $'/'$, $'\&'$ or $'\%'$ (left-associative)
- $@$, $\&$, $^$, $\backslash$, all remaining infix operators except sequences of colons and all remaining backtick operators (left-associative)
- sequences of colons of length at least two (right-associative)

13.2 Type Expressions and Declarations

Type expressions are built up inductively by applying the following rules:

Syntax	Description
a	named type (a is an identifier)
$'a$	type variable (a is an identifier)
(t)	type expression in parentheses, to override precedence
$t_1 * t_2 * \dots$	tuple (or product) of type expressions (non-associative)
$t_1 \rightarrow t_2 \rightarrow \dots$	function type between type expressions (right-associative)
$t_1 \& t_2 \& \dots$	function type with implicit codomain (in a constructor for an inductive type or in a predicate declaration)
$t \ a$ $(t_1, \dots, t_n) \ a$	type constructor application (a is the type constructor; it is applied postfix) with multiple type arguments
$\text{glob } M$	type of tuples consisting of a value for each of the global variables of module M

Named type declarations have one of the following forms:

Syntax	Description
<code>type t.</code>	abstract type
<code>type $t = t'$.</code>	concrete type equivalent to the type expression t'
<code>type $rt = \{$ $f_1 : t_1; \dots \}$. <code></code></code>	record type field projection f_1 of type (expression) t_1
<code>type $dt = [$ $\ c_1$ $\ c_2 \text{ of } t$ $\ c_3 \text{ of } t \& dt$ <code>]</code>. <code></code></code>	inductive type (first $ $ is optional) nullary constructor (constant) c_1 of type dt constructor c_2 of type $t \rightarrow dt$ recursive constructor c_3 of curried type $t_1 \rightarrow dt \rightarrow dt$
<code>type $'a \ t$.</code>	abstract type constructor
<code>type $'a \ t = t'$.</code> <code>type $('a_1, \dots) \ t'$.</code>	type constructor with one type variable referenced in type expression t' with multiple type variables

Constructors and field projections are operators. Constructors can have symbolic operator names.

13.3 Operator and Predicate Declarations

Operator and predicate declarations have one of the following forms:

Syntax	Description
<code>op $f : t$.</code> <code>const $f : t$.</code>	abstract operator of type t same, but t must not be a functional type
<code>op $f ['a] : 'a \ t$.</code>	abstract operator whose type is a type constructor
<code>op $f : t_1 \rightarrow t_2$.</code>	abstract operator of a functional type

<pre>op f (x : t₁) : t₂. op f ['a 'b] (x : 'a) : 'b.</pre>	preferred alternative syntax polymorphic abstract operator
<pre>op f (x₁ : t₁) (x₂ : t₂) : t₃ op f (x₁ : t₁, x₂ : t₂) : t₃</pre>	abstract operator of a nested functional type, alternative syntax another alternative syntax
<pre>op [tag1 tag2] f : t.</pre>	abstract operator with tags (concrete ones may also have tags)
<pre>op f : t = e. const f : t = e. op f ['a] : 'a list = []<:'a>.</pre>	concrete operator equivalent to the expression <i>e</i> same, but <i>t</i> must not be a functional type concrete operator whose type is a type constructor
<pre>op f : t₁ -> t₂ = fun (x : t₁) => e. op f (x : t₁) : t₂ = e. op f x = e.</pre>	concrete operator of a functional type pedantic syntax preferred alternative syntax with parameter and return types inferred from <i>e</i>
<pre>op f (x : dt) : t = with x = c₁ => e₁ with x = c₂ y => e₂ with x = c₃ y₁ y₂ => e₃.</pre>	operator over an inductive type (type <i>dt</i> is defined above) pattern matching over nullary constructor <i>y</i> is a pattern variable for the parameter of <i>c</i>₂ pattern variables for curried constructor <i>c</i>₃
<pre>pred p : t.</pre>	abstract predicate of type <i>t</i> -> bool
<pre>pred p ['a] : 'a t.</pre>	abstract polymorphic predicate of type 'a t -> bool
<pre>pred p : t₁ & t₂.</pre>	abstract predicate of curried type t₁ -> t₂ -> bool rather than (t₁ -> t₂) -> bool
<pre>pred p ['a 'b] : 'a & 'b. pred p : t = fun (x : t) => e. pred p (x : t) = e. pred p x = e.</pre>	abstract polymorphic predicate of curried type concrete predicate equivalent to the bool expression <i>e</i> pedantic syntax preferred alternative syntax with parameter type inferred from <i>e</i>
<pre>abbrev f (x : t₁) : t₂ = e. abbrev f = g. abbrev unzip1 ['a 'b] (s : ('a * 'b) list) : 'a list = map fst s.</pre>	declare <i>f</i> (<i>x</i>) as an abbreviation for the expression <i>e</i> declare <i>f</i> as an abbreviation for the operator <i>g</i> abbreviation over polymorphic operators and type constructors

Operators, predicates, constructors, field projections and abbreviations share one namespace, called either `op` or `pred` in a print step.

A partial projection operator (a.k.a. destructor) is generated automatically for each constructor of an inductive type. For type *dt*, above, they are

```
op get_as_c1 (the : dt) : unit option =
match the with
| c1 => Some tt
| c2 x => None
| c3 x y => None
end.
op get_as_c2 (the : dt) : t option =
match the with
| c1 => None
```

```

    | c2 x => Some x
    | c3 x y => None
  end.
op get_as_c3 (the : dt) : (t * dt) option =
  match the with
  | c1 => None
  | c2 x => None
  | c3 x y => Some (x, y)
  end.

```

13.4 Typed Expressions

Typed *expressions* are built up inductively from the following forms. Expressions of type `bool` are also called *formulas* (e.g., in some system messages).

Syntax	Description
0, 1.5, tt, true, false	literal values of type <code>int</code> , <code>real</code> , <code>unit</code> , <code>bool</code> and <code>bool</code> , respectively
5%r	convert <code>int</code> 5 to <code>real</code>
x	variable (global, local, logical or parameter)
(e)	typed expression in parentheses, to override precedence
(e ₁ , e ₂ , ...)	tuple expression
e.`1	access first component of a tuple expression (a kind of operator application)
{ f ₁ = e ₁ ; ... }	record expression (all fields must be assigned values)
{ r with f ₁ = e ₁ ; ... }	record equal to <code>r</code> except for specified fields
e.`f ₁	access a field of a record expression (a kind of operator application)
e%T	interpret expression <code>e</code> in the scope of theory <code>T</code>
fun (x : t) => e	anonymous function of type <code>t -> t'</code> where <code>t'</code> is the type of expression <code>e</code> and <code>e</code> can reference <code>x</code>
fun (x : t ₁) (y : t ₂) => e	nested function of type <code>t₁ -> t₂ -> t'</code> , alt. syntax
fun (x : t ₁ , y : t ₂) => e	nested function of type <code>t₁ -> t₂ -> t'</code> , alt. syntax
(fun (x : t) => e ₁) e ₂	function application
let x : t = e ₂ in e ₁	let expression; alternative syntax for function application
f e ₁ ... e _n	apply operator (or constructor) <code>f</code> to argument expressions <code>e₁ ... e_n</code> where <code>f</code> has at least <code>n</code> parameters If <code>f</code> is the unqualified name of a named or symbolic unary, binary or mixfix operator, it may be applied in prefix, infix or mixfix notation, respectively
f<:t ₁ , t ₂ > e ₁ ... e _n	apply polymorphic operator <code>f</code> , instantiating its type variables with argument types <code>t₁</code> and <code>t₂</code>
p e ₁ ... e _n	apply predicate <code>p</code> to argument expressions <code>e₁ ... e_n</code> where <code>p</code> has at least <code>n</code> parameters
p<:t ₁ , t ₂ > e ₁ ... e _n	apply polymorphic predicate <code>p</code> , instantiating its type variables with argument types <code>t₁</code> and <code>t₂</code>
b ? e ₁ : e ₂ if b then e ₁ else e ₂	conditional expression, where <code>b</code> has type <code>bool</code> , and <code>e₁</code> and <code>e₂</code> have the same type, which is the type of the expression
match e with pattern ₁ => e ₁	match expression, where <code>e</code> has an inductive type, each <code>pattern_i</code> is a constructor of that type applied to the

<code> pattern₂ => e₂ end</code>	appropriate number of pattern variables, all constructors must be listed and each result expression e_i must have the same type, which is the type of the expression
<code>forall (x : t), φ forall &m, φ forall (M <: T), φ forall (M <: T{$\pm N_1, \dots$}), φ</code>	universally quantified variable x of type t , where φ is an expression of type <code>bool</code> that may reference x universally quantified memory universally quantified module M over a module type T with memory restrictions: - $N_i, +N_i$: <code>glob M intersect glob N_i</code> must be empty or may be non-empty, respectively - $N_i.x, +N_i.x$: <code>glob M</code> must exclude or may include global variable $N_i.x$, respectively
<code>exists <same as forall></code>	existentially quantified variable; otherwise same as <code>forall</code>
<code>e{m}</code>	the expression e in which every reference u to a variable in <code>&m, res</code> or <code>glob M</code> is replaced by its value in <code>&m</code>
<code>{0,1}</code>	the full, uniform and lossless distribution over <code>bool</code>
<code>[5..10]</code>	the uniform and lossless distribution over <code>int</code> with support $\{i : \text{int} \mid 5 \leq i \leq 10\}$
<code>Pr[M.p(e₁, ..., e_n) @ &m : φ]</code>	probability expression, where M is a module and p a procedure in M running in memory <code>&m</code> , equal to the real probability that $M.p$ terminates with <code>&m</code> satisfying φ
<code>hoare [M.p : $\varphi \Rightarrow \psi$]</code>	Hoare logic judgement, where M is a module, p is a procedure in M and φ and ψ are predicates; if $M.p$ is run with an initial memory satisfying φ and it terminates, the final memory satisfies ψ
<code>ehoare [M.p : $\varphi \Rightarrow \psi$]</code>	Hoare logic judgement, where M is a module, p is a procedure in M and φ and ψ are predicates; if $M.p$ is run with an initial memory satisfying φ and it terminates, the final memory satisfies ψ
<code>phoare [M.p : $\varphi \Rightarrow \psi$] $\diamond e$</code>	probabilistic Hoare logic judgement, where M is a module, p is a procedure in M , φ and ψ are predicates, $\diamond$ is a comparison operator ($<, \leq, =, \geq$ or $>$), and e is a real-valued expression; if $M.p$ is run with an initial memory satisfying φ , then the probability that it terminates and the final memory satisfies ψ has the indicated relationship to e
<code>equiv [M.p ~ N.q : $\varphi \Rightarrow \psi$]</code>	probabilistic relational Hoare logic judgement, where M is a module, p is a procedure in M , φ and ψ are predicates and $M.p$ and $N.q$ run in separate memories, <code>&1</code> and <code>&2</code> , respectively; if $M.p$ and $N.q$ are run and <code>&1</code> and <code>&2</code> initially satisfy φ , then the sub-distributions of the final memories satisfy ψ
<code>islossless M.p</code>	lossless assertion, where M is a module and p is a procedure in M ; $M.p$ always terminates

An *expression pattern* is an expression with holes (underscores), such as `_` or `(f ( _ + _ ) _)`.

13.5 Axioms and Lemmas

Axioms and lemmas have one of the following forms:

Syntax	Description
<pre>axiom ax ['a] (x : t) : φ. axiom ax ['a] : forall (x : t), φ.</pre>	axiom where type variable(s) and parameter(s) are optional and φ may reference the parameters equivalent form with <code>forall</code> instead of parameters
<pre>lemma phoo ['a] (x : t) : φ. proof. <proof steps> qed.</pre>	lemma, which has the same syntax as an axiom but must be followed by a proof
<pre>lemma phoo ['a] (x : t) : φ by τ. by [].</pre>	lemma with an immediate proof τ is a tactic chain equivalent to <code>by (trivial; smt())</code>
<pre>op p : {t φ} as p_ax.</pre>	shorthand for <pre>op p : t. axiom p_ax : φ.</pre>
<pre>op f (x : t₁) : t₂ = e axiomatized by f_def.</pre>	shorthand for <pre>op f (x : t₁) : t₂ = e. axiom f_def : forall (x : t₁), f x = e.</pre>
<pre>op [full uniform lossless] d : t distr.</pre>	shorthand for <pre>op d : t distr. axiom d_ll : is_lossless d. axiom d_uni : is_uniform d. axiom d_fu : is_full d.</pre> Any combination of the tags in any order is legal. Axiom names are generated automatically.
<pre>op [opaque smt_opaque] f (x : t₁) : t₂ = e.</pre>	operator declarator where axioms and lemmas that use <code>f</code> will not be selected by the <code>smt</code> tactic automatically (but can still be selected manually); any combination of the tags in any order is legal.
<pre>hoare phoo (x : t) : [M.p : φ ==> ψ].</pre>	shorthand for <pre>lemma phoo (x : t) : hoare [M.p : φ ==> ψ]</pre>
<pre>ehoare phoo (x : t) : [M.p : φ ==> ψ].</pre>	shorthand for <pre>lemma phoo (x : t) : ehoare [M.p : φ ==> ψ]</pre>
<pre>phoare phoo (x : t) : [M.p : φ ==> ψ] $\diamond$ e</pre>	shorthand for <pre>lemma phoo (x : t) : phoare [M.p : φ ==> ψ] $\diamond$ e</pre>
<pre>equiv phoo (x : t) : [M.p ~ N.q : φ ==> ψ]</pre>	shorthand for <pre>lemma phoo (x : t) : equiv [M.p ~ N.q : φ ==> ψ]</pre>

An induction axiom is generated automatically for a record type. Its name is the type name followed by “_ind”. For type `rt`, above, it is

```
axiom rt_ind : forall (P : rt -> bool),
  (forall (x1 : t1, ...), P {| f1 = x1; ... |}) =>
  forall (x : rt), P x.
```

Two induction axioms are generated automatically for an inductive type. Their names are generated automatically. Each has an induction hypothesis for each constructor. They differ only for recursive constructors. For type `dt`, above, they are


```

axiom dt_case : forall (P : dt -> bool),
  P c1 =>
    (forall (x : t), P (c2 x)) =>
      (forall (x : t, y : dt), P (c3 x y)) =>
        forall (x : dt), P x.
axiom dt_ind : forall (P : dt -> bool)
  P c1 =>
    (forall (x : t), P (c2 x)) =>
      (forall (x : t, y : dt), P y => P (c3 x y)) =>
        forall (x : dt), P x.

```

13.6 Ambient Logic Proofs

After a `lemma` step, the proof engine is started. The optional `proof.` step emphasizes that a proof has begun. The `abort.` step terminates the proof engine unconditionally, abandoning the lemma. The `qed.` step terminates the proof engine and saves the lemma if there are no active goals and the name of the lemma is valid (e.g., not a duplicate); otherwise, it fails and the proof engine remains active. The `admitted` step terminates the proof engine and saves the lemma if the name of the lemma is valid, regardless of any active goals.

13.6.1 Proof Tactics

A proof tactic is a procedure that transforms (or *reduces*) a goal into zero or more *subgoals*. A tactic *solves* or *closes* a goal if it generates no subgoals for it.

Syntax	Description
<code>admit</code>	Closes (solves) the current goal by accepting it without proof.
<code>algebra</code>	Tries to solve the current goal by applying a set of axioms and lemmas governing rings and fields. Fails if it cannot solve the goal. Uses the rewrite databases <code>Ring.rw_algebra</code> and <code>Ring.inj_algebra</code> , which the user can add hints to.
<code>apply</code> <code>apply p</code> <code>apply p in H</code> <code>apply : p</code> <code> {a₁ ... a_m}</code> <code> Π₁ ... Π_n</code> <code>apply : p</code> <code> {a₁ ... a_m}</code> <code> Π₁ ... Π_n</code> <code> in H</code> <code>apply /p₁ ... /p_r</code>	<code>p</code> and <code>p₁ ... p_r</code> are proof terms. <code>apply p</code> Clears assumptions <code>a₁ ... a_m</code> from the context from left to right. <code>apply p in H</code> Generalizes patterns <code>Π₁ ... Π_n</code> from right to left. <code>apply : p</code> If no proof terms are present, applies the top assumption to the rest of the goal's conclusion using backward reasoning and clears the assumption. <code>apply : p</code> If one proof term is specified and <code>in H</code> is specified, applies <code>p</code> to hypothesis <code>H</code> using forward reasoning. <code>apply : p</code> If one proof term is specified and <code>in H</code> is not specified, applies <code>p</code> to the goal's conclusion using backward reasoning. <code>apply /p₁ ... /p_r</code> If multiple proof terms are specified, applies each one from left to right to the last subgoal generated by the previous proof term, or to the current goal for the first one. If no subgoals remain after applying a proof term, it ignores the remaining proof terms.
<code>assumption</code>	Searches in the context for a hypothesis that is convertible to the goal's conclusion, thus solving the goal. Fails if there is no such assumption.
<code>case</code> <code> /p₁ ... /p_r</code> <code> :</code> <code> {a₁ ... a_m}</code>	All of the arguments are optional but must be in the order shown. The underscore is optional, and if not present, the colon is optional. <code>case</code> Clears assumptions <code>a₁ ... a_m</code> from the context from left to right.

$\Pi_1 \dots \Pi_n$	Generalizes patterns $\Pi_1 \dots \Pi_n$ from right to left, with two exceptions. If all of the following conditions are met: 1. There are no proof terms 2. There is exactly one generalization pattern, Π_1 3. Π_1 is an expression pattern 4. Π_1 has no occurrence selector 5. Π_1 has no holes 6. Π_1 is of type <code>bool</code> then it performs excluded middle analysis using Π_1, generating two subgoals, $\Pi_1 \Rightarrow G$ and $! \Pi_1 \Rightarrow G$, where G is the conclusion of the current goal with all occurrences of Π_1 replaced with <code>true</code> or <code>false</code>, respectively. Otherwise, if the underscore is present and Π_1 is an expression pattern, it replaces Π_1 with the patterns $\{-1\} \Pi_1$ and $(eq_refl \Pi_1)$. If Π_1 has an occurrence selector, it is merged with $\{-1\}$ in the first replacement pattern. Applies proof terms $p_1 \dots p_r$ from left to right to the top assumption using forward reasoning. Destructs the top assumption based on its form as follows: • If (it simplifies to) <code>false</code>, solves (deletes) the goal • Replaces $p \ / \ q \Rightarrow \varphi$ or $p \ \&\& \ q \Rightarrow \varphi$ with $p \Rightarrow q \Rightarrow \varphi$ • Replaces $(p \ <=> \ q) \Rightarrow \varphi$ with the subgoals $(p \Rightarrow q) \Rightarrow \varphi$ and $(q \Rightarrow p) \Rightarrow \varphi$ • Replaces $p \ \backslash / \ q \Rightarrow \varphi$ with the subgoals $p \Rightarrow \varphi$ and $q \Rightarrow \varphi$ • Replaces $p \ \ q \Rightarrow \varphi$ with the subgoals $p \Rightarrow \varphi$ and $!p \Rightarrow q \Rightarrow \varphi$ • Replaces $a \ <= \ b \Rightarrow \varphi$ with the subgoals $a \ < \ b \Rightarrow \varphi$ and $a = b \Rightarrow \varphi$ • Replaces $(if \ a \ then \ bt \ else \ bf) \Rightarrow \varphi$ or $a \ ? \ bt : bf \Rightarrow \varphi$ with the subgoals $a \Rightarrow bt \Rightarrow \varphi$ and $!a \Rightarrow bf \Rightarrow \varphi$ • Replaces $(exists \ (x : t), \ \psi \ x) \Rightarrow \varphi$ with $(forall \ (x : t), \ \psi \ x) \Rightarrow \varphi$ • Replaces a universally quantified variable of a tuple type with a universally quantified variable for each of its components • Replaces a universally quantified variable of a record type with a universally quantified variable for each of its components • Replaces $forall \ (x : unit), \ \varphi \ x$ with $\varphi \ tt$ • Replaces $forall \ (x : bool), \ \varphi \ x$ with the subgoals $\varphi \ true$ and $\varphi \ false$ • Replaces $forall \ (x : int), \ 0 \ <= \ x \Rightarrow \varphi \ x$ with the subgoals $\varphi \ 0$ and $forall \ (i : int), \ 0 \ <= \ i \Rightarrow \varphi \ i \Rightarrow \varphi \ (i + 1)$ • Replaces a variable of an inductive type with a subgoal for each of its constructors (i.e., for each hypothesis in the <code>t_case</code> axiom generated for the type)
---------------------	---

	Not to be confused with the program logic <code>case</code> tactic, which has fewer options. Also not to be confused with the <i>case pattern</i>, which is an introduction pattern.
<code>change ϕ</code>	ϕ is a <code>bool</code> expression. Replaces the current goal's conclusion with ϕ , as long as ϕ is convertible to it.
<code>clear $a_1 \dots a_m$</code>	Clears (removes) assumptions $a_1 \dots a_m$ from the current goal's context from left to right. Fails if any assumption cannot be cleared. Not to be confused with the non-proof <code>clear</code> step, which deletes lemmas from a theory.
<code>congr</code>	If the current goal's conclusion has the form $f \ t_1 \dots t_n = f \ u_1 \dots u_n$ it replaces it with n subgoals having conclusions $t_i = u_i$ for $1 \leq i \leq n$ and closes subgoals solvable by <code>reflexivity</code>. Otherwise, it fails.
<code>done</code>	Applies the <code>trivial</code> tactic and fails if the current goal is not closed.
<code>elim /L : $\Pi_1 \dots \Pi_n$</code>	All arguments and the colon are optional but must be in the order shown. Generalizes patterns $\Pi_1 \dots \Pi_n$ from right to left. If L is provided, it eliminates the top assumption using L. This requires the top assumption to be a universally quantified variable and L to be an induction principle for its type. If L is not provided, it eliminates the top assumption like <code>case</code> except on inductive types, where it applies the induction lemma <code>t_ind</code> generated for the type.
<code>exact</code> <code>exact p</code> <code>exact : p $\Pi_1 \dots \Pi_n$</code> <code>exact /p₁ ... /p_r</code>	Equivalent to <code>by apply</code> with the arguments provided. Note that not all forms of <code>apply</code> are allowed.
<code>exists e</code>	If the current goal's conclusion has the form $\text{exists } (x : t), \phi \ x$ and e is an expression of type t, it replaces the conclusion with $\phi \ e$. Otherwise, it fails.
<code>field db₁ ... db_n</code> <code>fieldeq db₁ ... db_n</code>	Tries to solve the current goal by applying a set of axioms and lemmas governing fields, where <code>db₁ ... db_n</code> are zero or more names of rewrite databases to apply as well.
<code>gen have H : / ϕ</code> <code>gen have H, ι</code> <code> : / ϕ</code> <code>gen have H</code> <code> : $a_1 \dots a_m$ / ϕ</code> <code>gen have H, ι</code> <code> : $a_1 \dots a_m$ / ϕ</code>	ι is zero or more introduction patterns. $a_1 \dots a_m$ are names of assumptions in the context. H is an identifier not used in the context. ϕ is a <code>bool</code> expression without holes (an anonymous lemma). Adds the anonymous lemma ϕ as a cut, with the option of generalizing it. That is, it replaces the current goal with two subgoals that have the same context as the current goal. Let G be the conclusion of the current goal. The first subgoal has conclusion ϕ. The second subgoal depends on $H_1 \dots H_n$ and the comma: • If $a_1 \dots a_m$ are absent, ϕ is added to the context as H. • If $a_1 \dots a_m$ are present, ϕ is generalized as if by reverting assumptions a_1 through a_m from right to left and then added to

	the context as H. The context of the second subgoal is not affected by these notional reversions. • If the comma is absent, the conclusion is G • If the comma is present, the conclusion is $\varphi \Rightarrow G$ and any ι present are applied to it.
<pre>have $\iota : \varphi$ have $\iota : \varphi$ by τ have $\iota := p$</pre>	ι is zero or more introduction patterns. In the forms with $:$, φ is a <code>bool</code> expression without holes (an anonymous lemma) and τ is a tactic chain. The tactic adds the anonymous lemma φ as a cut. That is, it replaces the current goal with two subgoals that have the same context as the current goal. Let G be the conclusion of the current goal. The first has conclusion φ and the second has conclusion $\varphi \Rightarrow G$. If ι is present, it applies ι to the second subgoal. If <code>by τ</code> is present, it applies τ to the first subgoal and fails if <code>done</code> fails to close it. Note that <code>by []</code> is not allowed. In the form with <code>:=</code>, p is a proof term for a named (i.e., not anonymous) lemma. The proof term's outer parentheses may be omitted. The tactic instantiates the lemma according to p and adds it as the new top assumption for the original conclusion. If ι is present, it applies ι to this goal. Depending on p, it may generate additional subgoals but does not apply ι to them.
<code>idtac s</code>	The identity tactic, which does not change the current goal. s is an optional string that is printed as a warning message.
<code>left</code>	Replaces the conclusion of the current goal, which must be a disjunction, $p \ \backslash / \ q$ or $p \ \ q$, with p .
<pre>move /$p_1 \dots /p_r$: {$a_1 \dots a_m$} $\Pi_1 \dots \Pi_n$</pre>	All arguments are optional but must be in the order shown. The colon is required if the braces or any Π_i are present. Equivalent to $\text{idtac } \langle @ \{a_1 \dots a_m\} \ \Pi_1 \dots \Pi_n \Rightarrow \ /p_1 \dots /p_r$ where each p_i is a proof term to be applied to the top assumption using forward reasoning (see the introduction tactic), each a_i is an assumption to clear and each Π_i is a generalization pattern (see the generalization tactical for pattern syntax).
<code>pose occ $x := e$</code>	<code>occ</code> is an optional occurrence selector, x is an identifier not used in the context and e is an expression pattern. Finds the first subexpression of the goal's conclusion that matches e, to instantiate any holes it has. If e has no holes, a match is not needed. Adds the local definition $x : t := e$ to the context, where t is the type of the matched subexpression. Replaces all occurrences of e in the goal with x, or selected occurrences if <code>occ</code> is present. See below for occurrence selector syntax.
<pre>progress τ progress @[options] τ</pre>	Both arguments are optional. τ is a <i>core tactic</i>. <code>options</code> is a list of one or more option names separated by spaces. An option preceded by <code>-</code> turns the option off. The <code>@</code> is optional and meaningless. Repeatedly applies <code>simplify</code>, <code>split</code>, <code>assumption</code>, <code>subst</code>, <code>unfolding</code>, introducing assumptions and parts of <code>case</code> to make progress toward solving the goal. From <code>case</code> it matches the false,

	conjunction, existential and tuple equality patterns. It never fails, but may leave the goal unchanged. If specified, applies τ <i>tentatively</i> (i.e., without failing if τ does) after each step. The options and their meanings are <code>solve</code> – apply the <code>assumption</code> tactic <code>split</code> – apply the <code>split</code> tactic <code>subst</code> – apply the <code>subst</code> tactic to the top assumption <code>disjunct</code> – when applying <code>case</code>, include the disjunction pattern <code>delta : case</code> – when applying <code>case</code>, also try unfolding <code>delta : split</code> – when applying <code>split</code>, also try unfolding <code>delta</code> – a shorthand for <code>delta : case</code> and <code>delta : split</code> By default, <code>solve</code>, <code>split</code>, <code>subst</code> and <code>delta : split</code> are selected
<code>reflexivity</code>	Solves a goal with a conclusion of the form $b = b$ (up to computation/conversion/simplification)
<pre>rewrite $\rho_1 \dots \rho_n$ rewrite $\rho_1 \dots \rho_n$ in H</pre>	H is the name of a hypothesis in the context of the current goal. Each rewrite argument ρ_i is either • One of the introduction patterns <code>//</code>, <code>/=</code>, <code>/~=</code>, <code>//=</code>, <code>//~=</code>, <code>/#</code> or <code>//#</code>, with the same meaning as with the introduction tactical. • A proof term, with optional direction indicator, repetition marker, occurrence selector and match pattern prefixed in that order. If the conclusion of the lemma has the form $f1 = f2$ or $f1 <=> f2$, the tactic rewrites all occurrences of the first subexpression of the current goal's conclusion that matches $f1$ as $f2$ (a.k.a. rewriting from left to right); otherwise, it rewrites all occurrences of the first match of the lemma's conclusion as <code>true</code>. Turns unproven hypotheses of the proof term into additional subgoals with the context of the current goal. • The name of a rewrite database, with optional direction indicator, repetition marker, occurrence selector and match pattern prefixed in that order. The tactic tries each lemma in the database as a proof term with no arguments until one of them succeeds, failing if all of them fail. • A list of proof terms and rewrite databases in parentheses and separated by commas, with optional direction indicator, repetition marker, occurrence selector and match pattern prefixed in that order. Each list item may also have a direction indicator, which takes precedence. The tactic tries each list item in the order provided until one of them succeeds, failing if all of them fail. • An expression pattern prefixed by <code>/</code>, with optional direction indicator, repetition marker and occurrence selector before the <code>/</code>. The root of the pattern must be a defined symbol (concrete operator/predicate or local definition; not an abstract operator/predicate, constructor or projection). The tactic unfolds all occurrences of the first subexpression of the current goal's conclusion that matches the pattern.

	• $\& p$, where p is a proof term that the tactic applies to the current goal with the <code>apply</code> tactic. If <code>in H</code> is specified, it applies all patterns to H rather than the goal's conclusion. Each subgoal generated has the context of the current goal up to but not including H. For $i > 1$, it is equivalent to <code>rewrite ρ_1; ...; rewrite ρ_n</code>, meaning each ρ_i is applied to each of the subgoals generated by the preceding pattern. If a given subgoal is solved, the remaining patterns are ignored for that branch of the proof tree. Any pattern may be preceded by a focus index and a colon to specify the subgoals to which to apply it. See <i>Focus Indices</i>. The direction indicator is <code>-</code> and means to instantiate f_2 of a proof term and replace occurrences of it with f_1 (a.k.a. rewriting from right to left) or to fold a symbol rather than unfold it. A repetition marker indicates how many times to apply a pattern. See <i>Repetition Markers</i>. An occurrence selector specifies to which occurrences of the match to apply a pattern. See Occurrence Selectors. A match pattern is an expression pattern in brackets. The first sub-expression in the conclusion or H that matches the pattern is used to instantiate the proof term. If instantiation fails, the proof term fails.
<code>right</code>	Replaces the conclusion of the current goal, which must be a disjunction, $p \ \backslash / \ q$ or $p \ \ q$, with q or $!p \Rightarrow q$, respectively.
<code>ring db₁ ... db_n</code> <code>ringeq db₁ ... db_n</code>	Tries to solve the current goal by applying a set of axioms and lemmas governing rings, where $db_1 \dots db_n$ are zero or more names of rewrite databases to apply as well.
<code>simplify</code> <code>simplify delta</code> <code>simplify op₁ ... op_n</code> One or more of <code>beta</code> , <code>delta</code> , <code>delta op₁ ...</code> <code>op_n</code> , <code>iota</code> , <code>logic</code> or <code>zeta</code>	Reduces the goal's conclusion to its $\beta\iota\zeta\Lambda$ -head normal form, optionally followed by one step of parallel strong δ -reduction (unfolding), optionally restricted to a list of operator names. The tactics <code>beta</code> , <code>iota</code> , <code>zeta</code> and <code>logic</code> reduce the conclusion to the β -, ι -, ζ - and Λ -head normal form, respectively, and can be combined in one proof step. The <code>delta</code> tactic unfolds all or a specified list of operator names. Operators to be unfolded must be <i>defined</i> , meaning concrete operators or predicates that are not constructors, field projections or defined inductively; or local definitions.
<code>smt</code> <code>smt options</code>	Tries to solve the goal using one or more external satisfiability modulo theories (SMT) provers. The provers used and other options are specified by <code>options</code>. Options include • <code>prover=[prover-selector]</code>: which provers to run, as a list of strings in square brackets and separated by spaces. ○ Prover names can specify all, part or none of the version, such as "Alt-Ergo", "Alt-Ergo2" or "Alt-Ergo2.4.2" ○ To modify the global selection, a string can be prefixed with <code>+</code> to add the prover or <code>-</code> to remove it. ○ An unrecognized prover causes the tactic to fail. ○ <code>prover=</code> is optional • <code>timeout=n</code>: maximum time (in seconds) each prover has to respond

smt (dbhint)	• <code>maxprovers=n</code>: maximum number of provers to run in parallel • <code>quorum=n</code>: the minimum number of provers that must return success for the tactic to succeed • <code>verbose</code>: generates more output, such as how long it ran • <code>debug</code>: reports each solver that succeeds, ignoring the rest • <code>dump in="file"</code>: writes a copy of the data sent to Why3 to a file named <code>0000-file</code> in the current folder. Relative or absolute file paths are not allowed. • Lemma selection: A default set of axioms and lemmas to send to the prover(s) is selected automatically. Axioms and lemmas that use operators tagged <code>smt_opaque</code> are not selected. The following <code>smt</code> options further influence selection: ◦ <code>wantedlemmas=(dbhint)</code> ◦ <code>unwantedlemmas=(dbhint)</code> ◦ <code>all</code>: select all available axioms/lemmas except <code>unwantedlemmas</code> and those for <code>smt_opaque</code> operators (<code>ignore wantedlemmas</code>) ◦ <code>maxlemmas=n</code>: set the maximum number of axioms/lemmas selected ◦ <code>iterate</code>: incrementally augment the number of selected axioms/lemmas ◦ <code>selected</code>: displays selected lemmas <i>dbhint</i> is a list of axiom and lemma names separated by spaces • <code>@T</code> denotes all axioms and lemmas in theory <code>T</code> • <code>+</code> or <code>-</code> before a name selects or deselects the name SMT options can also be set globally with any of the steps • <code>prover options.</code> • <code>timeout n.</code> • <code>timeout * n.</code> (* use at most <code>n%</code> of CPU *) Shorthand for <code>smt wantedlemmas=(dbhint)</code>
split	Breaks the current goal into subgoals that have the context of the current goal. The conclusion of the current goal must have one of these “intrinsically conjunctive” forms: • Replaces $\varphi_1 \wedge \varphi_2$ with the subgoals φ_1 and φ_2 • Replaces $\varphi_1 \&\& \varphi_2$ with the subgoals φ_1 and $!\varphi_1 \Rightarrow \varphi_2$ • Replaces $\varphi_1 \Leftrightarrow \varphi_2$ with the subgoals $\varphi_1 \Rightarrow \varphi_2$ and $\varphi_2 \Rightarrow \varphi_1$ • Replaces equality or inequality between <i>n</i>-tuples with <i>n</i> subgoals comparing their components • Replaces equality or inequality between records with subgoals comparing their components • Replaces <code>(if b then φ_1 else φ_2)</code> or <code>b ? φ_1 : φ_2</code> with the subgoals <code>b => φ_1</code> and <code>!b => φ_2</code> • Closes any goal that is convertible to <code>true</code> or provable by the <code>reflexivity</code> tactic
subst subst x	<i>x</i> is the name of a variable in the context. Searches the context for the first hypothesis of the form <code>x = t</code> or <code>t = x</code>, replaces all occurrences of <i>x</i> with <i>t</i> in the context and the conclusion, then clears the hypothesis

	and the variable x. Fails if there is no such hypothesis. If x is not specified, repeatedly applies <code>subst</code> to all variables in the context and never fails.
<code>suff l : ϕ</code> <code>suff l : ϕ by τ</code>	l is zero or more introduction patterns. ϕ is a <code>bool</code> expression without holes (an anonymous lemma). τ is a tactic chain. Adds the anonymous lemma ϕ as a cut. That is, it replaces the current goal with two subgoals that have the same context as the current goal. Let G be the conclusion of the current goal. The first subgoal has conclusion $\phi \Rightarrow G$. If l is present, it applies l to this subgoal. If <code>by τ</code> is present, it also applies τ to this subgoal and fails if <code>done</code> fails to close it. Note that <code>by []</code> is not allowed. The second subgoal has conclusion ϕ. This is equivalent to have <code>l : ϕ [by τ]; first last</code>, except if <code>by τ</code> is present, the tactic applies it to $\phi \Rightarrow G$ rather than to ϕ.
<code>trivial</code>	Runs an unspecified combination of low-level tactics to the current goal, including <code>assumption</code>, <code>reflexivity</code>, <code>simplify</code>, <code>operator unfolding</code> and <code>rewriting</code> with lemmas listed with <code>hint exact</code>. If it can't solve the goal, it does not fail but leaves the goal unchanged. This tactic is run automatically by certain other tactics (e.g., <code>done</code> and <code>exact</code>), tacticals (e.g., <code>by</code>) and introduction patterns (e.g., <code>//</code>).
<code>wlog : $a_1 \dots a_m$ / ϕ</code>	$a_1 \dots a_m$ are optional names of assumptions in the context. ϕ is a <code>bool</code> expression without holes. Reduces a goal to a special case that is sufficient to prove the general case by adding the assumption ϕ. That is, it replaces the current goal with two subgoals that have the same context as the current goal. Let G be the conclusion of the current goal. If $a_1 \dots a_m$ are present, $\phi \Rightarrow G$ is generalized as if by reverting assumptions a_1 through a_m from right to left. The context of the subgoals is not affected by these notional reversions. Call this expression <code>gen_ϕ_G</code>. If $a_1 \dots a_m$ are absent, <code>gen_ϕ_G</code> = $\phi \Rightarrow G$. The first subgoal has conclusion <code>gen_ϕ_G</code> $\Rightarrow G$. The second subgoal has conclusion $\phi \Rightarrow G$.
<code>wlog suff :</code> <code> $a_1 \dots a_m$ / ϕ</code>	$a_1 \dots a_m$ are optional names of assumptions in the context. ϕ is a <code>bool</code> expression without holes. Adds the anonymous lemma ϕ as a cut, with the option of generalizing it. That is, it replaces the current goal with two subgoals that have the same context as the current goal. Let G be the conclusion of the current goal. If $a_1 \dots a_m$ are present, ϕ is generalized as if by reverting assumptions a_1 through a_m from right to left. The context of the subgoals is not affected by these notional reversions. Call this expression <code>gen_ϕ</code>. If $a_1 \dots a_m$ are absent, <code>gen_ϕ</code> = ϕ. The first subgoal has conclusion <code>gen_ϕ</code> $\Rightarrow G$. The second subgoal has conclusion ϕ. These are the same goals generated by <code>gen have</code>, but in the opposite order and without any introductions.

13.6.2 Proof Tacticals

Proof tacticals are used to apply multiple tactics in one proof step or otherwise augment the behavior of a tactic.

Syntax	Description
$\tau \Rightarrow l_1 \dots l_r$	Introduction. Each l_i is an introduction pattern. Apply τ to the current goal and then apply each l_i from left to right to the subgoals generated by the previous pattern. See the table of patterns below. Introduction patterns are also used with the <code>gen</code> , <code>have</code> , <code>have</code> and <code>suff</code> tactics. Selected introduction patterns are used with the <code>case</code> , <code>move</code> and <code>rewrite</code> tactics.
$\tau <@ \{a_1 \dots a_m\}$ $\Pi_1 \dots \Pi_n$	Generalization. Each a_i is the name of an assumption in the context. Each Π_i is a generalization pattern. Apply τ to the current goal and then, for each subgoal, clear $a_1 \dots a_m$ from left to right and process each Π_i from right to left, Π_n to Π_1 . See the table of patterns below. Generalization patterns are also used with the <code>apply</code> , <code>case</code> , <code>elim</code> , <code>exact</code> and <code>move</code> tactics.
$\tau_1 \mid \mid \dots \mid \mid \tau_n$	Alternative. Apply each τ_i in sequence to the current goal until one of them succeeds and fail if all of them fail.
$\tau_1; \tau_2$ $\tau_1; \text{first } \tau_2$ $\tau_1; \text{first } n \tau_2$ $\tau_1; \text{last } \tau_2$ $\tau_1; \text{last } n \tau_2$ $\tau_1; f: \tau_2$	Chain. Apply τ_1 to the current goal and τ_2 to all the subgoals it generates. Apply τ_1 to the current goal and τ_2 only to the first subgoal it generates. Apply τ_1 to the current goal and τ_2 only to the first n subgoals it generates. Apply τ_1 to the current goal and τ_2 only to the last subgoal it generates. Apply τ_1 to the current goal and τ_2 only to the last n subgoals it generates. Apply τ_1 to the current goal and τ_2 only to the subgoals specified by focus index f ; see <i>Focus Indices</i> .
$\tau; [\tau_1 \mid \dots \mid \tau_n]$ $\tau; [f_1: \tau_1 \mid \dots \mid f_k: \tau_k]$ $\tau; [f_1: \tau_1 \mid \dots \mid f_{k-1}: \tau_{k-1} \text{ else } \tau_k]$	Chain. Apply τ to the current goal and then τ_i to subgoal i , failing if there are not exactly n subgoals. For $k \leq n$, apply τ to the current goal and then τ_i to subgoals selected by focus index f_i ; <code>idtac</code> is applied to unselected goals. For $k \leq n$, apply τ to the current goal and then τ_i to subgoals selected by focus index f_i ; τ_k is applied to unselected goals. See <i>Focus Indices</i> .
$\tau; \text{first last}$ $\tau; \text{first } n \text{ last}$ $\tau; \text{last first}$ $\tau; \text{last } n \text{ first}$	Chain. Apply τ to the current goal and move the first subgoal to the end Apply τ to the current goal and move the first n subgoals to the end Apply τ to the current goal and move the last subgoal to the front Apply τ to the current goal and move the last n subgoals to the front
<code>by τ</code> <code>by []</code>	Apply τ to the current goal and then apply <code>done</code> to all subgoals. Apply <code>done</code> to the current goal.
<code>do ! τ</code> <code>do ? τ</code> <code>do n! τ</code> <code>do n? τ</code>	Apply τ to the current goal and then repeatedly apply it to successive lists of subgoals. See <i>Repetition Markers</i> .
<code>try τ</code>	Apply τ to the current goal but do not fail if τ fails.
<code>+ τ</code> <code>- τ</code> <code>* τ</code>	These are optional bullets akin to comments with no formal meaning. They are used informally to mark "significant" points in a proof. Other proof assistants use bullets to indicate the hierarchical structure of a proof.

13.6.3 Introduction Patterns

Most introduction patterns involve the top assumption of the current goal in some way, but some other functions have accreted to the tactical as well.

Syntax	Description
<code>name</code>	Introduces the top assumption with this name.
<code>n ?</code>	Introduces the top assumption anonymously (as <code>_</code> for a hypothesis or <code>`name</code> for a variable or local definition, which are invalid names); the assumption cannot be used by the user but tactics such as <code>trivial</code> and <code>assumption</code> still can. <code>n</code> is an optional non-negative integer saying how many assumptions to introduce anonymously.
<code>*</code>	Successively introduces all assumptions anonymously (see the <code>?</code> pattern).
<code>_</code> (underscore)	Clears (deletes) the top assumption.
<code>^</code> (hat)	Duplicates the top assumption, which must be a hypothesis: the conclusion $\psi \Rightarrow \phi$ becomes $\psi \Rightarrow \psi \Rightarrow \phi$. This is usually followed by a name pattern to move the duplicate to the context, but it needn't be.
<code>+</code>	Clears the top assumption and restores it in each active subgoal when the <code>-</code> pattern is encountered or after the last pattern. If the assumption is a let expression, it is restored as a universally quantified variable with no definition.
<code>-</code> (hyphen)	Restores any assumptions cleared earlier by the <code>+</code> pattern in each active subgoal. If this pattern does not appear, such assumptions are reverted after the last pattern. This pattern also forces a later case pattern to apply the <code>case</code> tactic (by ending the initial sequence of <code>//</code> and similar patterns).
<code>//</code>	Applies the <code>trivial</code> tactic to the goal's conclusion.
<code>/=</code>	Applies the <code>simplify</code> tactic to the goal's conclusion.
<code>/~=</code>	Applies a variant <code>simplify</code> tactic to the goal's conclusion.
<code>//=</code>	Applies the <code>simplify</code> and <code>trivial</code> tactics to the goal's conclusion (shorthand for <code>/= //</code>).
<code>//~=</code>	Applies the variant <code>simplify</code> and <code>trivial</code> tactics to the goal's conclusion (shorthand for <code>/~= //</code>).
<code>/#</code>	Applies the <code>smt()</code> tactic to the goal's conclusion.
<code>//#</code>	Applies the <code>trivial</code> tactic to the goal's conclusion and, if not solved, applies the <code>smt()</code> tactic (shorthand for <code>// /#</code>).
<code>rep ->></code>	Applies the <code>subst</code> tactic using the top assumption (which must be an equality with a variable on the left) left-to-right in the entire goal (i.e., context and conclusion), then clears the assumption and the variable. <code>rep</code> is an optional repetition marker to use multiple top assumptions; see <i>Repetition Markers</i> .
<code>rep <<-</code>	Applies the <code>subst</code> tactic using the top assumption (which must be an equality with a variable on the right) right-to-left in the entire goal (i.e., context and conclusion), then clears the assumption and the variable. <code>rep</code> is an optional repetition marker to use multiple top assumptions; see <i>Repetition Markers</i> .
<code>rep occ -></code>	Applies the <code>rewrite</code> tactic using the top assumption (which must be an equality) left-to-right to the rest of the conclusion, then clears the assumption. <code>rep</code> is an optional repetition marker to use multiple top assumptions; see <i>Repetition Markers</i> . <code>occ</code> is an optional occurrence selector; see <i>Occurrence Selectors</i> .

<code>rep occ <-</code>	Applies the <code>rewrite</code> tactic using the top assumption (which must be an equality) right-to-left to the rest of the conclusion, then clears the assumption. <code>rep</code> is an optional repetition marker to use multiple top assumptions; see <i>Repetition Markers</i> . <code>occ</code> is an optional occurrence selector; see <i>Occurrence Selectors</i> .
<code>n! <*></code> <code>n! <*>></code> <code>n! <<*></code>	Subst top patterns use as many top assumptions as possible, up to <code>n</code> , if <code>n!</code> is present, as substitutions Apply the <code>subst</code> tactic to each goal in either direction Apply the <code>subst</code> tactic to each goal favoring left-to-right Apply the <code>subst</code> tactic to each goal favoring right-to-left
<code> ></code> <code>/></code> <code> ></code> <code>//></code>	Crush mode tries many introduction patterns repeatedly to solve or make progress on as many subgoals as possible, including <code>//</code> , <code>/=</code> , <code>[]</code> and <code>-></code> . It uses <code>+</code> and <code>-</code> to move hypotheses out of the way temporarily. It tries the program logic tactics <code>conseq</code> and <code>auto</code> . Crush mode has two Boolean parameters, <i>unfold</i> and <i>solve</i> , that are encoded in the patterns <code>unfold = false, solve = false</code> <code>unfold = true, solve = false</code> <code>unfold = false, solve = true</code> <code>unfold = true, solve = true</code>
<code>{a₁, ..., a_n}</code>	Applies the <code>clear</code> tactic to the specified assumptions in the context.
<code>@ dir occ /op</code>	Applies the <code>rewrite</code> tactic to unfold occurrences of operator <code>op</code> in the goal's conclusion. If <code>dir</code> is present, <code>op</code> is folded rather than unfolded. <code>occ</code> is an optional occurrence selector; see <i>Occurrence Selectors</i> .
<code>/p</code>	Does <code>apply p</code> to the top assumption using forward reasoning. If <code>p</code> has the form <code>(_ p₁ ... p_n)</code> , it instead instantiates the top assumption's assumptions with <code>p₁ ... p_n</code> .
<code>[l₁₁ ... l_{1m1} ...</code> <code> l_{r1} ... l_{rmr}]</code>	Case pattern. If it is the first pattern after an initial sequence of 0 or more <code>//</code> , <code>/=</code> , <code>/~=</code> , <code>//=</code> , <code>//~=</code> , <code>/#</code> and <code>//#</code> patterns and if <code>τ</code> does something (i.e., is not <code>idtac</code> or <code>move</code>), it applies no tactic; otherwise, it applies the <code>case</code> tactic. If <code>r = 0</code> , it is done; otherwise, for each <code>i</code> , it applies the list of patterns <code>l₁₁ ... l_{1mi}</code> to subgoal <code>i</code> . Fails if there are not exactly <code>r</code> subgoals. See also the following discussion of case modes.

A case pattern may optionally have a **case mode** immediately following the left bracket.

Syntax	Description
<code>#</code>	Applies the conjunction rule only, as many times as possible, to the top assumption.
<code># </code>	Applies the conjunction and disjunction rules only, as many times as possible, to the top assumption. There must not be a space between <code>#</code> and <code> </code> .
<code>/ #</code> <code>/ # </code>	Unfolds operators if necessary to find conjunctions (and disjunctions) to destruct. The space after <code>/</code> is required.
<code>n!<mode></code>	Applies <code><mode></code> to up to <code>n</code> top assumptions, until it doesn't apply or there are no more assumptions. If <code>n</code> is omitted, it applies the <code>mode</code> to as many top assumptions as possible. <code><mode></code> is any of the preceding modes.

13.6.4 Generalization Patterns

A generalization pattern adds a new top assumption to the conclusion of the current goal.

Syntax	Description
<code>name</code>	Reverts the assumption named <code>name</code> ; that is, moves it from the context to the conclusion of the current goal as the new top assumption. Fails if <code>name</code> is used by other assumptions in the context. If <code>name</code> is a local definition, reverts it as a universally quantified variable and omits its definition.
<code>@name</code>	Reverts the assumption named <code>name</code> , which must be a local definition, as a let expression.
<code>p</code>	Instantiates a lemma (including an axiom, hypothesis or anonymous lemma) using proof term <code>p</code> and add it as a hypothesis. If the proof term is for an anonymous lemma, adds a subgoal to prove it using the current goal's context.
<code>occ e</code>	Abstracts a subexpression that matches expression pattern <code>e</code> as a new universally quantified variable and replaces occurrences of the matched expression with the variable. <code>occ</code> is an optional occurrence selector; see <i>Occurrence Selectors</i> .

13.6.5 Proof Terms

A proof term instantiates a named or anonymous lemma (including an axiom or a hypothesis in the context of a goal) or the top assumption of a goal, by providing an argument for each of its assumptions. The following table lists forms of proof terms:

Syntax	Description
<code>name</code> <code>name <:tva></code>	The name of a lemma with no arguments <code>tva</code> is one or more type expression arguments separated by commas to instantiate type variables
<code>(name arg₁ ... arg_n)</code> <code>(name <:tva> arg₁ ... arg_n)</code>	The name of a lemma with zero or more arguments <code>tva</code> is one or more type expression arguments separated by commas to instantiate type variables
<code>(_: φ)</code>	An anonymous lemma, <code>φ</code> , with no arguments; the underscore is optional and meaningless
<code>((_: φ) arg₁ ... arg_n)</code>	An anonymous lemma, <code>φ</code> , with zero or more arguments; the underscore is optional and meaningless
<code>_</code>	The top assumption, with no arguments
<code>(_ arg₁ ... arg_n)</code>	The top assumption, with zero or more arguments

The `/p` pattern used by the introduction tactical and the `move` and `case` tactics accepts all six forms. The `apply`, `exact` and `rewrite` tactics accept the first four forms. The generalization tactical and the `apply`, `case`, `elim`, `exact` and `move` tactics use the same syntax as the first four forms as generalization patterns for instantiating lemmas, except the `_` is required for an anonymous lemma. The `:=` mode of the `have` tactic accepts only the first two forms to instantiate a named lemma.

Proof term arguments must have a form compatible with the corresponding assumption:

- The argument for a universally quantified variable must be an expression pattern of the appropriate type or a hole, except
 - The argument for a universally quantified memory must be a memory identifier

- The argument for a universally quantified module variable must have the form `(< : mod)`, where `mod` is the name of a module that satisfies the type of the variable and abides by its access restrictions. If the module is parameterized, it can have arguments.
- The argument for a hypothesis must be a sub-proof term for a lemma that applies to it (i.e., with the `apply` tactic), a hole or one of the introduction patterns `//`, `/ #` or `// #`, with the same meaning as with the introduction tactical
 - A sub-proof term is a proof term for a named lemma (one of the first two forms above)
- Let expressions do not take arguments
- Trailing holes may be omitted

Proof terms for named and anonymous lemmas with arguments may have an optional `@` following the innermost `(`. This undoes the effect of the `+implicit` pragma.

13.6.6 Occurrence Selectors

An occurrence selector specifies which occurrences of a match are replaced. Occurrences are numbered starting at 1 following a prefix traversal of an expression's abstract syntax tree. An occurrence selector has one of the following forms:

Syntax	Description
<code>{ i₁ ... i_n }</code>	The occurrences listed
<code>{ -i₁ ... i_n }</code>	All except the occurrences listed
<code>{ + }</code>	All occurrences; the absence of a selector also means all occurrences

13.6.7 Focus Indices

A focus index is used to indicate to which subgoals a tactic or `rewrite` pattern is to be applied. If there are `N` subgoals, they are numbered from first to last as 1 to `N` and from last to first as `-1` to `-N`. Ranges can mix positive and negative indices, such as `3 .. -2` (the third to the next-to-last). Repetitions and overlapping ranges do not matter. A focus index has one of the following forms:

Syntax	Description
<code>N</code>	subgoal <code>N</code>
<code>M .. N</code>	subgoals <code>M</code> through <code>N</code> , inclusive
<code>M ..</code>	subgoals <code>M</code> through the last subgoal, inclusive
<code>.. N</code>	subgoals 1 through <code>N</code> , inclusive
<code>f₁, ..., f_n</code>	subgoals in the union of the indexes and ranges listed
<code>~ f₁, ..., f_n</code>	subgoals not in the union of the indexes and ranges listed
<code>f₁, ..., f_n ~ g₁, ..., g_m</code>	subgoals in the union of the indexes and ranges listed before <code>~</code> and not in the union of those listed after <code>~</code>

13.6.8 Repetition Markers

A repetition marker indicates how many times a tactic, `rewrite` pattern or introduction pattern is applied to each subgoal.

Syntax	Description
<code>!</code>	apply it until it fails and fail if it does not apply at all
<code>n!</code>	apply it <code>n</code> times and fail if any application fails
<code>?</code>	apply it until it fails, even if that is zero times
<code>n?</code>	apply it up to <code>n</code> times or it fails, even if that is zero times

13.7 Print, Search and Locate

The `print` step displays the definitions of objects with a given name and category. It has the form

```
print category name.
```

where `category` is optional; if omitted, objects from all categories except `hint` are displayed. The categories are as follows:

Syntax	Description
<code>*</code>	all categories except <code>hint</code>
<code>type</code>	types
<code>op</code> <code>pred</code>	operands, predicates, constructors, field projections and abbreviations
<code>axiom</code> <code>lemma</code>	axioms and lemmas
<code>module</code>	modules
<code>module type</code>	module types
<code>glob</code>	the global variables of a module
<code>goal</code>	In proof mode, takes a positive integer argument and displays the corresponding goal
<code>theory</code>	theories
<code>rewrite</code>	rewrite databases
<code>solve</code>	solve databases
<code>hint simplify</code> <code>hint solve</code> <code>hint rewrite</code> <code>hint</code>	lists user reductions in the (unnamed) simplify database lists all solve databases and the lemmas they contain lists all rewrite databases and the lemmas they contain all of the above

The name can be qualified. For the `op` and `pred` category, the name can be a symbolic operator name written in prefix form (e.g., `(+)`, `[!]` or `"[!]"`). For the `glob` category, the name is a procedure of a module. For the `goal` category, the name is a positive integer. The step

```
print axiom.
```

with no name specified prints all axioms (only; not lemmas) in imported theories and sub-theories.

The `search` step displays a list of axioms and lemmas that contain a list of operators or, more generally, match a list of expression patterns. It has the form

```
search e1 ... en.
```

To search for an abbreviation, you may have to search for its definition as an expression pattern. See *Searching for and Locating Lemmas*. You cannot search for field projections.

The `locate` step displays where a type, operator or lemma is defined.

13.8 Modules

A **module** is a unit of computation consisting of mutable global variables and procedures with side effects. Procedures are written in a simple but Turing-complete, probabilistic, imperative syntax.

Syntax	Description
<code>module M = {}.</code>	An empty module named <i>M</i>
<pre> module M = { var v : t proc p (x : t₁) : t = { <local variables> <statements> } module M1 = { ... } module M2 = N proc q = M3.v include M4 include M5 [+q, r] include M6 [-s, t] }. </pre>	A module named <i>M</i> with - A global variable <i>v</i> of type <i>t</i>, initialized to <code>witness<:t></code> - A procedure <i>p</i> with a parameter <i>x</i> of type <i>t</i>₁ and return type <i>t</i> • Parentheses are required; if there are no parameters, an implicit parameter of type <code>unit</code> is inferred for typing purposes • The return type is optional; if “nothing” is returned, it is <code>unit</code> - A sub-module - A local alias <i>M2</i> for module <i>N</i> - A local alias <i>q</i> for procedure <i>M3.v</i> - A local alias for each procedure in <i>M4</i> - A local alias for procedures <i>q</i> and <i>r</i> in <i>M5</i> - A local alias for each procedure in <i>M6</i> except <i>s</i> and <i>t</i> Variables, procedures, sub-modules and inclusions may be freely intermixed
Procedure body syntax	
<pre> var x, y : t₁; var x, y <- e; var x, y; </pre>	Declares local variables <i>x</i> and <i>y</i> of type <i>t</i> with initial value <i>e</i> and type inferred from <i>e</i> with types to be inferred from the statements of the procedure. The value of an uninitialized variable is undefined
<pre> v <- e; v <\$ d; v <@ M.p (x); M.p (x); </pre>	Assignment statements: Assigns variable <i>v</i> the value of expression <i>e</i> in the ambient logic Assigns variable <i>v</i> a value randomly sampled from sub-distribution <i>d</i> Assigns variable <i>v</i> the value returned by calling procedure <i>p</i> of module <i>M</i> with argument <i>x</i>. The parentheses are required even if no arguments are needed. <i>M</i> is optional if <i>p</i> is in the same module as the calling procedure. Note. <i>M.p (x)</i> is <i>not</i> an expression. Calls <i>M.p (x)</i> but does not store its return value The left-hand side of an assignment may also be a tuple or a map access (e.g., <i>m. [k]</i>) but not any other kind of expression
<pre> if (b) { <statements> } else { <statements> } if (e is c₁ v₁ ...) ... </pre>	Conditional statement: executes the first statement block if the Boolean expression <i>b</i> is <code>true</code> and the second one otherwise. The <code>else</code> clause is optional. Shorthand syntax for a <code>match</code> statement, where <i>e</i> is an expression of an inductive type and <i>c</i>₁ <i>v</i>₁ ... is a pattern for one of its constructors. The

	first statement block can use the pattern variables v_i. If either statement block is a single statement, the braces around it may be omitted. If the <code>else</code> block consists of a conditional statement, <code>else if</code> may be abbreviated <code>elif</code>.
<pre>match e with c₁ v₁ ... => { <statements> } ... end;</pre>	Match statement, where <code>e</code> is an expression of an inductive type and there is a pattern for each of its constructors with a pattern variable or underscore for each of its parameters If any of the statement blocks is a single statement, the braces around it may be omitted. The <code> </code> before the first pattern is optional.
<pre>while (b) { <statements> }</pre>	Iteration statement: if Boolean expression <code>b</code> is <code>true</code>, executes the body and repeats; otherwise, skips to the statement following the closing brace If the body is a single statement, the braces may be omitted
<pre>assert b;</pre>	Asserts that Boolean expression <code>b</code> is <code>true</code>; verified during proofs.
<pre>return e; return;</pre>	Returns the value of expression <code>e</code>. Returns <code>tt</code> when the return type is <code>unit</code>; optional If present, <code>return</code> must be the last statement.

A module defines separate namespaces for its global variables, procedures (including aliases) and sub-modules (including aliases). A procedure defines a shared namespace for its local variables and parameters.

A procedure may only call procedures declared before it (including those in required theories) and therefore may not be recursive, directly or indirectly.

A procedure's local variables and parameters hold values that may be modified during its execution. These changes are not visible outside the procedure but may be reasoned about during the procedure's execution.

Each statement in a procedure is identified by its **code position** in a hierarchical fashion, reflecting the nested block structure of the program:

- Local variable declarations and the `return` statement are ignored
- Statements in a block are numbered sequentially starting at 1
 - Statements in the body of a `while` statement start at `c . 1`, where `c` is the code position of the `while` statement
 - Statements in the `then` branch of an `if` statement start at `c . 1`, where `c` is the code position of the `if` statement; statements in the `else` branch start at `c?1`
 - Statements in a branch of a `match` statement start at `c#ci . 1`, where `c` is the code position of the `match` statement and `ci` is a constructor
 - Example: `3 . 1?2#Some . 1` is the first statement in the `Some` branch of the second statement in the `else` branch of the first statement in the body or `then` branch of the

third statement from the start of the procedure, which must be a `while` or `if` statement

- A negative integer counts from the end of the block
 - Example: `-5` is the fifth statement from the end of the procedure
 - Example: `3?-2` is the second statement from the end of the `else` branch of the third statement from the start of the procedure
- Instead of a number, you can specify a kind of statement
 - `^if`, `^while`, `^match` match the first `if`, `while` or `match` statement
 - `^<-`, `^<$`, `^<@`, `^()<-` match the first assignment statement (to a tuple in the last case)
 - `^x<-`, `^x<$`, `^x<@`, `^(x)<-` match the first assignment statement to variable `x` (to a tuple containing `x` in the last case)
 - Example: `^match#(::).^a<$` is the first random assignment to `a` in the `(: :)` branch of the first `match` statement in the procedure
- A code position can be followed by a signed offset within the same code block (the space after `&` is required)
 - Example: `5 & +2` is the second statement after the fifth one, which is the same as `7`
 - Example: `^while & -1` is the statement before the first `while` loop in the procedure

A module can be defined as a modification of an existing module.

Syntax	Description
<pre>module Mod1 = M with { -var u₁, ..., u_n var v₁, ..., v_k : t proc f [<see below>] }</pre>	Defines <code>Mod1</code> as a variation of module <code>M</code>. All modifications are optional but must be in this order: Deletes global variables <code>u₁</code> through <code>u_n</code>. At most one occurrence. Adds global variables <code>v₁ ... v_k</code> of type <code>t</code>. Any number of occurrences. Modifies procedure <code>f</code>. At most one for each procedure. At least one statement or condition modification must be inside the brackets.
<pre>var x, y : t</pre>	Adds local variables <code>x</code> and <code>y</code> of type <code>t</code>. Initial values cannot be provided. Optional.
<pre>c + { s } c + ^ { s } c ~ { s } c -</pre>	Statement modification Inserts <code>s</code> after code position <code>c</code> Inserts <code>s</code> before code position <code>c</code> Replaces the statement at code position <code>c</code> with <code>s</code> Deletes the statement at code position <code>c</code>
<pre>c + e c + ^ e c ~ e</pre>	Condition modification Inserts an <code>if</code> statement after code position <code>c</code> with condition <code>e</code>, an empty <code>then</code> branch and no <code>else</code> branch Inserts an <code>if</code> statement at code position <code>c</code> with condition <code>e</code>, the existing statement at <code>c</code> as its <code>then</code> branch and no <code>else</code> branch Changes the condition of the statement at code position <code>c</code> to <code>e</code>. The statement must be an <code>if</code> or <code>while</code> statement.

<code>res ~ e</code>	Optionally follows the closing right brace. Changes the return value of the procedure to <code>e</code> .
----------------------	---

The code position in a statement or condition modification can be a range:

- `[c1 .. c2]` refers to the statements from the one matching `c1` to the one matching `c2`, inclusive; both statements must be in the same block
- `[c1 - c2]` means the same thing, but `c2` must consist of a single-level pattern. For example, `[^while.1 - 3]` is a shorthand for `[^while.1 .. ^while.3]`

All code positions or ranges refer to the original procedure body before any changes have been made.

The set of **global variables of a module *M*** is the union of

- the set of global variables that are declared in *M*; and
- the set of global variables declared in other modules that could be read or written by a series of procedure calls beginning with a call to one of *M*'s procedures. By "could" we mean the read/write analysis includes the execution of all branches of conditionals, all branches of a match statement, the execution of while loop bodies, and the termination of while loops.

For every module *M*, there is a corresponding type, `glob M`, of tuples consisting of a value for each of the global variables of *M*.

13.9 Module Types and Functors

A **module type** consists of a sequence of **signatures** (procedure declarations without bodies). A module **satisfies** a module type if it contains a procedure for each signature of the type, matching its name, return type and the number and types of its parameters (parameter names do not matter). A module may have more procedures than the types it satisfies, and any global variables and sub-modules whatsoever.

A module may be parameterized on one or more module types, meaning it depends on procedures that can be provided by any module that satisfies the corresponding type. To call a procedure of a parameterized module, an argument must be provided for each module parameter. The parameterized module can only access the procedures of its arguments that are defined by the parameters' types, and therefore not their global variables, submodules or other procedures.

Syntax	Description
<code>module type T = {}.</code>	An empty module type named <i>T</i>
<code>module type T = { proc f (x : t₁) : t include T1 include T2 [+r, s] include T3 [-t, u] }.</code>	A module type named <i>T</i> that contains A signature <i>f</i> with a parameter of type <i>t₁</i> and return type <i>t</i> A copy of all signatures in module type <i>T1</i> A copy of signatures <i>r</i> and <i>s</i> of module type <i>T2</i> A copy of module type <i>T3</i> except signatures <i>t</i> and <i>u</i> Signatures and inclusions can be freely intermixed. Duplicate procedure names result in errors.
<code>module O : T = { ... }</code>	A module <i>O</i> that satisfies module type <i>T</i> A module may satisfy a module type without being declared to do so
<code>module P (N : T) = { ... }</code>	A module with a module of module type <i>T</i> as a parameter

<code>v <@ N.f (y) ;</code>	A call to procedure <code>f</code> of parameter <code>N</code> from inside <code>P</code>
<code>v <@ P(O).p (...);</code>	A call to procedure <code>p</code> of module <code>P</code> with argument <code>O</code>
<code>module (P : T₁) (N : T) = { ... }</code>	A module <code>P</code> that satisfies module type <code>T₁</code> and has a parameter of module type <code>T</code>
<code>module Q = P (O) .</code>	An alias for module <code>P</code> with module <code>O</code> as its argument

A module type with parameters is called a **functor**. A functor places access restrictions on which procedures of each parameter each signature may call. The default is that they can call all of them. A module satisfies a functor if it has the same parameter types in the same order (the parameter names do not matter) and contains a procedure that satisfies each signature of the functor, including its access restrictions. A functor may include a module type but not a functor.

A module may be parameterized on one or more functors, meaning its arguments must be modules that satisfy the corresponding functors.

Syntax	Description
<code>module type U (N : T) = { }.</code>	An empty functor named <code>U</code> with a parameter <code>N</code> of module type <code>T</code>
<code>module type U (N : T) = { proc g (x : t₁) : t {N.f, N.r} proc h () : unit { include T4 {N.s} include T5 [+a, b] include T6 [-c, d] {N.u} } }.</code>	A functor named <code>U</code> with a parameter <code>N</code> of module type <code>T</code> that contains A signature <code>g</code> with a parameter of type <code>t₁</code> and return type <code>t</code> that may call <code>N.f</code> and <code>N.r</code> only A signature <code>h</code> with no parameters and return type <code>unit</code> that may not call any procedure of <code>N</code> A copy of module type <code>T4</code> where all signatures may only call <code>N.s</code> A copy of signatures <code>a</code> and <code>b</code> of module type <code>T5</code> that may call any procedure of <code>N</code> A copy of module type <code>T6</code> except signatures <code>c</code> and <code>d</code> that may only call <code>N.u</code> Signatures and inclusions may be freely intermixed. Duplicate procedure names result in errors.
<code>module R : U = { ... }</code>	A module that satisfies functor <code>U</code> A module may satisfy a module type without being declared to do so
<code>module S (V : U) = { ... }</code>	A module <code>S</code> with a module of functor <code>U</code> as a parameter
<code>module (S : T₁) (V : U) = { ... }</code>	A module <code>S</code> that satisfies module type <code>T₁</code> and has a parameter <code>V</code> of functor <code>U</code>
<code>module W = S (R) .</code>	<code>W</code> is an alias for module <code>S</code> with module <code>R</code> as its argument

13.10 Theories and Sections

A top-level theory `T` is a sequence of steps in a script file named `T.ec` (for a concrete theory) or `T.eca` (for an abstract theory). A sub-theory is a sequence of steps within another theory. A theory is either abstract or concrete. An abstract theory can be required but not imported or exported and its elements cannot be accessed. The only step that can be performed on an abstract theory is to clone it.

Syntax	Description
<code>theory S.</code>	Marks the beginning of concrete sub-theory <code>S</code> .
<code>abstract theory S.</code>	Marks the beginning of abstract sub-theory <code>S</code> .

<code>clear [* , T.*].</code>	Designates one or more theories to be cleared when the active theory is closed. Clearing a theory deletes its lemmas. If the result is empty, the theory is deleted. * by itself refers to the active theory. Not to be confused with the <code>clear</code> tactic.
<code>end S.</code> <code>end [* , T.*] S.</code>	Marks the end of sub-theory S, which closes it. S must be open and have no open sub-theories or sections. Designates theories to be cleared and closes sub-theory S.
<code>Top</code>	A notional global namespace for top-level theories. Used in qualified names.
<code>Self</code>	Refers to a top-level theory from within the file that contains it. Used in qualified names.
<code>require T₁ ... T_n.</code>	Loads each top-level theory from file T _i .ec or T _i .eca, unless it has already been loaded. Theories must be loaded before they can be used.
<code>import T₁ ... T_n.</code>	Makes the elements of each theory or sub-theory T _i visible without qualification, when such names are not ambiguous. Also makes it possible to use symbolic operator names without enclosing them with <code>()</code> , <code>[]</code> , or <code>" "</code> and with their associated syntax (prefix, infix or mixfix), precedence and associativity. An abstract theory cannot be imported
<code>export T₁ ... T_n.</code>	Same as <code>import T₁ ... T_n</code> and any other theory that requires the theory containing this step automatically imports each T _i . An abstract theory cannot be exported
<code>require import T₁ ... T_n.</code> <code>require export T₁ ... T_n.</code>	Requires and imports a top-level theory in one step. Requires and exports a top-level theory in one step.
Cloning	
<code>clone T.</code> <code>clone [abstract] T.</code> <code>clone T as U.</code> <code>clone import T.</code> <code>clone export T.</code> <code>clone include T.</code>	Adds a copy of theory T named T as a concrete sub-theory of the active theory. Same as <code>clone</code> , but sub-theory T is abstract. Clones T and names the copy U. Can combine with <code>[abstract]</code> . Clones T and imports the copy. Can combine with <code>as U</code> . Clones T and exports the copy. Can combine with <code>as U</code> . Adds a copy of the elements of T to the active theory. Cannot be used with <code>as U</code> because no sub-theory is created <code>import</code> , <code>export</code> and <code>include</code> cannot be used with <code>[abstract]</code> because you can't access elements of an abstract theory.
<code>clone T with</code> <code>type 'a t = texp,</code> <code>op f ['a] : t = e,</code> <code>pred p ['a] <- e,</code>	Optional clause overrides elements separated by commas: Overrides type t with a type expression; type variables are only used if t is polymorphic; if t is not abstract, texp must unify with its body Overrides operator f with an expression; type variables are only used if f is polymorphic; : t is optional; if f is not abstract, e must unify with its body

<pre> axiom a = l, lemma a = l, module type M = N, theory S = R, Substitute <= or <- for = </pre>	Overrides predicate <code>p</code> with a <code>bool</code> expression; type variables are only used if <code>p</code> is polymorphic; if <code>p</code> is not abstract, <code>e</code> must unify with its body If <code>a</code> is an axiom <code>a</code>, the <code>exact</code> tactic must solve it with lemma <code>l</code>; otherwise, <code>l</code> can be any lemma (but there's no reason to override a lemma) Overrides module type or functor <code>M</code> with the module type or functor named <code>N</code>; <code>M</code> and <code>N</code> must have the same parameter types and signatures Overrides sub-theory <code>S</code> with the theory named <code>R</code> by overriding each element of <code>S</code> with the element of <code>R</code> with the same name; extra elements of <code>R</code> are ignored For types, operators and predicates, <code>=</code> adds the definition to the declaration <code><=</code> adds the definition to the declaration and inlines the definition elsewhere <code><-</code> deletes the declaration and inlines the definition elsewhere For axioms, <code>=</code> keeps it, <code><=</code> marks it (<code>* hidden *</code>) (e.g., when printed) and <code><-</code> deletes it For module types or theories, <code>,</code> <code>=</code> and <code><=</code> keep it and <code><-</code> deletes it
<pre> clone T rename "old" as "new" rename [type, op, ...] "old" as "new" remove abbrev f proof * proof a₁, ..., a_n proof a₁ by τ₁, * by τ₂ </pre>	After optional <code>with</code> clause, zero or more of each of these clause types in any order: Replaces matches for regular expression “old” with string “new” in all element names; see regex syntax below. Matching can be limited to specified namespaces. Removes the definition of abbreviation <code>f</code> States intent to prove all axioms as lemmas (see note on <code>*</code> below). States intent to prove selected axioms as lemmas. Any axiom or <code>*</code> can be proved inline by a core tactic; you cannot use a chain tactical or <code>by []</code> here. <code>*</code> refers to all axioms not named in this or other <code>proof</code> clauses, regardless of its position in the list.
<pre> realize a by τ. realize a by []. realize a. proof. <proof steps> qed. </pre>	Immediately following the <code>clone</code> step, selects an axiom and proves it is a lemma in the copy.

The table below shows the regular expression syntax (a subset of Perl's) used by the `rename` clause of a `clone` step. There is no way to capture a pattern and use it in the substitution.

Metacharacter	Meaning
.	Any character
^	Start of string
\$	End of string
?	The previous pattern is optional

+	One or more occurrences of the previous pattern
*	Zero or more occurrences of the previous pattern
{n}	Exactly n occurrences of the previous pattern
{m,n}	From m to n occurrences of the previous pattern
re ₁ re ₂	One occurrence of either re ₁ or re ₂
(...)	Grouping to override operator precedence

A **subtype** is a type that is related to another type by inclusion: every element of the subtype is also an element of the so-called **supertype**. A **subtype declaration** is syntactic sugar for cloning the library theory Subtype (in the structure folder). To use it, the file must require Subtype.

Syntax	Description
<pre> subtype s as S = { x : t p x } rename "f", "g". </pre>	Equivalent to <pre> type s. clone Subtype as S with type sT <- s, type T <- t, op P <- fun (x : t) => p x rename "val" as "g" "insub" as "f" proof inhabited. </pre> Must be followed by a <code>realize</code> step to prove <code>inhabited</code> (from theory Subtype). The renaming clause is optional.

An **instance** of a type class is a type and a set of operators that satisfy the axioms of the class. Three type classes are built-in to EasyCrypt: ring, bring (Boolean ring) and field.

Syntax	Description
<pre> instance ring with t op rzero = op₁ op rone = op₂ op add = op₃ op opp = op₄ op sub = op₅ op mul = op₆ op expr = op₇ op ofint = op₈ proof Ax by τ ... </pre>	An instance of an Integer ring, where <code>t</code> is a type expression and <code>op_i</code> is the name of an operator. <code>sub</code> and <code>expr</code> are optional. If <code>t = int</code>, <code>ofint</code> is not allowed. Axioms for each operator listed must be realized in <code>proof</code> clauses of the <code>instance</code> step or in <code>realize</code> steps immediately following. Each <code>proof</code> clause can name only one axiom and <code>*</code> is not allowed. EasyCrypt will list all axioms needing proof.
<pre> instance ring with t n p ... </pre>	An instance of a ring, where <code>t</code> is a type expression, <code>2 <= n</code> is the coefficient modulus or an underscore and <code>2 <= p</code> is the power modulus or an underscore. If <code>n = p = 2</code>, the ring is Boolean and <code>opp</code> is optional.
<pre> instance bring with t </pre>	An instance of a Boolean ring
<pre> instance field with t </pre>	An instance of a field

A **section** textually delimits part of a theory. Sections are opened by the `section` step and closed by the `end section` step. Section names are optional and must begin with an uppercase letter. Sections may be nested. A section defines a naming scope that permits creating **local** declarations. When the section is closed, any global declarations made therein (such as lemmas) are generalized with respect to the local ones:

- Local types become type variables
- Local operators become universally quantified variables
- Local axioms become hypotheses

Syntax	Description
<code>section.</code>	Marks the beginning of a section.
<code>section S.</code>	Marks the beginning of a section named <i>S</i> .
<code>end section.</code>	Marks the end of an open section and closes it. The section must not have any open theories.
<code>declare type t.</code> <code>declare op f : t.</code> <code>declare axiom ax (x : t) : ϕ x</code>	A declaration that is local to the section. May be an abstract type, an abstract operator (including a constant or a predicate) or an axiom.
<code>local <declaration>.</code>	A declaration that is local to the section but may not be referenced by a global declaration in the section. Almost any declaration can be local except an axiom and a user reduction (simplify hint).

13.11 Pragmas

Pragmas are meta-steps that configure how EasyCrypt operates or take an immediate action.

```
pragma silent.
```

A pragma is either a command to execute, a member of a set of mutually exclusive options or a Boolean option turned on with `+` and turned off with `-`. Unlike regular steps, pragmas do not increment the current step number and they cannot be undone. Pragmas can also be set from the command line:

```
$ easycrypt -pragma +implicits, Goals:printall.
```

Name	Kind	Meaning
<code>noop</code>	command	N/A
<code>compact</code>	command	runs the garbage collector
<code>reset</code>	command	resets EasyCrypt to its initial state, as if no input has been processed, but preserves pragma settings
<code>restart</code>	command	resets EasyCrypt to its initial state (see <code>reset</code>) and sets the <code>verbose</code> , <code>Goals</code> , <code>Proofs</code> and <code>PrPo</code> pragmas to their defaults
<code>silent</code>	one of a set	in a proof, do not display goal(s) after each command
<code>verbose</code>	one of a set	in a proof, display goal(s) after each command (default)
<code>Goals:printall</code>	one of a set	in a proof, always display all goals
<code>Goals:printone</code>	one of a set	in a proof, only display the current goal (default)
<code>Proofs:check</code>	one of a set	unknown (default)

Proofs:weak	one of a set	unknown
Proofs:report	one of a set	unknown
PrPo:pr:raw	one of a set	unknown (default)
PrPo:pr:ands	one of a set	unknown
PrPo:po:raw	one of a set	unknown (default)
PrPo:po:ands	one of a set	unknown
±implicits	Boolean	be more aggressive in inferring arguments to fill holes in proof terms (default false)
±redlogic	Boolean	apply user-defined reduction rules (simplify hints) in the <code>logic</code> tactic (default true)
±und_delta	Boolean	report when an unfolding operation has no effect in the <code>rewrite</code> tactic or <code>@ intro</code> pattern (default false)
±oldip	Boolean	for backward compatibility (default false)
±old_mem_restr	Boolean	for backward compatibility (default false)

13.12 Hoare Logic Tactics

The tactics in this section act on goals whose conclusion have one of these forms:

```
hoare [M.p : P ==> Q]
hoare [C : P ==> Q]
```

where `M` is a module, `p` is a procedure of `M`, `C` is a program (sequence of statements) and `P` and `Q` are expressions of type `bool`.

13.12.1 Program Reasoning Tactics

These tactics reason about a goal by consuming or modifying parts of the program and making corresponding changes to the specification.

Syntax	Description
<code>call (_ : P ==> Q)</code>	If the goal's conclusion is a statement judgement with precondition <code>P0</code> and postcondition <code>Q0</code> and ending with a procedure assignment or call to <code>N.q</code>, reasons about the called procedure abstractly instead of inlining it. Formulas <code>P</code> and <code>Q</code> define a Hoare triple for <code>N.q</code>. Reduces the goal to two subgoals: 1. <code>Hoare [N.q : P ==> Q]</code> I.e., verify the procedure does what the user claims. This is a procedure judgement, not a statement judgement. 2. <code>Hoare [C' : P0 ==> P' /\ forall v, Q(v) ==> Q0(v)]</code>, where <code>C'</code> represents <code>C</code> with its last statement removed, <code>P'</code> represents <code>P</code> instantiated with the arguments of the call and <code>v</code> stands for all variables modified by the procedure call and its return value. I.e., verifies that having the procedure do what the user asserted is what the calling program needs. Note: the <code>_</code> is optional.
<code>call p</code>	Proof term <code>p</code> refers to a lemma whose conclusion is a HL judgement involving the procedure called. Reduces the goal to just the second

call (_ : I)	subgoal above. I.e., the lemma provides P and Q for the second subgoal. As a special case of P and Q, the called procedure can be characterized by an invariant, I: a formula that the procedure maintains even though it may modify its variables. In this case, the second subgoal's postcondition is $I' \wedge \text{forall } v, I(v) \Rightarrow Q0(v)$. If the called procedure is concrete, this is a shorthand for <code>call (_ : I ==> I); first proc</code> to inline the called procedure of the first subgoal. If the called procedure is abstract, this is a shorthand for <code>call (_ : I ==> I); first proc I</code> except it prunes the first two subgoals generated by <code>proc</code>.
if	If the goal's conclusion is a statement judgement beginning with an <code>if</code> statement, reduces the goal to two subgoals: 1. Replaces the <code>if</code> statement with the <code>then</code> part and adds the <code>if</code> condition to the precondition 2. Replaces the <code>if</code> statement with the <code>else</code> part and adds the negation of the <code>if</code> condition to the precondition
proc* proc proc I	The <code>proc</code> tactic is for getting started: it is the only tactic that reduces a procedure judgement to a statement judgement. Reduces a procedure judgement to a statement judgement consisting of a call to the procedure. Reduces a procedure judgement involving a concrete procedure to a statement judgement on its body. Shorthand for <code>proc*; inline</code>. Reduces a procedure judgement involving an abstract procedure of a functor to two ambient logic subgoals, to verify that the judgement's pre- and postconditions allow such a reduction, and one procedure judgement for each procedure of the functor's parameters that the method may call. Note: the ambient subgoals come first, upon which <code>call (: I)</code> relies. 1. $P \Rightarrow I$ 2. $I \Rightarrow Q$ 3. For each procedure $N.q$ that may be called, $\{I\} N.q \{I\}$ The tactic fails if M is able to access any of the global variables referenced by I.
rnd	If the goal's conclusion is a statement judgement that ends with a random assignment, reduces the goal by consuming the assignment and replacing the goal's postcondition with the assignment's possibilistic weakest precondition, which is that the postcondition holds no matter what value the distribution returns (and ignoring the probabilities thereof).
seq n : R	Splits a program into a sequence of two programs with intermediate condition R. For a statement judgement with precondition P, postcondition Q and a program of at least n statements, reduces the goal to two statement judgements as follows: 1. Precondition P, the first n statements of the program and postcondition R. 2. Precondition R, all but the first n statements of the program and postcondition Q.

skip	Reduces an empty statement judgement $\{P\} \{Q\}$ to an ambient logic subgoal with $P \Rightarrow Q$ as its conclusion.
sp	Reduces a statement judgement with precondition P by consuming the longest prefix of statements built from ordinary assignment ($\leftarrow$) and <code>if</code> statements and replacing the goal's precondition with the strongest postcondition R such that $\{P\} \text{ prefix } \{R\}$ holds
sp n	Consumes the first n statements
while I	If the goal's conclusion is a statement judgement with precondition P and postcondition Q ending with a <code>while</code> statement with condition b , reduces the goal to two subgoals: $\{I \wedge b\} \text{ while body } \{I\}$ $\{P\} \text{ program} - \text{while statement } \{I \wedge \text{forall } v, ! b(v) \Rightarrow I(v) \Rightarrow Q(v)\}$, where v stands for all variables modified by the body of the <code>while</code> statement. I is the loop's <i>invariant</i> , as verified by the first subgoal.
wp	Reduces a statement judgement with postcondition Q by consuming the longest suffix of statements built from ordinary assignment ($\leftarrow$) and <code>if</code> statements and replacing the goal's postcondition with the weakest precondition R such that $\{R\} \text{ suffix } \{Q\}$ holds. It also consumes assertions by adding the asserted expression to the postcondition.
wp n	Consumes the last n statements

13.12.2 Program Transformation Tactics

These tactics apply only to statement judgements and reduce a goal to a single subgoal with a modified program without affecting its specification. The variable c ranges over code positions.

Syntax	Description
alias c with x	Introduces a new variable, x , as a copy of an existing one. If the goal's conclusion is a statement judgement in which statement c is an assignment statement (ordinary, random or procedure call), replaces the statement with the following two statements: $x \leftarrow \text{rhs}$ (or $\leftarrow \\$ or $\leftarrow @$ following original) $\text{lhs} \leftarrow x$
alias c alias c x = e	Equivalent to <code>alias c with x</code> . Insert $x \leftarrow e$ before statement c . In all variants, if x is already a local variable, it generates a fresh name by adding digits to the end.
cfold c ! m	"Folds" a constant. If the goal's conclusion is a statement judgement in which statement c is an ordinary assignment statement in which constant values are assigned to local identifiers, and the following statement block of length m does not write any of those identifiers, then replaces all occurrences of the assigned identifiers in that statement block with the constants assigned to them and moves the assignment statement to after the modified statement block. (If the moved statement is now superfluous, see <code>kill</code> .)
fission c!l @m,n	Breaks a loop into two independent loops over the same control variables. If the goal's conclusion is a statement judgement in which statement c is a <code>while</code> statement, let s_1 be the l statements prior to position c at its level

	• <code>e</code> be the <code>while</code> condition • <code>s2</code> be the first <code>m</code> statements of the <code>while</code> body • <code>s3</code> be the next <code>n - m</code> statements of the <code>while</code> body • <code>s4</code> be the rest of the <code>while</code> body where • <code>e</code> does not reference the variables written by <code>s2</code> and <code>s3</code> • <code>s1</code> and <code>s4</code> do not reference the variables written by <code>s2</code> and <code>s3</code> • <code>s2</code> and <code>s3</code> do not write the variables written by <code>s1</code> and <code>s4</code> • <code>s2</code> and <code>s3</code> do not reference the variables written by each other then it replaces <pre>s1 while (e) { s2 s3 s4 }</pre> with <pre>s1 while (e) { s2 s4 } s1 while (e) { s3 s4 }</pre> Equivalent to <code>fission c!1 @m, n.</code>
<pre>fission c @m, n fusion c!1 @m, n</pre>	Fuses independent loops over the same control variables into one loop. If the goal's conclusion is a statement judgement in which statement <code>c</code> is a <code>while</code> statement and the statements starting <code>l</code> positions before <code>c</code> match <pre>s1 while (e) { s2 s4 } s1 while (e) { s3 s4 }</pre> where • <code>s1</code> has length <code>l</code> • <code>s2</code> has length <code>m</code> • <code>s3</code> has length <code>n</code> • <code>e</code> does not reference the variables written by <code>s2</code> and <code>s3</code> • <code>s1</code> and <code>s4</code> do not reference the variables written by <code>s2</code> and <code>s3</code> • <code>s2</code> and <code>s3</code> do not write the variables written by <code>s1</code> and <code>s4</code> • <code>s2</code> and <code>s3</code> do not reference the variables written by each other then it replaces them with <pre>s1 while (e) { s2 s3 s4 }</pre> Equivalent to <code>fusion c!1 @m, n.</code>
<pre>fusion c @m, n inline M1.p1 ... inline* inline occ M.p</pre>	Repeatedly replaces calls to the named concrete procedures with their bodies, instantiated with the calls' arguments, until no more calls remain. Inlines all concrete procedures until no concrete procedure calls remain. Inlines specified occurrences of <code>M.p</code>. See <i>Occurrence Selectors</i> for occurrence selector syntax. TODO Are the parameters copied into new variables and local variables renamed as necessary to avoid side effects on the calling program's state?
<pre>kill c ! m</pre>	Removes superfluous code. If the goal's conclusion is a statement judgement with a statement block starting at position <code>c</code> and having length <code>m</code> and the variables written by this block are not used in the judgement's postcondition or read by the rest of the program, then it reduces the goal to two subgoals: 1. <code>Pr[block : true ==> true] = 1%</code> 2. The original goal with the block removed
<pre>match c_i n</pre>	If the goal's conclusion is a statement judgement whose <code>n</code>th statement is a <code>match</code> statement, reduces the goal to two subgoals:

	1. a statement judgement with the same precondition, statements 1 to $n - 1$ and a post condition that the match variable matches the pattern for constructor c_i 2. a statement judgement equal to the original goal but with the <code>match</code> statement replaced by its branch for c_i
<code>rcondf n</code>	If the goal's conclusion is a statement judgement whose nth statement is an <code>if</code> statement, reduces the goal to two subgoals: 3. a statement judgement with the same precondition, statements 1 to $n - 1$ and a post condition equal to the negation of the <code>if</code> condition 4. a statement judgement equal to the original goal but with the <code>if</code> statement replaced by its <code>else</code> part If the goal's conclusion is a statement judgement whose nth statement is a <code>while</code> statement, reduces the goal to two subgoals: 1. a statement judgement with the same precondition, statements 1 to $n - 1$ and a post condition equal to the negation of the <code>while</code> condition 2. a statement judgement equal to the original goal but with the <code>while</code> statement removed
<code>rcondt n</code>	If the goal's conclusion is a statement judgement whose nth statement is an <code>if</code> statement, reduces the goal to two subgoals: 1. a statement judgement with the same precondition, statements 1 to $n - 1$ and a post condition equal to the <code>if</code> condition 2. a statement judgement equal to the original goal but with the <code>if</code> statement replaced by its <code>then</code> part If the goal's conclusion is a statement judgement whose nth statement is a <code>while</code> statement, reduces the goal to two subgoals: 1. a statement judgement with the same precondition, statements 1 to $n - 1$ and a post condition equal to the <code>while</code> condition 2. a statement judgement equal to the original goal but with the <code>while</code> statement preceded by a copy of its body
<code>splitwhile c : e</code>	If the goal's conclusion is a statement judgement whose cth statement is a <code>while</code> statement and e is a <code>bool</code> expression in the statement's context, inserts before the <code>while</code> statement a copy of the <code>while</code> statement with e added as a conjunct to the <code>while</code> condition.
<code>swap n m l</code> <code>swap [n..m] k</code> <code>swap n k</code> <code>swap k</code>	Swaps code at positions n through $m - 1$ with code positions m through l. If $k \geq 0$, moves positions n through m down k positions; otherwise, moves them up k positions. Same as <code>swap [n..n] k</code>. If $k \geq 0$, same as <code>swap 1 k</code>; otherwise, same as <code>swap n k</code>, where n is the length of the program. All variants fail if the blocks of statements swapped aren't independent or if the old or new code positions or ranges aren't valid
<code>unroll c</code>	If the goal's conclusion is a statement judgement in which statement c is a <code>while</code> statement, inserts before that statement an <code>if</code> statement whose condition is the <code>while</code> condition and whose <code>then</code> part is the <code>while</code> body.

13.12.3 Program Specification Tactics

These tactics act on a goal's specification without affecting its program.

Syntax	Description
<code>bypr</code>	Reduces a procedure judgement $\text{hoare}[M.p : P \Rightarrow Q]$ to the probability assertion $\text{forall } \&m, P\{m\} \Rightarrow \text{Pr}[M.p(v) \ @\&m : !Q] = 0\%$ where v represents the values in $\&m$ of $M.p$'s formal parameters.
<code>case e</code>	Reduces a statement judgement by splitting the goal into two subgoals: 1. the goal with e added to the precondition 2. the goal with $!e$ added to the precondition
<code>conseq (_ : P $\Rightarrow$ Q)</code> <code>conseq p</code>	If the goal's conclusion is a procedure or statement judgement with precondition P' and postcondition Q' , reduces it to one with precondition P and postcondition Q and adds two subgoals: 1. $P' \Rightarrow P$ 2. $P \wedge Q \Rightarrow Q'$ If P or Q is $_$, the pre- or postcondition, respectively, is left unchanged. If the goal's conclusion is a procedure judgement with precondition P' and postcondition Q' and p is a proof term for a lemma whose conclusion is a judgement with precondition P and postcondition Q for the same procedure, reduces the goal to the two subgoals above.
<code>elim*</code>	If the goal's conclusion is a procedure or statement judgement whose precondition has the form $\text{exists } (x_1 \dots x_n), P$ changes the quantifier to <code>forall</code> .
<code>exists* e₁, ..., e_n</code>	If the goal's conclusion is a procedure or statement judgement with precondition P , change the precondition to $\text{exists } (x_1 \dots x_n), x_1 = e_1 \wedge \dots \wedge x_n = e_n \wedge P$
<code>hoare</code>	If the goal's conclusion is a judgement $\text{hoare}[M.p : P \Rightarrow Q]$ with $P = \text{true}$, reduce it to the probability assertion $\text{forall } \&m, P\{m\} \Rightarrow \text{Pr}[M.p(v) \ @\&m : !Q] = 0\%$ See also <code>bypr</code> , which does the same thing. Otherwise, reduce it to the pHL judgement $\{P\} M.p \{!Q\} \leq 0\%$ The RM says, "[It] can also be used to derive pHL judgments and certain probability inequalities by automatically applying <code>conseq</code> ," but not how.

13.12.4 Automatic Tactics

These tactics apply other tactics in combination automatically.

Syntax	Description
<code>auto</code>	If the current goal is a statement judgement, uses various program logic tactics to reduce the goal to a simpler one. Never fails, but may fail to make progress.

exfalse	Combines <code>conseq</code> , <code>byequiv</code> , <code>byphoare</code> , <code>hoare</code> and <code>bypr</code> to strengthen the precondition to <code>false</code> and discharges the resulting trivial goal.
---------	--

13.13 Probabilistic Hoare Logic Tactics

The tactics in this section act on goals whose conclusion have one of these forms:

```
phoare [M.p : P ==> Q] ◇ e
phoare [C : P ==> Q] ◇ e
```

where `M` is a module, `p` is a procedure of `M`, `C` is a program, `P` and `Q` are predicates, `e` is a real number and `◇` is one of `<`, `<=`, `=`, `>=` or `>`.

13.13.1 Program Reasoning Tactics

These tactics reason about a goal by consuming or modifying parts of the program and making corresponding changes to the specification.

Syntax	Description
<code>call</code>	This works as for Hoare logic.
<code>if</code>	There is no pHL version documented in the RM.
<code>proc*</code> <code>proc</code>	Reduces a judgement to a one-statement judgement consisting of a call to the procedure. Reduces a judgement involving a concrete procedure to a statement judgement on its body. Equivalent to <code>proc*</code> ; <code>inline</code> .
<code>rnd E</code>	If the program ends with a random assignment, reduces the goal by consuming the assignment and replacing the goal's postcondition with the assignment's probabilistic weakest precondition with respect to the predicate <code>E</code> . If <code>E</code> is omitted, it is inferred from the postcondition.
<code>seq n : C δ₁ δ₂ ρ₁ ρ₂ R</code>	<code>C</code> and <code>R</code> are <code>bool</code> expressions; <code>δ₁</code> , <code>δ₂</code> , <code>ρ₁</code> and <code>ρ₂</code> are real expressions. Splits a program into a sequence of two programs with intermediate condition <code>R</code> and an event <code>C</code> that may or may not result from the first part. Let <code>c1</code> be the first <code>n</code> statements of the program and <code>c2</code> the remainder and reduce the goal to the following six subgoals: 1. <code>hoare [c1 : P ==> R]</code> 2. <code>phoare [c1 : P ==> C] ◇ δ₁</code> 3. <code>phoare [c2 : R ∧ C ==> Q] ◇ δ₂</code> 4. <code>phoare [c1 : P ==> ! C] ◇ ρ₁</code> 5. <code>phoare [c2 : R ∧ ! C ==> Q] ◇ ρ₂</code> 6. <code>δ₁δ₂ + ρ₁ρ₂ ◇ e</code> When <code>δ₁</code> or <code>δ₂</code> (respectively, <code>ρ₁</code> or <code>ρ₂</code>) is 0, the other can be replaced with a wildcard <code>_</code> and the corresponding subgoal is not generated. If no probabilities are given, the defaults are <code>δ₁ = e</code> , <code>δ₂ = 1</code> , <code>ρ₁ = 0</code> , <code>ρ₂ = _</code> . <code>R</code> is optional and defaults to <code>true</code> .
<code>skip</code>	Reduces an empty statement judgement <code>{P} {Q}</code> to an ambient logic subgoal with <code>P ==> Q</code> as its conclusion.

sp	Reduces a statement judgement with precondition P by consuming the longest prefix of statements built from ordinary assignment ($<-$) and <code>if</code> statements and replacing the goal's precondition with the strongest postcondition R such that $\{P\}$ prefix $\{R\}$ holds Consume the first n statements
sp n	
while I	There is only a placeholder for the pHL version in the RM
wp	Reduces a statement judgement with postcondition Q by consuming the longest suffix of statements built from ordinary assignment ($<-$) and <code>if</code> statements and replacing the goal's postcondition with the weakest precondition R such that $\{R\}$ suffix $\{Q\}$ holds Consume the last n statements
wp n	

13.13.2 Program Transformation Tactics

These tactics apply only to statement judgements and reduce a goal to a single subgoal with a modified program without affecting its specification. The variable c ranges over code positions.

Syntax	Description
inline M1.p1 ... inline* inline occ M.p	Repeatedly replaces calls to the named concrete procedures with their bodies, instantiated with the calls' arguments, until no more calls remain Inlines all concrete procedures until no concrete procedure calls remain Inlines specified occurrences of $M.p$. See <i>Occurrence Selectors</i> for occurrence selector syntax.
interleave	
kill c ! m	Remove superfluous code. If the goal's conclusion is a statement judgement whose program has a statement block starting at position c and having length m and the variables written by this block are not used in the judgement's postcondition or read by the rest of the program, then reduce the goal to two subgoals: 1. $\text{Pr}[\text{block} : \text{true} ==> \text{true}] = 1\%r$ 2. The original goal with the block removed Not used in EasyCrypt library
outline	
swap n m l swap [n..m] k swap n k swap k	Swap statements n through $m - 1$ with statements m through l If $k \geq 0$, move statements n through m down k positions; otherwise, move them up k positions Same as <code>swap [n..n] k</code> If $k \geq 0$, same as <code>swap 1 k</code> ; otherwise, same as <code>swap n k</code> , where n is the length of the program All variants fail if the blocks of statements swapped aren't independent or if the old or new statement positions or ranges aren't valid

13.13.3 Program Specification Tactics

These tactics act on the program's specification without affecting its statements.

Syntax	Description
--------	-------------

case e	Reduces a statement judgement by splitting the goal into two subgoals: 1. The goal with e added to the precondition 2. The goal with !e added to the precondition
elim*	If the goal's conclusion is a judgement or statement judgement whose precondition has the form $\text{exists } (x_1 \dots x_n), P$ changes the quantifier to forall
exists*	
phoare split $\delta_A \delta_B \delta_{AB}$	Splits a judgment whose postcondition is a conjunction or disjunction of A and B into three judgments using the definition $\text{Pr}[A \ \backslash / \ B] = \text{Pr}[A] + \text{Pr}[B] - \text{Pr}[A \ /\ \ B]$ plus an ambient logic goal: 1. $\delta_A + \delta_B - \delta_{AB} \diamond \delta$ 2. $\{P\} \text{ c } \{A\} \diamond \delta_A$ 3. $\{P\} \text{ c } \{B\} \diamond \delta_B$ 4. $\{P\} \text{ c } \{A \ (\backslash / \text{ or } /\ \text{, respectively}) B\} \diamond^{-1} \delta_{AB}$
phoare split ! $\delta_T \delta_I$	Splits a judgment into two judgments using the definition $\text{Pr}[A] = 1 - \text{Pr}[!A]$ plus an ambient logic goal:: 1. $\delta_T - \delta_I \diamond \delta$ 2. $\{P\} \text{ c } \{\text{true}\} \diamond \delta_T$ 3. $\{P\} \text{ c } \{!Q\} \diamond^{-1} \delta_I$
phoare split $\delta_A \delta_{!A} : A$	Splits a judgment into two judgments plus an ambient logic goal according to whether event A happens: 1. $\delta_A + \delta_{!A} \diamond \delta$ 2. $\{P\} \text{ c } \{Q \ /\ \ A\} \diamond \delta_A$ 3. $\{P\} \text{ c } \{Q \ /\ \ !A\} \diamond \delta_{!A}$

13.13.4 Automatic Tactics

These tactics apply other tactics in combination automatically.

Syntax	Description
auto	If the current goal is a statement judgement, uses various program logic tactics to reduce the goal to a simpler one. Never fails, but may fail to make progress.
exfalse	Combines <code>conseq</code> , <code>byequiv</code> , <code>byphoare</code> , <code>hoare</code> and <code>bypr</code> to strengthen the precondition to false and discharges the resulting trivial goal.
sim	

13.14 Probabilistic Relational Hoare Logic Tactics

The tactics in this section act on goals whose conclusion have one of these forms:

```
equiv [M.p ~ N.q : P ==> Q]
equiv [C ~ D : P ==> Q]
```

where M and N are names of modules, p and q are names of procedures in M and N, respectively, C and D are programs and P and Q are predicates.

13.14.1 Program Reasoning Tactics

These tactics reason about a goal by consuming or modifying parts of the program and making corresponding changes to the specification.

Syntax	Description
<code>call (_ : P ==> Q)</code> <code>call p</code> <code>call (_ : I)</code> <code>call (_ : B, I)</code> <code>call (_ : B, I, J)</code>	Reduce a pRHL goal's conclusion that ends with a procedure assignment or call (must it be abstract?) to 2 subgoals: 1. A pHL judgement $\{P\}$ procedure call $\{Q\}$ with probability 1 (i.e., verify user-provided semantics for the call are valid) 2. A pRHL judgement without the procedure call and postconditions P and $Q \Rightarrow$ the original postcondition, with P and Q instantiated with the arguments of the call. <code>p</code> is a proof term whose conclusion is a judgement involving the procedure called, and only the second subgoal is generated. The next four (the first for a concrete procedure and all three for an abstract one) all say "use the specification argument to <code>call</code> generated from [the bad event B and] the invariant[s] I [and J] and automatically apply <code>proc</code> [B] [I] [J] to its first subgoal, pruning the first two subgoals ..." What is a "specification argument"? These are the only occurrences of the term in the RM. How do you generate one from I and/or B and/or J? It sounds like it invokes <code>call</code> (but how, since that's what we're describing) and then invokes <code>proc</code> on its first subgoal. Do you invoke <code>call (_ : I ==> I)</code>?
<code>if</code> <code>if {1} {2}</code>	If the goal's conclusion is a HL statement judgement that begins with an <code>if</code> statement, reduce it to two subgoals: 1. Replace the <code>if</code> statement with the <code>then</code> part and add the condition to the precondition 2. Replace the <code>if</code> statement with the <code>else</code> part and add the negation of the condition to the precondition If the goal's conclusion is a pRHL statement judgement where both programs begin with an <code>if</code> statement, reduce it to three subgoals: 1. The ambient logic formula asserting that the two <code>if</code> conditions are equivalent in the context of the goal's precondition 2. Replace both <code>if</code> statements with their <code>then</code> parts and add the first program's <code>if</code> condition to the precondition 3. Replace both <code>if</code> statements with their <code>else</code> parts and add the negation of the first program's <code>if</code> condition to the precondition Reduce only the specified program as in HL.
<code>proc</code> <code>proc I</code> <code>proc B I</code>	Reduce a HL, pHL or pRHL judgement involving concrete procedure(s) to a statement judgement on their body(ies). Equivalent to <code>proc* ; inline</code>. Reduce a pRHL judgement involving the same abstract procedure to judgements on each procedure it may call. That is, $M.p$ is abstract, so M is a module type. The procedures that $M.p$ may call are limited by the restrictions placed on its access to M's parameter, if any, and its global variables (<code>glob M</code>). Each of these generates a subgoal. I is an invariant that is intended to help somehow as both pre- and postcondition. The RM gives examples where the tactic "fails" because the restrictions are too weak, but I think it just means the subgoals are unprovable.

<pre>proc B I J proc*</pre>	Same as previous up to failure event B Same as previous where invariant J holds after failure Reduce a HL (pRHL) judgement to a HL (pRHL) one-statement judgement consisting of call(s) to the procedure(s). Useful in pRHL when one procedure is abstract and the other concrete.
<pre>rnd rnd f rnd f g rnd{1} rnd{2}</pre>	Reduce a statement judgement that ends with random assignments $x_1 <\\$ d_1$ and $x_2 <\\$ d_2$ by consuming them and replacing the goal's postcondition with the assignments' probabilistic weakest precondition. f maps the type & range of d_1 to that of d_2; omit if f & g are identity g maps the type & range of d_2 to that of d_1; omit if same as f TODO Finish this
<pre>seq n₁ n₂ : R</pre>	If the goal's conclusion is a statement judgement with precondition P, postcondition Q and programs of at least n_1 and n_2 statements, reduce the goal to two pRHL statement judgements as follows: 1. Precondition P, program is the first n_1 statements of the first program and the first n_2 statements of the second program, and postcondition R. 2. Precondition R, the program is all but the first n_1 statements of the first program and all but the first n_2 statements of the second program and postcondition R.
<pre>skip</pre>	Reduce an empty statement judgement $\{P\} \{Q\}$ to an ambient logic subgoal with $P \Rightarrow Q$ as its conclusion.
<pre>sp sp n sp n₁ n₂</pre>	Reduce a statement judgement by consuming all trailing ordinary assignment and conditional statements and applying their strongest postconditions to the goal's precondition. In pHL and HL, consume exactly n statements. In pRHL, consume n_1 statements of program 1 and n_2 of program 2.
<pre>while I</pre>	
<pre>wp wp n wp n₁ n₂</pre>	Reduce a statement judgement by consuming all leading ordinary assignment and conditional statements and applying their weakest precondition to the goal's postcondition (i.e., as conditional expressions and substitutions.) In pHL and HL, consume exactly n statements. In pRHL, consume n_1 statements of program 1 and n_2 of program 2.

13.14.2 Program Transformation Tactics

These tactics apply only to statement judgements and reduce a goal to a single subgoal with a modified program without affecting the program's specification. The variable c ranges over code positions.

Syntax	Description
alias	
cfold	
fission	
fusion	
inline M1.p1 ...	Replace calls to the named concrete procedures with their bodies instantiated with the calls' arguments, recursively
inline*	Inline all concrete procedures, recursively
inline occs M.p	Inline specified occurrences of M.p.

kill	
rcondf n	If the goal's conclusion is a HL statement judgement whose nth statement is an <code>if</code> statement, reduce the goal to two subgoals: 1. a statement judgement with the same precondition, statements 1 to $n - 1$ and a post condition equal to the negation of the <code>if</code> condition 2. a statement judgement equal to the original goal but with the <code>if</code> statement replaced by its <code>else</code> part If the goal's conclusion is a HL statement judgement whose nth statement is an <code>while</code> statement, reduce the goal to two subgoals: 1. a statement judgement with the same precondition, statements 1 to $n - 1$ and a post condition equal to the negation of <code>while</code> condition 2. a statement judgement equal to the original goal but with the <code>while</code> statement removed
rcondf{1} n	Same, but for the left program of a pRHL statement judgement If the precondition mentions the other program, the subgoal will be quantified over a memory of that program
rcondf{2} n	Same, but for the right program of a pRHL statement judgement
rcondt n	If the goal's conclusion is a HL statement judgement whose nth statement is an <code>if</code> statement, reduce the goal to two subgoals: 1. a statement judgement with the same precondition, statements 1 to $n - 1$ and a post condition equal to the <code>if</code> condition 2. a statement judgement equal to the original goal but with the <code>if</code> statement replaced by its <code>then</code> part If the goal's conclusion is a HL statement judgement whose nth statement is an <code>while</code> statement, reduce the goal to two subgoals: 1. a statement judgement with the same precondition, statements 1 to $n - 1$ and a post condition equal to the <code>while</code> condition 2. a statement judgement equal to the original goal but with the <code>while</code> statement preceded by a copy of its body
rcondt {1} n	Same, but for the left program of a pRHL statement judgement If the precondition mentions the other program, the subgoal will be quantified over a memory of that program
rcondt {2} n	Same, but for the right program of a pRHL statement judgement
splitwhile	
swap m n k	swap blocks of statements, which must be independent of each other.
swap [m..n] k	
swap k	
unroll	

13.14.3 Program Specification Tactics

These tactics act on the program's specification without affecting its statements.

Syntax	Description
bypr e_1 e_2	
case	
conseq ($_ : P \implies Q$)	Reduce a pRHL or HL (statement) judgement with pre P' and post Q' to one with pre P and post Q plus two subgoals: 1. $P' \implies P$

conseq p	2. $P \wedge Q \Rightarrow Q'$ p is a proof term that matches the judgement, which can't be a statement judgement.
elim*	If the goal's conclusion is a judgement or statement judgement whose precondition has the form $\text{exists } (x_1 \dots x_n), P$ change the quantifier to forall
exists*	
hoare	
outline	
phoare	
replace	Related to the transitivity tactic. It is never used in the library.
symmetry	
transitivity	

13.14.4 Automatic Tactics

These tactics apply other tactics in combination automatically.

Syntax	Description
auto	
exfalse	
sim	

13.14.5 Advanced Program Tactics

Syntax	Description
eager	A family of tactics

13.15 Other Program Logic Tactics

The tactics in this section act on goals whose conclusions have one of these forms:

$\text{Pr}[M.p(e_1, \dots, e_n) \ @ \ \&m : \phi]$
Is there a statement judgement form?

TODO Is there a full set of tactics for probability judgements? Is there anything else to cover?

13.15.1 Program Reasoning Tactics

These tactics reason about a goal by consuming or modifying parts of the program and making corresponding changes to the specification.

Syntax	Description
call (_ : $P \Rightarrow Q$)	
if	
proc	
rnd	
seq $n_1 \ n_2 : R$	

skip	
sp	
while I	
wp	

13.15.2 Program Transformation Tactics

These tactics apply only to statement judgements and reduce a goal to a single subgoal with a modified program without affecting the program's specification. The variable c ranges over code positions.

Syntax	Description
alias	
cfold	
fission	
fusion	
inline M1.p1 ...	Replace calls to the named concrete procedures with their bodies instantiated with the calls' arguments, recursively
inline*	Inline all concrete procedures, recursively
inline occ M.p	Inlines specified occurrences of M.p. See <i>Occurrence Selectors</i> for occurrence selector syntax.
kill	
rcondf n	
rcondt n	
splitwhile	
swap m n k	swap blocks of statements, which must be independent of each other
swap [m..n] k	
swap k	
unroll	

13.15.3 Program Specification Tactics

These tactics act on the program's specification without affecting its statements.

Syntax	Description
byequiv ($_ : P ==> Q$)	A goal's conclusion of the form $\text{Pr}[M_1.p_1(a_{11}, \dots) \ @ \ \&m_1 : E_1]$ $= \text{Pr}[M_2.p_2(a_{21}, \dots) \ @ \ \&m_2 : E_2]$ is reduced to 3 subgoals: 1. equiv $[M_1.p_1 \sim M_2.p_2 : P ==> Q]$ 2. An ambient logic goal whose conclusion is P, where references to $\&1$ and $\&2$ are replaced with $\&m_1$ and $\&m_2$ and the parameters of $M_1.p_1$ and $M_2.p_2$ are replaced with their arguments 3. An ambient logic goal whose conclusion is $Q \Rightarrow E_1\{1\} \Leftrightarrow E_2\{2\}$ A goal's conclusion of the form $\text{Pr}[M_1.p_1(a_{11}, \dots) \ @ \ \&m_1 : E_1]$ $\leq \text{Pr}[M_2.p_2(a_{21}, \dots) \ @ \ \&m_2 : E_2]$ is also reduced to 3 subgoals, where the third's conclusion is: $3. Q \Rightarrow E_1\{1\} \Rightarrow E_2\{2\}$ A goal's conclusion of the form

<pre> byequiv (_: P ==> Q): B₁ byequiv p [: B₁] </pre>	$\text{Pr}[M_1.p_1(a_{11}, \dots) @ \&m_1 : E_1]$ $\leq \text{Pr}[M_2.p_2(a_{21}, \dots) @ \&m_2 : E_2]$ $+ \text{Pr}[M_2.p_2(a_{21}, \dots) @ \&m_2 : B_2]$ is also reduced to 3 subgoals, where the third's conclusion is: $3. Q \Rightarrow !B_2\{2\} \Rightarrow E_1\{1\} \Rightarrow E_2\{2\}$ A goal's conclusion of the form $\text{Pr}[M_1.p_1(a_{11}, \dots) @ \&m_1 : E_1]$ $- \text{Pr}[M_2.p_2(a_{21}, \dots) @ \&m_2 : E_2] $ $\leq \text{Pr}[M_2.p_2(a_{21}, \dots) @ \&m_2 : B_2]$ is also reduced to 3 subgoals, where the third's conclusion is: $3. Q \Rightarrow (B_1\{1\} \Leftrightarrow B_2\{2\}) /\ \ !B_2\{2\}$ $\Rightarrow (E_1\{1\} \Leftrightarrow E_2\{2\})$ In any of the above forms, P or Q can be <code>_</code> to be inferred. No first argument means both are <code>_</code>. Any of the above forms can use a proof term <code>p</code> in place of the first argument and the first subgoal is not generated.
<pre> byphoare (_: P ==> Q) byphoare p </pre>	A goal's conclusion of the form $\text{Pr}[M.p(a_1, \dots, a_n) @ \&m : E] = e$ is reduced to 3 subgoals: 1. <code>phoare [M.p : P ==> Q] = e</code> 2. An ambient logic goal whose conclusion is P, where references are looked up in <code>&m</code> and the parameters of <code>M.p</code> are replaced with their arguments. 3. An ambient logic goal whose conclusion is $Q \Leftrightarrow E$. A goal's conclusion of the form $e \leq \text{Pr}[M.p(a_1, \dots, a_n) @ \&m : E]$ is also reduced to 3 subgoals, where the first is: 1. <code>phoare [M.p : P ==> Q] >= e</code> A goal's conclusion of the form $\text{Pr}[M.p(a_1, \dots, a_n) @ \&m : E] \leq e$ is also reduced to 3 subgoals, where the first is: 1. <code>phoare [M.p : P ==> Q] <= e</code> In any of the above forms, P or Q can be <code>_</code> to be inferred. No first argument means both are <code>_</code>. Any of the above forms can use a proof term <code>p</code> in place of the first argument and the first subgoal is not generated.

13.15.4 Advanced Program Tactics

Syntax	Description
<pre> fel init ctr stepub bound bad conds inv </pre>	The name stands for “failure event lemma”. A goal's conclusion of the form $\text{Pr}[M.p(a_1, \dots, a_n) @ \&m : \phi] \leq ub$ where • <code>ub</code> (“upper bound”) is a <code>real</code> expression • <code>ctr</code> (“counter”) is an <code>int</code> expression involving program variables • <code>bad</code> (“bad event”) is a <code>bool</code> expression involving program variables • <code>inv</code> (“invariant”) is an optional <code>bool</code> expression involving program variables that defaults to <code>true</code>

	• <code>init</code> ("initial") is a natural number no larger than the length of <code>M.p</code> • <code>bound</code> is an <code>int</code> expression such that $\varphi \wedge \text{inv} \Rightarrow \text{bad} \wedge \text{ctr} \leq \text{bound}$ • <code>conds</code> ("(pre)conditions") is a list of the form <code>N_i.p_i : φ_i</code> where the <code>N_i.p_i</code> are procedures and the φ_i are <code>bool</code> expressions involving program variables and procedure parameters; blah blah • <code>stepub</code> ("step upper bound") is a function of type <code>int -> real</code> • ... is reduced to submission:
--	---